APOSTILA TÉCNICA

Manipulação de Arquivos IFC

openBIM, esquema IFC, IfcOpenShell, IDS e interoperabilidade: do conceito à automação da informação em modelos IFC.

PROJETO / CLIENTE
Material didático | Arven LOD

CÓDIGO DO DOCUMENTO
ARV-000-RT-001

REVISÃO
Rev. 00

DATA DE EMISSÃO
19/09/2026

Sumário

Apresentação **7**
Sobre esta apostila 7
Público e pré-requisitos 7
Organização 8
Convenções 8
Como estudar 9
Material de apoio 9

0 Preparação do ambiente de trabalho **10**
0.1 Conceitos básicos para quem está começando 10
0.2 A cadeia de ferramentas 12
0.3 Python e o interpretador 12
0.4 Por que usar um ambiente virtual 13
0.5 Criação e ativação do ambiente virtual 14
0.6 Bibliotecas, pip e dependências 15
0.7 Editor de código: VS Code 17
0.8 Visualizador de IFC: Blender e Bonsai 17
0.9 Arquivos IFC de exemplo 18
0.10 Organização da pasta de trabalho 20
0.11 Verificação do ambiente 20
0.12 Diagnóstico de problemas comuns 24
0.13 Resumo do capítulo 24
0.14 Vocabulário técnico 25
0.15 Exercícios 25
0.16 Respostas dos exercícios 26
0.17 Referências 26

1 Fundamentos openBIM **28**
1.1 O que é BIM 28
1.2 O problema da troca de informação 29
1.3 O que é openBIM 29
1.4 A buildingSMART International 30
1.5 Os padrões openBIM 30
1.6 Como os padrões trabalham juntos 32
1.7 As versões do IFC 33
1.8 MVDs oficiais 35
1.9 IFC e a gestão da informação: a série ISO 19650 35
1.10 Contexto normativo brasileiro 36
1.11 Resumo do capítulo 37
1.12 Vocabulário técnico 37
1.13 Exercícios 38
1.14 Respostas dos exercícios 38
1.15 Referências 39

2 Anatomia do arquivo IFC **41**
2.1 Introdução: por que ler o arquivo em texto 41
2.2 O formato STEP 42

2.3 O cabeçalho (HEADER) ... 42
2.4 Os dados (DATA): instâncias ... 43
2.5 Os tipos de valores ... 44
2.6 Lendo uma linha: atributos por posição ... 45
2.7 Seguindo as referências: o arquivo completo de uma viga ... 46
2.8 Exemplos de várias áreas ... 48
2.9 Variantes do arquivo ... 49
2.10 Leitura com um script ... 50
2.11 Resumo do capítulo ... 51
2.12 Vocabulário técnico ... 52
2.13 Exercícios ... 52
2.14 Respostas dos exercícios ... 53
2.15 Referências ... 54

3 O esquema IFC ... 55
3.1 Introdução: o que é um esquema de dados ... 55
3.2 A linguagem EXPRESS ... 57
3.3 A hierarquia principal ... 59
3.4 Herança em várias áreas ... 60
3.5 Atributos herdados e diferenças entre versões ... 61
3.6 GlobalId ... 62
3.7 OwnerHistory ... 63
3.8 Tipos e ocorrências ... 64
3.9 PredefinedType e ObjectType ... 67
3.10 Relações objetificadas e atributos inversos ... 69
3.11 Consulta com um script ... 71
3.12 Resumo do capítulo ... 73
3.13 Vocabulário técnico ... 74
3.14 Exercícios ... 74
3.15 Respostas dos exercícios ... 75
3.16 Referências ... 76

4 Estrutura espacial e relacionamentos ... 77
4.1 Introdução: organizar um modelo ... 77
4.2 A estrutura espacial ... 78
4.3 Agregação: IfcRelAggregates ... 82
4.4 Contenção e referência espacial ... 83
4.5 Aninhamento: IfcRelNests ... 84
4.6 O alinhamento na estrutura do projeto (IFC 4.3) ... 85
4.7 Tipos e declarações ... 85
4.8 Grupos, sistemas e zonas ... 85
4.9 A mesma rodovia em IFC2x3, IFC4 e IFC 4.3 ... 87
4.10 Inspeção com um script ... 88
4.11 Resumo do capítulo ... 90
4.12 Vocabulário técnico ... 90
4.13 Exercícios ... 91
4.14 Respostas dos exercícios ... 91
4.15 Referências ... 93

5 Informação não geométrica ... 94
5.1 Introdução: a ficha técnica do objeto ... 94

5.2 Conjuntos de propriedades ... 95
5.3 Tipos de propriedade ... 96
5.4 Conjuntos padronizados e personalizados ... 98
5.5 Quantidades ... 100
5.6 Unidades ... 101
5.7 Materiais ... 103
5.8 Classificação ... 104
5.9 Documentos, aprovações e o registro da revisão ... 106
5.10 Do BEP ao IFC: mapear requisitos de informação ... 107
5.11 Leitura com um script ... 108
5.12 Resumo do capítulo ... 111
5.13 Vocabulário técnico ... 111
5.14 Exercícios ... 112
5.15 Respostas dos exercícios ... 112
5.16 Referências ... 113

6 Domínios: instalações, estruturas, arquitetura e infraestrutura ... 114
6.1 Introdução: disciplinas dentro de um único esquema ... 114
6.2 Instalações e processos: elementos de distribuição ... 115
6.3 Conectividade: portas e conexões ... 117
6.4 Estruturas de concreto e aço ... 122
6.5 Arquitetura ... 124
6.6 Infraestrutura (IFC 4.3) ... 125
6.7 IfcBuildingElementProxy: o último recurso ... 126
6.8 Resumo do capítulo ... 127
6.9 Vocabulário técnico ... 127
6.10 Exercícios ... 128
6.11 Respostas dos exercícios ... 129
6.12 Referências ... 129

7 Geometria e posicionamento ... 131
7.1 Introdução: onde está e que forma tem ... 131
7.2 Posicionamento local ... 132
7.3 Contextos e representações ... 133
7.4 Tipos de geometria ... 135
7.5 Alinhamentos e posicionamento linear (IFC 4.3) ... 137
7.6 Georreferenciamento ... 139
7.7 Consulta com um script ... 141
7.8 Resumo do capítulo ... 144
7.9 Vocabulário técnico ... 144
7.10 Exercícios ... 145
7.11 Respostas dos exercícios ... 145
7.12 Referências ... 146

8 Python essencial para IFC ... 147
8.1 Introdução: o que é programar ... 147
8.2 O primeiro programa ... 148
8.3 Variáveis e tipos básicos ... 148
8.4 Coleções: listas, tuplas, dicionários e conjuntos ... 150
8.5 Decisões e repetições ... 151
8.6 Funções ... 153

8.7 Módulos e bibliotecas ... 153
8.8 Arquivos e caminhos ... 154
8.9 Erros: ler e tratar ... 155
8.10 Classes e objetos ... 157
8.11 Planilhas Excel ... 158
8.12 Argumentos de linha de comando ... 160
8.13 Exemplo integrador: do IFC para o Excel ... 160
8.14 Boas práticas ... 162
8.15 Resumo do capítulo ... 162
8.16 Vocabulário técnico ... 162
8.17 Exercícios ... 163
8.18 Respostas dos exercícios ... 164
8.19 Referências ... 164

9 IfcOpenShell I: leitura e consulta ... 165
9.1 Introdução: o que é o IfcOpenShell ... 165
9.2 Abrir o arquivo e encontrar instâncias ... 166
9.3 Ler atributos ... 168
9.4 Navegar pelas relações ... 169
9.5 Funções prontas: ifcopenshell.util ... 169
9.6 Consultas em texto: o seletor ... 171
9.7 Exportar para planilha ... 174
9.8 Consultas que funcionam em todas as versões ... 175
9.9 Resumo do capítulo ... 176
9.10 Vocabulário técnico ... 176
9.11 Exercícios ... 177
9.12 Respostas dos exercícios ... 177
9.13 Referências ... 178

10 IfcOpenShell II: edição e metadados em lote ... 180
10.1 Introdução: por que editar por código ... 180
10.2 A API de edição ... 181
10.3 Edição básica ... 182
10.4 Corrigir a classe de elementos ... 185
10.5 Metadados em lote a partir de planilha ... 187
10.6 Outras operações ... 194
10.7 Resumo do capítulo ... 194
10.8 Vocabulário técnico ... 195
10.9 Exercícios ... 195
10.10 Respostas dos exercícios ... 196
10.11 Referências ... 197

11 Geometria por código ... 198
11.1 Introdução: da descrição à forma ... 198
11.2 Medir elementos ... 199
11.3 Criar geometria ... 201
11.4 Exportar malhas ... 204
11.5 Detectar interferências ... 205
11.6 Resumo do capítulo ... 207
11.7 Vocabulário técnico ... 207
11.8 Exercícios ... 207

11.9 Respostas dos exercícios ... 208
11.10 Referências ... 210

12 Requisitos e validação: IDS, IfcTester e BCF ... 211
12.1 Introdução: o que significa validar ... 211
12.2 Validação do esquema ... 212
12.3 IDS: requisitos de informação legíveis por máquina ... 215
12.4 Verificação com o IfcTester ... 221
12.5 Comparar revisões com o ifcdiff ... 231
12.6 BCF: comunicar os problemas ... 234
12.7 Resumo do capítulo ... 236
12.8 Vocabulário técnico ... 237
12.9 Exercícios ... 237
12.10 Respostas dos exercícios ... 238
12.11 Referências ... 241

13 Interoperabilidade com softwares Autodesk ... 242
13.1 Introdução: exportar é traduzir ... 242
13.2 Método de diagnóstico: o raio X do arquivo exportado ... 243
13.3 Revit 2025 ... 249
13.4 Civil 3D 2026 ... 253
13.5 Plant 3D ... 256
13.6 Navisworks 2026 ... 259
13.7 Autodesk Forma e Autodesk Tandem ... 259
13.8 Problemas frequentes e correções ... 260
13.9 Resumo do capítulo ... 261
13.10 Vocabulário técnico ... 261
13.11 Exercícios ... 262
13.12 Respostas dos exercícios ... 263
13.13 Referências ... 266

14 Projeto integrador: do BEP ao relatório de não conformidades ... 268
14.1 O problema e a solução ... 268
14.2 A planilha do BEP ... 269
14.3 O modelo de exemplo ... 272
14.4 O módulo da planilha ... 273
14.5 O integrador ... 275
14.6 Execução e leitura dos resultados ... 279
14.7 O ciclo de correção ... 281
14.8 Como adaptar ao seu projeto ... 281
14.9 Resumo do capítulo ... 282
14.10 Exercícios ... 282
14.11 Respostas dos exercícios ... 283
14.12 Referências ... 286

Controle de revisões ... 287

Apresentação

Sobre esta apostila

Esta apostila ensina a ler, interpretar, consultar e modificar arquivos IFC (Industry Foundation Classes), o formato aberto de troca de informação da construção. O percurso vai do conceito à prática: primeiro o contexto do openBIM e a estrutura do arquivo, depois o esquema de dados e, nos blocos seguintes, a manipulação dos arquivos por código com a biblioteca IfcOpenShell, a validação de requisitos com IDS e a interoperabilidade com os softwares de autoria.

O texto foi escrito para estudo individual. Cada capítulo explica o assunto por completo, traz exemplos resolvidos passo a passo, um resumo e exercícios com respostas comentadas.

Os exemplos não se restringem a um tipo de obra. O IFC 4.3, versão de referência desta apostila, descreve edificações e infraestrutura no mesmo esquema, e os exemplos percorrem várias áreas.

ÁREA	EXEMPLOS DE ENTIDADES IFC 4.3 USADAS NA APOSTILA
Edificações (arquitetura)	`IfcBuilding`, `IfcBuildingStorey`, `IfcSpace`, `IfcWall`, `IfcSlab`, `IfcRoof`
Estruturas	`IfcBeam`, `IfcColumn`, `IfcMember`, `IfcFooting`, `IfcPlate`
Instalações prediais e climatização	`IfcDuctSegment`, `IfcAirTerminal`, `IfcPipeSegment`, `IfcCableSegment`
Rodovias	`IfcRoad`, `IfcRoadPart`, `IfcCourse`, `IfcPavement`, `IfcEarthworksFill`, `IfcAlignment`
Pontes	`IfcBridge`, `IfcBridgePart`
Ferrovias	`IfcRailway`, `IfcRailwayPart`, `IfcTrackElement`, `IfcRail`
Instalações industriais	`IfcPipeSegment`, `IfcPump`, `IfcTank`, `IfcValve`, `IfcDistributionSystem`
Paisagismo e entorno	`IfcSite`, `IfcGeographicElement`

Tabela A.1: Áreas de aplicação cobertas pelos exemplos.

Público e pré-requisitos

A apostila se destina a profissionais que já trabalham com modelagem BIM em algum software de autoria (por exemplo, Revit, Civil 3D ou similares) e querem entender o que acontece dentro do arquivo IFC e automatizar tarefas sobre ele.

Não é necessário conhecimento prévio de IFC nem de programação. Os capítulos iniciais não usam código. A partir do Bloco 3, um capítulo dedicado apresenta o essencial da linguagem Python antes do uso da biblioteca IfcOpenShell.

Organização

BLOCO	CAPÍTULOS	CONTEÚDO
1	0 a 3	Ambiente de trabalho, fundamentos openBIM, anatomia do arquivo IFC e esquema IFC
2	4 a 7	Estrutura espacial e relacionamentos, informação não geométrica, domínios de infraestrutura e de instalações, geometria e posicionamento
3	8	Python essencial para IFC
4	9 a 14	IfcOpenShell (leitura, edição e geometria), requisitos e validação (IDS, IfcTester, BCF), interoperabilidade com softwares Autodesk e projeto integrador

Tabela A.2: Organização da apostila.

Cada capítulo segue a mesma estrutura:

- objetivos de aprendizagem;
- conteúdo, com exemplos resolvidos;
- resumo do capítulo;
- vocabulário técnico em inglês e português;
- exercícios em três níveis: fixação (conceitos), aplicação (uso direto) e desafio (análise);
- respostas comentadas dos exercícios;
- referências.

Convenções

- **Termos do IFC** ficam no original, em fonte monoespaçada (`IfcWall`), e são explicados em português na primeira ocorrência.
- **Versões do esquema** são indicadas como IFC2x3, IFC4 e IFC4.3. A existência de cada entidade e de cada atributo citado foi conferida nas definições de esquema distribuídas com a biblioteca IfcOpenShell 0.8.5.
- **Linhas de arquivo IFC** são reproduzidas exatamente como estão nos arquivos de exemplo.
- **Código Python** foi executado com Python 3.11 e 3.13, IfcOpenShell 0.8.5, IfcTester 0.8.5 e ifcdiff 0.8.5, nos arquivos indicados, com resultados idênticos nas duas versões do Python. A saída mostrada em cada exemplo é a obtida no teste. Os testes foram feitos em Linux; por isso, onde a saída mostra um caminho de arquivo, ele aparece com barras normais (`/`), e não com as barras invertidas do Windows. Quando uma saída varia a cada execução (horários, identificadores gerados aleatoriamente), o texto avisa.
- **Caminhos** seguem a pasta de trabalho sugerida no Capítulo 0 (`C:\IFC_Curso`), com uma subpasta por capítulo em `C:\IFC_Curso\exercicios`. Se você usar outra pasta, ajuste os caminhos.
- **Arquivos originais** nunca são alterados pelos scripts: todo resultado é gravado num arquivo novo, e os scripts recusam gravar sobre a entrada.

> **OBSERVAÇÃO**
>
> Normas, versões de software e atos legais mudam. As informações desta edição foram conferidas nas fontes oficiais em setembro de 2026. Nos capítulos sobre softwares comerciais, as versões consideradas estão indicadas no início do capítulo. Antes de usar um requisito em contrato, confira a versão vigente.

Como estudar

Os capítulos foram escritos para leitura em sequência: cada um usa conceitos e arquivos dos anteriores. Nos Blocos 1 e 2, abra os arquivos de amostra num visualizador e num editor de texto enquanto lê; nos Blocos 3 e 4, digite e execute os scripts, compare a sua saída com a do texto e só depois passe aos exercícios. As respostas comentadas ficam no fim de cada capítulo; tente resolver antes de consultá-las.

Material de apoio

Os exemplos usam arquivos oficiais de amostra da buildingSMART International, de acesso livre. O Capítulo 0 mostra como obtê-los e organizá-los. Os demais arquivos (a rede de tubulação do Capítulo 6, as planilhas de metadados, os arquivos IDS e os modelos de exemplo dos Capítulos 12 a 14) são criados pelos próprios scripts da apostila.

0 Preparação do ambiente de trabalho

Este capítulo monta o conjunto de ferramentas usado em toda a apostila: a linguagem Python, um ambiente isolado para as bibliotecas, o editor de código, o visualizador de IFC e os arquivos de exemplo. Ao final, você terá um ambiente testado e saberá identificar e corrigir os problemas mais comuns.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar a função de cada peça do ambiente e a dependência entre elas;
- instalar o Python e identificar qual instalação um terminal está usando;
- criar, ativar e conferir um ambiente virtual;
- instalar bibliotecas com o `pip` e reconhecer dependências;
- configurar o VS Code para executar scripts no ambiente correto;
- instalar o Blender com o complemento Bonsai e abrir um arquivo IFC;
- obter e organizar os arquivos de exemplo da buildingSMART;
- diagnosticar os erros mais frequentes.

0.1 Conceitos básicos para quem está começando

Esta seção apresenta cinco ideias usadas em todo o capítulo. Quem já usa terminal e scripts pode passar direto para a seção 0.2.

0.1.1 Arquivo, pasta e caminho

Todo arquivo do computador fica dentro de uma pasta, que pode estar dentro de outra pasta, e assim por diante. O **caminho** é o endereço completo de um arquivo ou pasta, escrito da unidade de disco até o nome final, com as pastas separadas por barra invertida (`\`) no Windows:

```
C:\IFC_Curso\amostras\bSI\Certification-datasets-main\README.md
│  │         │        │   │                           └ arquivo
│  │         │        │   └ pasta dentro de bSI
│  │         │        └ pasta dentro de amostras
│  │         └ pasta dentro de IFC_Curso
│  └ pasta na raiz do disco
└ unidade de disco
```

A **extensão** é a parte do nome depois do último ponto (`.md`, `.ifc`, `.py`). Ela indica o tipo do arquivo e qual programa costuma abri-lo. O Windows pode esconder as extensões; para vê-las, no Explorador de Arquivos, marque Exibir > Mostrar > Extensões de nomes de arquivos.

Quando um caminho contém espaços, ele deve ser escrito entre aspas nos comandos: `cd "C:\Pasta com espaço"`.

0.1.2 Terminal e comandos

O **terminal** é uma janela em que se dão ordens ao computador por escrito, em vez de clicar em menus. No Windows, o terminal padrão é o **PowerShell**. Para abri-lo, clique no menu Iniciar, digite "PowerShell" e pressione Enter.

Cada ordem é um **comando**: uma linha de texto que começa com o nome do programa ou da ação, seguido de **argumentos** (informações complementares). Exemplo:

```
cd C:\IFC_Curso
```

- `cd` (change directory) é o comando: mudar a pasta atual do terminal;
- `C:\IFC_Curso` é o argumento: para qual pasta ir.

O terminal sempre "está" em uma pasta, mostrada no início da linha (`PS C:\IFC_Curso>`). Comandos que citam arquivos sem caminho completo procuram esses arquivos na pasta atual.

COMANDO	O QUE FAZ
`cd <pasta>`	Muda a pasta atual
`cd ..`	Sobe um nível de pasta
`dir`	Lista o conteúdo da pasta atual
`mkdir <pasta>`	Cria uma pasta
`Get-Content <arquivo>`	Mostra o conteúdo de um arquivo de texto
`cls`	Limpa a tela

Tabela 0.1: Comandos básicos do PowerShell.

Para executar um comando, digite-o (ou cole-o) e pressione Enter. A resposta aparece logo abaixo. Mensagens em vermelho indicam erro; leia a primeira linha da mensagem, que costuma dizer a causa.

> **ATENÇÃO**
>
> Nesta apostila, os comandos aparecem em blocos de código. Copie apenas o conteúdo do bloco. Se o arquivo Markdown for aberto como texto puro, aparecem linhas com três crases (```) antes e depois dos comandos: elas marcam o início e o fim do bloco e não devem ser coladas no terminal.

0.1.3 Programa, script e linguagem de programação

Uma **linguagem de programação** é um conjunto de regras para escrever instruções que o computador executa. Um **script** é um arquivo de texto com essas instruções. Nesta apostila, os scripts são escritos na linguagem **Python** e têm a extensão `.py`.

Quem já escreveu rotinas AutoLISP ou gráficos no Dynamo já programou: uma rotina `.lsp` é um script na linguagem AutoLISP, executado pelo AutoCAD. Um script Python é executado por um programa chamado **interpretador Python** (seção 0.3).

0.1.4 Biblioteca

Uma **biblioteca** é um conjunto de funções prontas, escritas por outras pessoas, que um script pode usar. Em vez de programar do zero a leitura de um arquivo IFC, o script usa a biblioteca **IfcOpenShell**, que já sabe fazer isso. Bibliotecas são instaladas no computador uma vez e usadas por quantos scripts forem necessários.

0.1.5 Editor de código e visualizador

Um **editor de código** é um programa para escrever scripts, com recursos como cores para destacar a sintaxe e execução com um clique. Esta apostila usa o **VS Code** (Visual Studio Code), gratuito.

Um **visualizador de IFC** é um programa que abre o modelo e mostra a geometria e as informações. Esta apostila usa o **Blender** com o complemento **Bonsai**, ambos gratuitos.

0.2 A cadeia de ferramentas

Trabalhar com IFC por código exige várias ferramentas encadeadas. Cada uma depende da anterior:

```
Windows
  └─ Python (interpretador: programa que executa o código)
       └─ Ambiente virtual (Python exclusivo do projeto)
            └─ Bibliotecas (ifcopenshell, ifctester, openpyxl, pandas)
                 └─ Scripts .py, escritos e executados no VS Code
                      └─ Arquivos IFC, lidos e gravados pelos scripts
                           └─ Blender + Bonsai, para visualizar e conferir
```

Figura 0.1: Cadeia de ferramentas do ambiente de trabalho.

A regra que explica a maioria dos problemas de ambiente é simples: **um script só enxerga as bibliotecas instaladas no Python que o executa**. Se a biblioteca foi instalada num Python e o script roda em outro, o script informa que a biblioteca "não existe", embora ela esteja no computador.

O raciocínio é conhecido de quem usa AutoCAD e Dynamo: uma rotina AutoLISP só funciona se o arquivo `.lsp` foi carregado na sessão em uso, e um gráfico do Dynamo só funciona se os pacotes estão instalados na versão do Dynamo que o executa.

0.3 Python e o interpretador

Python é uma linguagem de programação. O **interpretador** é o programa que lê um arquivo `.py` e executa as instruções. Quando se diz "o Python instalado", fala-se desse programa.

Um mesmo computador pode ter várias instalações do Python, com versões diferentes ou até iguais, vindas de origens diferentes. No Windows, as origens mais comuns são:

ORIGEM	LOCAL TÍPICO DA INSTALAÇÃO	OBSERVAÇÃO
Python Install Manager (python.org)	`%LocalAppData%\Python\pythoncore-<versão>-64\`	Forma recomendada pela documentação oficial a partir do Python 3.14; instala por usuário, sem administrador
Microsoft Store	`%LocalAppData%\Microsoft\WindowsApps\`	Às vezes instalada automaticamente ao digitar `python` num terminal
Instalador tradicional (python.org)	`%LocalAppData%\Programs\Python\`	Descontinuado a partir do Python 3.16, segundo a documentação oficial

Tabela 0.2: Origens comuns do Python no Windows.

Para saber o que existe e o que cada comando usa, abra o PowerShell (menu Iniciar, digite "PowerShell") e execute:

```
py --version        # versão que o lançador py usa por padrão
py -0p              # todas as instalações registradas, com o caminho
python --version    # versão que o comando python usa
where.exe python    # ordem em que o Windows encontra o python
```

- O **lançador** `py` escolhe entre as instalações registradas. A marcada com `*` na lista de `py -0p` é a padrão.
- O comando **`python`** usa a primeira instalação encontrada na lista de caminhos do sistema. A primeira linha de `where.exe python` mostra qual é.
- No PowerShell, escreva `where.exe`. O comando `where` sozinho tem outro significado nesse terminal.

Exemplo resolvido 0.1: duas instalações da mesma versão

Suponha que os comandos acima respondam:

```
PS> py --version
Python 3.13.15
PS> py -0p
 -V:3.13          C:\Users\<usuário>\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.
3.13_qbz5n2kfra8p0\python.exe
 -V:3.13[-64] *   C:\Users\<usuário>\AppData\Local\Python\pythoncore-3.13-64\python.exe
PS> python --version
Python 3.13.14
PS> where.exe python
C:\Users\<usuário>\AppData\Local\Microsoft\WindowsApps\python.exe
```

Leitura:

1. Há duas instalações do 3.13: uma da Microsoft Store (3.13.14) e outra do Python Install Manager (3.13.15).
2. O `py` usa por padrão a do Install Manager (marcada com `*`).
3. O `python` usa a da Store, porque ela aparece primeiro na lista de caminhos.
4. As duas têm o mesmo rótulo (`3.13`). Um comando como `py -V:3.13` pode escolher qualquer uma delas.

Conclusão: para criar o ambiente do projeto a partir de uma instalação específica, use o caminho completo do executável (seção 0.5). Não é preciso desinstalar nada.

Faixa de versões. A biblioteca IfcOpenShell 0.8.5 é distribuída para Python 3.10 a 3.14. Fora dessa faixa, a instalação falha. Se não houver uma versão compatível, instale a 3.13 pelo Python Install Manager (disponível em python.org/downloads) e, num PowerShell novo, execute:

```
py install 3.13
py -0p
```

0.4 Por que usar um ambiente virtual

Um **ambiente virtual** (virtual environment) é uma pasta com um Python próprio de um projeto. Ele é criado a partir de uma instalação base, mas guarda as suas bibliotecas separadas das demais.

Três razões justificam o uso:

1. **Isolamento de versões.** Cada projeto pode exigir versões diferentes de uma biblioteca. Atualizar uma biblioteca para um projeto não quebra outro.
2. **Reprodutibilidade.** A lista de bibliotecas do ambiente descreve exatamente do que o projeto depende.
3. **Descarte seguro.** Se o ambiente se corromper, basta apagar a pasta e recriá-la, sem afetar o sistema nem outros projetos.

A comparação com o dia a dia de projeto é direta. É como manter um perfil do software de autoria com um conjunto específico de complementos carregados para um tipo de trabalho, sem alterar a configuração usada nos demais projetos.

0.5 Criação e ativação do ambiente virtual

0.5.1 Pasta de trabalho

Esta apostila usa a pasta `C:\IFC_Curso` para tudo: ambiente, amostras e exercícios. Duas recomendações orientam a escolha:

- **Fora de pastas sincronizadas** (OneDrive, Dropbox e similares). O ambiente virtual tem milhares de arquivos, e a sincronização pode bloqueá-los durante o uso.
- **Caminho curto.** O Windows, por padrão, não grava arquivos cujo caminho completo tenha 260 caracteres ou mais (seção 0.9.3).

0.5.2 Criação

A partir de uma instalação específica (recomendado quando há mais de uma):

```
mkdir C:\IFC_Curso
C:\Users\<usuário>\AppData\Local\Python\pythoncore-3.13-64\python.exe -m venv C:\IFC_Curso\.venv
```

Ou, quando há uma única instalação compatível:

```
py -V:3.13 -m venv C:\IFC_Curso\.venv
```

O arquivo `pyvenv.cfg`, criado dentro do ambiente, registra a origem. A linha `home` indica a instalação base:

```
Get-Content C:\IFC_Curso\.venv\pyvenv.cfg
```

```
home = C:\Users\<usuário>\AppData\Local\Python\pythoncore-3.13-64
include-system-site-packages = false
version = 3.13.15
```

0.5.3 Ativação

```
C:\IFC_Curso\.venv\Scripts\Activate.ps1
```

A ativação faz o comando `python` daquele terminal apontar para o ambiente. A linha de comando passa a começar com `(.venv)`. A ativação vale apenas para a janela em que foi feita: cada novo PowerShell precisa ser ativado de novo.

Na primeira tentativa, o Windows costuma bloquear a ativação com a mensagem de que "a execução de scripts foi desabilitada neste sistema". O motivo é que `Activate.ps1` é um script do PowerShell, e o Windows bloqueia scripts por padrão. A liberação é feita uma única vez, apenas para a sua conta, e não exige administrador:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

A política `RemoteSigned` libera scripts criados no próprio computador e continua exigindo assinatura digital dos baixados da internet. O PowerShell pede confirmação; responda `S`.

> **OBSERVAÇÃO**
> Se a política de execução não puder ser alterada (por exemplo, por regra da empresa), use o Prompt de Comando (`cmd`) e ative o ambiente com `C:\IFC_Curso\.venv\Scripts\activate.bat`.

0.5.4 Conferência

Com o ambiente ativo:

```
python --version
where.exe python
```

A versão deve ser a da instalação base escolhida, e a primeira linha de `where.exe python` deve ser `C:\IFC_Curso\.venv\Scripts\python.exe`. As demais instalações continuam na lista, depois do ambiente, e só são usadas fora dele.

0.6 Bibliotecas, pip e dependências

0.6.1 O pip

O **`pip`** é o instalador de bibliotecas do Python. Ele baixa pacotes do **PyPI** (Python Package Index), o repositório público oficial, e os instala no ambiente ativo. O papel é o mesmo do gerenciador de pacotes do Dynamo.

```
python -m pip install --upgrade pip
python -m pip install ifcopenshell ifctester ifcdiff openpyxl pandas pytest
python -m pip list
```

Escrever `python -m pip`, e não apenas `pip`, garante que a instalação vá para o mesmo Python que o comando `python` usa, isto é, para o ambiente ativo.

BIBLIOTECA	VERSÃO DE REFERÊNCIA	FUNÇÃO
`ifcopenshell`	0.8.5	Ler, consultar, editar e criar arquivos IFC
`ifctester`	0.8.5	Validar modelos IFC contra requisitos IDS

BIBLIOTECA	VERSÃO DE REFERÊNCIA	FUNÇÃO
`ifcdiff`	0.8.5	Comparar duas revisões de um modelo IFC (Capítulo 12)
`openpyxl`	3.1.5	Ler e gravar planilhas Excel
`pandas`	3.0.6	Organizar dados em tabelas e gerar relatórios
`pytest`	9.1.1	Ferramenta de testes; o validador do IfcOpenShell a usa para executar as regras do esquema (opção `--rules`, Capítulos 10 a 12)

Tabela 0.3: Bibliotecas usadas na apostila.

0.6.2 Dependências

Ao instalar as seis bibliotecas, o `pip` instala cerca de quarenta pacotes. Os demais são **dependências**: bibliotecas de que as bibliotecas pedidas precisam para funcionar. O `pip` as identifica e instala automaticamente.

```
ifcopenshell ──> numpy, shapely, lark, isodate, python-dateutil (──> six), typing_extensions
ifctester ─────> ifcopenshell, xmlschema (──> elementpath), bcf-client (──> xsdata, requests),
                 odfpy (──> defusedxml), pystache, flask (──> componentes internos)
ifcdiff ───────> ifcopenshell, deepdiff (──> orderly-set, cachebox)
openpyxl ──────> et_xmlfile
pandas ────────> numpy, python-dateutil, tzdata
pytest ────────> pluggy, iniconfig, packaging, pygments (e colorama, no Windows)
```

Figura 0.2: Árvore de dependências das bibliotecas da apostila.

DEPENDÊNCIA	FUNÇÃO
`numpy`	Cálculo com listas de números (coordenadas, vetores, matrizes)
`shapely`	Geometria 2D: áreas, polígonos, interseções
`lark`	Interpretação da linguagem de consulta usada para selecionar elementos
`isodate`, `python-dateutil`	Datas no padrão ISO 8601, o formato de data do IFC
`xmlschema`	Conferência de arquivos IDS (que são XML) contra o esquema oficial do IDS
`bcf-client`	Gravação de relatórios no formato BCF
`odfpy`, `pystache`	Relatórios de validação em planilha `.ods` e em página HTML
`flask`	Interface web do IfcTester
`deepdiff`	Comparação detalhada de estruturas de dados, usada pelo `ifcdiff`

Tabela 0.4: Principais dependências e suas funções.

Não desinstale dependências: a biblioteca que depende da removida deixa de funcionar. Para conferir uma biblioteca, use `python -m pip show <nome>`. A resposta informa a versão, o local da instalação (`Location`) e as dependências (`Requires`).

0.7 Editor de código: VS Code

O **VS Code** é um editor de código. Ele escreve e organiza os scripts, mas não executa nada sozinho: usa o interpretador que estiver selecionado para a pasta aberta.

0.7.1 Configuração

1. Instale a extensão **Python**, publicada pela Microsoft, pelo painel de extensões (`Ctrl+Shift+X`) ou pelo terminal: `code --install-extension ms-python.python`.
2. Abra a pasta de trabalho inteira, e não um arquivo isolado: menu Arquivo > Abrir Pasta > `C:\IFC_Curso`. Confirme que confia nos autores da pasta; sem isso, o editor abre em modo restrito e não executa scripts.
3. Selecione o interpretador: clique na indicação da versão do Python no canto inferior direito da janela e escolha **Enter Interpreter Path...** (Inserir o caminho do interpretador). Cole, na caixa que abre no topo da janela, `C:\IFC_Curso\.venv\Scripts\python.exe`.
4. Confira: o canto inferior direito passa a indicar `.venv` e a versão do Python.
5. Para executar um script aberto, use o botão de execução no canto superior direito (Run Python File).

0.7.2 Armadilhas frequentes

SITUAÇÃO	O QUE ACONTECE	COMO EVITAR
Interpretador não selecionado	O editor usa outro Python, que não tem as bibliotecas	Conferir o canto inferior direito antes de executar
Comando `Python: Create Environment` ou opção Quick Create	Cria um novo ambiente vazio dentro da pasta aberta	Usar `Python: Select Interpreter`
Paleta de comandos sem o sinal `>`	A caixa busca arquivos e responde "No matching results"	Abrir a paleta com `Ctrl+Shift+P`, que já inclui o `>`
Caminho colado no terminal	Abre o Python interativo (`>>>`)	Sair com `exit()` e colar o caminho na caixa do topo
Linhas com três crases coladas no PowerShell	Erro "o termo não é reconhecido"	Copiar só o conteúdo dos blocos de código

Tabela 0.5: Erros comuns de configuração do editor.

0.8 Visualizador de IFC: Blender e Bonsai

O **Blender** é um software 3D gratuito e de código aberto. O **Bonsai** (antigo BlenderBIM) é um complemento do Blender que o transforma em visualizador e editor de IFC. Ele faz parte do projeto IfcOpenShell e usa a mesma biblioteca que os scripts desta apostila.

0.8.1 Por que o Bonsai

Visualizadores comerciais, como o Navisworks, convertem o IFC para o próprio formato e mostram os dados já interpretados. O Bonsai trabalha sobre o arquivo IFC como ele é e mostra o esquema, a MVD declarada, a hierarquia espacial, as classes, os tipos e os conjuntos de propriedades. Isso permite conferir, no visualizador, exatamente o que o texto do arquivo contém.

0.8.2 Compatibilidade de versões

A versão 0.8.5 do Bonsai é compatível do Blender 4.2 ao 5.0 e não é suportada no Blender 5.1 ou superior. Entre as versões de suporte longo (LTS, long-term support) do Blender disponíveis em setembro de 2026, 4.5 e 5.2, só a 4.5 atende. Por isso, esta apostila usa o **Blender 4.5 LTS**.

A situação ilustra um princípio que vale para qualquer software, inclusive os de autoria: **a versão certa é a compatível com o conjunto de ferramentas do fluxo de trabalho**, e não necessariamente a mais nova. Recursos como a exportação IFC mudam de uma versão para outra.

0.8.3 Instalação

1. Baixe o Blender 4.5 LTS em blender.org/download/lts/4-5/. Com permissão de administrador, use o instalador `.msi`; sem ela, a versão portátil (`.zip`). Evite as versões da Microsoft Store e do Steam, que se atualizam sozinhas para a série 5.x.
2. No Blender (interface em inglês): Edit > Preferences > Get Extensions > Allow Online Access. Pesquise "Bonsai" e clique em Install.
3. Confira: o menu File passa a mostrar a opção Open IFC Project.

Notebooks com duas placas de vídeo (integrada e dedicada) podem executar o Blender na integrada, com desempenho menor. No Windows 11, a tela de preferência de placa por aplicativo abre com `Win + R` > `ms-settings:display-advancedgraphics`. Adicione o `blender.exe` e escolha **Alto desempenho**. O nome do item no menu Configurações varia com a tradução do Windows (Vídeo, Tela ou Exibir); o atalho evita a procura.

0.8.4 Cuidados

- O campo de classe no painel de decomposição espacial (por exemplo, `IfcBridge`) define a classe de **novos** elementos espaciais. Ele não descreve o arquivo aberto.
- Ao fechar, o Blender pergunta se deseja salvar o arquivo `.blend`, que é o formato próprio do Blender. Para não gravar nada, escolha Don't Save.

0.9 Arquivos IFC de exemplo

0.9.1 Origem

A buildingSMART International publica arquivos de exemplo de acesso livre em dois repositórios:

REPOSITÓRIO (NOME ATUAL)	CONTEÚDO	ARQUIVOS IFC
Certification-datasets	Modelos simples separados por versão do esquema: `IFC 2.3.0.1 (IFC 2x3 TC1)`, `IFC 4.0.2.1 (IFC 4 ADD2 TC1)` e `IFC 4.3.2.0 (IFC 4.3 ADD2)`	35
IFC4.x-specification-models	Modelos que ilustram a especificação: geometria, elementos construtivos, alinhamento, posicionamento linear e georreferenciamento	46

Tabela 0.6: Repositórios de exemplo da buildingSMART (contagem de setembro de 2026).

Os endereços antigos, github.com/buildingSMART/Sample-Test-Files e github.com/buildingSMART/IFC4.3.x-sample-models, redirecionam para os nomes atuais. Em cada página, use o botão Code > Download ZIP.

0.9.2 Conteúdo por área

A pasta `Simple-Scene` do IFC 4.3 tem um modelo por área:

ARQUIVO	ÁREA	PRINCIPAIS ENTIDADES (QUANTIDADE)
`Building-Architecture.ifc`	Arquitetura	`IfcWall` (4), `IfcSlab` (3), `IfcSpace` (2), `IfcRoof`, `IfcFurniture`
`Building-Structural.ifc`	Estrutura	`IfcBeam` (6), `IfcWall` (4), `IfcFooting`, `IfcDiscreteAccessory` (2)
`Building-Hvac.ifc`	Climatização	`IfcAirTerminal` (2), `IfcDuctSegment`
`Building-Landscaping.ifc`	Paisagismo	`IfcGeographicElement` (4)
`Infra-Bridge.ifc`	Ponte	`IfcBridge` (3), `IfcBridgePart` (18), `IfcBeam`, `IfcColumn`, `IfcFooting`, `IfcSign`
`Infra-Electrical.ifc`	Instalações elétricas	`IfcElectricAppliance`, `IfcDistributionChamberElement` (2)
`Infra-Plumbing.ifc`	Drenagem	`IfcPipeSegment` (24), `IfcDistributionChamberElement` (2)
`Infra-Rail.ifc`	Ferrovia	`IfcRailway` (2), `IfcTrackElement` (66), `IfcRail` (4), `IfcCourse` (2)
`Infra-Road.ifc`	Rodovia	`IfcRoad` (5), `IfcRoadPart` (26), `IfcCourse` (16), `IfcEarthworksFill` (16)
`Infra-Landscaping.ifc`	Paisagismo de infraestrutura	`IfcGeographicElement` (76), `IfcSign` (10)

Tabela 0.7: Modelos IFC 4.3 da pasta Simple-Scene. Contagens obtidas por leitura dos arquivos com o IfcOpenShell 0.8.5.

0.9.3 Extração e o limite de 260 caracteres

O Explorador de Arquivos do Windows, por padrão, não grava arquivos cujo caminho completo tenha 260 caracteres ou mais, e a falha pode passar despercebida durante a extração. Os repositórios têm nomes longos, como `alignment-geometries-and-linear-positioning\linear-placement-of-signal\linear-placement-of-signal.ifc`. Para evitar o problema:

1. Grave os ZIPs em `C:\IFC_Curso\amostras\bSI`.
2. Na extração (botão direito > Extrair Tudo), apague o nome de pasta que o Windows acrescenta ao final do destino. Cada ZIP já contém uma pasta com o nome do repositório. Se o nome sugerido for mantido, a pasta fica duplicada (`Certification-datasets-main\Certification-datasets-main`), e o caminho cresce.
3. Confira a quantidade de arquivos IFC extraídos: 35 no primeiro repositório e 46 no segundo.

0.10 Organização da pasta de trabalho

LOCAL	CONTEÚDO	REGRA
`C:\IFC_Curso\.venv`	Ambiente virtual	Não editar à mão
`C:\IFC_Curso\amostras`	Arquivos IFC de referência	**Somente leitura**
`C:\IFC_Curso\exercicios`	Scripts e arquivos gerados	Sempre com nome novo

Tabela 0.8: Estrutura sugerida para a pasta de trabalho.

A regra de somente leitura tem dois motivos. Os arquivos originais servem de referência para comparar o antes e o depois, e um erro num script não pode destruir o único exemplar de um arquivo. Em projetos reais, vale o mesmo: scripts leem as pastas de referência e gravam o resultado em outro lugar, com nome novo.

0.11 Verificação do ambiente

O script abaixo confere o ambiente de ponta a ponta:

1. informa a versão do Python e o interpretador;
2. confere as bibliotecas;
3. cria um arquivo IFC 4.3 mínimo com projeto, terreno, rodovia, pavimento e um conjunto de propriedades;
4. relê o arquivo gravado;
5. grava um relatório em texto.

Grave-o como `C:\IFC_Curso\exercicios\cap0\teste_ambiente.py`.

```
# -*- coding: utf-8 -*-
"""
teste_ambiente.py | Capítulo 0: verificação do ambiente de trabalho.

O que este script faz:
  1. Informa a versão do Python e o interpretador em uso (confirma o ambiente virtual).
  2. Verifica se as bibliotecas estão instaladas e informa as versões.
  3. Cria um arquivo IFC mínimo no esquema IFC4X3_ADD2 (IFC 4.3) com a estrutura
     IfcProject > IfcSite > IfcRoad, um IfcPavement contido na rodovia e um
     conjunto de propriedades customizado com metadados de teste.
  4. Reabre o arquivo gravado, lista o conteúdo e mostra as primeiras linhas do texto STEP.
  5. Grava um relatório em texto (UTF-8) com toda a saída, para conferência.

Nenhum arquivo existente é alterado. O script grava apenas, na mesma pasta dele:
  - teste_ambiente_IFC4X3.ifc
  - teste_ambiente_relatorio.txt

Versões consideradas: Python 3.10 a 3.14; ifcopenshell 0.8.5
(API atual ifcopenshell.api.<módulo>.<função>).
Dependências: ifcopenshell, ifctester, openpyxl, pandas.

Execução (com o ambiente virtual ativado, dentro da pasta do script):
    python teste_ambiente.py
"""

import sys
import traceback
from importlib import metadata
from pathlib import Path
```

```
PASTA = Path(__file__).resolve().parent
ARQUIVO_IFC = PASTA / "teste_ambiente_IFC4X3.ifc"
ARQUIVO_RELATORIO = PASTA / "teste_ambiente_relatorio.txt"

linhas_relatorio = []


def log(*partes):
    """Imprime na tela e guarda a linha para o relatório final."""
    texto = " ".join(str(p) for p in partes)
    print(texto)
    linhas_relatorio.append(texto)


def verificar_python():
    log("=" * 60)
    log("1. PYTHON")
    log("   Versão.........:", sys.version.split()[0])
    log("   Interpretador..:", sys.executable)
    if not ((3, 10) <= sys.version_info[:2] <= (3, 14)):
        log("   ATENÇÃO: o ifcopenshell 0.8.5 exige Python entre 3.10 e 3.14.")
    if sys.prefix == sys.base_prefix:
        log("   ATENÇÃO: o ambiente virtual NÃO está ativo.")
    else:
        log("   Ambiente virtual ativo: sim")


def verificar_bibliotecas():
    log("=" * 60)
    log("2. BIBLIOTECAS")
    faltando = []
    for pacote in ["ifcopenshell", "ifctester", "openpyxl", "pandas"]:
        try:
            log(f"   {pacote:14s} {metadata.version(pacote)}")
        except metadata.PackageNotFoundError:
            log(f"   {pacote:14s} NÃO INSTALADO")
            faltando.append(pacote)
    if faltando:
        if sys.prefix == sys.base_prefix:
            # Causa mais provável: o script foi executado por um Python fora do ambiente virtual.
            # Instalar as bibliotecas nesse Python seria o erro: elas ficariam fora do ambiente.
            raise RuntimeError(
                "Bibliotecas não encontradas porque o script foi executado FORA do ambiente virtual.\n"
                "   NÃO instale as bibliotecas neste Python. Em vez disso:\n"
                "   - no PowerShell, ative o ambiente: C:\\IFC_Curso\\.venv\\Scripts\\Activate.ps1\n"
                "   - no VS Code, selecione o interpretador: C:\\IFC_Curso\\.venv\\Scripts\\python.exe\n"
                "   Depois, execute o script novamente."
            )
        raise RuntimeError(
            "Ambiente virtual ativo, mas faltam pacotes. Instale-os com: python -m pip install "
            + " ".join(faltando)
        )


def criar_ifc():
    import ifcopenshell
    import ifcopenshell.api.root
    import ifcopenshell.api.unit
    import ifcopenshell.api.context
    import ifcopenshell.api.aggregate
    import ifcopenshell.api.spatial
    import ifcopenshell.api.pset

    log("=" * 60)
    log("3. CRIAÇÃO DE ARQUIVO IFC4X3_ADD2")

    modelo = ifcopenshell.file(schema="IFC4X3_ADD2")

    # Projeto, unidades e contexto geométrico.
    # assign_unit sem argumentos aplica as unidades padrão do IfcOpenShell
    # (comprimento em milímetros).
    projeto = ifcopenshell.api.root.create_entity(modelo, ifc_class="IfcProject", name="Curso IFC - Teste")
```

```
    ifcopenshell.api.unit.assign_unit(modelo)
    ifcopenshell.api.context.add_context(modelo, context_type="Model")

    # Estrutura espacial: IfcProject > IfcSite > IfcRoad (relação IfcRelAggregates)
    terreno = ifcopenshell.api.root.create_entity(modelo, ifc_class="IfcSite", name="Terreno de teste")
    rodovia = ifcopenshell.api.root.create_entity(modelo, ifc_class="IfcRoad", name="Rodovia de teste")
    ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)
    ifcopenshell.api.aggregate.assign_object(modelo, products=[rodovia], relating_object=terreno)

    # Elemento contido na rodovia (relação IfcRelContainedInSpatialStructure)
    pavimento = ifcopenshell.api.root.create_entity(
        modelo, ifc_class="IfcPavement", name="Pavimento de teste", predefined_type="FLEXIBLE"
    )
    ifcopenshell.api.spatial.assign_container(modelo, products=[pavimento], relating_structure=rodovia)

    # Conjunto de propriedades customizado (o prefixo "Pset_" é reservado aos Psets padrão)
    pset = ifcopenshell.api.pset.add_pset(modelo, product=pavimento, name="ARVEN_Metadados")
    ifcopenshell.api.pset.edit_pset(
        modelo, pset=pset, properties={"TAG": "PAV-001", "REVISAO": "0", "STATUS": "TESTE"}
    )

    modelo.write(str(ARQUIVO_IFC))
    log("   Gravado em:", ARQUIVO_IFC)


def ler_ifc():
    import ifcopenshell
    import ifcopenshell.util.element

    log("=" * 60)
    log("4. LEITURA DO ARQUIVO GRAVADO")
    lido = ifcopenshell.open(str(ARQUIVO_IFC))
    log("   Esquema.........:", lido.schema_identifier)
    log("   Nº de instâncias:", len(list(lido)))
    for classe in ["IfcProject", "IfcSite", "IfcRoad", "IfcPavement"]:
        for ent in lido.by_type(classe):
            log(f"   {classe:12s} Name='{ent.Name}'  GlobalId={ent.GlobalId}")
    pav = lido.by_type("IfcPavement")[0]
    log("   PredefinedType do IfcPavement:", pav.PredefinedType)
    log("   Psets do IfcPavement:", ifcopenshell.util.element.get_psets(pav))

    log("=" * 60)
    log("5. PRIMEIRAS LINHAS DO TEXTO STEP")
    with open(ARQUIVO_IFC, encoding="utf-8") as f:
        for i, linha in enumerate(f):
            if i >= 12:
                break
            log("   " + linha.rstrip())


if __name__ == "__main__":
    try:
        verificar_python()
        verificar_bibliotecas()
        criar_ifc()
        ler_ifc()
        log("=" * 60)
        log("AMBIENTE OK.")
    except Exception:
        log("=" * 60)
        log("ERRO:")
        log(traceback.format_exc())
    finally:
        with open(ARQUIVO_RELATORIO, "w", encoding="utf-8") as f:
            f.write("\n".join(linhas_relatorio) + "\n")
        print("Relatório gravado em:", ARQUIVO_RELATORIO)
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap0
python teste_ambiente.py
```

Saída obtida em um Windows 11 com Python 3.13.15 (os GlobalId e a data mudam a cada execução):

```
============================================================
1. PYTHON
   Versão.........: 3.13.15
   Interpretador..: C:\IFC_Curso\.venv\Scripts\python.exe
   Ambiente virtual ativo: sim
============================================================
2. BIBLIOTECAS
   ifcopenshell   0.8.5
   ifctester      0.8.5
   openpyxl       3.1.5
   pandas         3.0.6
============================================================
3. CRIAÇÃO DE ARQUIVO IFC4X3_ADD2
   Gravado em: C:\IFC_Curso\exercicios\cap0\teste_ambiente_IFC4X3.ifc
============================================================
4. LEITURA DO ARQUIVO GRAVADO
   Esquema.........: IFC4X3_ADD2
   Nº de instâncias: 21
   IfcProject   Name='Curso IFC - Teste'  GlobalId=16ExQx0_v1Le6wrC1zsfVj
   IfcSite      Name='Terreno de teste'  GlobalId=2QjmYwb857BPNdG0peKPR3
   IfcRoad      Name='Rodovia de teste'  GlobalId=0yE21Gwn1EbwmtkrNOv277
   IfcPavement  Name='Pavimento de teste'  GlobalId=0Fs4ynj5r3VQtZPlNwZWpO
   PredefinedType do IfcPavement: FLEXIBLE
   Psets do IfcPavement: {'ARVEN_Metadados': {'TAG': 'PAV-001', 'REVISAO': '0', 'STATUS': 'TESTE', 'id': 17}}
...
AMBIENTE OK.
```

O arquivo gerado também serve para testar o Bonsai: abra-o por File > Open IFC Project. Como ele não tem geometria, a área 3D fica vazia; o critério de sucesso é abrir sem erro e mostrar, no painel do projeto, o esquema IFC4X3 e a árvore Projeto > Terreno > Rodovia.

Exemplo resolvido 0.2: o mesmo script, dois resultados

Ao executar o script pelo VS Code antes de selecionar o interpretador, a saída foi:

```
1. PYTHON
   Versão.........: 3.13.14
   Interpretador..: C:\Users\<usuário>\AppData\Local\Microsoft\WindowsApps\...\python.exe
   ATENÇÃO: o ambiente virtual NÃO está ativo.
2. BIBLIOTECAS
   ifcopenshell   NÃO INSTALADO
   ifctester      NÃO INSTALADO
```

Diagnóstico:

1. A versão (3.13.14) e o caminho (`WindowsApps`) mostram que o script rodou no Python da Microsoft Store.
2. As bibliotecas foram instaladas no ambiente virtual, e não nesse Python. Por isso, aparecem como não instaladas.
3. Instalar as bibliotecas no Python da Store "resolveria" o sintoma, mas criaria um segundo conjunto de bibliotecas fora do ambiente, com versões que podem divergir.

Correção: selecionar o interpretador `C:\IFC_Curso\.venv\Scripts\python.exe` no VS Code (seção 0.7.1) e executar de novo. A saída passa a mostrar a versão 3.13.15, o caminho do `.venv` e as quatro bibliotecas.

0.12 Diagnóstico de problemas comuns

SINTOMA	CAUSA PROVÁVEL	SOLUÇÃO
`ModuleNotFoundError: No module named 'ifcopenshell'`	Execução fora do ambiente virtual	Ativar o ambiente ou selecionar o interpretador no VS Code
"A execução de scripts foi desabilitada neste sistema"	Política de execução do PowerShell	`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`
"O termo '```powershell' não é reconhecido"	Marcações de bloco de código coladas junto com o comando	Copiar só o conteúdo do bloco
`python --version` mostra outra versão	Ambiente não ativado naquela janela	Ativar o ambiente
Instalação do `ifcopenshell` falha	Python fora da faixa 3.10 a 3.14	Criar o ambiente com um Python compatível
Arquivos faltando após extrair um ZIP	Caminho com 260 caracteres ou mais	Pasta de destino curta, sem subpasta duplicada
Bonsai não aparece no Blender	Versão do Blender incompatível	Blender 4.2 a 5.0 (apostila: 4.5 LTS)

Tabela 0.9: Sintomas, causas e soluções.

0.13 Resumo do capítulo

- Um script só enxerga as bibliotecas instaladas no Python que o executa. A maior parte dos erros de ambiente vem de executar o código no Python errado.
- O ambiente virtual isola as bibliotecas de um projeto; fica numa pasta curta, fora de pastas sincronizadas, e precisa ser ativado em cada terminal novo.
- `python -m pip install` instala bibliotecas no ambiente ativo, junto com as dependências.
- No VS Code, a execução usa o interpretador selecionado para a pasta, indicado no canto inferior direito.
- O Bonsai 0.8.5 exige Blender entre 4.2 e 5.0. A escolha de versão segue a compatibilidade do conjunto, e não a novidade.
- As amostras da buildingSMART cobrem várias áreas e três versões do esquema; ficam numa pasta de somente leitura.

0.14 Vocabulário técnico

INGLÊS	PORTUGUÊS
interpreter	interpretador
virtual environment	ambiente virtual
package, library	pacote, biblioteca
dependency	dependência
execution policy	política de execução
command palette	paleta de comandos
long-term support (LTS)	suporte de longo prazo
path	caminho
add-on, extension	complemento, extensão

Tabela 0.10: Vocabulário do capítulo.

0.15 Exercícios

Fixação

1. Você vai fazer outro curso que exige uma versão diferente do `pandas`. O que você faz para não afetar o ambiente deste?
2. Em um PowerShell novo, o comando `python -c "import ifcopenshell"` responde que o módulo não foi encontrado, embora a instalação tenha sido concluída ontem. Qual é a causa provável e como resolver?
3. Ao instalar o `ifctester`, o `pip` baixou também o `xmlschema`, que você não pediu. Por quê?
4. Sai uma versão nova do Bonsai que funciona apenas com o Blender 5.x. O que você verifica antes de atualizar?
5. Você vai inserir conjuntos de propriedades em uma cópia do `Infra-Road.ifc`. De que pasta você lê o original e onde grava o resultado?

Aplicação

6. Abra um PowerShell novo, sem ativar o ambiente, entre na pasta `C:\IFC_Curso\exercicios\cap0` e execute `python teste_ambiente.py`. Explique a mensagem obtida. Depois, ative o ambiente, execute de novo e compare as seções 1 e 2 das duas saídas.
7. Com o ambiente ativo, execute `python -m pip show ifcopenshell`. Identifique a versão instalada, o local da instalação e as dependências. O local está dentro de `C:\IFC_Curso\.venv`?

Desafio

8. Abra no Bonsai o `Building-Architecture.ifc` e o `Infra-Road.ifc` da pasta `IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene`. Para cada um, anote o esquema e a MVD mostrados no painel do projeto e

descreva a árvore de decomposição espacial até o segundo nível. Qual é a diferença entre as classes espaciais de uma edificação e as de uma rodovia?

0.16 Respostas dos exercícios

1. Criar um segundo ambiente virtual, em outra pasta, para o outro curso, e instalar lá a versão exigida do `pandas`. Cada ambiente mantém as próprias bibliotecas, e nenhum afeta o outro.
2. O ambiente virtual não foi ativado naquela janela. A ativação vale só para o terminal em que foi feita, e um PowerShell novo usa o Python padrão do sistema, que não tem o `ifcopenshell`. Solução: executar `C:\IFC_Curso\.venv\Scripts\Activate.ps1` e repetir o comando. Instalar a biblioteca de novo seria o erro: ela iria para o Python errado.
3. O `xmlschema` é uma dependência do `ifctester`. O IfcTester lê arquivos IDS, que são XML, e usa o `xmlschema` para conferir se o arquivo segue o esquema oficial do IDS. O `pip` lê a lista de dependências de cada pacote e instala tudo o que falta.
4. Verifique se o Blender instalado é compatível com a nova versão do Bonsai. Se a nova versão exigir o Blender 5.x, a atualização exige também trocar o Blender, e isso só vale a pena se o novo par (Blender e Bonsai) for compatível e estável. O princípio é o de compatibilidade do conjunto (seção 0.8.2).
5. Lê de `C:\IFC_Curso\amostras\bSI\...` (somente leitura) e grava a cópia modificada em `C:\IFC_Curso\exercicios\...`, com nome novo, por exemplo `Infra-Road_com_psets.ifc`. O original nunca é alterado.
6. Sem o ambiente ativo, o script mostra "o ambiente virtual NÃO está ativo" e "Bibliotecas não encontradas porque o script foi executado FORA do ambiente virtual": o comando `python` chamou o Python padrão do sistema, sem as bibliotecas. Com o ambiente ativo, a seção 1 mostra o interpretador `C:\IFC_Curso\.venv\Scripts\python.exe` e a seção 2 lista as quatro bibliotecas com as versões.
7. A resposta mostra `Version: 0.8.5`, `Location:` com um caminho dentro de `C:\IFC_Curso\.venv\Lib\site-packages` e `Requires: isodate, lark, numpy, python-dateutil, shapely, typing-extensions`. Se o local estiver fora do `.venv`, o comando foi executado sem o ambiente ativo.
8. Os dois arquivos mostram o esquema IFC4X3 e a MVD `ReferenceView`. Na edificação, a árvore é Projeto > Terreno (`IfcSite`) > Edificação (`IfcBuilding`) > Pavimento (`IfcBuildingStorey`). Na rodovia, é Projeto > Terreno (`IfcSite`) > Rodovia (`IfcRoad`) > Partes da rodovia (`IfcRoadPart`). A diferença está na classe da instalação e das suas partes: edificação e pavimentos de um lado, rodovia e partes de rodovia do outro. Os dois usam o mesmo mecanismo de decomposição, estudado no Capítulo 4.

0.17 Referências

- IfcOpenShell no PyPI: https://pypi.org/project/ifcopenshell/
- Python no Windows (documentação oficial): https://docs.python.org/3/using/windows.html
- Instalação do Bonsai: https://docs.bonsaibim.org/quickstart/installation.html
- Bonsai na plataforma de extensões do Blender: https://extensions.blender.org/add-ons/bonsai/

- Versões LTS do Blender: https://www.blender.org/download/lts/
- Amostras da buildingSMART: https://github.com/buildingSMART/Sample-Test-Files e https://github.com/buildingSMART/IFC4.3.x-sample-models

1 Fundamentos openBIM

Antes de abrir um arquivo IFC, é preciso entender por que ele existe, quem o define e como ele se relaciona com os outros padrões e com as normas de gestão da informação. Este capítulo apresenta esse contexto, sem exigir conhecimento prévio.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que é BIM e o problema de troca de informação que o openBIM resolve;
- descrever o papel da buildingSMART e de cada um dos seus padrões: IFC, IDM, MVD, IDS, bSDD e BCF;
- distinguir as versões do esquema IFC e saber o que mudou entre elas;
- relacionar o IFC à gestão da informação da série ISO 19650;
- situar o uso de BIM e de formatos abertos no contexto normativo brasileiro.

1.1 O que é BIM

BIM (Building Information Modelling, modelagem da informação da construção) é a forma de projetar, construir e operar obras com base em modelos digitais que contêm, além da geometria, a informação sobre cada objeto.

Num desenho CAD tradicional, uma parede é um conjunto de linhas. Quem olha o desenho sabe que é uma parede, mas o arquivo não sabe. Num modelo BIM, a parede é um **objeto** que o software reconhece como parede e que carrega informações: espessura, material, resistência ao fogo, pavimento em que está, fase de construção, custo. A mesma ideia vale para um tubo, uma viga, uma camada de pavimento rodoviário ou um equipamento industrial.

Três consequências práticas decorrem disso:

1. **Consultas:** é possível perguntar ao modelo "quais tubos são de aço carbono?" ou "qual o volume de concreto das fundações?".
2. **Coerência:** plantas, cortes, tabelas e quantitativos são gerados do mesmo modelo, e uma alteração se propaga para todos.
3. **Continuidade:** a informação produzida no projeto pode seguir para a obra e para a operação do ativo.

A terceira consequência só se realiza se a informação puder passar de um software para outro e de uma empresa para outra. É aqui que surge o problema tratado neste capítulo.

1.2 O problema da troca de informação

1.2.1 Interoperabilidade

Um empreendimento envolve muitas disciplinas (arquitetura, estrutura, instalações, geotecnia, infraestrutura) e muitas empresas, cada uma com os seus softwares. **Interoperabilidade** é a capacidade de dois sistemas trocarem informação e usarem essa informação sem retrabalho. Em BIM, o problema tem duas partes:

PARTE	PERGUNTA	EXEMPLO DE FALHA
Sintática	O arquivo pode ser lido?	O software de destino não abre o formato recebido
Semântica	O significado se mantém?	O arquivo abre, mas o tubo chega como um sólido genérico, sem diâmetro, sem material e sem indicação de que é um tubo

Tabela 1.1: As duas partes da interoperabilidade.

A falha semântica é a mais perigosa, porque não é percebida à primeira vista: a geometria aparece correta, mas a informação que dava sentido ao objeto se perdeu.

1.2.2 Formato proprietário e formato neutro

Um **formato proprietário** é definido por um fabricante de software, que decide o que ele contém e como muda de uma versão para outra. Um **formato neutro** (ou aberto) é definido por uma organização independente, com especificação pública, e pode ser implementado por qualquer fabricante.

Exemplo resolvido 1.1: quantas conversões são necessárias?

Situação: cinco softwares diferentes precisam trocar modelos entre si.

Sem formato neutro: cada software precisa de um conversor específico para cada outro. O número de conversões diretas é N x (N - 1) = 5 x 4 = **20 conversores**, cada um mantido por alguém e com as suas perdas próprias.

Com formato neutro: cada software precisa apenas de um exportador e de um importador para o formato comum: 2 x N = 2 x 5 = **10 componentes**, e cada fabricante mantém somente os seus.

Conclusão: o formato neutro reduz o número de conversões e, sobretudo, cria uma referência única. Todos sabem o que o arquivo deve conter, e é possível verificar se um exportador cumpre essa referência.

1.3 O que é openBIM

openBIM é a abordagem de trabalho em BIM baseada em padrões abertos e neutros, mantidos por uma organização independente dos fabricantes de software. O objetivo é permitir que profissionais e empresas colaborem usando os softwares que preferirem, sem ficar presos a um único fornecedor.

O oposto é o fluxo fechado, em que todas as partes precisam usar o mesmo fornecedor e os formatos proprietários dele. O fluxo fechado pode funcionar dentro de uma empresa, mas se torna um problema quando o empreendimento reúne várias empresas, dura décadas ou precisa ser verificado por terceiros, como em contratações públicas.

openBIM não significa abandonar os softwares proprietários. O modelo continua sendo produzido no software de autoria (Revit, Civil 3D, Tekla e outros), e o formato aberto é usado nas trocas entre as partes e na entrega da informação.

1.4 A buildingSMART International

A **buildingSMART International** é a organização que desenvolve e mantém os padrões openBIM.

FATO	INFORMAÇÃO
Origem	1994, como consórcio chamado Industry Alliance for Interoperability (IAI), formado por empresas norte-americanas a convite da Autodesk
Nome atual	buildingSMART, desde 2005
Natureza	Organização internacional sem fins lucrativos, organizada em capítulos nacionais
Padrões mantidos	IFC, IDS, BCF, bSDD, entre outros
No Brasil	buildingSMART Brasil, capítulo pleno desde 2023, sediado pelo BIM Fórum Brasil (buildingsmart.org.br)

Tabela 1.2: Dados principais da buildingSMART International.

1.5 Os padrões openBIM

Cada padrão responde a uma pergunta diferente sobre a troca de informação. Esta seção apresenta um de cada vez.

1.5.1 IFC: como descrever o modelo

O **IFC** (Industry Foundation Classes) é o **esquema de dados** que descreve o ativo construído: quais objetos existem (paredes, vigas, tubos, rodovias), como se relacionam (esta parede está neste pavimento; este tubo é deste tipo), quais propriedades têm e qual é a sua geometria.

"Esquema de dados" significa um conjunto de regras que define **o que pode ser escrito** e **como**. O IFC define, por exemplo, que existe uma entidade chamada `IfcWall` para paredes, quais informações ela pode ter e em que ordem. Um arquivo `.ifc` é um modelo escrito segundo essas regras.

O IFC é publicado pela ISO como a norma **ISO 16739-1**. A edição vigente, ISO 16739-1:2024, corresponde ao IFC 4.3. Os Capítulos 2 e 3 tratam do arquivo e do esquema em detalhe.

1.5.2 IDM: que informação é necessária, e quando

O **IDM** (Information Delivery Manual, manual de entrega de informação) é uma metodologia para documentar um processo de trabalho e as informações que precisam ser trocadas em cada etapa: quem entrega, o quê, quando e para quê. Por exemplo: "na entrega do projeto executivo, o projetista de instalações entrega ao orçamentista os tubos com diâmetro, material e comprimento".

O IDM é um documento para pessoas, publicado como ISO 29481-1 (edição de 2025).

1.5.3 MVD: o que um software deve exportar

O IFC é muito amplo, e nenhum fluxo de trabalho usa tudo. Uma **MVD** (Model View Definition, definição de vista de modelo) recorta do esquema a parte necessária para um tipo de troca e define regras de implementação. Ela é usada, principalmente, para orientar e certificar os exportadores e importadores dos softwares. A seção 1.8 lista as MVDs oficiais.

1.5.4 IDS: o que um projeto exige

O **IDS** (Information Delivery Specification, especificação de entrega de informação) é um arquivo legível por computador que descreve os requisitos de informação de um projeto específico. Em linguagem comum, um requisito IDS tem a forma:

> EXEMPLO DE REQUISITO
>
> Todos os objetos da classe `IfcPipeSegment` devem ter a propriedade `TAG` preenchida no conjunto `Metadados_Projeto`, e o valor deve seguir o padrão definido no plano de execução BIM.

Um programa de validação lê o arquivo IDS e o modelo IFC e informa quais objetos cumprem e quais não cumprem cada requisito. O IDS 1.0 é padrão oficial da buildingSMART desde junho de 2024 e é tratado no Capítulo 12.

1.5.5 bSDD: o significado das palavras

O **bSDD** (buildingSMART Data Dictionary, dicionário de dados da buildingSMART) é um serviço on-line que reúne dicionários de classes, propriedades e classificações, cada item com um identificador único na internet (URI). Ele resolve o problema de dois projetos usarem nomes diferentes para a mesma coisa, ou o mesmo nome para coisas diferentes. Um requisito IDS pode apontar para uma propriedade do bSDD, garantindo que todos entendam o mesmo significado.

1.5.6 BCF: como comunicar problemas

O **BCF** (BIM Collaboration Format, formato de colaboração BIM) registra apontamentos sobre o modelo (uma interferência, uma informação faltante, uma dúvida) com a posição da câmera, a imagem, os objetos envolvidos e os comentários, sem trocar o modelo inteiro. É usado em arquivos `.bcfzip` ou por meio de um servidor na internet (API). As versões em uso são a 2.1 e a 3.0.

PADRÃO	PERGUNTA QUE RESPONDE	FORMATO	SITUAÇÃO
IFC	Como descrever o modelo?	`.ifc` (texto), `.ifcXML`, `.ifcZIP`	IFC 4.3, ISO 16739-1:2024

PADRÃO	PERGUNTA QUE RESPONDE	FORMATO	SITUAÇÃO
IDM	Que informação é necessária em cada etapa?	Documento	ISO 29481-1:2025
MVD	O que um software deve exportar para uma finalidade?	Especificação técnica	Vinculada a cada versão do IFC
IDS	O que este projeto exige do modelo?	XML (`.ids`)	IDS 1.0, padrão oficial desde junho de 2024
bSDD	O que significa cada termo?	Serviço web e API	Em operação
BCF	Como comunicar os problemas encontrados?	`.bcfzip` ou API	Versões 2.1 e 3.0

Tabela 1.3: Resumo dos padrões openBIM.

1.6 Como os padrões trabalham juntos

```
IDM ........ descreve o processo e a informação necessária em cada etapa
  |
IDS ........ transforma a necessidade em requisitos verificáveis
  |          (usando termos e classificações do bSDD)
IFC ........ carrega o modelo entregue
  |
Validação .. o modelo IFC é conferido contra o IDS
  |
BCF ........ comunica as não conformidades aos autores do modelo
```

Figura 1.1: Encadeamento dos padrões openBIM num projeto.

Exemplo resolvido 1.2: requisitos de metadados em um projeto de tubulação

Situação: o plano de execução BIM de uma unidade industrial exige que cada tubo tenha TAG, revisão, status, fluido, pressão e temperatura de projeto. O modelo é produzido num software de autoria e entregue ao cliente.

Encadeamento:

1. **IDM:** o plano de execução define que, na emissão para construção, o projetista entrega ao cliente o modelo de tubulação com esses dados.
2. **IDS:** os requisitos viram um arquivo `.ids`: "todo `IfcPipeSegment` deve ter as propriedades TAG, REVISAO, STATUS, FLUIDO, PRESSAO e TEMPERATURA preenchidas", com os padrões de valor aceitos.
3. **IFC:** o projetista exporta o modelo em IFC 4.3.
4. **Validação:** o cliente (ou o próprio projetista, antes de entregar) valida o IFC contra o IDS e obtém a lista de tubos com dados faltantes.
5. **BCF:** as pendências voltam ao projetista como apontamentos BCF, cada um ligado ao tubo com problema.

Conclusão: cada padrão tem um papel e nenhum substitui o outro. O IDS não descreve o modelo, e o IFC não diz o que é obrigatório.

1.7 As versões do IFC

1.7.1 O que é uma versão do esquema

Como qualquer norma, o IFC evolui. Cada **versão** do esquema acrescenta entidades, corrige problemas e, às vezes, renomeia ou retira elementos. Um arquivo IFC sempre declara a versão em que foi escrito, e o software que o lê precisa conhecer essa versão. Um arquivo IFC4.3 aberto num programa que só conhece o IFC2x3 perde tudo o que é novo.

1.7.2 Situação oficial

VERSÃO	NOME	PUBLICAÇÃO	ISO	SITUAÇÃO
2.3.0.1	IFC 2x3 TC1	07/2007	ISO/PAS 16739:2005	Oficial
4.0.2.1	IFC 4 ADD2 TC1	10/2017	ISO 16739-1:2018	Oficial
4.1.0.0	IFC 4.1	06/2018		Retirada de uso
4.2.0.0	IFC 4.2	04/2019		Retirada de uso
4.3.2.0	IFC 4.3 ADD2	04/2024	ISO 16739-1:2024	Oficial
4.4	IFC 4.4.x			Em desenvolvimento
5	IFC 5			Em desenvolvimento

Tabela 1.4: Situação das versões do IFC publicada pela buildingSMART (consulta em 18/09/2026).

As siglas nos nomes indicam correções: **TC** (Technical Corrigendum, correção técnica) e **ADD** (Addendum, adendo). As versões anteriores às listadas estão aposentadas.

Cada versão grava no arquivo um identificador próprio, na linha `FILE_SCHEMA` do cabeçalho (Capítulo 2):

VERSÃO	IDENTIFICADOR NO ARQUIVO
IFC 2x3 TC1	`IFC2X3`
IFC 4 ADD2 TC1	`IFC4`
IFC 4.3 ADD2	`IFC4X3_ADD2`

Tabela 1.5: Identificadores de esquema.

1.7.3 O que mudou

DE	PARA	PRINCIPAIS MUDANÇAS
IFC2x3	IFC4	Classes específicas de instalações para as ocorrências (`IfcPipeSegment` , `IfcDuctSegment` , `IfcPump` , `IfcValve`); atributo `PredefinedType` nas ocorrências; histórico de autoria (`OwnerHistory`) opcional; relação de tipo com atributo próprio (`IsTypedBy`); geometria por malhas de triângulos (`IfcTriangulatedFaceSet`); georreferenciamento (`IfcMapConversion` , `IfcProjectedCRS`); sistemas de distribuição (`IfcDistributionSystem`)

DE	PARA	PRINCIPAIS MUDANÇAS
IFC4	IFC4.3	Domínios de infraestrutura: instalações (`IfcFacility`) do tipo rodovia (`IfcRoad`), ponte (`IfcBridge`), ferrovia (`IfcRailway`) e obra portuária (`IfcMarineFacility`); partes (`IfcRoadPart` , `IfcBridgePart` , `IfcRailwayPart`); alinhamento (`IfcAlignment`) e posicionamento linear (`IfcLinearPlacement` , `IfcReferent`); terraplenagem (`IfcEarthworksFill` , `IfcEarthworksCut`); camadas e pavimento (`IfcCourse` , `IfcPavement` , `IfcKerb`); sinalização (`IfcSign`); `IfcBuildingElement` renomeado para `IfcBuiltElement`

Tabela 1.6: Principais mudanças entre versões. Existência das entidades conferida no esquema.

No IFC 4.3, a edificação (`IfcBuilding`) passa a ser um dos tipos de instalação (`IfcFacility`), ao lado de rodovia, ponte, ferrovia e obra portuária. Edificação e infraestrutura ficam no mesmo esquema.

Exemplo resolvido 1.3: a mesma rodovia em três versões

A buildingSMART publica o arquivo `Infra-Road.ifc` com a mesma rodovia escrita em IFC2x3, IFC4 e IFC4.3. A leitura dos três arquivos com o IfcOpenShell dá o seguinte resultado:

ASPECTO	IFC2X3	IFC4	IFC4.3
MVD declarada	`CoordinationView_V2.0`	`ReferenceView_V1.2`	`ReferenceView`
Classe da rodovia	`IfcBuilding` (5)	`IfcBuilding` (5)	`IfcRoad` (5)
Partes da rodovia	não há	não há	`IfcRoadPart` (26)
Camadas do pavimento e aterros	`IfcSlab` (32, "lajes")	`IfcSlab` (32, "lajes")	`IfcCourse` (16) e `IfcEarthworksFill` (16)
Marcações de pista	`IfcAnnotation` (20)	`IfcSurfaceFeature` (20)	`IfcSurfaceFeature` (20)
Total de instâncias	3.608	1.051	894

Tabela 1.7: Comparação do arquivo `Infra-Road.ifc` nas três versões.

Interpretação: antes do IFC 4.3, uma rodovia só podia ser descrita com classes de edificação: a rodovia vira um "prédio" e as camadas do pavimento viram "lajes". A geometria chega ao destino, mas o significado se perde (falha semântica, seção 1.2.1). No IFC 4.3, uma consulta por `IfcCourse` encontra exatamente as camadas do pavimento, e uma consulta por `IfcEarthworksFill` encontra os aterros. O mesmo vale para pontes e ferrovias.

Conclusão: para obras de infraestrutura, e para modelos que combinam edificação e infraestrutura, o IFC 4.3 é a versão a usar. É a versão de referência desta apostila.

1.7.4 O futuro: IFC 4.4 e IFC 5

O **IFC 4.4** está em desenvolvimento como extensão do 4.3, com correções e novas funcionalidades. O **IFC 5** é uma reestruturação maior e está em fase alfa (alpha): os exemplos oficiais usam o formato JSON (arquivos IFCX), e o esquema é definido na linguagem TypeSpec. O próprio repositório da buildingSMART informa que esses exemplos são preliminares e não servem para uso em produção.

1.8 MVDs oficiais

VERSÃO	MVD	FINALIDADE
IFC2x3	Coordination View 2.0	Coordenação entre arquitetura, estrutura e instalações
IFC2x3	Space Boundary Addon View	Limites de espaços para análise energética e quantitativos
IFC2x3	Basic FM Handover View	Entrega de informação para sistemas de manutenção
IFC2x3	Structural Analysis View	Modelos de análise estrutural
IFC4	Reference View 1.2	Representação geométrica e relacional simplificada, para coordenação
IFC4	IFC4 Precast	Troca de peças pré-moldadas entre projeto e fabricação
IFC4.3	Reference View	Versão atualizada da Reference View para o IFC 4.3
IFC4.3	Alignment-based View	Alinhamentos em projetos de infraestrutura

Tabela 1.8: MVDs oficiais publicadas pela buildingSMART.

A MVD é declarada no cabeçalho do arquivo, como `ViewDefinition [ReferenceView]`. Nos softwares de autoria, a escolha da versão e da MVD aparece nas configurações de exportação IFC (Capítulo 13).

> **OBSERVAÇÃO**
>
> MVD e IDS são complementares. A MVD garante que o software sabe exportar certos tipos de informação. O IDS verifica se um modelo específico contém a informação que o projeto exige.

1.9 IFC e a gestão da informação: a série ISO 19650

1.9.1 Introdução

Ter um formato aberto não basta. Também é preciso definir quem pede qual informação, quem a produz, como ela é verificada, onde é guardada e como é entregue. A série de normas **ISO 19650** trata dessa **gestão da informação** ao longo do ciclo de vida do ativo construído.

PARTE	TEMA	ANO
19650-1	Conceitos e princípios	2018
19650-2	Fase de entrega dos ativos	2018
19650-3	Fase de operação dos ativos	2020
19650-4	Troca de informação	2022
19650-5	Segurança da informação	2020
19650-6	Informação de saúde e segurança	2025

Tabela 1.9: Partes da série ISO 19650.

1.9.2 Conceitos principais

CONCEITO	SIGNIFICADO
Requisitos de informação	O que o contratante precisa saber, em níveis: da organização (OIR), do ativo (AIR), do projeto (PIR) e de cada troca (EIR)
Plano de execução BIM (BEP)	Documento em que a equipe de entrega explica como atenderá aos requisitos
Ambiente comum de dados (CDE)	Repositório único em que a informação é produzida, verificada, compartilhada e publicada
Contêiner de informação	Unidade de informação com identificação própria, como um arquivo de modelo, um desenho ou uma planilha
Modelo de informação do projeto (PIM) e do ativo (AIM)	Conjunto da informação produzida na entrega e na operação, respectivamente

Tabela 1.10: Conceitos centrais da ISO 19650.

1.9.3 Onde o IFC entra

A ISO 19650 não obriga um formato de arquivo. Ela define o processo. O IFC entra como o formato aberto dos contêineres de informação do modelo, e o IDS entra como a forma verificável de expressar os requisitos de troca (EIR). Os parâmetros que um BEP exige dos objetos, como TAG, tipo de objeto, revisão, status, fase, dimensões e propriedades técnicas, são requisitos de informação. Nesta apostila, eles aparecem como conjuntos de propriedades no Capítulo 5, são inseridos em lote no Capítulo 10 e são verificados com IDS no Capítulo 12.

1.10 Contexto normativo brasileiro

INSTRUMENTO	CONTEÚDO RELEVANTE
Lei nº 14.133/2021 (licitações e contratos), art. 19, § 3º	"Nas licitações de obras e serviços de engenharia e arquitetura, sempre que adequada ao objeto da licitação, será preferencialmente adotada a Modelagem da Informação da Construção (Building Information Modelling - BIM) ou tecnologias e processos integrados similares ou mais avançados que venham a substituí-la."
Decreto nº 10.306/2020 (BIM em obras e serviços de engenharia federais)	Em vigor. Implantação em três fases: a partir de 01/01/2021, projetos de arquitetura e engenharia; a partir de 01/01/2024, também orçamentação, planejamento e controle da execução; a partir de 01/01/2028, também gerenciamento e manutenção após a construção. Exige a entrega dos arquivos também "em formato aberto (não proprietário)". Especifica os órgãos e os tipos de obra abrangidos
Decreto nº 11.888/2024 (Estratégia BIM BR)	Institui a Estratégia Nacional de Disseminação do BIM e o seu Comitê Gestor; revoga o Decreto nº 9.983/2019. Entre os objetivos, incentivar especificações técnicas abertas para a interoperabilidade
ABNT NBR 15965 (Sistema de classificação da informação da construção)	Norma em sete partes, todas publicadas: 1 (2011), 2 (2012), 3 (2014), 4 (2021), 5 (2022), 6 (2022) e 7 (2015), segundo os boletins de normas técnicas da CBIC. Confira no catálogo da ABNT a situação atual
ABNT NBR ISO 12006-2	Estrutura para classificação da informação da construção

INSTRUMENTO	CONTEÚDO RELEVANTE
ABNT NBR ISO 19650-1:2022	Adoção brasileira da ISO 19650-1. Confira no catálogo da ABNT a situação das demais partes

Tabela 1.11: Instrumentos normativos brasileiros relacionados a BIM (situação conferida em 18/09/2026).

Nenhum dos atos legais acima cita o IFC pelo nome. Eles exigem formato aberto ou especificações abertas, e o IFC é o formato aberto padronizado internacionalmente para essa finalidade. Órgãos contratantes publicam documentos técnicos próprios, como os requisitos BIM do DNIT para infraestrutura de transportes. Em cada contrato, consulte os documentos do contratante.

> **OBSERVAÇÃO**
> Esta seção tem caráter informativo e não substitui a leitura dos textos legais e normativos vigentes nem a orientação jurídica em contratos.

1.11 Resumo do capítulo

- BIM trabalha com objetos que carregam informação; o valor dessa informação depende de ela passar entre softwares e empresas sem perdas.
- Interoperabilidade tem uma parte sintática (ler o arquivo) e uma semântica (manter o significado); a falha semântica é a mais difícil de perceber.
- openBIM usa padrões abertos, mantidos pela buildingSMART: IFC (o modelo), IDM (o processo), MVD (o que o software exporta), IDS (o que o projeto exige), bSDD (o significado dos termos) e BCF (a comunicação de problemas).
- As versões oficiais do IFC são IFC2x3, IFC4 e IFC4.3. O IFC4.3 (ISO 16739-1:2024) cobre edificações e infraestrutura e é a versão de referência.
- A ISO 19650 organiza a gestão da informação; o IFC e o IDS são os instrumentos abertos para cumprir os requisitos de informação.
- No Brasil, a Lei nº 14.133/2021 e o Decreto nº 10.306/2020 estimulam o BIM e exigem formato aberto nas entregas federais abrangidas.

1.12 Vocabulário técnico

INGLÊS	PORTUGUÊS
interoperability	interoperabilidade
open standard	padrão aberto
vendor-neutral	neutro em relação ao fornecedor
information requirement	requisito de informação
information container	contêiner de informação
common data environment (CDE)	ambiente comum de dados
BIM execution plan (BEP)	plano de execução BIM

INGLÊS	PORTUGUÊS
model view	vista de modelo
data dictionary	dicionário de dados
issue	apontamento, pendência
retired / withdrawn	aposentada / retirada de uso
facility	instalação, obra

Tabela 1.12: Vocabulário do capítulo.

1.13 Exercícios

Fixação

1. Explique, com um exemplo da sua área, a diferença entre interoperabilidade sintática e semântica.
2. Qual é a diferença de finalidade entre uma MVD e um IDS?
3. Quais são as três versões oficiais do IFC, e qual identificador cada uma grava no arquivo?
4. O que o IFC 4.3 trouxe que permite descrever uma ponte com o significado correto?
5. A ISO 19650 obriga o uso de IFC? Justifique.

Aplicação

6. Abra no VS Code os três arquivos `Infra-Road.ifc` (um de cada versão) da pasta `Certification-datasets-main`. Sem alterar nada, localize em cada um as linhas `FILE_DESCRIPTION` e `FILE_SCHEMA` e confira os valores com a Tabela 1.7.
7. Nos três arquivos `Infra-Bridge.ifc`, use a busca do VS Code (`Ctrl+F`) para procurar o texto `IFCBRIDGE(`. Em quais versões ele aparece, e por quê?

Desafio

8. Um contratante público federal pede, para uma obra de ponte, a entrega do modelo em formato aberto e a verificação automática de que todos os elementos estruturais têm TAG e fase preenchidos. Monte o encadeamento de padrões (seção 1.6) para atender o pedido: versão do IFC, MVD, onde ficam os requisitos, como são verificados e como as não conformidades são comunicadas. Indique o fundamento normativo brasileiro para a exigência de formato aberto.

1.14 Respostas dos exercícios

1. A interoperabilidade sintática trata de o arquivo poder ser lido; a semântica, de o significado ser preservado. Exemplo: um modelo de drenagem exportado em formato que o software de destino abre (sintática atendida), mas em que as galerias chegam como sólidos genéricos, sem diâmetro nem material (semântica falhou). O projetista de destino vê a geometria, mas não consegue filtrar os tubos nem obter o quantitativo por diâmetro.

2. A MVD define o que um **software** deve saber exportar e importar para um tipo de troca; serve para orientar e certificar exportadores. O IDS define o que um **projeto** exige de um modelo específico e permite verificar o arquivo entregue.
3. IFC 2x3 TC1 (`IFC2X3`), IFC 4 ADD2 TC1 (`IFC4`) e IFC 4.3 ADD2 (`IFC4X3_ADD2`).
4. Entidades próprias de infraestrutura: `IfcBridge` (a ponte como instalação) e `IfcBridgePart` (suas partes), além do alinhamento e do posicionamento linear. Nas versões anteriores, a ponte precisava ser descrita como edificação.
5. Não. A ISO 19650 define o processo de gestão da informação (requisitos, planos, ambiente comum de dados, entregas) sem prescrever formato. O formato é definido nos requisitos de cada contrato; o IFC é a escolha aberta e padronizada para os modelos.
6. IFC2x3: `ViewDefinition [CoordinationView_V2.0]` e `IFC2X3` ; IFC4: `ViewDefinition [ReferenceView_V1.2]` e `IFC4` ; IFC4.3: `ViewDefinition [ReferenceView]` e `IFC4X3_ADD2` .
7. Só no arquivo IFC4.3, com 3 ocorrências. `IfcBridge` não existe no IFC2x3 nem no IFC4; nesses arquivos, a mesma ponte é descrita com outras classes, e o texto `IFCBRIDGE(` não aparece.
8. IFC 4.3, porque tem as entidades de ponte (`IfcBridge` , `IfcBridgePart`). MVD: Reference View do IFC 4.3, adequada à coordenação e à entrega de referência. Requisitos: escritos em um arquivo IDS (por exemplo, "todo `IfcBeam` , `IfcColumn` , `IfcMember` , `IfcFooting` e `IfcSlab` deve ter TAG e FASE preenchidos"), com os termos, se possível, referenciados no bSDD. Verificação: validação automática do IFC contra o IDS, com um programa como o IfcTester (Capítulo 12). Comunicação: não conformidades enviadas ao projetista em BCF. Fundamento normativo: o Decreto nº 10.306/2020 exige, nas obras federais abrangidas, a entrega também em formato aberto (não proprietário); a Lei nº 14.133/2021 estabelece a preferência pelo BIM nas licitações de obras e serviços de engenharia, quando adequado ao objeto.

1.15 Referências

- buildingSMART, situação das versões do IFC: https://technical.buildingsmart.org/standards/ifc/ifc-schema-specifications/
- buildingSMART, IFC: https://www.buildingsmart.org/standards/bsi-standards/industry-foundation-classes/
- ISO 16739-1:2024: https://www.iso.org/standard/84123.html
- ISO 29481-1:2025 (IDM): https://www.iso.org/standard/88515.html
- buildingSMART, banco de MVDs: https://technical.buildingsmart.org/standards/ifc/mvd/mvd-database/
- IDS 1.0: https://github.com/buildingSMART/IDS/releases/tag/v1.0.0
- BCF: https://technical.buildingsmart.org/standards/bcf/
- bSDD: https://search.bsdd.buildingsmart.org
- IFC 5 (desenvolvimento): https://github.com/buildingSMART/IFC5-development
- buildingSMART Brasil: https://www.buildingsmart.org.br
- Lei nº 14.133/2021: https://www.planalto.gov.br/ccivil_03/_ato2019-2022/2021/lei/l14133.htm
- Decreto nº 10.306/2020: https://www.planalto.gov.br/ccivil_03/_ato2019-2022/2020/decreto/d10306.htm

- Decreto nº 11.888/2024: https://www.planalto.gov.br/ccivil_03/_ato2023-2026/2024/decreto/d11888.htm
- CBIC, boletins de normas técnicas publicadas (NBR 15965-4 em 2021; NBR 15965-5 e 15965-6 em 2022): https://cbic.org.br/wp-content/uploads/2023/01/alteracoes-de-normas-tecnicas-setor-da-construcao-civil-2022.pdf

2 Anatomia do arquivo IFC

Um arquivo IFC é um arquivo de texto. Qualquer editor comum consegue abri-lo, e quem conhece as regras de escrita consegue ler, linha por linha, o que o software de origem gravou. Este capítulo ensina essas regras, a partir de arquivos reais.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- reconhecer a estrutura de um arquivo `.ifc`: cabeçalho, dados e encerramento;
- interpretar o cabeçalho: vista de modelo (MVD), dados do arquivo e esquema;
- ler uma linha de dados: número da instância, entidade e atributos por posição;
- identificar cada tipo de valor: texto, número, enumeração, lógico, lista, referência, vazio (`$`), derivado (`*`) e valor tipado;
- seguir as referências entre instâncias para reconstruir um objeto;
- distinguir as variantes `.ifc`, `.ifcXML` e `.ifcZIP`.

2.1 Introdução: por que ler o arquivo em texto

2.1.1 Arquivo de texto e arquivo binário

Arquivos de computador são de dois tipos principais. Um **arquivo binário** (como um `.dwg` ou um `.rvt`) guarda a informação num código que só o programa de origem, ou programas preparados para aquele formato, conseguem interpretar. Aberto num editor de texto, mostra símbolos sem sentido. Um **arquivo de texto** guarda a informação em caracteres legíveis, como este parágrafo. O `.ifc` é um arquivo de texto.

Isso tem uma consequência prática importante: **nenhum software é necessário para saber o que um IFC contém**. Basta um editor de texto e o conhecimento das regras.

2.1.2 Para que serve ler o texto

Visualizadores mostram uma interpretação do arquivo. O texto mostra o que foi realmente gravado. Ler o texto resolve dúvidas como estas:

- uma propriedade que existia no software de autoria sumiu no IFC: ela foi exportada?
- um elemento chegou com a classe errada: qual classe o exportador gravou?
- o identificador de um objeto mudou entre duas revisões: o que o arquivo diz?

Também é a base para entender a programação dos capítulos seguintes: a biblioteca IfcOpenShell lê e grava exatamente as linhas estudadas aqui.

2.1.3 Como abrir um IFC como texto

No VS Code: Arquivo > Abrir Arquivo e escolha o `.ifc`. Para buscar um trecho, use `Ctrl+F`. Arquivos grandes (dezenas de megabytes) podem demorar a abrir; para estudo, prefira arquivos pequenos, como as amostras da buildingSMART.

> **ATENÇÃO**
>
> Nunca edite à mão os arquivos da pasta de amostras. Para testar uma alteração, copie o arquivo para a pasta de exercícios, com outro nome.

2.2 O formato STEP

O IFC usa um formato de gravação criado para a indústria em geral: o **STEP Physical File** (arquivo físico STEP), definido pela norma **ISO 10303-21**. STEP é uma família de normas para troca de dados de produtos industriais, e o IFC aproveita a sua forma de escrever arquivos.

A estrutura do arquivo é sempre a mesma:

```
ISO-10303-21;          <- abertura do arquivo
HEADER;                <- início do cabeçalho
  ...                  <- descrição, dados do arquivo e esquema
ENDSEC;                <- fim do cabeçalho
DATA;                  <- início dos dados
  #1=...;              <- uma instância por linha
  ...
ENDSEC;                <- fim dos dados
END-ISO-10303-21;      <- encerramento do arquivo
```

Figura 2.1: Estrutura geral de um arquivo IFC em formato STEP.

Três regras gerais:

- toda instrução termina com ponto e vírgula;
- comentários ficam entre `/*` e `*/` e são ignorados na leitura;
- os nomes das entidades são gravados em letras maiúsculas (`IFCWALL`). A documentação e a biblioteca IfcOpenShell usam a grafia do esquema (`IfcWall`). As duas se referem à mesma entidade.

2.3 O cabeçalho (HEADER)

O cabeçalho identifica o arquivo. Tem três registros obrigatórios:

REGISTRO	CONTEÚDO
`FILE_DESCRIPTION`	Descrição do arquivo. No IFC, traz a vista de modelo declarada (MVD), como `ViewDefinition [ReferenceView]`, e o nível de implementação do formato (`'2;1'`)
`FILE_NAME`	Sete campos, nesta ordem: nome do arquivo, data e hora, autor, organização, pré-processador, sistema de origem e autorização
`FILE_SCHEMA`	Versão do esquema usada nos dados: `IFC2X3`, `IFC4` ou `IFC4X3_ADD2`

Tabela 2.1: Registros do cabeçalho.

Exemplo real, do arquivo `Infra-Road.ifc` (IFC 4.3), exportado de um software de modelagem genérico:

```
ISO-10303-21;
HEADER;
FILE_DESCRIPTION(('ViewDefinition [ReferenceView]'),'2;1');
FILE_NAME('Infra-Road.ifc','2026-06-23T11:54:08',(''),(''),'Sketchup-IFC-manager 5.6.0 / SketchUp 2026
(26.2.242)','BIM_Tools - Sketchup_IFC_manager - 5.6.0','None');
```

```
FILE_SCHEMA(('IFC4X3_ADD2'));
ENDSEC;
```

A amostra `beam-extruded-solid.ifc` traz os nomes dos sete campos de `FILE_NAME` em comentários, o que ajuda a memorizar a ordem:

```
FILE_NAME(
/* name */ 'beam-extruded-solid.ifc',
/* time_stamp */ '2016-02-04T08:11:04',
/* author */ ('redacted'),
/* organization */ ('redacted'),
/* preprocessor_version */ 'redacted',
/* originating_system */ 'redacted - redacted - 3.14159',
/* authorization */ 'None');
```

> **DICA**
>
> Ao receber um IFC, leia primeiro o cabeçalho. Ele informa a versão do esquema (que determina as classes disponíveis), a vista de modelo pretendida e o software que gerou o arquivo, que é o primeiro suspeito quando algo chega errado.

2.4 Os dados (DATA): instâncias

2.4.1 O que é uma instância

Cada linha da seção DATA é uma **instância**: um objeto concreto do modelo, como uma viga, uma unidade de medida, um ponto ou uma relação entre objetos. Uma linha tem três partes:

```
#211= IFCBEAM('0EF5_zZRv0pQPddeofU3KT',$,'ExampleBeamName','ExampleBeamDescription',$,#213,#210,'Tag',$);
|       |      └──────────────────────────── atributos, na ordem definida pelo esquema ──────────────────┘
|       └── entidade (classe do objeto)
└── número da instância
```

Figura 2.2: Partes de uma linha de dados.

- **Número da instância (`#211`):** identifica a linha dentro deste arquivo. Não é permanente: ao regravar o arquivo, um software pode renumerar tudo. O identificador permanente do objeto é o GlobalId (Capítulo 3).
- **Entidade (`IFCBEAM`):** a classe do objeto no esquema IFC. Aqui, uma viga.
- **Atributos:** os valores entre parênteses, separados por vírgulas.

2.4.2 Três regras sobre as instâncias

1. **A numeração não precisa ser contínua.** Na amostra da viga, os números saltam de `#5` para `#50` e de `#105` para `#200`.
2. **A ordem das linhas não tem significado.** Uma instância pode citar outra que aparece mais abaixo no arquivo.
3. **Os atributos são posicionais.** Não há rótulos como "nome =" ou "posição =". O significado de cada valor vem da sua posição na lista, na ordem definida pelo esquema para aquela entidade. Por isso, ler uma linha exige conhecer a lista de atributos da entidade.

2.5 Os tipos de valores

Os atributos podem conter valores de vários tipos, cada um com uma forma própria de escrita:

ESCRITA	SIGNIFICADO	EXEMPLO REAL
`'texto'`	Texto (string)	`'ExampleBeamName'`
`''` dentro do texto	Um apóstrofo	`'Caixa D''\X2\00E1\X0\gua'` = Caixa D'água
`\X2\...\X0\`	Caracteres fora do alfabeto básico, em código hexadecimal	`'Pavimenta\X2\00E700E3\X0\o'` = Pavimentação
`12`, `1000.0`, `0.`, `1.0000000000000001E-05`	Números inteiros e reais; o separador decimal é o ponto	`IFCEXTRUDEDAREASOLID(#202,#206,#207,1000.0)`
`.VALOR.`	Enumeração: um valor de uma lista fechada	`.SOLIDWALL.`, `.MILLI.`, `.ELEMENT.`
`.T.`, `.F.`, `.U.`	Lógico: verdadeiro, falso, desconhecido	`IFCINDEXEDPOLYCURVE(#200,(...),.F.)`
`#n`	Referência a outra instância	`#213`
`( ... )`	Lista (conjunto ou sequência)	`(#102,#103,#104)`
`((...),(...))`	Lista de listas	`((2.8,-79.5),(2.8,79.5),...)`
`$`	Atributo opcional sem valor	`IFCBEAM(...,$,#213,...)`
`*`	Atributo derivado: o valor é calculado pelo esquema e não é gravado	`IFCSIUNIT(*,.LENGTHUNIT.,.MILLI.,.METRE.)`
`IFCTIPO(valor)`	Valor tipado: o valor com o seu tipo declarado	`IFCLABEL('PAV-001')`, `IFCLINEINDEX((1,2))`

Tabela 2.2: Tipos de valores no formato STEP.

Observações sobre a tabela:

- **Vazio (`$`)** só é permitido em atributo opcional. Em atributo obrigatório, o arquivo fica inválido.
- **Números:** `0.` é o número zero escrito como real. `1.0000000000000001E-05` é notação científica: 1 x 10^{-5}, ou 0,00001.
- **Enumerações** são sempre escritas entre pontos e em maiúsculas. O conjunto de valores válidos é definido no esquema para cada atributo.
- **Valores tipados** aparecem em atributos que aceitam mais de um tipo de dado. O valor de uma propriedade, por exemplo, pode ser um rótulo, um número inteiro ou uma medida de comprimento, e por isso o tipo precisa ser declarado: `IFCPROPERTYSINGLEVALUE('TAG',$,IFCLABEL('PAV-001'),$)`.

Exemplo resolvido 2.1: decodificar um texto com acentos

Situação: um arquivo traz o nome `'Pavimenta\X2\00E700E3\X0\o'`. Qual é o texto?

Solução:

1. O trecho entre `\X2\` e `\X0\` é `00E700E3`. Cada caractere ocupa quatro dígitos hexadecimais: `00E7` e `00E3`.
2. Pela tabela Unicode, `00E7` é "ç" e `00E3` é "ã".
3. Juntando o texto: `Pavimenta` + `ç` + `ã` + `o` = **Pavimentação**.

CARACTERE	CÓDIGO	CARACTERE	CÓDIGO
á	`00E1`	ç	`00E7`
â	`00E2`	é	`00E9`
ã	`00E3`	ê	`00EA`
í	`00ED`	ó	`00F3`
õ	`00F5`	ú	`00FA`
Ç	`00C7`	É	`00C9`

Tabela 2.3: Códigos Unicode dos caracteres acentuados mais comuns no português.

Conclusão: acentos em nomes, descrições e valores de propriedades aparecem codificados no texto do arquivo. Um visualizador mostra "Pavimentação"; o texto mostra `Pavimenta\X2\00E700E3\X0\o`. Os dois exemplos da Tabela 2.2 foram gravados com o IfcOpenShell 0.8.5 e relidos com o texto original.

2.6 Lendo uma linha: atributos por posição

A viga `#211` da amostra, com os atributos na ordem definida pelo esquema IFC 4.3:

POSIÇÃO	ATRIBUTO	VALOR	LEITURA
1	GlobalId	`'0EF5_zZRv0pQPddeofU3KT'`	Identificador global único (22 caracteres)
2	OwnerHistory	`$`	Sem histórico de autoria (opcional no IFC4 e no IFC4.3)
3	Name	`'ExampleBeamName'`	Nome
4	Description	`'ExampleBeamDescription'`	Descrição
5	ObjectType	`$`	Vazio
6	ObjectPlacement	`#213`	Posicionamento: siga a referência
7	Representation	`#210`	Geometria: siga a referência
8	Tag	`'Tag'`	Identificador do elemento no software de origem
9	PredefinedType	`$`	Tipo predefinido não informado

Tabela 2.4: Leitura dos atributos da viga `#211`.

As posições 1 a 4 são comuns a quase todas as entidades do IFC: objetos, tipos, relações e conjuntos de propriedades começam sempre por GlobalId, OwnerHistory, Name e Description. Isso acontece porque todas herdam esses atributos de uma entidade comum, `IfcRoot`. A posição 5 vem de `IfcObject`, a 6 e a 7 de `IfcProduct`, a 8 de `IfcElement` e a 9 de `IfcBeam`. A herança é o assunto do Capítulo 3.

As entidades de geometria e de recursos, como `IfcCartesianPoint` (ponto) ou `IfcSIUnit` (unidade), não derivam de `IfcRoot` e não têm GlobalId.

2.7 Seguindo as referências: o arquivo completo de uma viga

O arquivo `beam-extruded-solid.ifc` (2.711 bytes, 28 instâncias), da pasta `IFC4.x-specification-models-main\models\building-elements\beam-extruded-solid`, está reproduzido abaixo na íntegra:

```
ISO-10303-21;
HEADER;
FILE_DESCRIPTION(('ViewDefinition [NotAssigned]'),'2;1');
FILE_NAME(
/* name */ 'beam-extruded-solid.ifc',
/* time_stamp */ '2016-02-04T08:11:04',
/* author */ ('redacted'),
/* organization */ ('redacted'),
/* preprocessor_version */ 'redacted',
/* originating_system */ 'redacted - redacted - 3.14159',
/* authorization */ 'None');

FILE_SCHEMA (('IFC4X3_ADD2'));
ENDSEC;

DATA;
/* general entities required for all IFC data sets, defining the context for the exchange */
#1= IFCGEOMETRICREPRESENTATIONCONTEXT($,'Model',3,0.0001,#3,$);
#2= IFCCARTESIANPOINT((0.0,0.0,0.0));
#3= IFCAXIS2PLACEMENT3D(#2,$,$);
#4= IFCGEOMETRICREPRESENTATIONSUBCONTEXT('Axis','Model',*,*,*,*,#1,$,.MODEL_VIEW.,$);
#5= IFCGEOMETRICREPRESENTATIONSUBCONTEXT('Body','Model',*,*,*,*,#1,$,.MODEL_VIEW.,$);
/* defines the default building (as required as the minimum spatial element)  */
#50= IFCBUILDING('39t4Pu3nTC4ekXYRIHJB9W',$,'IfcBuilding',$,$,$,$,$,$,$,$);
#51= IFCRELCONTAINEDINSPATIALSTRUCTURE('3Sa3dTJGn0H8TQIGiuGQd5',$,'Building','Building Container for Elements',
(#211),#50);
#52= IFCAXIS2PLACEMENT3D(#2,$,$);
#100= IFCPROJECT('0$WU4A9R19$vKWO$AdOnKA',$,'IfcProject',$,$,$,$,(#1),#101);
#101= IFCUNITASSIGNMENT((#102,#103,#104));
#102= IFCSIUNIT(*,.LENGTHUNIT.,.MILLI.,.METRE.);
#103= IFCSIUNIT(*,.PLANEANGLEUNIT.,$,.RADIAN.);
#104= IFCSIUNIT(*,.TIMEUNIT.,$,.SECOND.);
#105= IFCRELAGGREGATES('091a6ewbvCMQ2Vyiqspa7a',$,'Project Container','Project Container for Buildings',#100,
(#50));
#200= IFCCARTESIANPOINTLIST2D(((2.8,-79.5),(2.8,79.5),(6.31471899999999,87.985281),(14.8,91.5),(50.0,91.5),
(50.0,100.0),(-50.0,100.0),(-50.0,91.5),(-14.8,91.5),(-6.31471899999997,87.985281),(-2.79999999999999,79.5),
(-2.8,-79.5),(-6.31471899999999,-87.985281),(-14.8,-91.5),(-50.0,-91.5),(-50.0,-100.0),(50.0,-100.0),
(50.0,-91.5),(14.8,-91.5),(6.31471899999997,-87.985281)),$);
#201= IFCINDEXEDPOLYCURVE(#200,
(IFCLINEINDEX((1,2)),IFCARCINDEX((2,3,4)),IFCLINEINDEX((4,5,6,7,8,9)),IFCARCINDEX((9,10,11)),IFCLINEINDEX((11,12)
),IFCARCINDEX((12,13,14)),IFCLINEINDEX((14,15,16,17,18,19)),IFCARCINDEX((19,20,1))),.F.);
#202= IFCARBITRARYCLOSEDPROFILEDEF(.AREA.,'IPE200',#201);
#203= IFCCARTESIANPOINT((0.0,0.0,0.0));
#204= IFCDIRECTION((0.0,1.0,0.0));
#205= IFCDIRECTION((1.0,0.0,0.0));
#206= IFCAXIS2PLACEMENT3D(#203,#204,#205);
#207= IFCDIRECTION((0.0,0.0,1.0));
#208= IFCEXTRUDEDAREASOLID(#202,#206,#207,1000.0);
#209= IFCSHAPEREPRESENTATION(#5,'Body','SweptSolid',(#208));
#210= IFCPRODUCTDEFINITIONSHAPE($,$,(#209));
#211= IFCBEAM('0EF5_zZRv0pQPddeofU3KT',$,'ExampleBeamName','ExampleBeamDescription',$,#213,#210,'Tag',$);
```

```
#212= IFCAXIS2PLACEMENT3D(#2,$,$);
#213= IFCLOCALPLACEMENT($,#212);
ENDSEC;

END-ISO-10303-21;
```

Figura 2.3: Arquivo `beam-extruded-solid.ifc` completo (amostra da buildingSMART).

2.7.1 Leitura por grupos

GRUPO	INSTÂNCIAS	O QUE DEFINEM
Contexto geométrico	`#1` a `#5`	O sistema de coordenadas do modelo (3D, precisão 0,0001) e dois subcontextos: eixo (`Axis`) e corpo (`Body`). Os `*` nos subcontextos são atributos que, nessa entidade, são derivados do contexto principal
Projeto e unidades	`#100` a `#104`	O projeto (`IfcProject`) e as unidades: comprimento em milímetro, ângulo em radiano, tempo em segundo
Estrutura espacial	`#50`, `#105`	Uma edificação (`IfcBuilding`) ligada ao projeto pela relação de agregação `IfcRelAggregates`
Contenção	`#51`	A viga `#211` contida na edificação `#50` pela relação `IfcRelContainedInSpatialStructure`
Geometria	`#200` a `#210`	Um perfil I (`'IPE200'`) desenhado por pontos (`#200`) e por segmentos retos e em arco (`#201`), extrudado por 1.000 mm (`#208`)
Elemento	`#211` a `#213`	A viga, com posicionamento `#213` e geometria `#210`

Tabela 2.5: Grupos de instâncias do arquivo da viga.

Exemplo resolvido 2.2: reconstruir a geometria da viga

Pergunta: a partir do texto, como a viga `#211` tem a sua forma definida?

Solução: partindo da viga, siga o atributo 7 (Representation) e, em cada linha encontrada, a referência que leva à geometria:

```
#211 IFCBEAM
 └─ Representation  #210 IFCPRODUCTDEFINITIONSHAPE
     └─ #209 IFCSHAPEREPRESENTATION  contexto #5 ('Body'), tipo 'SweptSolid'
         └─ #208 IFCEXTRUDEDAREASOLID  profundidade 1000.0, direção #207 (0,0,1)
             └─ SweptArea #202 IFCARBITRARYCLOSEDPROFILEDEF  perfil 'IPE200'
                 └─ OuterCurve #201 IFCINDEXEDPOLYCURVE
                     └─ Points #200 IFCCARTESIANPOINTLIST2D  (20 pontos)
```

Leitura:

1. `#209` diz que a representação é do tipo `'SweptSolid'` (sólido gerado por varredura) e pertence ao contexto de corpo (`Body`).
2. `#208` é um sólido extrudado: uma área (`#202`) empurrada na direção `(0,0,1)` por 1000.0. Como a unidade de comprimento é o milímetro (`#102`), o comprimento é 1.000 mm.
3. `#202` é o contorno do perfil, chamado `'IPE200'`, descrito pela curva `#201`, que liga os 20 pontos de `#200` com trechos retos (`IFCLINEINDEX`) e em arco (`IFCARCINDEX`).

Conclusão: a viga e a sua geometria ocupam 14 das 28 instâncias do arquivo (`#200` a `#213`). É comum que a geometria ocupe a maior parte de um IFC. As informações de negócio (nome, tipo, propriedades) ocupam poucas linhas, mas são elas que os requisitos de informação cobram.

2.7.2 A ordem dos atributos nas relações

Em `#51`, `IFCRELCONTAINEDINSPATIALSTRUCTURE` grava primeiro os elementos contidos `(#211)` e depois a estrutura que os contém `#50`. Em `#105`, `IFCRELAGGREGATES` grava primeiro o objeto que agrega `#100` e depois os agregados `(#50)`. A ordem vem do esquema de cada entidade e não segue regra geral. Na dúvida, confira a lista de atributos (seção 2.10).

2.8 Exemplos de várias áreas

As linhas abaixo foram extraídas das amostras IFC 4.3 da pasta `Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene`. Todas começam pelos mesmos quatro atributos. O que muda é o final da linha, definido pela entidade.

Arquitetura (`Building-Architecture.ifc`):

```
#45=IFCBUILDINGSTOREY('1Ano2ZUxnEIvVQ_beukl8b',#1,'00 groundfloor','The ground floor, forming the base level of
the building.','buildingstorey',#47,$,$,.ELEMENT.,$);
#85=IFCSPACE('0xY$LvXaDEswJDk_VU74C_',#1,'living room','A cozy space, perfect for relaxation and family
gatherings.','living area',#87,#113,$,.ELEMENT.,$,0.);
#158=IFCWALL('1AQAupaRP1txwK1AGiN61V',#1,'house - outer wall - house right front','A solid outer wall, forming
the right front side of the house.','solidwall',#167,#179,'454425.1027891.979946.932083.920023',$);
```

- `IfcBuildingStorey` (pavimento) termina com `LongName`, `CompositionType` (`.ELEMENT.`) e `Elevation` (`$`).
- `IfcSpace` (ambiente) termina com `LongName`, `CompositionType`, `PredefinedType` e `ElevationWithFlooring` (`0.`).

Climatização (`Building-Hvac.ifc`) e **estrutura** (`Building-Structural.ifc`):

```
#81=IFCDUCTSEGMENT('38WbwIGD90nB_3T2BTU5Ed',#1,'building element',
$,'rigidsegment',#84,#96,'454425.1027891.979946.932083.2023772.884510.880032',$);
#175=IFCBEAM('0fqX614OH1YO1Njdxms2$Q',#1,'girder','A strong girder, providing essential support for
structures.','girder_segment',#184,#196,'454425.1027891.979946.932084.902513',$);
```

Ponte (`Infra-Bridge.ifc`):

```
#37=IFCBRIDGE('3vzNAxKB5AJA_Ehx4miINx',#1,'road river bridge','A sturdy bridge, spanning the river and connecting
roads.','girder',#39,$,$,.ELEMENT.,.GIRDER.);
#44=IFCBRIDGEPART('04kO$szQnDTAhzkZMdfItT',#1,'road rail bridge - approach','The approach to the road rail
bridge, ensuring a smooth transition.','approach',#46,$,$,.COMPLEX.,.LATERAL.,.NOTDEFINED.);
```

Ferrovia (`Infra-Rail.ifc`), **drenagem** (`Infra-Plumbing.ifc`) e **rodovia** (`Infra-Road.ifc`):

```
#110=IFCTRACKELEMENT('0mApyqgbP3kwWGcMHpBgdC',#1,'sleeper wood','Durable wooden sleepers, supporting the railway
tracks.','sleeper',#113,#125,'454425.1027894.2183506.2042690.955696',$);
#35=IFCPIPESEGMENT('1F4KLUGJf1aAa0YXxTvOR5',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.','culvert',#39,#51,'454425.1027892.1880634.1093597',$);
#41=IFCROADPART('13nXdzaiLCcBMI_327t3$B',#1,'road parking - road segment','A segment of the road, connecting
various parts of the infrastructure.','roadsegment',#43,$,$,.COMPLEX.,.LATERAL.,.ROADSEGMENT.);
```

Nesses arquivos, o atributo 2 (OwnerHistory) de todos os objetos aponta para `#1`, uma única instância `IFCOWNERHISTORY` compartilhada:

```
#1=IFCOWNERHISTORY(#2,#5,$,.ADDED.,1782208448,#2,#5,1782208448);
```

As datas do histórico são gravadas como número de segundos desde 01/01/1970 (tipo `IfcTimeStamp`). O valor `1782208448` corresponde a 23/06/2026, 09:54:08, no horário universal (UTC).

Exemplo resolvido 2.3: ler uma linha de ponte

Situação: interpretar `#44=IFCBRIDGEPART(...)` da ponte acima.

Solução: a lista de atributos de `IfcBridgePart` no IFC 4.3 é: GlobalId, OwnerHistory, Name, Description, ObjectType, ObjectPlacement, Representation, LongName, CompositionType, UsageType e PredefinedType. Associando por posição:

POSIÇÃO	ATRIBUTO	VALOR	LEITURA
1	GlobalId	`'04kO$szQnDTAhzkZMdfItT'`	Identificador
2	OwnerHistory	`#1`	Histórico compartilhado
3	Name	`'road rail bridge - approach'`	Nome: acesso da ponte rodoferroviária
4	Description	`'The approach to ...'`	Descrição
5	ObjectType	`'approach'`	Texto livre de tipo
6	ObjectPlacement	`#46`	Posicionamento
7	Representation	`$`	Sem geometria própria (é uma parte espacial)
8	LongName	`$`	Vazio
9	CompositionType	`.COMPLEX.`	Agrupa outras partes
10	UsageType	`.LATERAL.`	Divisão no sentido lateral
11	PredefinedType	`.NOTDEFINED.`	Tipo de parte não definido

Tabela 2.6: Leitura da parte de ponte `#44`.

Conclusão: o mesmo método (lista de atributos e associação por posição) lê qualquer linha, de qualquer área.

2.9 Variantes do arquivo

EXTENSÃO	FORMATO	OBSERVAÇÃO
`.ifc`	Texto STEP (ISO 10303-21)	Formato padrão, usado nesta apostila
`.ifcXML`	XML	Mesmo conteúdo, escrito em XML; arquivos maiores. O IfcOpenShell 0.8.5 lê `.ifcXML`, mas não grava: a tentativa retorna o erro `NotImplementedError: Writing .ifcXML files is not supported`

EXTENSÃO	FORMATO	OBSERVAÇÃO
`.ifcZIP`	Arquivo `.ifc` (ou `.ifcXML`) compactado em ZIP	Em teste, o `Infra-Road.ifc` com 263.708 bytes resultou em `.ifcZIP` com 48.292 bytes (cerca de 18% do original); o IfcOpenShell reabriu o `.ifcZIP` com as mesmas 894 instâncias

Tabela 2.7: Variantes do arquivo IFC.

O IFC 5, ainda em fase alfa, usa JSON (Capítulo 1) e não é compatível com esses formatos.

2.10 Leitura com um script

O script abaixo faz, por programa, a mesma leitura da seção 2.6: mostra o cabeçalho e traduz cada atributo de uma instância para o nome correspondente. Ele só lê o arquivo; não grava nada. Os comandos usados (`ifcopenshell.open`, `by_id`, `attribute_name`) são explicados no Capítulo 9. Aqui, o objetivo é conferir que a leitura manual coincide com a do programa.

Grave-o como `C:\IFC_Curso\exercicios\cap2\leitura_step.py`.

```
# -*- coding: utf-8 -*-
"""
leitura_step.py | Capítulo 2: leitura do cabeçalho e de uma instância de um arquivo IFC.

O que faz:
  1. Abre um arquivo IFC (somente leitura; nada é gravado).
  2. Mostra o cabeçalho: esquema, MVD declarada e software de origem.
  3. Mostra a linha STEP de uma instância e cada atributo com posição, nome e valor.

Uso (ambiente virtual ativado):
    python leitura_step.py                      -> usa o arquivo e a instância padrão abaixo
    python leitura_step.py <arquivo.ifc> <id>   -> outro arquivo e outro número de instância

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell

# Arquivo padrão: amostra da buildingSMART com uma única viga (IFC 4.3).
ARQUIVO_PADRAO = (
    r"C:\IFC_Curso\amostras\bSI\IFC4.x-specification-models-main\models"
    r"\building-elements\beam-extruded-solid\beam-extruded-solid.ifc"
)
ID_PADRAO = 211  # a instância #211 é a IfcBeam

arquivo = sys.argv[1] if len(sys.argv) > 1 else ARQUIVO_PADRAO
numero = int(sys.argv[2]) if len(sys.argv) > 2 else ID_PADRAO

modelo = ifcopenshell.open(arquivo)

print("=== CABEÇALHO (HEADER) ===")
print("Esquema (FILE_SCHEMA).........:", modelo.schema_identifier)
print("Descrição/MVD (FILE_DESCRIPTION):", modelo.header.file_description.description)
print("Software de origem (FILE_NAME):", modelo.header.file_name.originating_system)
print("Total de instâncias (DATA)....:", len(list(modelo)))

print(f"\n=== INSTÂNCIA #{numero} ===")
inst = modelo.by_id(numero)
print("Linha STEP:", inst)
print("Entidade..:", inst.is_a())
print("\nPos. | Atributo             | Valor")
for i in range(len(inst)):
    valor = inst[i]
    if valor is None:
        texto = "$ (vazio)"
    elif isinstance(valor, ifcopenshell.entity_instance):
```

```
        texto = f"referência a #{valor.id()} ({valor.is_a()})" if valor.id() else f"valor tipado {valor}"
    else:
        texto = repr(valor)
    print(f"{i + 1:>4} | {inst.attribute_name(i):<20} | {texto}")
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap2
python leitura_step.py
```

Saída obtida no teste:

```
=== CABEÇALHO (HEADER) ===
Esquema (FILE_SCHEMA).........: IFC4X3_ADD2
Descrição/MVD (FILE_DESCRIPTION): ('ViewDefinition [NotAssigned]',)
Software de origem (FILE_NAME): redacted - redacted - 3.14159
Total de instâncias (DATA)....: 28

=== INSTÂNCIA #211 ===
Linha STEP: #211=IfcBeam('0EF5_zZRv0pQPddeofU3KT',$,'ExampleBeamName','ExampleBeamDescription',$,#213,#210,'Tag',
$)
Entidade..: IfcBeam

Pos. | Atributo             | Valor
   1 | GlobalId             | '0EF5_zZRv0pQPddeofU3KT'
   2 | OwnerHistory         | $ (vazio)
   3 | Name                 | 'ExampleBeamName'
   4 | Description          | 'ExampleBeamDescription'
   5 | ObjectType           | $ (vazio)
   6 | ObjectPlacement      | referência a #213 (IfcLocalPlacement)
   7 | Representation       | referência a #210 (IfcProductDefinitionShape)
   8 | Tag                  | 'Tag'
   9 | PredefinedType       | $ (vazio)
```

A linha "Linha STEP" mostra a entidade na grafia do esquema (`IfcBeam`), e não em maiúsculas como no arquivo. É a forma como a biblioteca apresenta as instâncias.

Para outro arquivo e outra instância, informe os dois como argumentos. Exemplo com o tubo `#35` do modelo de drenagem:

```
python leitura_step.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-
Scene\Infra-Plumbing.ifc" 35
```

Trecho da saída obtida:

```
Entidade..: IfcPipeSegment
   2 | OwnerHistory         | referência a #1 (IfcOwnerHistory)
   3 | Name                 | 'sewer pipe'
   5 | ObjectType           | 'culvert'
   8 | Tag                  | '454425.1027892.1880634.1093597'
   9 | PredefinedType       | $ (vazio)
```

2.11 Resumo do capítulo

- O `.ifc` é um arquivo de texto no formato STEP (ISO 10303-21), com cabeçalho (HEADER) e dados (DATA).
- O cabeçalho informa a vista de modelo (MVD), o software de origem e a versão do esquema.

- Cada linha de dados é uma instância: número (válido só no arquivo), entidade e atributos por posição.
- `$` é vazio (só em atributo opcional); `*` é derivado; `.VALOR.` é enumeração; `#n` é referência; `IFCTIPO(valor)` é valor tipado.
- Acentos são codificados como `\X2\...\X0\`.
- Um objeto se reconstrói seguindo as referências; a geometria costuma ocupar a maior parte do arquivo.
- As variantes `.ifcXML` e `.ifcZIP` guardam o mesmo conteúdo em XML e compactado.

2.12 Vocabulário técnico

INGLÊS	PORTUGUÊS
physical file	arquivo físico
header / data section	cabeçalho / seção de dados
entity instance	instância de entidade
attribute	atributo
positional	posicional
unset (`$`)	não informado
derived (`*`)	derivado
enumeration	enumeração
select type	tipo de seleção
typed value	valor tipado
extruded solid	sólido extrudado
timestamp	carimbo de data e hora

Tabela 2.8: Vocabulário do capítulo.

2.13 Exercícios

Fixação

1. Quais são os três registros do cabeçalho, e que informação cada um traz?
2. O que significam `$` e `*`? Em que tipo de atributo cada um pode aparecer?
3. Por que o número `#211` não serve para identificar a viga de uma revisão para a outra do modelo?
4. Como aparece no arquivo a palavra "Canalização"?
5. Em `IfcRelAggregates` e em `IfcRelContainedInSpatialStructure`, qual atributo vem primeiro: o todo ou as partes?

Aplicação

6. Abra `Building-Architecture.ifc` (IFC 4.3) no VS Code. Localize a linha `#85=` (um ambiente). Seguindo a referência do atributo 6 (ObjectPlacement), encontre a instância de posicionamento e, a partir dela, a instância indicada no primeiro atributo (`PlacementRelTo`). Repita até chegar a um posicionamento cujo primeiro atributo seja `$`. A que objetos pertencem os posicionamentos encontrados?
7. Execute `leitura_step.py` para a instância `#37` de `Infra-Bridge.ifc` (IFC 4.3) e identifique a posição, o nome e o valor de cada enumeração.

Desafio

8. Usando somente o texto do arquivo da viga (Figura 2.3), responda: qual é o comprimento da viga, em que unidade, e quais são a altura e a largura do perfil? Indique as instâncias e os valores usados.

2.14 Respostas dos exercícios

1. `FILE_DESCRIPTION`: descrição e vista de modelo (MVD) declarada. `FILE_NAME`: nome, data e hora, autor, organização, pré-processador, sistema de origem e autorização. `FILE_SCHEMA`: versão do esquema (`IFC2X3`, `IFC4` ou `IFC4X3_ADD2`).
2. `$` é um atributo opcional sem valor; só pode aparecer em atributos opcionais. `*` indica um atributo derivado, cujo valor é calculado pelo esquema a partir de outros dados e por isso não é gravado; aparece nos atributos declarados como derivados naquela entidade.
3. O número da instância só vale dentro do arquivo e pode ser alterado a cada gravação. O identificador permanente é o GlobalId, primeiro atributo da linha.
4. `'Canaliza\X2\00E700E3\X0\o'`. O "ç" é `00E7` e o "ã" é `00E3`; caracteres consecutivos fora do alfabeto básico ficam dentro do mesmo trecho `\X2\...\X0\`.
5. Em `IfcRelAggregates`, o todo (`RelatingObject`) vem antes das partes (`RelatedObjects`). Em `IfcRelContainedInSpatialStructure`, as partes (`RelatedElements`) vêm antes do todo (`RelatingStructure`). Não há regra geral; é preciso conferir cada entidade.
6. A cadeia é: `#87=IFCLOCALPLACEMENT(#47,#88)`, do ambiente; `#47=IFCLOCALPLACEMENT(#40,#7)`, do pavimento "00 groundfloor"; `#40=IFCLOCALPLACEMENT(#30,#41)`, da edificação "Single-family house"; `#30=IFCLOCALPLACEMENT(#23,#31)`, do terreno "house - site"; `#23=IFCLOCALPLACEMENT($,#24)`, do terreno "environment - site", cujo `$` indica que não há posicionamento acima. Cada objeto é posicionado em relação ao que o contém, até a origem do modelo. O ponto de inserção do ambiente é `#89=IFCCARTESIANPOINT((200.,2000.,0.))`, em milímetros, relativo ao pavimento. O assunto é aprofundado no Capítulo 7.
7. A linha é `#37=IFCBRIDGE(...,.ELEMENT.,.GIRDER.)`. A posição 9 é `CompositionType` = `.ELEMENT.` (um elemento único, não um conjunto nem uma parte) e a posição 10 é `PredefinedType` = `.GIRDER.` (ponte de vigas).
8. O comprimento vem de `#208=IFCEXTRUDEDAREASOLID(#202,#206,#207,1000.0)`: profundidade de extrusão 1000.0. A unidade de comprimento do arquivo é o milímetro (`#102=IFCSIUNIT(*,.LENGTHUNIT.,.MILLI.,.METRE.)`), logo o comprimento é 1.000 mm. O perfil está em `#200`: as coordenadas vão de -100 a 100 no segundo eixo, o que dá **200 mm de altura**, e de -50

a 50 no primeiro eixo, o que dá **100 mm de largura** da mesa. Esses valores coincidem com as dimensões nominais do perfil IPE 200, indicado no nome do perfil em `#202`.

2.15 Referências

- ISO 10303-21:2016 (STEP, arquivo físico): https://www.iso.org/standard/63141.html
- Documentação do IFC 4.3 (versão oficial IFC 4.3.2.0): https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/
- Amostra da viga: `IFC4.x-specification-models-main\models\building-elements\beam-extruded-solid`
- Documentação do IfcOpenShell: https://docs.ifcopenshell.org/
- Tabela de caracteres Unicode (Latin-1 Supplement): https://www.unicode.org/charts/PDF/U0080.pdf

3 O esquema IFC

O Capítulo 2 mostrou como um arquivo IFC é escrito. Este capítulo mostra por que ele é escrito assim. Cada linha do arquivo obedece a um conjunto de regras chamado **esquema**, que define quais objetos podem existir, que informações cada um carrega e como eles se ligam. Quem sabe consultar o esquema consegue responder, sem tentativa e erro, a perguntas como "este atributo existe nesta versão?", "onde devo gravar esta informação?" e "por que o programa rejeitou este arquivo?".

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que é um esquema de dados e os conceitos de classe, instância, atributo e herança;
- ler uma definição de entidade escrita em EXPRESS;
- percorrer a hierarquia principal do IFC e explicar por que os atributos aparecem sempre na mesma ordem;
- explicar o GlobalId e o OwnerHistory, com as diferenças entre versões;
- distinguir **tipo** e **ocorrência** e reconhecer a relação que os liga;
- aplicar as regras de `PredefinedType` e `ObjectType`;
- explicar o que são relações objetificadas e atributos inversos;
- consultar o esquema e as relações de um objeto com um script.

3.1 Introdução: o que é um esquema de dados

3.1.1 Um formulário com regras

Imagine o cadastro de pacientes de uma clínica. Antes de cadastrar alguém, a clínica define o **formulário**: nome (obrigatório), data de nascimento (obrigatória, no formato dia/mês/ano), telefone (opcional), convênio (escolhido de uma lista). Cada paciente cadastrado é uma **ficha preenchida** segundo esse formulário. O formulário não é um paciente; ele diz como qualquer paciente deve ser descrito.

Um **esquema de dados** é exatamente isso: a definição formal de como a informação deve ser organizada. O esquema IFC é um formulário muito grande, com centenas de modelos de ficha, um para cada tipo de coisa que pode existir num modelo da construção: parede, viga, tubo, rodovia, pavimento, conjunto de propriedades, material, relação entre objetos.

CONCEITO	NO EXEMPLO DA CLÍNICA	NO IFC
Esquema	O conjunto de formulários da clínica	O esquema IFC de uma versão (IFC2X3, IFC4, IFC4X3_ADD2)
Entidade (classe)	O modelo de ficha "Paciente"	`IfcWall`, `IfcPipeSegment`, `IfcRoad`
Atributo	O campo "Nome" da ficha	`Name`, `GlobalId`, `PredefinedType`
Tipo de dado	"texto", "data", "item de uma lista"	`IfcLabel`, `IfcLengthMeasure`, `IfcWallTypeEnum`

CONCEITO	NO EXEMPLO DA CLÍNICA	NO IFC
Instância	Uma ficha preenchida	Uma linha do arquivo: `#158=IFCWALL(...)`

Tabela 3.1: Conceitos de um esquema de dados.

No vocabulário do IFC, o modelo de ficha se chama **entidade** (em programação, é comum o termo **classe**; neste livro, os dois são usados como sinônimos). A ficha preenchida se chama **instância** ou **objeto**. Cada linha da seção DATA de um arquivo IFC é uma instância de alguma entidade do esquema.

3.1.2 Atributos e tipos de dados

Cada entidade tem uma lista de **atributos**, isto é, os campos da ficha. Cada atributo tem:

- um **nome** (por exemplo, `Name`);
- um **tipo de dado**, que diz que valores são aceitos (texto, número, referência a outra instância, item de uma lista);
- uma indicação de **obrigatoriedade**: obrigatório ou opcional.

No arquivo, os atributos aparecem por posição, na ordem definida pelo esquema (Capítulo 2). É o esquema que diz que a nona posição de um `IfcWall` no IFC 4.3 é o `PredefinedType`.

3.1.3 Herança: classificar do geral para o particular

Classificar do geral para o particular é algo que fazemos o tempo todo. Um veículo pode ser terrestre ou aquático; um veículo terrestre pode ser um automóvel ou um caminhão. Tudo o que vale para "veículo" (ter um proprietário, uma placa ou registro) vale para "automóvel", que acrescenta características próprias (número de portas).

Em um esquema, essa organização se chama **herança**. Uma entidade mais geral é o **supertipo**; uma mais específica, derivada dela, é o **subtipo**. O subtipo **herda** todos os atributos do supertipo e acrescenta os seus. No IFC:

- toda parede é um elemento (`IfcElement`); todo elemento é um produto (`IfcProduct`), isto é, algo que pode ter posição e forma; todo produto é um objeto; e assim por diante até a raiz, `IfcRoot`;
- por isso, uma parede tem `GlobalId` (herdado de `IfcRoot`), posição (herdada de `IfcProduct`), `Tag` (herdado de `IfcElement`) e `PredefinedType` (próprio de `IfcWall`).

Algumas entidades existem apenas para organizar a classificação e não podem ser usadas diretamente. São as entidades **abstratas**. Ninguém cadastra um "veículo" genérico, e sim um automóvel ou um caminhão. Da mesma forma, nenhum arquivo contém uma linha `IFCELEMENT(...)`: `IfcElement` é abstrata, e o que aparece no arquivo são instâncias de entidades **concretas**, como `IfcWall` ou `IfcPipeSegment`.

3.1.4 Por que o esquema importa na prática

Saber consultar o esquema resolve problemas reais:

- **Exportação.** Uma parede exportada chega com `PredefinedType` vazio. Isso é erro? O esquema responde: depende de existir um tipo associado (seção 3.9).

- **Onde gravar informação.** Uma propriedade é comum a centenas de elementos iguais (tubos de uma mesma especificação, vigas de um mesmo perfil, luminárias de um mesmo modelo). O esquema oferece um lugar para ela: o **tipo** (seção 3.8).
- **Compatibilidade de versões.** Um script escrito para IFC4 falha num arquivo IFC2x3 porque o atributo usado não existe na versão antiga (seção 3.5).
- **Validação.** Validadores e requisitos de informação (Capítulo 12) verificam o arquivo contra as regras do esquema.

3.2 A linguagem EXPRESS

3.2.1 Uma linguagem para escrever esquemas

O esquema IFC não é descrito em texto livre, e sim numa linguagem formal de modelagem de dados chamada **EXPRESS**, definida pela norma **ISO 10303-11**. É a mesma família de normas do formato STEP do Capítulo 2 (ISO 10303-21): a parte 11 define como escrever o esquema, e a parte 21 define como gravar os dados que seguem o esquema.

A definição de `IfcRoot`, a entidade raiz, conforme a documentação oficial do IFC 4.3:

```
ENTITY IfcRoot
ABSTRACT SUPERTYPE OF (ONEOF
(IfcObjectDefinition
,IfcPropertyDefinition
,IfcRelationship));
GlobalId : IfcGloballyUniqueId;
OwnerHistory : OPTIONAL IfcOwnerHistory;
Name : OPTIONAL IfcLabel;
Description : OPTIONAL IfcText;
UNIQUE
UR1 : GlobalId;
END_ENTITY;
```

Figura 3.1: Definição de `IfcRoot` em EXPRESS (IFC 4.3).

Leitura, trecho por trecho:

TRECHO	SIGNIFICADO
`ENTITY IfcRoot ... END_ENTITY;`	Início e fim da definição da entidade
`ABSTRACT`	Entidade abstrata: não pode ter instâncias próprias no arquivo, só subtipos
`SUPERTYPE OF (ONEOF (...))`	Subtipos diretos; `ONEOF` indica que uma instância pertence a um único deles
`GlobalId : IfcGloballyUniqueId;`	Atributo obrigatório, seguido do tipo de dado
`OPTIONAL`	Atributo opcional: pode ficar vazio (`$` no arquivo)
`UNIQUE UR1 : GlobalId;`	Regra de unicidade: duas instâncias não podem ter o mesmo GlobalId

Tabela 3.2: Leitura da definição de `IfcRoot`.

Outras palavras-chave que aparecem nas definições:

PALAVRA-CHAVE	SIGNIFICADO
`SUBTYPE OF (...)`	Indica o supertipo, de quem a entidade herda os atributos
`DERIVE`	Atributo calculado a partir de outros; aparece como `*` no arquivo
`INVERSE`	Atributo inverso: não é gravado, é deduzido das relações (seção 3.10)
`WHERE`	Regras que cada instância deve cumprir (seção 3.9)
`TYPE ... END_TYPE;`	Definição de um tipo de dado (seção 3.2.2)

Tabela 3.3: Outras palavras-chave do EXPRESS.

No IFC2x3, a definição de `IfcRoot` é igual, exceto por um detalhe: `OwnerHistory` não tem a palavra `OPTIONAL`, isto é, é obrigatório (seção 3.7).

3.2.2 Tipos de dados

Além das entidades, o esquema declara **tipos de dados**. Eles dizem que valores um atributo aceita. Há três famílias principais:

FAMÍLIA	O QUE É	EXEMPLOS	COMO APARECE NO ARQUIVO
Tipo definido	Um nome com significado para um tipo básico (texto, número real, lógico)	`IfcLabel` (texto curto: nomes, rótulos), `IfcText` (texto longo), `IfcIdentifier` (código), `IfcLengthMeasure` (comprimento, número real), `IfcPositiveLengthMeasure` (comprimento maior que zero)	`'Parede externa'`, `1000.`
Enumeração	Uma lista fechada de valores permitidos	`IfcWallTypeEnum` (`SOLIDWALL`, `PARAPET`, `RETAININGWALL` ...), `IfcPipeSegmentTypeEnum` (`CULVERT`, `RIGIDSEGMENT` ...)	`.SOLIDWALL.`
Seleção (select)	Uma lista de tipos alternativos; o valor pode ser de qualquer um deles	`IfcValue` (reúne medidas e valores simples)	`IFCLENGTHMEASURE(2.5)`, o valor tipado do Capítulo 2

Tabela 3.4: Famílias de tipos de dados do esquema.

O uso de tipos definidos dá significado aos números. Um `IfcLengthMeasure` e um `IfcMassMeasure` são ambos números reais, mas um é comprimento e o outro é massa. O programa sabe, pelo tipo, em que unidade interpretar o valor (Capítulo 5).

O tamanho do esquema em cada versão, contado no IfcOpenShell 0.8.5:

VERSÃO	ENTIDADES	DAS QUAIS ABSTRATAS	TIPOS DEFINIDOS	ENUMERAÇÕES	SELEÇÕES
IFC2X3	653	97	117	164	46
IFC4	776	123	130	207	60
IFC4X3_ADD2	876	133	132	243	61

Tabela 3.5: Tamanho do esquema por versão.

Ninguém precisa decorar centenas de entidades. O que importa é entender a organização, que se repete: conhecendo a hierarquia principal, qualquer entidade nova é lida da mesma forma.

Exemplo resolvido 3.1: ler a enumeração de um tipo de parede

Problema. Um projeto exige classificar muros de arrimo como tal no IFC. Existe um valor próprio para isso? Em que versão?

Solução. O atributo `PredefinedType` de `IfcWall` usa a enumeração `IfcWallTypeEnum`. Consultando os valores no esquema de cada versão:

VERSÃO	VALORES DE `IFCWALLTYPEENUM`
IFC2X3	`STANDARD`, `POLYGONAL`, `SHEAR`, `ELEMENTEDWALL`, `PLUMBINGWALL`, `USERDEFINED`, `NOTDEFINED`
IFC4	`MOVABLE`, `PARAPET`, `PARTITIONING`, `PLUMBINGWALL`, `SHEAR`, `SOLIDWALL`, `STANDARD`, `POLYGONAL`, `ELEMENTEDWALL`, `USERDEFINED`, `NOTDEFINED`
IFC4X3_ADD2	`ELEMENTEDWALL`, `MOVABLE`, `PARAPET`, `PARTITIONING`, `PLUMBINGWALL`, `POLYGONAL`, `RETAININGWALL`, `SHEAR`, `SOLIDWALL`, `STANDARD`, `WAVEWALL`, `USERDEFINED`, `NOTDEFINED`

Tabela 3.6: Valores de `IfcWallTypeEnum` por versão.

Conclusão. O valor `RETAININGWALL` (muro de arrimo) só existe no IFC 4.3. Em IFC4, a solução é `USERDEFINED`, com o texto "muro de arrimo" no atributo `ObjectType` (regra da seção 3.9). Em IFC2x3, `IfcWall` nem tem `PredefinedType` (seção 3.5), e a especialização fica só no tipo (`IfcWallType`).

Todas as enumerações do IFC terminam com dois valores especiais: `USERDEFINED` (definido pelo usuário, quando nenhum valor da lista serve) e `NOTDEFINED` (não definido, quando a especialização é desconhecida).

3.3 A hierarquia principal

A árvore abaixo mostra os subtipos diretos dos níveis mais importantes do IFC 4.3, conferidos no esquema. Os nomes entre parênteses são os atributos introduzidos em cada nível.

```
IfcRoot  (GlobalId, OwnerHistory, Name, Description)
 ├─ IfcObjectDefinition ........... tudo o que é "coisa" (objetos, tipos, contexto)
 │   ├─ IfcObject  (ObjectType) ... ocorrências individuais
 │   │   ├─ IfcProduct  (ObjectPlacement, Representation) .. tem posição e/ou forma
 │   │   │   ├─ IfcElement  (Tag) ............ elementos físicos (paredes, tubos, vigas)
 │   │   │   ├─ IfcSpatialElement ............ estrutura espacial (terreno, edificação,
 │   │   │   │                                 rodovia, ponte, pavimento, ambiente)
 │   │   │   ├─ IfcPositioningElement ........ alinhamentos, eixos, referentes (IFC 4.3)
 │   │   │   ├─ IfcPort ..................... pontos de conexão de redes (Capítulo 6)
 │   │   │   └─ IfcAnnotation, IfcLinearElement, IfcStructuralActivity, IfcStructuralItem
 │   │   ├─ IfcProcess .... tarefas e eventos (planejamento da obra)
 │   │   ├─ IfcResource ... recursos: mão de obra, equipamentos, materiais de construção
 │   │   ├─ IfcControl .... controles: custos, cronogramas, licenças, solicitações
 │   │   ├─ IfcGroup ...... grupos, sistemas e zonas
 │   │   └─ IfcActor ...... pessoas e organizações envolvidas
 │   ├─ IfcTypeObject ...... tipos (seção 3.8)
 │   └─ IfcContext ......... contexto: o projeto (IfcProject) e bibliotecas
 ├─ IfcPropertyDefinition .. conjuntos de propriedades e de quantidades (Capítulo 5)
 └─ IfcRelationship ........ relações entre objetos (seção 3.10)
```

Figura 3.2: Hierarquia principal do IFC 4.3.

Três observações:

1. **`IfcRoot`, `IfcObjectDefinition`, `IfcObject`, `IfcProduct` e `IfcElement` são abstratas** nas três versões. No arquivo só aparecem entidades concretas.
2. **Nem tudo deriva de `IfcRoot`.** As entidades de geometria (`IfcCartesianPoint`, `IfcExtrudedAreaSolid`), de unidades (`IfcSIUnit`), de posicionamento (`IfcLocalPlacement`) e de outros recursos ficam fora dessa árvore. Elas não têm GlobalId: são "peças de apoio" que só existem para descrever os objetos de `IfcRoot`. Por isso, no Capítulo 2, a linha `#208=IFCEXTRUDEDAREASOLID(...)` não começava com um identificador entre aspas.
3. **Os três ramos de `IfcRoot` organizam todo o modelo.** Objetos (o que existe), definições de propriedades (o que se sabe sobre eles) e relações (como se ligam). Os próximos capítulos seguem essa divisão: estrutura espacial e relações (Capítulo 4), propriedades (Capítulo 5), elementos por domínio (Capítulo 6) e geometria (Capítulo 7).

3.4 Herança em várias áreas

Cadeias de herança de entidades de diferentes áreas, conferidas no esquema IFC 4.3:

ÁREA	ENTIDADE	CADEIA ATÉ `IFCROOT`
Arquitetura	`IfcWall` (parede)	`IfcWall` > `IfcBuiltElement` > `IfcElement` > `IfcProduct` > `IfcObject` > `IfcObjectDefinition` > `IfcRoot`
Arquitetura	`IfcDoor` (porta)	`IfcDoor` > `IfcBuiltElement` > `IfcElement` > … > `IfcRoot`
Estrutura	`IfcBeam` (viga)	`IfcBeam` > `IfcBuiltElement` > `IfcElement` > … > `IfcRoot`
Rodovia	`IfcCourse` (camada de pavimento)	`IfcCourse` > `IfcBuiltElement` > `IfcElement` > … > `IfcRoot`
Ferrovia	`IfcTrackElement` (elemento de via)	`IfcTrackElement` > `IfcBuiltElement` > `IfcElement` > … > `IfcRoot`
Climatização	`IfcDuctSegment` (trecho de duto)	`IfcDuctSegment` > `IfcFlowSegment` > `IfcDistributionFlowElement` > `IfcDistributionElement` > `IfcElement` > … > `IfcRoot`
Hidráulica, drenagem, industrial	`IfcPipeSegment` (trecho de tubo)	`IfcPipeSegment` > `IfcFlowSegment` > `IfcDistributionFlowElement` > `IfcDistributionElement` > `IfcElement` > … > `IfcRoot`
Elétrica	`IfcLightFixture` (luminária)	`IfcLightFixture` > `IfcFlowTerminal` > `IfcDistributionFlowElement` > `IfcDistributionElement` > `IfcElement` > … > `IfcRoot`
Arquitetura	`IfcSpace` (ambiente)	`IfcSpace` > `IfcSpatialStructureElement` > `IfcSpatialElement` > `IfcProduct` > … > `IfcRoot`
Ponte	`IfcBridge` (ponte)	`IfcBridge` > `IfcFacility` > `IfcSpatialStructureElement` > `IfcSpatialElement` > `IfcProduct` > … > `IfcRoot`
Traçado	`IfcAlignment` (alinhamento)	`IfcAlignment` > `IfcLinearPositioningElement` > `IfcPositioningElement` > `IfcProduct` > … > `IfcRoot`

ÁREA	ENTIDADE	CADEIA ATÉ `IFCROOT`
Planejamento	`IfcTask` (tarefa)	`IfcTask` > `IfcProcess` > `IfcObject` > `IfcObjectDefinition` > `IfcRoot`
Propriedades	`IfcPropertySet`	`IfcPropertySet` > `IfcPropertySetDefinition` > `IfcPropertyDefinition` > `IfcRoot`
Relações	`IfcRelAggregates`	`IfcRelAggregates` > `IfcRelDecomposes` > `IfcRelationship` > `IfcRoot`

Tabela 3.7: Cadeias de herança em várias áreas (IFC 4.3).

O que a herança ensina na prática:

- **Tudo o que vale para um nível superior vale para os de baixo.** Todo `IfcElement` tem `Tag`. Todo `IfcProduct` pode ter posição e geometria. Todo `IfcRoot` tem GlobalId. Uma tarefa de cronograma (`IfcTask`) tem GlobalId, mas não tem posição, porque não é um `IfcProduct`.
- **Uma consulta pelo supertipo alcança todos os subtipos.** Buscar `IfcFlowSegment` retorna tubos, dutos, cabos, eletrocalhas e trechos de transportadores (`IfcPipeSegment`, `IfcDuctSegment`, `IfcCableSegment`, `IfcCableCarrierSegment`, `IfcConveyorSegment`). Buscar `IfcBuiltElement` retorna paredes, lajes, vigas, pilares, portas, camadas de pavimento, trilhos e dezenas de outros elementos construídos. Esse recurso é usado a todo momento nos scripts (Capítulo 9).
- **No IFC 4.3, a edificação é uma instalação.** `IfcBuilding` é subtipo de `IfcFacility`, ao lado de `IfcRoad`, `IfcBridge`, `IfcRailway` e `IfcMarineFacility`. Uma consulta por `IfcFacility` retorna todas as instalações do modelo, de qualquer natureza.

Exemplo resolvido 3.2: escolher o nível certo de consulta

Problema. Deseja-se contar, num modelo IFC 4.3 de um conjunto hospitalar, (a) todos os trechos de tubulação, (b) todos os trechos lineares de qualquer rede (tubos, dutos, cabos) e (c) todos os elementos físicos. Por qual entidade consultar em cada caso?

Solução.

- (a) `IfcPipeSegment`: só tubos.
- (b) `IfcFlowSegment`: o supertipo comum a todos os trechos de rede.
- (c) `IfcElement`: o supertipo de todos os elementos físicos. Como é abstrata, a consulta retorna instâncias dos seus subtipos concretos.

Cuidado com a versão. Em IFC2x3, a entidade `IfcPipeSegment` não existe (seção 3.8). Tubos aparecem como `IfcFlowSegment`, e a distinção entre tubo e duto fica no tipo associado. A consulta (a), feita num arquivo IFC2x3, retornaria erro ou zero.

3.5 Atributos herdados e diferenças entre versões

A ordem dos atributos de uma entidade segue a herança: primeiro os de `IfcRoot`, depois os de cada nível intermediário, por último os da própria entidade. Comparação de `IfcWall` nas três versões, conferida no esquema:

POS.	ATRIBUTO	DEFINIDO EM	IFC2X3	IFC4	IFC4.3
1	GlobalId	`IfcRoot`	obrigatório	obrigatório	obrigatório
2	OwnerHistory	`IfcRoot`	**obrigatório**	opcional	opcional
3	Name	`IfcRoot`	opcional	opcional	opcional
4	Description	`IfcRoot`	opcional	opcional	opcional
5	ObjectType	`IfcObject`	opcional	opcional	opcional
6	ObjectPlacement	`IfcProduct`	opcional	opcional	opcional
7	Representation	`IfcProduct`	opcional	opcional	opcional
8	Tag	`IfcElement`	opcional	opcional	opcional
9	PredefinedType	`IfcWall`	**não existe**	opcional	opcional

Tabela 3.8: Atributos de `IfcWall` por versão.

O supertipo imediato de `IfcWall` é `IfcBuildingElement` no IFC2x3 e no IFC4, e `IfcBuiltElement` no IFC 4.3. A troca de nome ("elemento de edificação" para "elemento construído") reflete a ampliação do esquema para a infraestrutura: uma camada de pavimento ou um trilho não são elementos de edificação.

Consequências práticas:

- um script que lê `parede.PredefinedType` funciona em IFC4 e IFC4.3, mas falha num arquivo IFC2x3;
- um arquivo IFC2x3 sem `OwnerHistory` é inválido; em IFC4 ou IFC4.3, é válido;
- um script que consulta `IfcBuiltElement` falha num arquivo IFC4, onde a entidade equivalente é `IfcBuildingElement`.

Por isso, todo script deste livro verifica a versão do arquivo quando o comportamento muda entre versões (seção 3.11).

3.6 GlobalId

3.6.1 O que é

O **GlobalId** (tipo `IfcGloballyUniqueId`) é o identificador permanente de toda instância derivada de `IfcRoot`. Pense no número de chassi de um automóvel: a placa pode mudar, o dono pode mudar, mas o chassi identifica aquele veículo durante toda a vida útil. O número `#35` do arquivo é como a posição do carro num estacionamento: vale só naquele momento, naquele arquivo.

Características:

- representa um número de 128 bits, chamado UUID (do inglês *universally unique identifier*, identificador universalmente único), gerado de forma que a chance de repetição seja desprezível;
- no arquivo, é gravado compactado em **22 caracteres**, com o alfabeto `0-9`, `A-Z`, `a-z`, `_` e `$`, nessa ordem. Essa codificação é própria do IFC; não é a Base64 comum;
- é obrigatório e deve ser único no arquivo (regra `UR1` de `IfcRoot`).

Exemplo com o tubo `#35` de `Infra-Plumbing.ifc` (IFC 4.3), convertido com o módulo `ifcopenshell.guid`:

FORMA	VALOR
Compactada (no arquivo IFC)	`1F4KLUGJf1aAa0YXxTvOR5`
Expandida (UUID em hexadecimal)	`4f11455e413a4190a9008a1edde586c5`
UUID no formato usual	`4f11455e-413a-4190-a900-8a1edde586c5`

Tabela 3.9: As formas do GlobalId.

A conversão nos dois sentidos devolve o mesmo valor. Programas diferentes podem mostrar o GlobalId em formas diferentes (alguns mostram o UUID), mas trata-se do mesmo identificador.

3.6.2 Por que a estabilidade importa

O GlobalId é o que permite reconhecer o mesmo objeto em arquivos diferentes. Ele liga:

- as revisões de um modelo: a parede da revisão 01 é a mesma da revisão 02?
- os apontamentos de coordenação em BCF, que citam os objetos pelo GlobalId (Capítulo 12);
- a comparação entre versões com ferramentas como o `ifcdiff` (Capítulo 12);
- o preenchimento de metadados em lote a partir de planilhas, em que cada linha da planilha precisa encontrar o seu objeto (Capítulo 10).

Nas amostras da buildingSMART, o mesmo modelo de drenagem foi exportado em IFC2x3 e em IFC 4.3. O tubo tem o mesmo GlobalId, `1F4KLUGJf1aAa0YXxTvOR5`, nos dois arquivos, embora o número da instância seja `#25` num e `#35` no outro. É o comportamento desejado.

> **ATENÇÃO**
>
> Se o software de autoria gera GlobalIds novos a cada exportação, todos esses vínculos se perdem. Antes de montar um fluxo que dependa do GlobalId, exporte o mesmo modelo duas vezes e compare os identificadores. O comportamento dos exportadores do Revit e do Civil 3D é tratado no Capítulo 13.

3.7 OwnerHistory

O atributo **OwnerHistory** aponta para uma instância `IfcOwnerHistory`, que registra quem criou ou alterou o objeto, com qual aplicativo e quando. É o "carimbo de autoria" da instância. Atributos, em ordem:

POS.	ATRIBUTO	CONTEÚDO	IFC2X3	IFC4 E IFC4.3
1	OwningUser	Pessoa e organização responsáveis	obrigatório	obrigatório
2	OwningApplication	Aplicativo que criou o objeto	obrigatório	obrigatório
3	State	Estado (por exemplo, somente leitura)	opcional	opcional
4	ChangeAction	Ação: adicionado, modificado, excluído	**obrigatório**	opcional
5	LastModifiedDate	Data da última modificação	opcional	opcional

POS.	ATRIBUTO	CONTEÚDO	IFC2X3	IFC4 E IFC4.3
6	LastModifyingUser	Quem modificou por último	opcional	opcional
7	LastModifyingApplication	Com qual aplicativo	opcional	opcional
8	CreationDate	Data de criação	obrigatório	obrigatório

Tabela 3.10: Atributos de `IfcOwnerHistory`.

Exemplo real (`Infra-Plumbing.ifc` , IFC 4.3):

```
#1=IFCOWNERHISTORY(#2,#5,$,.ADDED.,1782208448,#2,#5,1782208448);
#2=IFCPERSONANDORGANIZATION(#3,#4,$);
#3=IFCPERSON('3720f2e9-0107-4ce6-b699-e20d9bd03331','Jan B.',$,$,$,$,$);
#4=IFCORGANIZATION($,'buildingSMART International','buildingSMART is the worldwide industry body driving the
digital transformation of the built environment.',$,$);
#5=IFCAPPLICATION(#6,'5.6.0','IFC manager for sketchup','su_ifcmanager');
```

Figura 3.3: Registro de autoria de uma amostra.

As datas são gravadas como `IfcTimeStamp` : o número de segundos desde 1º de janeiro de 1970 (UTC). O valor `1782208448` corresponde a 23/06/2026, 09:54:08 UTC. Todos os objetos do arquivo apontam para o mesmo `#1` , o que é comum: o exportador registra uma única autoria para o arquivo inteiro.

VERSÃO	SITUAÇÃO DO ATRIBUTO `OWNERHISTORY` EM `IFCROOT`
IFC2x3	Obrigatório
IFC4 e IFC4.3	Opcional

Tabela 3.11: Obrigatoriedade do OwnerHistory.

Nas amostras, os dois casos aparecem: a viga do Capítulo 2 tem `$` na posição 2, e os modelos da Figura 3.3 apontam todos os objetos para `#1` . Na prática, o OwnerHistory raramente é usado para controle de revisão; esse controle costuma ser feito por propriedades (Capítulo 5) e pelo ambiente comum de dados.

3.8 Tipos e ocorrências

3.8.1 A ideia: o catálogo e a peça instalada

Pense num catálogo de fabricante de válvulas. O catálogo descreve a válvula gaveta de 4 polegadas, classe 150, corpo em aço-carbono: dimensões, peso, material, pressão admissível. Na obra, dezenas de válvulas daquele modelo são instaladas, cada uma num lugar, com uma identificação própria (TAG) e uma data de instalação. O catálogo descreve o que as válvulas têm **em comum**; cada válvula instalada tem o que é **só dela**.

O IFC organiza a informação da mesma forma:

- uma **ocorrência** é o objeto individual, posicionado no modelo: este trecho de tubo, esta parede, esta camada de pavimento;
- um **tipo** reúne o que é comum a várias ocorrências: a especificação do tubo, o tipo de parede, a camada padrão.

CONCEITO	ENTIDADE IFC	ANALOGIAS
Tipo	Subtipos de `IfcTypeObject`: `IfcWallType`, `IfcPipeSegmentType`, `IfcValveType`, `IfcCourseType`, `IfcBeamType`	Item de catálogo; definição de bloco no AutoCAD; tipo de família no Revit
Ocorrência	Subtipos de `IfcObject`: `IfcWall`, `IfcPipeSegment`, `IfcValve`, `IfcCourse`, `IfcBeam`	Peça instalada; referência de bloco inserida; instância no Revit
Ligação	`IfcRelDefinesByType` (atributos `RelatedObjects` e `RelatingType`)	A inserção aponta para a definição

Tabela 3.12: Tipo, ocorrência e ligação.

3.8.2 O que o tipo carrega

A cadeia do tipo de tubo no IFC 4.3 é paralela à da ocorrência: `IfcPipeSegmentType` > `IfcFlowSegmentType` > `IfcDistributionFlowElementType` > `IfcDistributionElementType` > `IfcElementType` > `IfcTypeProduct` > `IfcTypeObject` > `IfcObjectDefinition` > `IfcRoot`. Os atributos introduzidos nos níveis do tipo:

NÍVEL	ATRIBUTO	CONTEÚDO
`IfcTypeObject`	`ApplicableOccurrence`	Texto que indica a que ocorrências o tipo se aplica
`IfcTypeObject`	`HasPropertySets`	Conjuntos de propriedades comuns a todas as ocorrências
`IfcTypeProduct`	`RepresentationMaps`	Geometria compartilhada, definida uma vez e reaproveitada (Capítulo 7)
`IfcTypeProduct`	`Tag`	Identificação do tipo
`IfcElementType`	`ElementType`	Nome do tipo definido pelo usuário, quando `PredefinedType` é `USERDEFINED`
Tipo concreto (ex.: `IfcWallType`)	`PredefinedType`	Especialização, **obrigatória** nos tipos de elemento das três versões

Tabela 3.13: Atributos dos tipos.

As propriedades definidas no tipo valem para todas as ocorrências ligadas a ele, salvo quando a ocorrência define a mesma propriedade com outro valor. Nesse caso, prevalece o valor da ocorrência (Capítulo 5).

3.8.3 Exemplo real: 24 tubos, 1 tipo

No arquivo `Infra-Plumbing.ifc` (IFC 4.3):

```
#33=IFCPIPESEGMENTTYPE('080SXXxhz7KRyQXS17Pb41',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.',$,$,$,'1092611','culvert',.CULVERT.);
#34=IFCRELDEFINESBYTYPE('3OhsgF8sj9Z9q7RBjByRUz',#1,$,$,
(#35,#58,#69,#80,#91,#102,#113,#124,#252,#266,#277,#288,#299,#310,#318,#329,#361,#376,#387,#398,#409,#420,#428,#4
36),#33);
#35=IFCPIPESEGMENT('1F4KLUGJf1aAa0YXxTvOR5',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.','culvert',#39,#51,'454425.1027892.1880634.1093597',$);
```

Figura 3.4: Tipo, relação e ocorrência (IFC 4.3).

- `#33` é o tipo. Na posição 10, `PredefinedType = .CULVERT.` (galeria ou bueiro, segundo a documentação: canal coberto ou tubo de grande diâmetro que conduz água sob uma rodovia ou ferrovia). As posições 5, 6 e 7 (`ApplicableOccurrence`, `HasPropertySets`, `RepresentationMaps`) estão vazias.
- `#34` liga as 24 ocorrências ao tipo, numa única relação.
- `#35` é uma das ocorrências. O `PredefinedType` dela está vazio (`$`), porque o tipo já o define.

Uma informação comum aos 24 tubos, como a especificação de material ou a classe de pressão, deve ficar no tipo. Informações próprias de cada trecho (TAG, comprimento, posição, trecho da rede) ficam na ocorrência.

3.8.4 O mesmo modelo em IFC2x3

A versão IFC2x3 do mesmo modelo:

```
#23=IFCPIPESEGMENTTYPE('080SXXxhz7KRyQXS17Pb41',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.',$,$,$,'1092611','culvert',.USERDEFINED.);
#24=IFCRELDEFINESBYTYPE('3OhsgF8sj9Z9q7RBjByRUz',#1,$,$,(#25,#341,#383, ... ,#4329),#23);
#25=IFCFLOWSEGMENT('1F4KLUGJf1aAa0YXxTvOR5',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.',$,#29,#302,'454425.1027892.1880634.1093597');
```

Figura 3.5: O mesmo tipo em IFC2x3 (lista de ocorrências abreviada).

Três diferenças, todas explicadas pelo esquema:

1. **A ocorrência é `IFCFLOWSEGMENT`**, porque `IfcPipeSegment` não existe no IFC2x3. A especialização "tubo" fica só no tipo.
2. **A ocorrência tem 8 atributos, não 9**: não há `PredefinedType` nas ocorrências de IFC2x3.
3. **O tipo tem `PredefinedType = .USERDEFINED.` e `ElementType = 'culvert'`**. A enumeração `IfcPipeSegmentTypeEnum` do IFC2x3 não tem o valor `CULVERT` (ele entrou no IFC4). O exportador aplicou corretamente a regra: `USERDEFINED` e o nome no atributo textual.

3.8.5 Como o tipo é encontrado a partir da ocorrência

A relação `#34` aponta para a ocorrência, mas a ocorrência `#35` não tem nenhum atributo que aponte para a relação. Como, então, um programa descobre o tipo de um tubo? Pelos **atributos inversos** (seção 3.10):

VERSÃO	INVERSO NA OCORRÊNCIA	INVERSO NO TIPO
IFC2x3	`IsDefinedBy`: reúne as relações de tipo e de propriedades; é preciso filtrar as do tipo `IfcRelDefinesByType`	`ObjectTypeOf`
IFC4 e IFC4.3	`IsTypedBy`: exclusivo da relação com o tipo (no máximo uma)	`Types`

Tabela 3.14: Inversos entre tipo e ocorrência.

3.9 PredefinedType e ObjectType

3.9.1 Duas formas de especializar

Uma entidade como `IfcWall` cobre muitas paredes diferentes. Para dizer que uma parede é de contenção, divisória ou maciça, o IFC oferece dois atributos:

- **`PredefinedType`** especializa a entidade com um valor de **enumeração**: uma parede `.SOLIDWALL.`, um tubo `.CULVERT.`, uma camada `.PAVEMENT.`. Por ser uma lista fechada, é fácil de verificar e de filtrar por programa.
- **`ObjectType`** (herdado de `IfcObject`) é um **texto livre**. Segundo a documentação, ele guarda o tipo definido pelo usuário quando `PredefinedType` é `USERDEFINED`. Nos tipos, o atributo equivalente é `ElementType`.

3.9.2 A regra formal

A documentação do IFC 4.3 define, para `IfcPipeSegment`, a regra `CorrectPredefinedType`:

> REGRA CORRECTPREDEFINEDTYPE (IFC 4.3)
>
> "Either the *PredefinedType* attribute is unset (e.g. because an *IfcPipeSegmentType* is associated), or the inherited attribute *ObjectType* shall be provided, if the *PredefinedType* is set to USERDEFINED."
>
> Tradução: ou o atributo *PredefinedType* fica vazio (por exemplo, porque um *IfcPipeSegmentType* está associado), ou o atributo herdado *ObjectType* deve ser preenchido, se o *PredefinedType* for USERDEFINED.

Além da regra, a descrição do atributo reforça o padrão. Para `IfcBeam`, por exemplo, a documentação diz: "The *PredefinedType* shall only be used, if no *IfcBeamType* is assigned" (o `PredefinedType` só deve ser usado se nenhum `IfcBeamType` estiver associado).

Uma segunda regra, `CorrectTypeAssigned`, exige que, havendo tipo associado, ele seja um `IfcPipeSegmentType`. Um tubo não pode ser ligado a um tipo de parede. As demais entidades com `PredefinedType` seguem regras equivalentes, com os nomes trocados.

Resumo das regras:

SITUAÇÃO	ONDE FICA A ESPECIALIZAÇÃO
Ocorrência ligada a um tipo	No `PredefinedType` do **tipo**; o da ocorrência fica vazio
Ocorrência sem tipo, valor previsto na enumeração	No `PredefinedType` da ocorrência
Valor não previsto na enumeração	`PredefinedType` = `USERDEFINED` e o texto em `ObjectType` (na ocorrência) ou em `ElementType` (no tipo)
Especialização desconhecida	`PredefinedType` = `NOTDEFINED`

Tabela 3.15: Regras de especialização.

3.9.3 Exemplos nas amostras

ARQUIVO (IFC 4.3)	OCORRÊNCIA	TIPO	LEITURA
`Building-Architecture.ifc`	`IfcWall #158: PredefinedType $, ObjectType 'solidwall'`	`IfcWallType #156: .SOLIDWALL.`	Padrão correto; o `ObjectType` repete o valor por escolha do exportador
`Infra-Road.ifc`	`IfcCourse #57: PredefinedType $, ObjectType 'core'`	`IfcCourseType #55: .CORE.`	Padrão correto: a camada herda a especialização do tipo
`Infra-Plumbing.ifc`	`IfcPipeSegment #35: $, 'culvert'`	`IfcPipeSegmentType #33: .CULVERT.`	Padrão correto

Tabela 3.16: Especialização nas amostras.

Nas três amostras, o valor está no tipo, como a regra prevê. O `ObjectType` preenchido na ocorrência não é exigido nesse caso; é uma característica do exportador usado (IFC Manager para SketchUp).

Exemplo resolvido 3.3: diagnosticar a especialização

Problema. Avalie as situações abaixo, todas em IFC 4.3:

1. `IFCVALVE(...,'gaveta',...,.USERDEFINED.)`, sem tipo associado.
2. `IFCVALVE(...,$,...,.USERDEFINED.)`, sem tipo associado.
3. `IFCSLAB(...,$,...,$)`, ligada a um `IfcSlabType` com `.FLOOR.`.
4. `IFCBEAM(...,$,...,.BEAM.)`, ligada a um `IfcBeamType` com `.GIRDER_SEGMENT.`.

Solução.

1. Correto, embora pouco recomendável: `USERDEFINED` com `ObjectType` preenchido cumpre a regra. Porém, a enumeração de válvulas (`IfcValveTypeEnum`) tem o valor `ISOLATING` (válvula de bloqueio), que representaria melhor uma válvula gaveta; o texto "gaveta" poderia ir para uma propriedade.
2. **Incorreto**: `USERDEFINED` exige `ObjectType` preenchido. Um validador acusa a violação da regra `CorrectPredefinedType`.
3. Correto: a ocorrência vazia herda `.FLOOR.` do tipo.
4. Conflito: a ocorrência diz `BEAM` e o tipo diz `GIRDER_SEGMENT`. A documentação do atributo `PredefinedType` de `IfcBeam` determina: "The *PredefinedType* shall only be used, if no *IfcBeamType* is assigned" (o `PredefinedType` só deve ser usado se nenhum `IfcBeamType` estiver associado). A regra formal não detecta esse caso, porque só verifica a combinação `USERDEFINED` sem `ObjectType`; mesmo assim, a ocorrência preenchida contraria a documentação e cria informação contraditória. A correção é esvaziar o `PredefinedType` da ocorrência ou ligá-la a um tipo coerente.

3.10 Relações objetificadas e atributos inversos

3.10.1 Relações como objetos

Em muitos sistemas, uma ligação entre dois objetos é um campo de um deles: a ficha do tubo teria um campo "tipo" apontando para o catálogo. No IFC, as ligações entre objetos são, em geral, **instâncias próprias**, subtipos de `IfcRelationship`. Essas instâncias são chamadas de **relações objetificadas** (a relação vira um objeto). Vantagens:

- a relação tem GlobalId, nome e descrição, como qualquer entidade derivada de `IfcRoot`;
- uma única relação pode ligar muitos objetos (um tipo e 24 tubos; um pavimento e todas as paredes dele);
- o objeto não precisa ser alterado quando uma relação é criada ou removida. Associar uma classificação a mil elementos é criar uma relação, não editar mil linhas.

O Capítulo 4 estuda as relações mais usadas. Por ora, basta reconhecer o padrão: nas relações, os atributos com prefixo `Relating` apontam para o lado "um" (o todo, o tipo, o contêiner) e os com prefixo `Related` apontam para o lado "muitos" (as partes, as ocorrências, os elementos).

3.10.2 Atributos inversos: o caminho de volta

A relação `#157=IFCRELDEFINESBYTYPE(...,(#158),#156)` é gravada apontando para a parede `#158` e para o tipo `#156`. A parede não tem, no arquivo, nenhum campo que aponte para a relação. Mesmo assim, é natural perguntar à parede "qual é o seu tipo?".

Para isso, o esquema declara **atributos inversos**. `IsTypedBy`, por exemplo, é um inverso declarado em `IfcObject`: ele significa "as relações `IfcRelDefinesByType` que me citam em `RelatedObjects`". Os programas reconstroem os inversos durante a leitura. No IfcOpenShell, `parede.IsTypedBy` devolve a relação `#157`, e `parede.IsTypedBy[0].RelatingType` devolve o tipo `#156`.

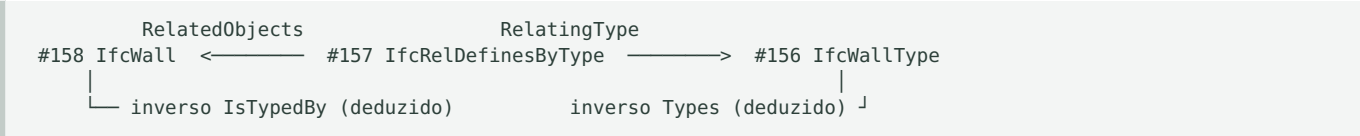


Figura 3.6: Atributos diretos (gravados) e inversos (deduzidos).

Principais inversos, conferidos no esquema:

NÍVEL	INVERSO	RELAÇÃO DE ORIGEM	VERSÕES	USO
`IfcObjectDefinition`	`Decomposes` / `IsDecomposedBy`	`IfcRelAggregates` (IFC4 e IFC4.3); `IfcRelDecomposes` (IFC2x3)	Todas	"Faço parte de" / "Sou composto por"

NÍVEL	INVERSO	RELAÇÃO DE ORIGEM	VERSÕES	USO
`IfcObjectDefinition`	`Nests` / `IsNestedBy`	`IfcRelNests`	IFC4, IFC4.3	Aninhamento: portas de conexão, componentes de um alinhamento
`IfcObjectDefinition`	`HasAssociations`	Associações (`IfcRelAssociates` e subtipos)	Todas	Material, classificação, documentos (Capítulo 5)
`IfcObjectDefinition`	`HasAssignments`	Atribuições (`IfcRelAssigns` e subtipos)	Todas	Grupos, sistemas, tarefas (Capítulo 4)
`IfcObject`	`IsDefinedBy`	`IfcRelDefinesByProperties` (IFC4+); `IfcRelDefines` (IFC2x3)	Todas	Conjuntos de propriedades; no IFC2x3, também o tipo
`IfcObject`	`IsTypedBy`	`IfcRelDefinesByType`	IFC4, IFC4.3	Tipo
`IfcTypeObject`	`Types` (IFC4+) / `ObjectTypeOf` (IFC2x3)	`IfcRelDefinesByType`	Todas	Ocorrências de um tipo
`IfcElement`	`ContainedInStructure`	`IfcRelContainedInSpatialStructure`	Todas	Contêiner espacial (no máximo um)
`IfcElement`	`HasOpenings` / `FillsVoids`	`IfcRelVoidsElement` / `IfcRelFillsElement`	Todas	Aberturas em paredes e lajes; portas e janelas que as preenchem
`IfcElement`	`HasSurfaceFeatures`	`IfcRelAdheresToElement`	IFC4.3	Marcações e tratamentos de superfície
`IfcSpatialElement`	`ContainsElements`	`IfcRelContainedInSpatialStructure`	Todas	Elementos contidos no espaço

Tabela 3.17: Principais atributos inversos.

No IFC2x3, os inversos de estrutura espacial estão em `IfcSpatialStructureElement`, porque `IfcSpatialElement` ainda não existia.

> **OBSERVAÇÃO**
>
> Os inversos **não aparecem no arquivo**. Eles existem no esquema e nos programas que leem o arquivo. No texto do arquivo, a informação está sempre do lado da relação. Para descobrir, lendo o texto, a que pavimento pertence uma parede, procure a relação `IFCRELCONTAINEDINSPATIALSTRUCTURE` que cita o número da parede.

3.11 Consulta com um script

O script abaixo reúne os conceitos do capítulo. A Parte A consulta o esquema: para uma entidade, informa se ela é abstrata, a cadeia de herança e os atributos em cada versão. A Parte B abre um arquivo e mostra, para uma instância, o GlobalId nas duas formas, o tipo associado, o contêiner espacial e o objeto do qual ela faz parte. O script só lê; não grava nada.

Os comandos de abertura e consulta (`ifcopenshell.open`, `by_id`) são explicados no Capítulo 9. O módulo `ifcopenshell.ifcopenshell_wrapper` dá acesso às definições do esquema e é de nível baixo; é usado aqui apenas para consulta ao esquema.

Grave-o como `C:\IFC_Curso\exercicios\cap3\esquema.py`.

```
# -*- coding: utf-8 -*-
"""
esquema.py | Capítulo 3: consulta ao esquema IFC e às relações de uma instância.

O que faz:
  Parte A: para uma entidade, mostra se ela é abstrata, a cadeia de herança
           e os atributos (obrigatório ou opcional) em IFC2X3, IFC4 e IFC4X3_ADD2.
  Parte B: abre um arquivo IFC (somente leitura; nada é gravado) e mostra, para
           uma instância: o GlobalId nas duas formas, o tipo associado, o
           contêiner espacial e o objeto do qual ela faz parte (agregação).

Uso (ambiente virtual ativado):
    python esquema.py                                  -> valores padrão abaixo
    python esquema.py <Entidade> <arquivo.ifc> <id>    -> outros valores

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.guid                               # conversão do GlobalId
import ifcopenshell.ifcopenshell_wrapper as wrapper    # acesso às definições do esquema

ENTIDADE_PADRAO = "IfcPipeSegment"
ARQUIVO_PADRAO = (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Infra-Plumbing.ifc"
)
ID_PADRAO = 35  # a instância #35 é um IfcPipeSegment

entidade = sys.argv[1] if len(sys.argv) > 1 else ENTIDADE_PADRAO
arquivo = sys.argv[2] if len(sys.argv) > 2 else ARQUIVO_PADRAO
numero = int(sys.argv[3]) if len(sys.argv) > 3 else ID_PADRAO

# ---------------------------------------------------------------- PARTE A
print(f"=== PARTE A: {entidade} nas três versões do esquema ===")
for esquema in ["IFC2X3", "IFC4", "IFC4X3_ADD2"]:
    definicoes = wrapper.schema_by_name(esquema)
    try:
        decl = definicoes.declaration_by_name(entidade)
    except Exception:
        print(f"\n{esquema}: a entidade não existe nesta versão.")
        continue
    cadeia = []                     # sobe na herança até a entidade sem supertipo
```

```
    d = decl
    while d is not None:
        cadeia.append(d.name())
        d = d.supertype()
    situacao = "abstrata" if decl.is_abstract() else "concreta"
    print(f"\n{esquema} ({situacao}): " + " > ".join(cadeia))
    for pos, atr in enumerate(decl.all_attributes(), start=1):
        obrig = "opcional" if atr.optional() else "OBRIGATÓRIO"
        print(f"   {pos}. {atr.name():<22} {obrig}")

# ------------------------------------------------------------------ PARTE B
print(f"\n=== PARTE B: instância #{numero} ===")
modelo = ifcopenshell.open(arquivo)
inst = modelo.by_id(numero)
print("Esquema do arquivo:", modelo.schema_identifier)
print("Instância.........:", inst.is_a(), "|", inst.Name)
print("GlobalId..........:", inst.GlobalId, "| expandido:", ifcopenshell.guid.expand(inst.GlobalId))

# Tipo: no IFC4 e no IFC4.3 o inverso é IsTypedBy;
# no IFC2X3 o tipo aparece em IsDefinedBy, junto com as propriedades.
if modelo.schema_identifier == "IFC2X3":
    rels_tipo = [r for r in inst.IsDefinedBy if r.is_a("IfcRelDefinesByType")]
else:
    rels_tipo = list(inst.IsTypedBy)
if not rels_tipo:
    print("Tipo..............: nenhum tipo associado")
for rel in rels_tipo:
    tipo = rel.RelatingType
    print("Tipo..............:", tipo.is_a(), "|", tipo.Name,
          "| PredefinedType do tipo:", getattr(tipo, "PredefinedType", None))
    print("Ocorrências do tipo:", len(rel.RelatedObjects), f"(relação #{rel.id()})")

# Contêiner espacial (no máximo um por elemento)
for rel in getattr(inst, "ContainedInStructure", None) or []:
    est = rel.RelatingStructure
    print("Contido em........:", est.is_a(), "|", est.Name, f"(relação #{rel.id()})")

# Agregação: de qual objeto esta instância é parte
for rel in inst.Decomposes:
    if rel.is_a("IfcRelAggregates"):
        todo = rel.RelatingObject
        print("Parte de..........:", todo.is_a(), "|", todo.Name, f"(relação #{rel.id()})")
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap3
python esquema.py
```

Saída obtida no teste:

```
=== PARTE A: IfcPipeSegment nas três versões do esquema ===

IFC2X3: a entidade não existe nesta versão.

IFC4 (concreta): IfcPipeSegment > IfcFlowSegment > IfcDistributionFlowElement > IfcDistributionElement >
IfcElement > IfcProduct > IfcObject > IfcObjectDefinition > IfcRoot
   1. GlobalId              OBRIGATÓRIO
   2. OwnerHistory          opcional
   3. Name                  opcional
   4. Description           opcional
   5. ObjectType            opcional
   6. ObjectPlacement       opcional
   7. Representation        opcional
   8. Tag                   opcional
   9. PredefinedType        opcional

IFC4X3_ADD2 (concreta): IfcPipeSegment > IfcFlowSegment > IfcDistributionFlowElement > IfcDistributionElement >
IfcElement > IfcProduct > IfcObject > IfcObjectDefinition > IfcRoot
   1. GlobalId              OBRIGATÓRIO
```

```
   2. OwnerHistory           opcional
   3. Name                   opcional
   4. Description            opcional
   5. ObjectType             opcional
   6. ObjectPlacement        opcional
   7. Representation         opcional
   8. Tag                    opcional
   9. PredefinedType         opcional

=== PARTE B: instância #35 ===
Esquema do arquivo: IFC4X3_ADD2
Instância.........: IfcPipeSegment | sewer pipe
GlobalId..........: 1F4KLUGJf1aAa0YXxTvOR5 | expandido: 4f11455e413a4190a9008a1edde586c5
Tipo..............: IfcPipeSegmentType | sewer pipe | PredefinedType do tipo: CULVERT
Ocorrências do tipo: 24 (relação #34)
Contido em........: IfcSite | road parking - site (relação #38)
```

Outra área: a parede `#158` de `Building-Architecture.ifc` (IFC 4.3). Comando e trecho da saída obtida:

```
python esquema.py IfcWall "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Building-Architecture.ifc" 158
```

```
=== PARTE B: instância #158 ===
Esquema do arquivo: IFC4X3_ADD2
Instância.........: IfcWall | house - outer wall - house right front
GlobalId..........: 1AQAupaRP1txwK1AGiN61V | expandido: 4a68ae3391b641dfbe9404a42c5c605f
Tipo..............: IfcWallType | house - outer wall - house right front | PredefinedType do tipo: SOLIDWALL
Ocorrências do tipo: 1 (relação #157)
Contido em........: IfcBuildingStorey | 00 groundfloor (relação #67)
```

Mesmo modelo de drenagem, em IFC2x3 (instância `#25`), trecho da saída obtida:

```
=== PARTE B: instância #25 ===
Esquema do arquivo: IFC2X3
Instância.........: IfcFlowSegment | sewer pipe
GlobalId..........: 1F4KLUGJf1aAa0YXxTvOR5 | expandido: 4f11455e413a4190a9008a1edde586c5
Tipo..............: IfcPipeSegmentType | sewer pipe | PredefinedType do tipo: USERDEFINED
Ocorrências do tipo: 24 (relação #24)
Contido em........: IfcSite | environment - site (relação #28)
```

3.12 Resumo do capítulo

- O esquema define as entidades (classes), seus atributos, os tipos de dados e as regras. Cada linha do arquivo é uma instância de uma entidade.
- O esquema IFC é escrito em EXPRESS (ISO 10303-11). `ABSTRACT`, `SUBTYPE OF`, `OPTIONAL`, `DERIVE`, `INVERSE`, `WHERE` e `UNIQUE` são as palavras-chave essenciais.
- A herança organiza o esquema do geral para o particular. Os atributos aparecem na ordem da herança; consultas pelo supertipo alcançam os subtipos.
- `IfcRoot` dá a todos os objetos, propriedades e relações um GlobalId (22 caracteres, 128 bits), que deve ser estável entre exportações.
- `OwnerHistory` é obrigatório no IFC2x3 e opcional no IFC4 e no IFC 4.3.
- O tipo (`IfcTypeObject`) guarda o que é comum; a ocorrência (`IfcObject`), o que é individual; `IfcRelDefinesByType` liga os dois.
- Havendo tipo, o `PredefinedType` fica no tipo. `USERDEFINED` exige o texto em `ObjectType` (ocorrência) ou `ElementType` (tipo).

- As relações são objetos próprios. Os atributos inversos são deduzidos na leitura e não aparecem no arquivo.

3.13 Vocabulário técnico

INGLÊS	PORTUGUÊS
schema	esquema
entity	entidade
instance	instância
attribute	atributo
supertype / subtype	supertipo / subtipo
abstract / concrete	abstrata / concreta
inheritance	herança
defined type	tipo definido
enumeration	enumeração
select type	tipo de seleção
occurrence	ocorrência
type object	objeto de tipo
predefined type	tipo predefinido
user defined	definido pelo usuário
inverse attribute	atributo inverso
objectified relationship	relação objetificada
where rule / formal proposition	regra de validação / proposição formal
globally unique identifier	identificador global único
timestamp	carimbo de data e hora

Tabela 3.18: Vocabulário do capítulo.

3.14 Exercícios

Fixação

1. O que significa `ABSTRACT` numa definição EXPRESS? Cite três entidades abstratas do IFC.
2. Por que as quatro primeiras posições de um `IfcWall`, de um `IfcPropertySet` e de um `IfcRelAggregates` são iguais?
3. Qual atributo existe em `IfcWall` no IFC4 e não existe no IFC2x3? Qual atributo de `IfcRoot` deixou de ser obrigatório a partir do IFC4?
4. Em que situação o `PredefinedType` de uma ocorrência deve ficar vazio?

5. Por que o atributo `IsTypedBy` não aparece escrito em nenhuma linha do arquivo IFC?

Aplicação

6. Execute `esquema.py` para a entidade `IfcBeam`, com o arquivo `Building-Structural.ifc` (IFC 4.3) e uma instância de viga do arquivo. Para encontrar o número, busque `IFCBEAM(` no VS Code. Registre a cadeia de herança, o tipo associado e o `PredefinedType` do tipo. A linha "Contido em" não vai aparecer. Explique por quê, a partir da linha "Parte de", e descubra onde está contido o objeto do qual a viga faz parte.
7. Execute o mesmo script para `IfcCourse`, com `Infra-Road.ifc` (IFC 4.3) e a instância `#57`. Em que versões a entidade existe? Quantas camadas compartilham o tipo? Aplique a Tabela 3.15 e diga se o arquivo segue o padrão correto.

Desafio

8. Numa rede de tubulação industrial, várias peças de uma mesma linha recebem valores iguais (classe de tubulação, fluido, pressão e temperatura de projeto), enquanto outras informações variam por peça (TAG, comprimento, posição). Proponha uma organização em IFC 4.3: quais informações vão para o tipo, quais para a ocorrência, e qual relação liga os dois. Discuta o limite dessa solução: a pressão de projeto é uma característica do tubo (do catálogo) ou da linha em que ele está?

3.15 Respostas dos exercícios

1. `ABSTRACT` indica que a entidade não pode ter instâncias próprias; só os subtipos concretos aparecem no arquivo. Exemplos: `IfcRoot`, `IfcObjectDefinition`, `IfcObject`, `IfcProduct`, `IfcElement`.
2. Porque as três derivam de `IfcRoot` e, pela herança, os atributos começam pelos de `IfcRoot`: GlobalId, OwnerHistory, Name e Description.
3. `PredefinedType`, que não existe nas ocorrências de IFC2x3. O `OwnerHistory` de `IfcRoot`, obrigatório no IFC2x3, é opcional a partir do IFC4.
4. Quando a ocorrência está ligada a um tipo: a especialização fica no `PredefinedType` do tipo.
5. Porque é um atributo inverso: é deduzido, na leitura, a partir da relação `IfcRelDefinesByType` que cita a ocorrência em `RelatedObjects`. No arquivo, a informação só existe do lado da relação.
6. Com a instância `#175`, a saída obtida foi: cadeia `IfcBeam > IfcBuiltElement > IfcElement > IfcProduct > IfcObject > IfcObjectDefinition > IfcRoot` (concreta, 9 atributos, com `PredefinedType`); tipo `IfcBeamType` "house - girder", `PredefinedType` `GIRDER_SEGMENT`, compartilhado por 6 vigas (relação `#174`); "Parte de: IfcRoof | house - roof (relação #183)". A viga não tem contêiner espacial próprio porque faz parte de um conjunto, a cobertura `#166`, por agregação. Quem está contido na estrutura espacial é o todo: a cobertura `#166` está contida no terreno "house - site" (`#28`). Um elemento que é parte de outro não deve ter contêiner espacial próprio; herda a localização do todo. O assunto é aprofundado no Capítulo 4.
7. `IfcCourse` só existe no IFC 4.3 (no IFC2X3 e no IFC4, o script informa que a entidade não existe). Duas camadas (`#57` e `#270`) compartilham o tipo `#55`, pela relação `#56`. A ocorrência tem `PredefinedType` vazio e o tipo tem `.CORE.`: o padrão está correto. A camada está contida em `IfcRoadPart` "road - parking" (relação `#60`).

8. Uma organização possível: criar um tipo de tubo (`IfcPipeSegmentType`) por especificação de catálogo (diâmetro, material, classe de pressão do componente, norma), com essas propriedades em `HasPropertySets`; nas ocorrências (`IfcPipeSegment`), gravar TAG, comprimento e informações de montagem; ligar os dois por `IfcRelDefinesByType`. Isso reduz a repetição (a propriedade é escrita uma vez para todos os tubos iguais) e facilita a validação (basta verificar o tipo). O limite: fluido, pressão e temperatura de projeto não são características do tubo, e sim da **linha** (do sistema): o mesmo tubo de catálogo pode estar em linhas com fluidos diferentes. Essas informações pertencem a um agrupamento das peças da linha, como um sistema de distribuição (`IfcDistributionSystem`), assunto dos Capítulos 4 e 6. A boa modelagem da informação distingue três níveis: o catálogo (tipo), a peça (ocorrência) e o sistema (grupo).

3.16 Referências

- ISO 10303-11:2004 (linguagem EXPRESS): https://www.iso.org/standard/38047.html
- Documentação do IFC 4.3, `IfcRoot`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcRoot.htm
- Documentação do IFC 4.3, `IfcPipeSegment` (regras formais): https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcPipeSegment.htm
- Documentação do IFC 4.3, `IfcObject` (ObjectType, IsTypedBy, IsDefinedBy): https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcObject.htm
- Orientação sobre o GlobalId: https://technical.buildingsmart.org/resources/ifcimplementationguidance/ifc-guid/
- Documentação do IfcOpenShell: https://docs.ifcopenshell.org/

4 Estrutura espacial e relacionamentos

Um modelo com milhares de elementos só é útil se estiver organizado. Saber que existe uma válvula não basta; é preciso saber em que unidade, em que área, em que pavimento ou em que trecho da rodovia ela está, a que conjunto pertence e a que sistema serve. Este capítulo mostra como o IFC organiza os objetos: a **estrutura espacial**, que responde "onde está", e as **relações**, que respondem "do que faz parte", "com o que se liga" e "a que grupo pertence".

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar a diferença entre elemento espacial e elemento físico;
- montar e ler a estrutura espacial de edificações e de obras de infraestrutura (rodovia, ponte, ferrovia);
- usar corretamente agregação, contenção espacial, referência espacial e aninhamento;
- distinguir grupos, sistemas e zonas e saber quando usar cada um;
- reconhecer as diferenças entre IFC2x3, IFC4 e IFC 4.3 na estrutura espacial;
- inspecionar a estrutura de um arquivo com um script e detectar elementos sem localização.

4.1 Introdução: organizar um modelo

4.1.1 A analogia do endereço

Um endereço postal é uma organização hierárquica: país, estado, cidade, bairro, rua, número, apartamento. Cada nível contém o seguinte. Com o endereço, qualquer pessoa encontra uma casa entre milhões.

O IFC usa a mesma ideia. Os objetos físicos (paredes, tubos, vigas) são "endereçados" numa hierarquia de lugares: o projeto, o terreno, a edificação (ou a rodovia, ou a ponte), o pavimento (ou o trecho, ou a parte da ponte), o ambiente. Esses lugares não são objetos físicos; são **elementos espaciais**, criados para organizar o modelo.

4.1.2 Elementos espaciais e elementos físicos

	ELEMENTO ESPACIAL	ELEMENTO FÍSICO
O que é	Um lugar, uma divisão do projeto	Uma coisa construída ou instalada
Supertipo	`IfcSpatialElement` (IFC4 e IFC4.3); `IfcSpatialStructureElement` (IFC2x3)	`IfcElement`
Exemplos	Terreno, edificação, pavimento, ambiente, rodovia, trecho de rodovia, ponte, parte de ponte	Parede, laje, tubo, válvula, bomba, viga, camada de pavimento, trilho
Como se organizam entre si	Por **agregação**: o terreno agrega a edificação, que agrega os pavimentos	Por **contenção**: o pavimento contém as paredes

Tabela 4.1: Elemento espacial e elemento físico.

A distinção é importante porque as relações são diferentes. Um pavimento não "contém" outro pavimento; uma edificação é **composta** de pavimentos. Já uma parede não "compõe" um pavimento; ela está **contida** nele. Misturar as duas coisas é um dos erros mais comuns em arquivos exportados.

4.1.3 Relações: as ligações entre objetos

Além do "onde", um modelo precisa registrar outras ligações:

- **composição**: a escada é composta de lances e patamares; a cobertura, de vigas e telhas; o skid de bombeamento, de bombas, tubos e válvulas;
- **tipo**: estes 24 tubos seguem a mesma especificação (Capítulo 3);
- **agrupamento**: estes tubos e caixas formam o sistema de esgoto; estes ambientes formam a zona térmica;
- **conexão**: este tubo se conecta a esta válvula (Capítulo 6);
- **associação**: esta viga é de aço ASTM A572; esta laje tem uma classificação da ABNT NBR 15965 (Capítulo 5).

Como visto no Capítulo 3, cada uma dessas ligações é uma instância própria no arquivo, um subtipo de `IfcRelationship`.

4.2 A estrutura espacial

4.2.1 Os níveis

A estrutura espacial é uma árvore que começa no projeto e desce até os ambientes. No IFC 4.3, os níveis são:

NÍVEL	ENTIDADES	EXEMPLOS
Projeto	`IfcProject`	O empreendimento ou contrato
Terreno	`IfcSite`	A área do empreendimento, um lote, a faixa de domínio
Instalação	`IfcFacility` e subtipos: `IfcBuilding`, `IfcRoad`, `IfcBridge`, `IfcRailway`, `IfcMarineFacility`	Uma edificação, uma rodovia, uma ponte, uma ferrovia, um porto
Parte da instalação	`IfcBuildingStorey`; `IfcFacilityPart` e subtipos: `IfcRoadPart`, `IfcBridgePart`, `IfcRailwayPart`, `IfcMarinePart`, `IfcFacilityPartCommon`	Um pavimento; um trecho de rodovia, uma pista, um acostamento; o tabuleiro, um pilar; uma via permanente
Espaço	`IfcSpace`	Uma sala, um banheiro, uma vaga de estacionamento

Tabela 4.2: Níveis da estrutura espacial (IFC 4.3).

Segundo a documentação do IFC 4.3 (conceito *Spatial Decomposition*), os níveis de terreno, parte da instalação e espaço são opcionais, e um elemento espacial só deve ser parte de outro de nível igual ou superior. Diretamente abaixo de `IfcProject`, a documentação admite `IfcSite`, as instalações (`IfcFacility`, `IfcBuilding`, `IfcRoad`, `IfcBridge`, `IfcRailway`, `IfcMarineFacility`), `IfcSpace` e `IfcExternalSpatialElement`.

No IFC2x3 e no IFC4, só existem os níveis de edificação: `IfcProject`, `IfcSite`, `IfcBuilding`, `IfcBuildingStorey` e `IfcSpace`. Rodovias, pontes e ferrovias não têm entidades próprias nessas versões (seção 4.9).



Figura 4.1: Três estruturas espaciais típicas.

4.2.2 IfcProject: o contexto

`IfcProject` é a raiz de tudo. Além de ser o topo da árvore, ele define o **contexto** do arquivo:

- as **unidades padrão** (`UnitsInContext`): metro ou milímetro, graus ou radianos (Capítulo 5);
- os **contextos de representação geométrica** (`RepresentationContexts`): o sistema de coordenadas do projeto, a precisão, a direção do norte (Capítulo 7).

Regras da documentação do IFC 4.3:

- deve haver **no máximo um** `IfcProject` no arquivo (regra global `IfcSingleProjectInstance`);
- o atributo `Name` é obrigatório (regra `HasName`);
- o projeto é a raiz da decomposição e não pode ser parte de outro objeto (regra `NoDecomposition`).

VERSÃO	SUPERTIPO DE `IFCPROJECT`	`REPRESENTATIONCONTEXTS` E `UNITSINCONTEXT`
IFC2x3	`IfcObject`	Obrigatórios
IFC4 e IFC4.3	`IfcContext`	Opcionais no esquema

Tabela 4.3: `IfcProject` por versão.

4.2.3 IfcSite: o terreno

`IfcSite` representa a área de terreno onde a obra se situa. Atributos próprios:

ATRIBUTO	CONTEÚDO
`RefLatitude`, `RefLongitude`	Latitude e longitude de referência, em graus, minutos, segundos e milionésimos de segundo (tipo `IfcCompoundPlaneAngleMeasure`, lista de 3 ou 4 inteiros)
`RefElevation`	Cota de referência em relação ao nível do mar
`LandTitleNumber`	Número de registro do imóvel (matrícula)
`SiteAddress`	Endereço postal

Tabela 4.4: Atributos de `IfcSite`.

A latitude 18°55'07,5" sul, por exemplo, é gravada como `(-18,-55,-7,-500000)`: todos os componentes levam o sinal negativo. `RefLatitude` e `RefLongitude` dão apenas uma localização aproximada; o georreferenciamento completo usa outras entidades (Capítulo 7).

Um projeto pode ter vários terrenos, agregados a um terreno principal. Nas amostras da buildingSMART, o terreno "environment - site" (`COMPLEX`) agrega vários terrenos parciais (`PARTIAL`): o da casa, o do estacionamento, o de cada ponte.

4.2.4 As instalações

No IFC 4.3, as instalações (*facilities*) são subtipos de `IfcFacility`:

ENTIDADE	INSTALAÇÃO	`PREDEFINEDTYPE` (EXEMPLOS DE VALORES)
`IfcBuilding`	Edificação	Não tem; atributos `ElevationOfRefHeight`, `ElevationOfTerrain`, `BuildingAddress`
`IfcRoad`	Rodovia, via urbana	Só `USERDEFINED` e `NOTDEFINED`
`IfcBridge`	Ponte, viaduto	`ARCHED`, `CABLE_STAYED`, `CANTILEVER`, `CULVERT`, `FRAMEWORK`, `GIRDER`, `SUSPENSION`, `TRUSS`
`IfcRailway`	Ferrovia	Só `USERDEFINED` e `NOTDEFINED`
`IfcMarineFacility`	Obra portuária e hidroviária	`BREAKWATER`, `CANAL`, `JETTY`, `PORT`, `QUAY`, `SHIPLOCK`, entre outros

Tabela 4.5: Instalações do IFC 4.3.

Plantas industriais, estações de tratamento e subestações não têm entidade específica. As opções usuais são: `IfcSite` para a área da planta; `IfcBuilding` para as edificações (casa de controle, subestação abrigada); e, para unidades de processo ao ar livre, a própria `IfcFacility`, que no esquema IFC 4.3 não é abstrata (pode ser instanciada), com o `ObjectType` descrevendo a instalação, ou zonas espaciais (seção 4.2.6). Não há um padrão único: a escolha deve ser registrada no BEP e verificada com quem vai receber o arquivo.

4.2.5 As partes das instalações

Pavimentos. `IfcBuildingStorey` representa um pavimento. O atributo `Elevation` guarda a cota do pavimento.

Partes de instalações de infraestrutura. No IFC 4.3, `IfcFacilityPart` tem o atributo obrigatório `UsageType`, que diz como a parte subdivide a instalação. Definições da documentação:

`USAGETYPE`	DEFINIÇÃO (TRADUÇÃO)	EXEMPLO
`LONGITUDINAL`	Subdivisão ao longo de uma instalação linear, com início e fim definidos ao longo do alinhamento	Trecho de rodovia do km 10 ao km 15
`LATERAL`	Subdivisão lateral (esquerda, direita, centro) de um segmento longitudinal ou da instalação	Pista, acostamento, canteiro central
`VERTICAL`	Subdivisão no eixo Z, em geral um nível de elevação ou volume controlado	Superestrutura e infraestrutura de uma ponte

USAGETYPE	DEFINIÇÃO (TRADUÇÃO)	EXEMPLO
REGION	Subdivisão delimitada no plano XY	Uma praça de pedágio, um pátio

Tabela 4.6: Valores de IfcFacilityUsageEnum.

Cada subtipo tem sua enumeração de PredefinedType. Exemplos:

ENTIDADE	VALORES (SELEÇÃO)
IfcRoadPart	ROADSEGMENT (trecho), CARRIAGEWAY (pista), TRAFFICLANE (faixa), SHOULDER, HARDSHOULDER, SOFTSHOULDER (acostamentos), CENTRALRESERVE (canteiro central), SIDEWALK (calçada), INTERSECTION, ROUNDABOUT, TOLLPLAZA, BUS_STOP, PARKINGBAY
IfcBridgePart	SUPERSTRUCTURE, SUBSTRUCTURE, DECK, DECK_SEGMENT, PIER, PIER_SEGMENT, ABUTMENT (encontro), FOUNDATION, PYLON, SURFACESTRUCTURE
IfcRailwayPart	TRACK, TRACKPART, PLAINTRACK, TURNOUTTRACK, DILATIONTRACK, SUBSTRUCTURE, ABOVETRACK, LINESIDE
IfcFacilityPartCommon	SEGMENT, ABOVEGROUND, BELOWGROUND, JUNCTION, LEVELCROSSING, SUBSTRUCTURE, SUPERSTRUCTURE, TERMINAL

Tabela 4.7: Tipos de partes de instalação (IFC 4.3).

4.2.6 Espaços, zonas espaciais e o exterior

ENTIDADE	VERSÕES	USO
IfcSpace	Todas	Ambiente delimitado: sala, banheiro, vaga. No IFC4 e no IFC4.3 tem PredefinedType (SPACE, PARKING, GFA, INTERNAL, EXTERNAL…); no IFC2x3 tem InteriorOrExteriorSpace (obrigatório)
IfcSpatialZone	IFC4, IFC4.3	Zona espacial com forma própria, **não hierárquica e possivelmente sobreposta**: zona de construção, de segurança contra incêndio, de iluminação, térmica, de interferência. Os elementos são ligados a ela por referência, não por contenção
IfcExternalSpatialElement	IFC4, IFC4.3	O exterior da edificação, usado em análises energéticas (fronteiras com o ar externo, o solo, a água)

Tabela 4.8: Espaços e zonas.

4.2.7 CompositionType

Os elementos espaciais têm o atributo CompositionType (tipo IfcElementCompositionEnum), que indica se o objeto é um conjunto, um objeto único ou uma parte:

VALOR	DEFINIÇÃO (TRADUÇÃO)	EXEMPLO
COMPLEX	Um grupo ou agregação de elementos semelhantes	Um complexo de terrenos; um conjunto de edificações
ELEMENT	O elemento em si, não dividido	Uma edificação; um pavimento
PARTIAL	Uma subparte	Um terreno parcial; uma ala da edificação; um mezanino

Tabela 4.9: Valores de `CompositionType`.

O atributo é obrigatório no IFC2x3 e opcional a partir do IFC4. O valor mais comum é `ELEMENT`.

4.3 Agregação: IfcRelAggregates

4.3.1 O conceito

A **agregação** liga um todo às suas partes. É a relação "é composto de" / "faz parte de". Atributos de `IfcRelAggregates` (a partir da posição 5; as quatro primeiras são as de `IfcRoot`):

POS.	ATRIBUTO	CONTEÚDO
5	`RelatingObject`	O todo (um objeto)
6	`RelatedObjects`	As partes (conjunto de um ou mais objetos)

Tabela 4.10: Atributos de `IfcRelAggregates`.

No IFC2x3, os mesmos atributos são herdados de `IfcRelDecomposes`, na mesma ordem.

A agregação tem dois usos principais:

1. **Montar a estrutura espacial**: projeto agrega terreno, terreno agrega edificação, edificação agrega pavimentos, pavimento agrega ambientes.
2. **Compor elementos físicos**: uma escada (`IfcStair`) agrega lances (`IfcStairFlight`) e patamares; uma cobertura (`IfcRoof`) agrega vigas e placas; uma fachada-cortina (`IfcCurtainWall`) agrega montantes e painéis; um conjunto montado (`IfcElementAssembly`) agrega as peças de uma treliça, de um pórtico ou de um skid de equipamentos.

`IfcElementAssembly` merece destaque. É a entidade genérica para conjuntos montados, com `PredefinedType` como `TRUSS` (treliça), `RIGID_FRAME` (pórtico), `BRACED_FRAME`, `BEAM_GRID`, `GIRDER`, `PIER`, `DECK`, `SUPPORTINGASSEMBLY` (conjunto de suporte), `MAST`, `SIGNALASSEMBLY`, entre outros (IFC 4.3). Um skid de bombeamento, uma estrutura de suporte de tubulação ou um conjunto de sinalização são bons candidatos.

Exemplo resolvido 4.1: ler a estrutura espacial no texto do arquivo

Problema. Sem usar programa algum, descubra a estrutura espacial do arquivo `Building-Architecture.ifc` (IFC 4.3).

Solução. Busque `IFCRELAGGREGATES` no editor. As linhas encontradas, com os objetos citados:

```
#13=IFCPROJECT('2Ndyd$OSX7s9A04nc4lyye',#1,'ifc silly sample scene - project',...,(#11),#14);
#21=IFCSITE('23sFQGRy90RxVbRHD9iSE2',#1,'environment - site',...,.COMPLEX.,$,$,0.,$,$);
#22=IFCRELAGGREGATES('2ka3C6taL34OtYXU3vPJfQ',#1,'ifc silly sample scene - project container',$,#13,(#21));
#28=IFCSITE('1Pbuu0tu59NfhrTsztVBK1',#1,'house - site',...,.PARTIAL.,$,$,0.,$,$);
#29=IFCRELAGGREGATES('3_hAyBOOnCLv6QjTSqJvbS',#1,'environment - site container',$,#21,(#28));
#35=IFCBUILDING('0c$N1CTon2BB2Sp89385G8',#1,'Single-family house',...,.ELEMENT.,0.,0.,$);
#39=IFCRELAGGREGATES('1zbPFleBz8UfM9JDNDyRRr',#1,'house - site container',$,#28,(#35));
#45=IFCBUILDINGSTOREY('1Ano2ZUxnEIvVQ_beukl8b',#1,'00 groundfloor',...,.ELEMENT.,$);
#46=IFCRELAGGREGATES('3oup8TZ0X8NO3m6hJOemlz',#1,'Single-family house container',$,#35,(#45));
#86=IFCRELAGGREGATES('1GeoUeeV5DGerMWqWmKR_r',#1,'00 groundfloor container',$,#45,(#85,#135));
```

Figura 4.2: Relações de agregação da amostra (trechos abreviados com `...`).

Lendo cada relação pelo quinto atributo (o todo) e pelo sexto (as partes): `#22` diz que o projeto `#13` agrega o terreno `#21`; `#29`, que o terreno `#21` agrega o terreno `#28`; `#39`, que `#28` agrega a edificação `#35`; `#46`, que a edificação agrega o pavimento `#45`; `#86`, que o pavimento agrega os ambientes `#85` e `#135`. A árvore é:

```
#13 IfcProject
└─ #21 IfcSite 'environment - site' (COMPLEX)
   └─ #28 IfcSite 'house - site' (PARTIAL)
      └─ #35 IfcBuilding 'Single-family house' (ELEMENT)
         └─ #45 IfcBuildingStorey '00 groundfloor' (ELEMENT)
            ├─ #85 IfcSpace 'living room'
            └─ #135 IfcSpace 'entry hall'
```

Note que o exportador deu nomes às relações ("... container"). Isso é permitido (o `Name` vem de `IfcRoot`), mas não tem efeito: o significado está nos atributos 5 e 6.

4.4 Contenção e referência espacial

4.4.1 IfcRelContainedInSpatialStructure

A **contenção espacial** liga elementos físicos (e outros produtos, como anotações) ao elemento espacial onde estão. Atributos:

POS.	ATRIBUTO	CONTEÚDO
5	`RelatedElements`	Os elementos contidos (conjunto)
6	`RelatingStructure`	O elemento espacial que os contém

Tabela 4.11: Atributos de `IfcRelContainedInSpatialStructure`.

Atenção à ordem: aqui as partes vêm **antes** do todo, ao contrário de `IfcRelAggregates`. No exemplo da Figura 4.2, a linha `#67=IFCRELCONTAINEDINSPATIALSTRUCTURE(...,(#53,#158,#182,#201,#220,#228),#45);` diz que seis elementos (a laje, quatro paredes e a chaminé) estão contidos no pavimento `#45`.

Regras da documentação do IFC 4.3:

- **um elemento só pode estar contido em um único elemento espacial**. A contenção é hierárquica: a parede está no pavimento 00, que está na edificação; não é preciso (nem permitido) conter a parede também na edificação;
- a relação **não** deve ser usada para colocar elementos espaciais dentro de outros (regra `WR31`); para isso existe a agregação;
- um elemento que é **parte de um conjunto agregado não deve ser contido** na estrutura espacial. Na documentação do conceito *Spatial Containment*, para esse caso: "it should not be additionally contained in the project spatial hierarchy" (não deve ser adicionalmente contido na hierarquia espacial do projeto). O todo é contido; as partes herdam a localização do todo.

4.4.2 IfcRelReferencedInSpatialStructure

Alguns elementos pertencem a mais de um lugar: um pilar que atravessa três pavimentos, um poço de elevador, uma prumada de tubulação, uma escada entre dois níveis. Como a contenção é única, o IFC oferece a **referência espacial**: o elemento é contido no lugar principal (em geral, onde começa) e **referenciado** nos demais.

`IfcRelReferencedInSpatialStructure` tem os mesmos atributos, na mesma ordem (`RelatedElements`, `RelatingStructure`). Não é hierárquica e não tem limite de quantidade. No IFC 4.3, também é a relação usada para ligar alinhamentos e elementos a zonas espaciais (`IfcSpatialZone`) e a partes de instalações lineares.

SITUAÇÃO	CONTENÇÃO	REFERÊNCIA
Parede do pavimento 01	Pavimento 01	Nenhuma
Pilar que vai do pavimento 00 ao 02	Pavimento 00	Pavimentos 01 e 02
Prumada de esgoto de 10 pavimentos	Pavimento inferior	Demais pavimentos
Luminária numa zona de iluminação	Pavimento	`IfcSpatialZone` da zona de iluminação
Alinhamento de uma rodovia (IFC 4.3)	Nunca (regra da documentação)	`IfcRoad` ou suas partes

Tabela 4.12: Contenção e referência.

4.5 Aninhamento: IfcRelNests

O **aninhamento** é outra forma de composição, usada quando as partes têm **ordem** ou são componentes "encaixados" no todo. Atributos: `RelatingObject` (o todo) e `RelatedObjects` (as partes), com a diferença de que, no IFC4 e no IFC 4.3, `RelatedObjects` é uma **lista ordenada** (`LIST`), e não um conjunto (`SET`). A ordem das partes tem significado.

Usos principais:

- **portas de conexão** (`IfcDistributionPort`) de tubos, válvulas e equipamentos, no IFC4 e no IFC 4.3 (Capítulo 6);
- **alinhamentos** (IFC 4.3): o alinhamento aninha os seus leiautes, horizontal (`IfcAlignmentHorizontal`), vertical (`IfcAlignmentVertical`) e de superelevação (`IfcAlignmentCant`), e cada leiaute aninha os seus segmentos, em ordem;
- **referentes** (IFC 4.3): estacas e marcos quilométricos (`IfcReferent`) aninhados ao alinhamento;
- **tarefas** de cronograma, em ordem.

No IFC2x3, `IfcRelNests` também existe, mas as partes formam um conjunto sem ordem, e os inversos são os mesmos da agregação (`IsDecomposedBy`, `Decomposes`), herdados de `IfcRelDecomposes`.

4.6 O alinhamento na estrutura do projeto (IFC 4.3)

O alinhamento (`IfcAlignment`) é o eixo de referência de obras lineares: rodovias, ferrovias, adutoras, linhas de transmissão. Ele é um elemento de posicionamento (`IfcPositioningElement`), não um elemento espacial nem físico. Regras da documentação do IFC 4.3:

- "Every *IfcAlignment* must be related to *IfcProject* using the *IfcRelAggregates* relationship - either directly or indirectly" (todo alinhamento deve estar ligado ao `IfcProject` por agregação, direta ou indiretamente);
- alinhamentos podem ser ligados a elementos espaciais por `IfcRelReferencedInSpatialStructure`, mas "must not be related to spatial elements using *IfcRelContainedInSpatialStructure*" (não devem ser contidos);
- o alinhamento aninha (`IfcRelNests`) seus leiautes e seus referentes.

Estrutura da amostra `linear-placement-of-signal.ifc` (modelos de especificação da buildingSMART), obtida com o script da seção 4.10:

```
#7 IfcProject 'Stationing'
    #2268 IfcSite 'Default Site Name' (ELEMENT)
        #2273 IfcRailway 'Default Railway Name' [NOTDEFINED] (ELEMENT)
            contém (#3042): IfcSignal x2
    #2278 IfcAlignment 'Track alignment' [USERDEFINED]
        aninha (#2448): IfcAlignmentCant x1, IfcAlignmentHorizontal x1, IfcAlignmentVertical x1
        aninha (#3047): IfcReferent x23
```

Figura 4.3: Alinhamento agregado ao projeto, com leiautes e referentes aninhados.

O alinhamento está agregado ao projeto, ao lado do terreno. Ele aninha três leiautes (horizontal, vertical e superelevação, usual em ferrovias) e 23 referentes (estacas). Os dois sinais ferroviários estão contidos na ferrovia. O Capítulo 7 mostra como os sinais são posicionados ao longo do alinhamento.

4.7 Tipos e declarações

A relação com o tipo (`IfcRelDefinesByType`) foi vista no Capítulo 3. No IFC4 e no IFC 4.3, existe ainda `IfcRelDeclares`, que liga o projeto (`RelatingContext`) às definições que ele declara (`RelatedDefinitions`): tipos, modelos de conjuntos de propriedades e bibliotecas. É útil para bibliotecas de tipos que não têm nenhuma ocorrência no modelo. Muitos exportadores não a gravam.

4.8 Grupos, sistemas e zonas

4.8.1 O conceito de grupo

Um **grupo** reúne objetos por um critério funcional, sem hierarquia espacial. Um mesmo objeto pode estar em vários grupos. Exemplos: os tubos, válvulas e caixas de um sistema de esgoto; os ambientes de uma zona térmica; os elementos de uma etapa de obra; os ativos sob um contrato de manutenção.

A ligação é feita por `IfcRelAssignsToGroup`:

POS.	ATRIBUTO	CONTEÚDO
5	`RelatedObjects`	Os membros do grupo (conjunto)
6	`RelatedObjectsType`	Restrição do tipo de membro; em geral vazio
7	`RelatingGroup`	O grupo

Tabela 4.13: Atributos de `IfcRelAssignsToGroup`.

4.8.2 Entidades de grupo

ENTIDADE	VERSÕES	USO
`IfcGroup`	Todas	Grupo genérico
`IfcSystem`	Todas	Sistema: grupo de elementos que funcionam juntos
`IfcDistributionSystem`	IFC4, IFC4.3	Sistema de distribuição (redes). `PredefinedType` com valores como `DOMESTICCOLDWATER`, `DOMESTICHOTWATER`, `SEWAGE`, `STORMWATER`, `DRAINAGE`, `FIREPROTECTION`, `GAS`, `FUEL`, `OIL`, `CHEMICAL`, `COMPRESSEDAIR`, `VACUUM`, `CHILLEDWATER`, `HEATING`, `VENTILATION`, `EXHAUST`, `ELECTRICAL`, `LIGHTING`, `DATA`
`IfcBuildingSystem`	IFC4, IFC4.3	Sistema construtivo: fachada, fundação, estrutura portante, vedação externa (`FENESTRATION`, `FOUNDATION`, `LOADBEARING`, `OUTERSHELL`, `SHADING`, `TRANSPORT`)
`IfcBuiltSystem`	IFC4.3	Sistema construído, generalização para obras de infraestrutura
`IfcZone`	Todas	Zona: agrupamento de ambientes (`IfcSpace`), sem forma própria. Subtipo de `IfcGroup` no IFC2x3 e de `IfcSystem` a partir do IFC4
`IfcAsset`	Todas	Ativo, para gestão patrimonial e manutenção
`IfcStructuralAnalysisModel`	Todas	Modelo de análise estrutural

Tabela 4.14: Grupos, sistemas e zonas.

`IfcZone` ou `IfcSpatialZone`? A zona (`IfcZone`) é um grupo de ambientes: não tem forma nem posição, só lista os ambientes que reúne. A zona espacial (`IfcSpatialZone`, IFC4 e IFC4.3) é um elemento espacial com forma própria, que pode cortar ambientes e se sobrepor a outras zonas. Uma zona de ocupação formada por salas inteiras é uma `IfcZone`; uma área de canteiro desenhada sobre o terreno é uma `IfcSpatialZone`.

A relação `IfcRelServicesBuildings` liga um sistema aos elementos espaciais que ele atende (por exemplo, o sistema de água fria atende a edificação A e a edificação B).

Exemplo resolvido 4.2: sistema de esgoto em duas versões

Problema. Como a amostra `Infra-Plumbing.ifc` registra que os tubos e as caixas formam um sistema de esgoto, em IFC 4.3 e em IFC2x3?

Solução. Em IFC 4.3:

```
#31=IFCDISTRIBUTIONSYSTEM('2kFeQrGC1AX8h9FmQxZHo0',#1,'road wastewater system','A comprehensive wastewater
system, ensuring efficient sewage management.',$,$,.SEWAGE.);
```

```
#32=IFCRELASSIGNSTOGROUP('0goaJ2ifn16gBAMHWaeKNM',#1,$,$,
(#35,#58,#69,#80,#91,#102,#113,#124,#147,#185,#252, ... ,#436),$,#31);
```

O sistema `#31` é um `IfcDistributionSystem` com `PredefinedType` = `.SEWAGE.`. A relação `#32` reúne 26 membros: 24 tubos e 2 caixas de inspeção (`IfcDistributionChamberElement`). Em IFC2x3, o mesmo sistema aparece como `IfcSystem` genérico, sem `PredefinedType`, porque `IfcDistributionSystem` não existe nessa versão. O GlobalId do sistema é o mesmo nas duas versões.

Note que o sistema e a estrutura espacial são independentes: os tubos estão contidos em terrenos diferentes (o do estacionamento, o de uma ponte, o da rodovia) e, ao mesmo tempo, pertencem a um único sistema.

4.9 A mesma rodovia em IFC2x3, IFC4 e IFC 4.3

Como o IFC2x3 e o IFC4 não têm entidades de infraestrutura, obras lineares exportadas nessas versões recorrem a adaptações. A amostra `Infra-Road.ifc`, lida com o script da seção 4.10, mostra o problema com clareza:

VERSÃO	COMO A RODOVIA APARECE	COMO OS ELEMENTOS APARECEM
IFC2x3	`IfcBuilding` 'road parking - road' agregado ao terreno	`IfcElementAssembly` e `IfcBuildingElementProxy` genéricos; marcações como `IfcAnnotation`
IFC4	`IfcBuilding` 'road parking - road' dentro de um terreno parcial	Conjuntos genéricos; marcações como `IfcSurfaceFeature`
IFC 4.3	`IfcRoad` 'road parking - road' com `IfcRoadPart` para trecho, estacionamento, acostamentos e pista	`IfcCourse` (camadas de pavimento) e `IfcEarthworksFill` (aterro), contidos nas partes da rodovia

Tabela 4.15: A rodovia da amostra nas três versões.

Trechos da estrutura em IFC 4.3:

```
#24=IFCSITE('1adp27B_9CUfup2ojuKOng',#1,'road parking - site',...,.PARTIAL.,$,$,0.,$,$);
#31=IFCROAD('2MBfH6RyP3luv6spdwrCIJ',#1,'road parking - road',...,'road',#36,$,$,.ELEMENT.,.NOTDEFINED.);
#35=IFCRELAGGREGATES('356nZ76Ff6yxENKPBcVVR1',#1,'road parking - site container',$,#24,(#31));
#41=IFCROADPART('13nXdzaiLCcBMI_327t3$B',#1,'road parking - road segment',...,'roadsegment',#43,$,
$,.COMPLEX.,.LATERAL.,.ROADSEGMENT.);
#42=IFCRELAGGREGATES('2TADpZRN95kPPCGjT0LDn9',#1,'road parking - road container',$,#31,(#41,#257));
#48=IFCROADPART('2X7Dlo9gX5dgM3FX0gYZXP',#1,'road - parking',...,'parkingbay',#50,$,
$,.PARTIAL.,.LATERAL.,.PARKINGBAY.);
#49=IFCRELAGGREGATES('1ivxuPimX2NumKCDeDFkVD',#1,'road parking - road segment container',$,#41,
(#48,#124,#135,#146));
#60=IFCRELCONTAINEDINSPATIALSTRUCTURE('17jnsEMzb05QCKR5Z14RJU',#1,$,$,(#57,#76,#92,#108),#48);
```

Figura 4.4: Rodovia em IFC 4.3 (trechos abreviados).

Nas linhas de `IfcRoadPart`, os três últimos atributos são `CompositionType`, `UsageType` e `PredefinedType`. Em IFC2x3, um programa que procure rodovias não encontra nenhuma: encontra "edificações". Essa é a principal razão para exigir IFC 4.3 em obras de infraestrutura, quando todos os softwares envolvidos o suportam.

4.10 Inspeção com um script

O script abaixo mostra a árvore de decomposição a partir do projeto, os elementos contidos e referenciados em cada nível, os aninhamentos, os grupos e sistemas e, por fim, verifica se há elementos físicos "perdidos": sem contêiner espacial e sem pertencer a outro elemento. Esse é um dos problemas mais comuns em arquivos exportados: o elemento existe, mas não aparece na árvore do visualizador nem nas listagens por pavimento.

Os comandos de consulta são explicados no Capítulo 9. Grave-o como `C:\IFC_Curso\exercicios\cap4\estrutura_espacial.py`.

```
# -*- coding: utf-8 -*-
"""
estrutura_espacial.py | Capítulo 4: estrutura espacial, agrupamentos e verificação de contêiner.

O que faz (somente leitura; nada é gravado):
  1. Mostra a árvore de decomposição a partir do IfcProject (relações IfcRelAggregates),
     com o CompositionType e o PredefinedType de cada nível espacial.
  2. Para cada nível espacial, resume os elementos contidos (IfcRelContainedInSpatialStructure)
     e os referenciados (IfcRelReferencedInSpatialStructure).
  3. Mostra os objetos aninhados (IfcRelNests), como as partes de um alinhamento.
  4. Lista os grupos e sistemas (IfcRelAssignsToGroup) e o que cada um reúne.
  5. Verifica os elementos físicos sem contêiner espacial e que não são parte de outro elemento.

Uso (ambiente virtual ativado):
    python estrutura_espacial.py                 -> arquivo padrão abaixo
    python estrutura_espacial.py <arquivo.ifc>   -> outro arquivo

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; arquivos IFC2X3, IFC4 e IFC4X3_ADD2.
"""
import sys
from collections import Counter
import ifcopenshell

ARQUIVO_PADRAO = (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc"
)
arquivo = sys.argv[1] if len(sys.argv) > 1 else ARQUIVO_PADRAO
modelo = ifcopenshell.open(arquivo)
print("Arquivo:", arquivo)
print("Esquema:", modelo.schema_identifier)


def resumo(objetos):
    """Conta os objetos por entidade: 'IfcWall x4, IfcSlab x1'."""
    contagem = Counter(o.is_a() for o in objetos)
    return ", ".join(f"{classe} x{qtd}" for classe, qtd in sorted(contagem.items()))


def rotulo(obj):
    """Texto de uma linha para um objeto: número, entidade, nome e especializações."""
    texto = f"#{obj.id()} {obj.is_a()} '{obj.Name}'"
    tipo = getattr(obj, "PredefinedType", None)
    if tipo:
        texto += f" [{tipo}]"
    composicao = getattr(obj, "CompositionType", None)
    if composicao:
        texto += f" ({composicao})"
    return texto


def mostrar(obj, nivel=0):
    """Mostra um objeto e desce recursivamente pelas partes agregadas."""
    recuo = "    " * nivel
    print(recuo + rotulo(obj))
    for rel in getattr(obj, "ContainsElements", None) or []:
```

```
        print(recuo + f"    contém (#{rel.id()}): {resumo(rel.RelatedElements)}")
    for rel in getattr(obj, "ReferencesElements", None) or []:
        print(recuo + f"    referencia (#{rel.id()}): {resumo(rel.RelatedElements)}")
    for rel in getattr(obj, "IsNestedBy", None) or []:     # IfcRelNests (IFC4 e IFC4.3)
        print(recuo + f"    aninha (#{rel.id()}): {resumo(rel.RelatedObjects)}")
    for rel in obj.IsDecomposedBy:                          # IfcRelAggregates
        for parte in rel.RelatedObjects:
            mostrar(parte, nivel + 1)


print("\n=== 1 a 3. ÁRVORE DE DECOMPOSIÇÃO ===")
for projeto in modelo.by_type("IfcProject"):
    mostrar(projeto)

print("\n=== 4. GRUPOS E SISTEMAS ===")
grupos = modelo.by_type("IfcGroup")
if not grupos:
    print("Nenhum grupo no arquivo.")
for grupo in grupos:
    membros = [o for rel in grupo.IsGroupedBy for o in rel.RelatedObjects]
    print(rotulo(grupo), "->", resumo(membros) if membros else "vazio")

print("\n=== 5. ELEMENTOS SEM CONTÊINER ESPACIAL ===")
sem_conteiner = []
for elem in modelo.by_type("IfcElement"):
    if elem.is_a("IfcFeatureElement"):   # aberturas e similares ficam no elemento que alteram
        continue
    contido = bool(elem.ContainedInStructure)
    parte_de = any(r.is_a("IfcRelAggregates") for r in elem.Decomposes)
    aninhado = bool(getattr(elem, "Nests", None))
    if not (contido or parte_de or aninhado):
        sem_conteiner.append(elem)
print("Total de elementos físicos:", len(modelo.by_type("IfcElement")))
print("Sem contêiner e sem pertencer a outro elemento:", len(sem_conteiner))
for elem in sem_conteiner[:10]:
    print("   ", rotulo(elem))
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap4
python estrutura_espacial.py
```

Saída obtida no teste (o caminho do arquivo foi omitido):

```
Esquema: IFC4X3_ADD2

=== 1 a 3. ÁRVORE DE DECOMPOSIÇÃO ===
#13 IfcProject 'ifc silly sample scene - project'
    #21 IfcSite 'environment - site' (COMPLEX)
        contém (#356): IfcBuildingElementProxy x1
        #28 IfcSite 'house - site' (PARTIAL)
            contém (#253): IfcBuildingElementProxy x1, IfcEarthworksFill x1, IfcRoof x1, IfcSpatialZone x1
            #35 IfcBuilding 'Single-family house' (ELEMENT)
                #45 IfcBuildingStorey '00 groundfloor' (ELEMENT)
                    contém (#67): IfcChimney x1, IfcSlab x1, IfcWall x4
                    #85 IfcSpace 'living room' (ELEMENT)
                        contém (#119): IfcFurniture x1
                    #135 IfcSpace 'entry hall' (ELEMENT)

=== 4. GRUPOS E SISTEMAS ===
#81 IfcZone 'house - living space' -> IfcSpace x2

=== 5. ELEMENTOS SEM CONTÊINER ESPACIAL ===
Total de elementos físicos: 13
Sem contêiner e sem pertencer a outro elemento: 0
```

Três leituras: a cobertura (`IfcRoof`) está contida no terreno da casa, e não no pavimento, escolha do exportador; o mobiliário está contido no ambiente "living room"; a zona `#81` agrupa os dois ambientes. O script foi executado nas 30 amostras de certificação (10 modelos em cada uma das três versões) sem erros, e nenhuma tem elementos sem contêiner.

Com a amostra de drenagem (`Infra-Plumbing.ifc` , IFC 4.3), o trecho de grupos mostra o sistema do Exemplo resolvido 4.2:

```
=== 4. GRUPOS E SISTEMAS ===
#31 IfcDistributionSystem 'road wastewater system' [SEWAGE] -> IfcDistributionChamberElement x2, IfcPipeSegment
x24
```

4.11 Resumo do capítulo

- Elementos espaciais (lugares) organizam o modelo; elementos físicos (coisas) são localizados neles.
- A estrutura espacial no IFC 4.3: projeto, terreno, instalação (edificação, rodovia, ponte, ferrovia, obra marítima), parte da instalação (pavimento, parte de rodovia, de ponte, de ferrovia) e espaço. IFC2x3 e IFC4 só têm os níveis de edificação.
- `IfcRelAggregates` monta a árvore espacial e compõe elementos (todo antes das partes). `IfcRelContainedInSpatialStructure` localiza elementos (partes antes do todo), com um único contêiner por elemento.
- Elementos em vários lugares usam `IfcRelReferencedInSpatialStructure` . Partes de conjuntos agregados não devem ter contêiner próprio.
- `IfcRelNests` compõe com ordem: portas de conexão, leiautes e referentes de alinhamentos.
- O alinhamento é agregado ao projeto e referenciado (nunca contido) na estrutura espacial.
- Grupos, sistemas e zonas (`IfcRelAssignsToGroup`) reúnem objetos por função, independentemente da localização.

4.12 Vocabulário técnico

INGLÊS	PORTUGUÊS
spatial structure	estrutura espacial
spatial element	elemento espacial
site / building / storey	terreno / edificação / pavimento
facility / facility part	instalação / parte da instalação
space / spatial zone	ambiente (espaço) / zona espacial
aggregation / decomposition	agregação / decomposição
containment	contenção
reference	referência
nesting	aninhamento

INGLÊS	PORTUGUÊS
element assembly	conjunto montado
group / system / zone	grupo / sistema / zona
alignment / layout / referent	alinhamento / leiaute / referente (estaca)
carriageway / shoulder	pista de rolamento / acostamento
superstructure / substructure / pier / abutment	superestrutura / infraestrutura / pilar / encontro

Tabela 4.16: Vocabulário do capítulo.

4.13 Exercícios

Fixação

1. Explique a diferença entre agregação e contenção espacial, com um exemplo de cada.
2. Um pilar atravessa três pavimentos. Como representá-lo corretamente?
3. Por que uma viga que faz parte de uma cobertura agregada não deve ter contêiner espacial próprio?
4. Que entidades podem ficar diretamente abaixo de `IfcProject` no IFC 4.3?
5. Qual a diferença entre `IfcZone` e `IfcSpatialZone`?

Aplicação

6. Execute `estrutura_espacial.py` com `Infra-Bridge.ifc` (IFC 4.3). Liste as partes da ponte "road river bridge" com seus `PredefinedType` e diga quais elementos estão contidos em cada pilar. Em seguida, abra o arquivo no VS Code e verifique o `UsageType` das partes da ponte. Avalie o valor usado pelo exportador à luz da Tabela 4.6.
7. Execute o script com `Infra-Rail.ifc` nas três versões (IFC2x3, IFC4 e IFC 4.3). Compare como a ferrovia é representada em cada uma.

Desafio

8. Proponha a estrutura espacial, os conjuntos e os sistemas, em IFC 4.3, para uma unidade industrial com: uma área de tancagem com três tanques e um dique; uma casa de bombas com duas bombas montadas num skid; um pipe rack com as linhas de produto e de água de incêndio; e uma edificação administrativa de dois pavimentos. Indique entidades, relações e o que ficaria a critério do BEP.

4.14 Respostas dos exercícios

1. Agregação (`IfcRelAggregates`) liga um todo às suas partes: a edificação agrega os pavimentos; a escada agrega os lances. Contenção (`IfcRelContainedInSpatialStructure`) localiza um elemento físico num elemento espacial: a parede está contida no pavimento. A agregação responde "do que é feito"; a contenção, "onde está".
2. Contém-se o pilar no pavimento onde ele começa (em geral o inferior), com `IfcRelContainedInSpatialStructure`, e referencia-se nos outros dois, com `IfcRelReferencedInSpatialStructure`. A contenção é única; a referência pode ser múltipla.

3. Porque a localização é herdada do todo: a cobertura está contida (no pavimento ou no terreno) e a viga é parte dela. A documentação do IFC 4.3 diz que a parte "should not be additionally contained in the project spatial hierarchy". Conter também a viga criaria duas localizações possivelmente contraditórias.
4. `IfcSite`, `IfcFacility` e os subtipos (`IfcBuilding`, `IfcRoad`, `IfcBridge`, `IfcRailway`, `IfcMarineFacility`), `IfcSpace` e `IfcExternalSpatialElement`, segundo o conceito *Spatial Decomposition*. Os alinhamentos também são agregados ao projeto, mas não são elementos espaciais.
5. `IfcZone` é um grupo de ambientes, sem forma nem posição próprias, ligado aos ambientes por `IfcRelAssignsToGroup`. `IfcSpatialZone` (IFC4 e IFC4.3) é um elemento espacial com forma e posição próprias, não hierárquico e possivelmente sobreposto; os elementos são ligados a ela por referência espacial.
6. A ponte `#37` (`GIRDER`) agrega: `#44` e `#104` (`NOTDEFINED`, "approach"), cada uma contendo uma laje e agregando um encontro `ABUTMENT` que contém uma viga; `#149` (`DECK`), que contém duas guardas (`IfcRailing`) e uma laje; `#281` (`SUPERSTRUCTURE`), que contém três vigas; `#320` (`SUBSTRUCTURE`), que agrega três pilares `PIER`, cada um contendo um pilar (`IfcColumn`), uma sapata (`IfcFooting`) e uma viga (travessa). No arquivo, todas as partes têm `UsageType` = `.LATERAL.`. Pelas definições da Tabela 4.6, superestrutura e infraestrutura são subdivisões no eixo Z (`VERTICAL`), e pilares e encontros são subdivisões localizadas (`REGION`) ou ao longo da ponte (`LONGITUDINAL`); `LATERAL` (esquerda, direita, centro) não descreve essas partes. O arquivo é válido pelo esquema, mas a informação é pobre: é o tipo de problema que um IDS (Capítulo 12) pode detectar.
7. Em IFC2x3, cada via aparece como `IfcBuilding` 'Rail track', e os elementos como conjuntos genéricos (`IfcElementAssembly`) contidos no terreno; em IFC4, também como `IfcBuilding` 'Rail track', agora dentro de terrenos parciais; em IFC 4.3, como `IfcRailway`, que agrega uma `IfcRailwayPart` do tipo `TRACK` contendo 2 trilhos (`IfcRail`), 33 dormentes (`IfcTrackElement`) e o lastro (`IfcCourse`). A conclusão é a mesma da Tabela 4.15: sem IFC 4.3, a semântica de infraestrutura se perde e o receptor precisa "adivinhar" pelo nome.
8. Uma proposta: `IfcProject` > `IfcSite` (área da unidade). Sob o terreno: `IfcBuilding` "Edifício administrativo" > dois `IfcBuildingStorey` > `IfcSpace`; `IfcBuilding` "Casa de bombas" (se for abrigada) > `IfcBuildingStorey` único. Para a tancagem e o pipe rack, que não são edificações, duas opções a definir no BEP: `IfcFacility` genérica com `ObjectType` "Área de tancagem" e `IfcFacilityPartCommon`, ou `IfcSpatialZone` referenciando os elementos. Elementos: tanques (`IfcTank`) e dique (`IfcWall` ou `IfcEarthworksFill`, conforme a construção) contidos na área de tancagem; o skid como `IfcElementAssembly` (contido na casa de bombas) agregando as bombas (`IfcPump`), tubos e válvulas, que não recebem contêiner próprio; o pipe rack como `IfcElementAssembly` (`SUPPORTINGASSEMBLY`) agregando pilares e vigas. Sistemas: um `IfcDistributionSystem` por linha ou por serviço (por exemplo, `FIREPROTECTION` para a água de incêndio; `USERDEFINED` com `ObjectType` para a linha de produto, ou `CHEMICAL`/`OIL`, conforme o fluido), reunindo tubos, conexões, válvulas e equipamentos por `IfcRelAssignsToGroup`. A critério do BEP: a entidade da área de processo, a granularidade dos sistemas (por linha, por serviço ou por unidade) e o nível de contenção (área ou zona).

4.15 Referências

- Documentação do IFC 4.3, conceito *Spatial Decomposition*: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/concepts/Object_Composition/Aggregation/Spatial_Decomposition/content.html
- Documentação do IFC 4.3, conceito *Spatial Containment*: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/concepts/Object_Connectivity/Spatial_Structure/Spatial_Containment/content.html
- Documentação do IFC 4.3, `IfcProject`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcProject.htm
- Documentação do IFC 4.3, `IfcRelContainedInSpatialStructure`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcRelContainedInSpatialStructure.htm
- Documentação do IFC 4.3, `IfcAlignment`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcAlignment.htm
- Documentação do IFC 4.3, `IfcFacilityUsageEnum`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcFacilityUsageEnum.htm
- Documentação do IFC 4.3, `IfcSpatialZone`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcSpatialZone.htm

5 Informação não geométrica

A geometria mostra a forma de um objeto; a informação não geométrica diz o que ele é, de que é feito, quanto mede, quanto pesa, em que condições opera, como é classificado e em que situação se encontra. Na maioria dos usos de um modelo (orçamento, compras, fabricação, operação e manutenção), é essa informação que gera valor. Este capítulo mostra onde e como o IFC a guarda: propriedades, quantidades, unidades, materiais, classificações e documentos.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que são conjuntos de propriedades e de quantidades e como se ligam aos objetos;
- reconhecer os tipos de propriedade e escolher o adequado a cada informação;
- distinguir conjuntos padronizados (`Pset_`, `Qto_`) de conjuntos personalizados;
- interpretar as unidades de um arquivo e converter valores;
- ler e associar materiais, classificações e documentos;
- mapear requisitos de informação de um plano de execução BIM (BEP) para o IFC.

5.1 Introdução: a ficha técnica do objeto

5.1.1 O que é informação não geométrica

Pense na ficha técnica de uma bomba centrífuga num catálogo: fabricante, modelo, vazão, altura manométrica, potência, rotação, material da carcaça, peso. Nenhum desses dados é forma; todos são informação. Num modelo BIM, cada objeto pode carregar uma ficha dessas.

O IFC organiza essa ficha em blocos:

BLOCO	O QUE GUARDA	EXEMPLO
Atributos	Poucos dados fixos, definidos pelo esquema	`Name`, `Description`, `Tag`, `PredefinedType`
Propriedades	Características, em conjuntos nomeados	Resistência ao fogo, pressão de trabalho, fabricante
Quantidades	Medidas para quantificação	Comprimento, área, volume, peso
Material	Do que o objeto é feito	Aço ASTM A106 Gr. B, concreto C30
Classificação	Em que categoria o objeto se enquadra num sistema de classificação	Código de uma tabela da ABNT NBR 15965, Uniclass, Omniclass
Documentos	Referências a documentos externos	Folha de dados, manual, certificado

Tabela 5.1: Os blocos de informação não geométrica.

5.1.2 Por que não usar só atributos

Os atributos são fixos: estão no esquema e não mudam de projeto para projeto. Ninguém poderia prever, no esquema, todas as características de todos os objetos de todas as disciplinas. Por isso, o IFC deixa os atributos para o essencial e oferece um mecanismo flexível e extensível: os **conjuntos de propriedades**. Qualquer projeto pode acrescentar propriedades sem alterar o esquema.

5.2 Conjuntos de propriedades

5.2.1 Estrutura

Um **conjunto de propriedades** (`IfcPropertySet`, abreviado *Pset*) é um grupo nomeado de propriedades. Cada propriedade tem um nome e um valor. A ligação com os objetos é feita por uma relação:

```
  #53 IfcSlab  ──RelatedObjects──  #59 IfcRelDefinesByProperties  ──RelatingPropertyDefinition──>  #58
IfcPropertySet 'Pset_SlabCommon'
                                                                                                    ├─ #54
IfcPropertySingleValue 'IsExternal' = .T.
                                                                                                    ├─ #55
IfcPropertySingleValue 'LoadBearing' = .F.
                                                                                                    ├─ #56
IfcPropertySingleValue 'FireRating' = 'REI30'
                                                                                                    └─ #57
IfcPropertySingleValue 'AcousticRating' = '29dB Rw'
```

Figura 5.1: Propriedades de uma laje (`Building-Architecture.ifc`, IFC 4.3).

No arquivo:

```
#54=IFCPROPERTYSINGLEVALUE('IsExternal',$,IFCBOOLEAN(.T.),$);
#55=IFCPROPERTYSINGLEVALUE('LoadBearing',$,IFCBOOLEAN(.F.),$);
#56=IFCPROPERTYSINGLEVALUE('FireRating',$,IFCLABEL('REI30'),$);
#57=IFCPROPERTYSINGLEVALUE('AcousticRating',$,IFCLABEL('29dB Rw'),$);
#58=IFCPROPERTYSET('0Gf5Qdrg10b8jCLdimssaD',#1,'Pset_SlabCommon',$,(#54,#55,#56,#57));
#59=IFCRELDEFINESBYPROPERTIES('06_iHFF012I8fFw$7gtyur',#1,$,$,(#53),#58);
```

Figura 5.2: O mesmo conjunto no texto do arquivo.

Pontos importantes:

- o conjunto (`#58`) deriva de `IfcRoot`: tem GlobalId e nome. As propriedades (`#54` a `#57`) não derivam de `IfcRoot` e não têm GlobalId;
- o valor de cada propriedade é um **valor tipado** (Capítulo 2): `IFCBOOLEAN(.T.)`, `IFCLABEL('REI30')`. O tipo diz como interpretar o valor;
- a relação `IfcRelDefinesByProperties` (atributos `RelatedObjects` e `RelatingPropertyDefinition`) pode ligar o mesmo conjunto a vários objetos. Se cem lajes têm os mesmos valores, um único conjunto e uma única relação bastam;
- no conjunto, os nomes das propriedades devem ser únicos (regra `UniquePropertyNames`), e o nome do conjunto é obrigatório (regra `ExistsName`).

5.2.2 Propriedades no tipo e na ocorrência

Um conjunto pode estar ligado à ocorrência (pela relação acima) ou ao tipo (no atributo `HasPropertySets` de `IfcTypeObject`, Capítulo 3). Na mesma amostra, o tipo da laje também tem um `Pset_SlabCommon`:

```
#48=IFCSLABTYPE('0hnSKr4LD8eRixcnqcc6X1',#1,'house - groundfloor',...,$,(#52),$,'884513','slab on
grade',.FLOOR.);
#52=IFCPROPERTYSET('01sjjx6SP4Ig0PSPhgAg4W',#1,'Pset_SlabCommon',$,(#50,#51));
#50=IFCPROPERTYSINGLEVALUE('FireRating',$,IFCLABEL('REI60'),$);
#51=IFCPROPERTYSINGLEVALUE('SurfaceSpreadOfFlame',$,IFCLABEL('A2 s1 d0'),$);
```

Figura 5.3: Propriedades no tipo da laje.

O tipo diz `FireRating` = `REI60`; a ocorrência diz `REI30`. Qual vale? A regra é: **as propriedades do tipo valem para todas as ocorrências, e a ocorrência pode sobrescrever o valor de uma propriedade de mesmo nome, no conjunto de mesmo nome**. O conjunto efetivo da laje `#53` é:

PROPRIEDADE	VALOR EFETIVO	ORIGEM
`IsExternal`	`True`	Ocorrência
`LoadBearing`	`False`	Ocorrência
`FireRating`	`REI30`	Ocorrência (sobrescreve o `REI60` do tipo)
`AcousticRating`	`29dB Rw`	Ocorrência
`SurfaceSpreadOfFlame`	`A2 s1 d0`	Tipo

Tabela 5.2: Propriedades efetivas da laje `#53`.

A documentação chama isso de conjunto *type driven override* (orientado pelo tipo, com sobrescrita). Os modelos de conjuntos padronizados indicam se o conjunto se aplica ao tipo, à ocorrência ou a ambos (seção 5.4).

5.3 Tipos de propriedade

5.3.1 As entidades

Nem toda informação é um valor único. O IFC oferece seis tipos de propriedade simples e um composto:

ENTIDADE	USO	EXEMPLO
`IfcPropertySingleValue`	Um valor	Pressão de trabalho = 1.000.000 Pa
`IfcPropertyEnumeratedValue`	Um ou mais valores escolhidos de uma lista fechada (`IfcPropertyEnumeration`)	Status = `NEW` (entre `NEW`, `EXISTING`, `DEMOLISH`, `TEMPORARY`...)
`IfcPropertyBoundedValue`	Uma faixa: limite inferior, superior e, a partir do IFC4, valor de ajuste (`SetPointValue`)	Faixa de temperatura de 5 °C a 60 °C
`IfcPropertyListValue`	Uma lista de valores do mesmo tipo	Diâmetros disponíveis: 50, 75, 100 mm

ENTIDADE	USO	EXEMPLO
`IfcPropertyTableValue`	Uma tabela de pares (valor que define, valor definido)	Curva de uma bomba: vazão x altura manométrica
`IfcPropertyReferenceValue`	Uma referência a outra entidade	Um documento, uma pessoa, uma tabela de horários
`IfcComplexProperty`	Um grupo de propriedades dentro de outra (propriedade composta)	Dados de um flange: diâmetro, espessura, número de furos

Tabela 5.3: Tipos de propriedade.

Atributos de `IfcPropertySingleValue`:

POS.	ATRIBUTO	IFC2X3 E IFC4	IFC 4.3
1	Nome	`Name`	`Name`
2	Descrição ou especificação	`Description`	`Specification` (renomeado)
3	Valor	`NominalValue`	`NominalValue`
4	Unidade	`Unit`	`Unit`

Tabela 5.4: Atributos de `IfcPropertySingleValue`.

No IFC 4.3, o segundo atributo de todas as propriedades passou a se chamar `Specification`. A posição não mudou; um arquivo IFC4 e um IFC 4.3 gravam a mesma coisa no mesmo lugar. A mudança afeta scripts que acessam o atributo pelo nome.

5.3.2 Tipos de valor

O valor é sempre tipado. Os tipos mais comuns:

FAMÍLIA	TIPOS	O QUE REPRESENTAM
Texto	`IfcLabel` (curto), `IfcText` (longo), `IfcIdentifier` (código)	Nomes, descrições, códigos
Lógico	`IfcBoolean` (verdadeiro/falso), `IfcLogical` (verdadeiro/falso/desconhecido)	Sim ou não
Número puro	`IfcInteger`, `IfcReal`, `IfcCountMeasure`, `IfcRatioMeasure`, `IfcPositiveRatioMeasure`, `IfcNormalisedRatioMeasure` (entre 0 e 1)	Contagens, proporções
Medidas	`IfcLengthMeasure`, `IfcPositiveLengthMeasure`, `IfcAreaMeasure`, `IfcVolumeMeasure`, `IfcMassMeasure`, `IfcPressureMeasure`, `IfcThermodynamicTemperatureMeasure`, `IfcPlaneAngleMeasure`, `IfcPowerMeasure`, `IfcVolumetricFlowRateMeasure`, `IfcMassDensityMeasure`	Grandezas físicas, com unidade
Data e hora	`IfcDate`, `IfcDateTime` (IFC4 e IFC4.3); `IfcTimeStamp` (todas as versões); no IFC2x3, datas de calendário são entidades (`IfcCalendarDate`)	Datas

Tabela 5.5: Tipos de valor mais usados.

> **ATENÇÃO**
>
> Escolha o tipo pela natureza da informação. Um diâmetro gravado como `IfcLabel('100 mm')` é texto: nenhum programa consegue comparar, somar ou converter. Gravado como `IfcPositiveLengthMeasure(100.)`, num arquivo em milímetros, é uma medida: pode ser validada, convertida e usada em cálculos.

5.4 Conjuntos padronizados e personalizados

5.4.1 Conjuntos padronizados

A buildingSMART publica, junto com o esquema, centenas de **conjuntos de propriedades padronizados**, com nomes iniciados por `Pset_`, e **conjuntos de quantidades**, iniciados por `Qto_`. Cada um tem um **modelo** (*template*) que define as propriedades, o tipo de valor de cada uma e as entidades às quais o conjunto se aplica. Contagem nos modelos distribuídos com o IfcOpenShell 0.8.5:

VERSÃO	`PSET_`	`QTO_`
IFC2x3	317	0 (as quantidades não tinham conjuntos padronizados)
IFC4	420	93
IFC 4.3	645	115

Tabela 5.6: Conjuntos padronizados por versão.

Exemplos úteis, em várias áreas (IFC 4.3):

CONJUNTO	APLICA-SE A	PROPRIEDADES (SELEÇÃO)
`Pset_WallCommon`	`IfcWall`, `IfcWallType`	`Reference`, `Status`, `FireRating`, `AcousticRating`, `ThermalTransmittance`, `IsExternal`, `LoadBearing`
`Pset_BeamCommon`	`IfcBeam`, `IfcBeamType`	`Reference`, `Status`, `Span`, `Slope`, `IsExternal`, `LoadBearing`, `FireRating`
`Pset_PipeSegmentTypeCommon`	`IfcPipeSegment`, `IfcPipeSegmentType`	`Reference`, `Status`, `WorkingPressure`, `PressureRange`, `TemperatureRange`, `NominalDiameter`, `InnerDiameter`, `OuterDiameter`, `Length`
`Pset_ValveTypeCommon`	`IfcValve`, `IfcValveType`	`ValvePattern`, `ValveOperation`, `ValveMechanism`, `Size`, `TestPressure`, `WorkingPressure`
`Pset_PumpTypeCommon`	`IfcPump`, `IfcPumpType`	`FlowRateRange`, `ConnectionSize`, `TemperatureRange`, `NetPositiveSuctionHead`, `NominalRotationSpeed`

CONJUNTO	APLICA-SE A	PROPRIEDADES (SELEÇÃO)
`Pset_TankTypeCommon`	`IfcTank`, `IfcTankType`	`StorageType`, `NominalDepth`, `TankNominalCapacity`, `EffectiveCapacity`, `OperatingWeight`, `PatternType`
`Pset_CourseCommon`	`IfcCourse`, `IfcCourseType`	`NominalLength`, `NominalThickness`, `NominalWidth`
`Pset_ManufacturerTypeInformation`	`IfcElement`, `IfcElementType`	`Manufacturer`, `ModelReference`, `ModelLabel`, `ArticleNumber`, `ProductionYear`
`Pset_Warranty`	`IfcElement`, `IfcElementType`	`WarrantyIdentifier`, `WarrantyStartDate`, `WarrantyPeriod`
`Pset_Condition`	`IfcElement`, `IfcElementType`, `IfcSystem`, `IfcAsset`	`AssessmentDate`, `AssessmentCondition`, `NextAssessmentDate`
`Qto_WallBaseQuantities`	`IfcWall`, `IfcWallType`	`Length`, `Width`, `Height`, `GrossSideArea`, `NetSideArea`, `GrossVolume`, `NetVolume`, `GrossWeight`, `NetWeight`
`Qto_PipeSegmentBaseQuantities`	`IfcPipeSegment`, `IfcPipeSegmentType`	`Length`, `GrossCrossSectionArea`, `OuterSurfaceArea`, `GrossWeight`, `NetWeight`

Tabela 5.7: Conjuntos padronizados de várias áreas (IFC 4.3).

A propriedade `Status`, presente em quase todos os conjuntos `Pset_...Common` do IFC4 e do IFC 4.3, é uma enumeração com os valores `NEW` (novo), `EXISTING` (existente), `DEMOLISH` (a demolir), `TEMPORARY` (provisório), `OTHER`, `NOTKNOWN` e `UNSET`. Ela descreve a situação do elemento na obra, e não o status de aprovação do projeto.

Os conjuntos mudam entre versões. `Pset_PipeSegmentTypeCommon`, por exemplo:

VERSÃO	APLICA-SE A	PROPRIEDADES
IFC2x3	`IfcPipeSegmentType`	`Material`, `WorkingPressure`, `UnitWeight`, `PressureRange`, `TemperatureRange`, `NominalDiameter`, `InnerDiameter`, `OuterDiameter`, `EndStyleTreatment`
IFC4	`IfcPipeSegment`	`Reference`, `Status`, `WorkingPressure`, `PressureRange`, `TemperatureRange`, `NominalDiameter`, `InnerDiameter`, `OuterDiameter`
IFC 4.3	`IfcPipeSegment`, `IfcPipeSegmentType`	As do IFC4, mais `Length`

Tabela 5.8: Evolução de um conjunto padronizado.

Outro exemplo: a cor em `Pset_PipeSegmentOccurrence` chama-se `Color` no IFC4 e `Colour` no IFC 4.3. Detalhes assim fazem uma validação falhar. Sempre confira o conjunto na documentação da versão usada.

5.4.2 Conjuntos personalizados

Quando nenhum conjunto padronizado tem a informação exigida, cria-se um **conjunto personalizado**. A documentação do IFC 4.3 é clara: "Property sets that are not declared as part of the IFC specification shall have a Name value not including the 'Pset_' prefix" (conjuntos que não fazem parte da especificação não devem ter o prefixo `Pset_`).

Boas práticas para conjuntos personalizados:

- **prefixo próprio**, que identifique a origem: `ARV_Tubulacao`, `CLIENTE_Manutencao`, `BEP_Identificacao`;
- **nomes estáveis**, sem acentos nem espaços, definidos no BEP e no IDS (Capítulo 12);
- **tipo de valor correto** (Tabela 5.5): medida para medidas, lógico para sim ou não, texto só para texto;
- **não duplicar** o que já existe num conjunto padronizado. Se a pressão de trabalho cabe em `Pset_PipeSegmentTypeCommon.WorkingPressure`, use-a;
- **documentar** o significado e a unidade de cada propriedade.

A amostra `Infra-Landscaping.ifc` (IFC 4.3) tem um conjunto personalizado:

```
#85=IFCPROPERTYSET('3NAZCLJDHCXhv4f6CvNAk2',#1,'SizeSet',$,(#83,#84));
#83=IFCPROPERTYSINGLEVALUE('Height',$,IFCLENGTHMEASURE(167.),$);
#84=IFCPROPERTYSINGLEVALUE('Volume',$,IFCVOLUMEMEASURE(0.0040000000000000001),$);
```

Figura 5.4: Conjunto personalizado com valores tipados.

5.5 Quantidades

5.5.1 Conjuntos de quantidades

As **quantidades** são medidas usadas para quantificação e orçamento. Ficam em `IfcElementQuantity`, ligado ao objeto pela mesma relação `IfcRelDefinesByProperties`. Cada quantidade é uma entidade própria:

ENTIDADE	GRANDEZA	VERSÕES
`IfcQuantityLength`	Comprimento	Todas
`IfcQuantityArea`	Área	Todas
`IfcQuantityVolume`	Volume	Todas
`IfcQuantityWeight`	Peso (massa)	Todas
`IfcQuantityCount`	Contagem	Todas
`IfcQuantityTime`	Tempo	Todas
`IfcQuantityNumber`	Número sem unidade	IFC 4.3

Tabela 5.9: Entidades de quantidade.

Exemplo real, a parede `#158` de `Building-Architecture.ifc`:

```
#161=IFCQUANTITYVOLUME('NetVolume',$,$,1.2692649352635803,$);
#162=IFCQUANTITYLENGTH('Width',$,$,200.,$);
#163=IFCQUANTITYLENGTH('Length',$,$,1800.,$);
```

```
#164=IFCQUANTITYAREA('NetSideArea',$,$,6.3463246763178782,$);
#165=IFCELEMENTQUANTITY('0pwzncA99EPe8_AwN9lSZt',#1,'Qto_WallBaseQuantities',$,'BaseQuantities',
(#161,#162,#163,#164));
#166=IFCRELDEFINESBYPROPERTIES('0p3oIa1hj7Cf3Q4vp29ZgQ',#1,$,$,(#158),#165);
```

Figura 5.5: Quantidades de uma parede.

O quinto atributo de `IfcElementQuantity` é o método de medição (`MethodOfMeasurement`); aqui, `'BaseQuantities'`, as quantidades básicas definidas pela buildingSMART.

5.5.2 Bruto e líquido

Os conjuntos `Qto_` distinguem valores **brutos** (*gross*) e **líquidos** (*net*). O bruto ignora aberturas e recortes; o líquido os desconta. Numa parede com uma janela, `GrossSideArea` é a área da parede inteira, e `NetSideArea` é a área descontada a janela. Para o peso, `GrossWeight` e `NetWeight` seguem a mesma lógica.

> **ATENÇÃO**
>
> Quantidades gravadas no arquivo são calculadas pelo software de origem, no momento da exportação. Elas não são atualizadas se a geometria mudar depois. Para verificações críticas, recalcule a partir da geometria (Capítulo 11) e compare.

5.6 Unidades

5.6.1 Unidades do projeto

Um número só tem sentido com a unidade. No IFC, as **unidades padrão** do arquivo são declaradas no `IfcProject` (atributo `UnitsInContext`), numa `IfcUnitAssignment`. A amostra da parede:

```
#14=IFCUNITASSIGNMENT((#15,#16,#17));
#15=IFCSIUNIT(*,.LENGTHUNIT.,.MILLI.,.METRE.);
#16=IFCSIUNIT(*,.AREAUNIT.,$,.SQUARE_METRE.);
#17=IFCSIUNIT(*,.VOLUMEUNIT.,$,.CUBIC_METRE.);
```

Figura 5.6: Declaração de unidades.

Leitura: comprimentos em milímetros, áreas em metros quadrados, volumes em metros cúbicos. Por isso, na Figura 5.5, `Width` = 200 significa 200 mm, e `NetSideArea` = 6,35 significa 6,35 m². Misturar unidades assim é comum e correto, desde que declarado.

5.6.2 Tipos de unidade

ENTIDADE	USO	EXEMPLO
`IfcSIUnit`	Unidade do Sistema Internacional, com prefixo opcional	`IFCSIUNIT(*,.MASSUNIT.,.KILO.,.GRAM.)` = quilograma; `IFCSIUNIT(*,.PRESSUREUNIT.,$,.PASCAL.)` = pascal; `IFCSIUNIT(*,.THERMODYNAMICTEMPERATUREUNIT.,$,.DEGREE_CELSIUS.)` = grau Celsius

ENTIDADE	USO	EXEMPLO
`IfcConversionBasedUnit`	Unidade definida por conversão de outra	Grau (ângulo) = 0,0174532925199433 radiano; polegada = 0,0254 metro
`IfcDerivedUnit`	Combinação de unidades com expoentes	Massa específica: $kg \cdot m^{-3}$
`IfcMonetaryUnit`	Moeda	`IFCMONETARYUNIT('BRL')` (IFC4 e IFC4.3, texto); no IFC2x3, `IFCMONETARYUNIT(.BRL.)` (enumeração)

Tabela 5.10: Tipos de unidade.

O primeiro atributo de `IfcSIUnit` (`Dimensions`) é derivado (`*`): o esquema calcula as dimensões físicas a partir do nome.

5.6.3 Qual unidade vale para uma propriedade

A documentação de `IfcPropertySingleValue` define: o atributo `Unit` é a "unit for the nominal value, if not given, the default value for the measure type (given by the TYPE of nominal value) is used as defined by the global unit assignment at IfcProject" (se não for informado, vale a unidade padrão do projeto para o tipo de medida do valor).

SITUAÇÃO	UNIDADE DO VALOR
`Unit` preenchido na propriedade	A unidade informada
`Unit` vazio e o projeto declara unidade para a grandeza	A unidade do projeto
`Unit` vazio e o projeto não declara unidade para a grandeza	A unidade SI da grandeza, na interpretação usual dos programas; o IDS, por exemplo, compara valores em unidades SI (Capítulo 12)

Tabela 5.11: Unidade de uma propriedade.

Exemplo resolvido 5.1: converter valores para o SI

Problema. Num arquivo com as unidades da Figura 5.6, uma propriedade `NominalDiameter` vale `IFCPOSITIVELENGTHMEASURE(150.)` e uma propriedade `WorkingPressure` vale `IFCPRESSUREMEASURE(1000000.)`, ambas sem `Unit`. Quais são os valores em metros e em bar?

Solução. O diâmetro é um comprimento; a unidade de comprimento do projeto é o milímetro. Logo, 150 mm = 0,150 m. A pressão é uma grandeza para a qual o projeto não declara unidade; vale a unidade SI, o pascal. Logo, 1.000.000 Pa = 10 bar (1 bar = 100.000 Pa). Por programa, o fator do comprimento é obtido com `ifcopenshell.util.unit.calculate_unit_scale(modelo, "LENGTHUNIT")`, que retorna 0,001 (seção 5.10).

5.7 Materiais

5.7.1 Material simples

`IfcMaterial` representa uma substância. Atributos: `Name` (em todas as versões) e, a partir do IFC4, `Description` e `Category`. A documentação recomenda categorias em termos gerais: "'concrete', 'steel', 'aluminium', 'block', 'brick', 'stone', 'wood', 'glass', 'gypsum', 'plastic', 'earth'". A ligação com os objetos é feita por `IfcRelAssociatesMaterial` (atributos `RelatedObjects` e `RelatingMaterial`):

```
#160=IFCMATERIAL('stone_sand-lime',$,$);
#159=IFCRELASSOCIATESMATERIAL('0sAYN8OJzFKu1rNoJBdSki',#1,$,$,(#158,#182,#201,#220),#160);
```

Figura 5.7: Um material associado a quatro paredes.

5.7.2 Conjuntos de materiais

Muitos elementos não são de um único material. O IFC oferece conjuntos, cada um adequado a uma forma de construção:

CONJUNTO	VERSÕES	USO	EXEMPLO
`IfcMaterialLayerSet` e `IfcMaterialLayer`	Todas	Camadas com espessura, empilhadas	Parede de alvenaria com reboco; laje com contrapiso; pavimento com camadas
`IfcMaterialProfileSet` e `IfcMaterialProfile`	IFC4, IFC4.3	Material associado a um perfil de seção	Viga W 310x38,7 em aço ASTM A572 Gr. 50
`IfcMaterialConstituentSet` e `IfcMaterialConstituent`	IFC4, IFC4.3	Partes de um elemento, cada uma de um material	Janela: caixilho em alumínio, vidro laminado
`IfcMaterialList`	Todas; desaconselhada	Lista simples	"The entity IfcMaterialList is deprecated and shall no longer be used. Use IfcMaterialConstituentSet instead."

Tabela 5.12: Conjuntos de materiais.

As variantes com uso (`IfcMaterialLayerSetUsage`, `IfcMaterialProfileSetUsage`) ligam o conjunto à ocorrência e dizem como ele se posiciona em relação ao eixo do elemento: a direção das camadas e o deslocamento, no caso das camadas; o ponto de referência da seção (`CardinalPoint`), no caso dos perfis. Em geral, o conjunto fica no tipo e o uso fica na ocorrência.

Dois exemplos reais, dos modelos de especificação da buildingSMART (IFC 4.3). Primeiro, um pilar metálico IPE 200 em aço S355JR:

```
#200= IFCMATERIAL('S355JR','Steel',$);
#202= IFCISHAPEPROFILEDEF(.AREA.,'IPE200',$,100.0,200.0,5.6,8.5,12.0,$,$);
#203= IFCMATERIALPROFILE('IPE200',$,#200,#202,0,$);
#205= IFCMATERIALPROFILESET('IPE200',$,(#203),$);
#206= IFCRELASSOCIATESMATERIAL('0TDF0F6A574e3x4InqoSrm',#56,'MatAssoc','Material Associates',(#207),#205);
#207= IFCCOLUMNTYPE('3qJDCKcPj1tgEHrIL1MUed',#56,'IPE200',$,$,$,$,$,.COLUMN.);
```

```
#210= IFCCOLUMN('3S1GK_wA565RDoiWQEJc_l',#56,'IPE200',$,$,#211,#224,$,$);
#219= IFCMATERIALPROFILESETUSAGE(#205,5,$);
#220= IFCRELASSOCIATESMATERIAL('0$JavXVwHFKOejR2KlPyfE',#56,'MatAssoc','Material Associates',(#210),#219);
```

Figura 5.8: Conjunto de perfis no tipo e uso na ocorrência (`column-extruded-solid.ifc`).

O material `#200` (categoria "Steel") e o perfil `#202` (largura 100, altura 200, alma 5,6, mesa 8,5, raio 12, em milímetros) formam o perfil de material `#203`, reunido no conjunto `#205`. O conjunto é associado ao **tipo** `#207` (relação `#206`). A ocorrência `#210` recebe o **uso** `#219`, com ponto de referência 5 (centro da seção), pela relação `#220`.

Segundo, uma laje de concreto de 200 mm, com camadas:

```
#200= IFCMATERIAL('Concrete','Concrete',$);
#202= IFCMATERIALLAYER(#200,200.0,.U.,'Core',$,$,$);
#204= IFCMATERIALLAYERSET((#202),'200mm Concrete',$);
#310= IFCMATERIALLAYERSETUSAGE(#204,.AXIS3.,.POSITIVE.,-200.0,$);
#311= IFCRELASSOCIATESMATERIAL('2Npx$Qu3X5tPUqWORsru6k',#56,'MatAssoc','Material Associates',(#303),#310);
```

Figura 5.9: Conjunto de camadas com uso (`slab-openings.ifc`).

A camada `#202` tem 200 mm; o uso `#310` diz que as camadas se empilham ao longo do eixo Z local (`.AXIS3.`), no sentido positivo, a partir de um deslocamento de -200 mm em relação à face de referência.

5.7.3 Propriedades de materiais

Materiais também têm propriedades, em conjuntos próprios (`IfcMaterialProperties` , IFC4 e IFC4.3), com modelos padronizados como `Pset_MaterialCommon` (`MassDensity` , `Porosity`) e `Pset_MaterialSteel` (`YieldStress` , `UltimateStress` , `StructuralGrade`). A massa específica permite calcular o peso a partir do volume.

5.8 Classificação

5.8.1 O que é um sistema de classificação

Um **sistema de classificação** organiza os objetos da construção em categorias codificadas, segundo critérios padronizados. Ele permite, por exemplo, ligar os objetos do modelo às linhas de um orçamento, às especificações técnicas ou a um plano de manutenção, mesmo que cada software use nomes diferentes. Exemplos: a ABNT NBR 15965 (Brasil), a Uniclass (Reino Unido), a OmniClass (Estados Unidos), a CCI (Dinamarca e outros países).

A **ABNT NBR 15965** (Sistema de classificação da informação da construção) está organizada em sete partes e treze tabelas, identificadas por códigos de dois caracteres. Segundo levantamento publicado na revista *Ambiente Construído*:

PARTE	TEMA	TABELAS
2	Características dos objetos da construção	0M (materiais), 0P (propriedades)
3	Processos da construção	1F (fases), 1S (serviços), 1D (disciplinas)
4	Recursos da construção	2N (funções), 2Q (equipamentos), 2C (produtos)

PARTE	TEMA	TABELAS
5	Resultados da construção	3E (elementos), 3R (resultados de serviços)
6	Unidades e espaços da construção	4U (unidades), 4A (espaços)
7	Informação da construção	5I (informação)

Tabela 5.13: Tabelas da ABNT NBR 15965.

A Parte 1 trata da terminologia e da estrutura. Para classificar elementos de um modelo (paredes, tubos, vigas), a tabela mais usada é a 3E; para espaços, a 4A. Os códigos e títulos exatos devem ser obtidos nas partes publicadas da norma.

5.8.2 Classificação no IFC

A classificação é uma **associação**, como o material. Entidades:

ENTIDADE	PAPEL	ATRIBUTOS PRINCIPAIS
`IfcClassification`	O sistema (a "tabela")	`Source` (quem publica), `Edition`, `EditionDate`, `Name`, `Description` e, no IFC4, `Location` (renomeado `Specification` no IFC 4.3)
`IfcClassificationReference`	Um item do sistema (um código)	`Location` (URI), `Identification` (o código; `ItemReference` no IFC2x3), `Name` (título), `ReferencedSource` (o sistema ou o item pai)
`IfcRelAssociatesClassification`	A ligação com os objetos	`RelatedObjects`, `RelatingClassification`

Tabela 5.14: Entidades de classificação.

Exemplo real (`Building-Architecture.ifc`, IFC 4.3), com a classificação dinamarquesa CCI publicada no bSDD:

```
#36=IFCCLASSIFICATION('Molio','1.0','2023-01-23','CCI Construction',$,$,$);
#37=IFCCLASSIFICATIONREFERENCE('https://identifier.buildingsmart.org/uri/molio/cciconstruction/1.0/class/E-
AAA','E-AAA','Single-family house',#36,$,$);
#38=IFCRELASSOCIATESCLASSIFICATION('29uQ$2gsnCLeQxvJoA9Iv1',#1,'CCI Construction Classification',$,(#35),#37);
```

Figura 5.10: Classificação de uma edificação.

A edificação `#35` recebe o código `E-AAA` ("Single-family house") do sistema CCI Construction, edição 1.0, publicado pela Molio. O atributo `Location` guarda o endereço permanente do item no bSDD.

5.8.3 O bSDD

O **bSDD** (buildingSMART Data Dictionary) é um serviço da buildingSMART que reúne dicionários de dados (classificações, catálogos de propriedades) publicados por diferentes organizações, cada item com um endereço permanente (URI). O portal de busca é https://search.bsdd.buildingsmart.org/. O próprio IFC 4.3 está publicado como dicionário; o URI da classe `IfcWall`, por exemplo, é `https://identifier.buildingsmart.org/uri/buildingsmart/ifc/4.3/class/IfcWall`.

Ligar o modelo ao bSDD pelo URI tem uma vantagem: qualquer programa pode consultar a definição oficial do código, em vez de depender só do texto gravado no arquivo. No IFC 4.3, o URI de uma propriedade pode ser gravado no atributo `Specification` da própria propriedade.

> **OBSERVAÇÃO**
> Em consulta à interface do bSDD feita em setembro de 2026, a ABNT NBR 15965 aparece apenas como dicionário de demonstração, de caráter de pesquisa ("SICLIC (ABNT NBR 15965)", versão 0.1.2, situação *Preview*, publicado pela organização "DEMO"). Não há dicionário oficial da ABNT. Ao usar a NBR 15965 no IFC, grave o código em `Identification`, o título em `Name` e a identificação da norma em `IfcClassification`, e confira a situação do bSDD antes de usar URIs.

5.9 Documentos, aprovações e o registro da revisão

5.9.1 Documentos

Um objeto pode apontar para documentos externos: folha de dados, desenho, manual, certificado. As entidades:

ENTIDADE	PAPEL	ATRIBUTOS PRINCIPAIS
`IfcDocumentInformation`	O documento (metadados)	`Identification` (`DocumentId` no IFC2x3), `Name`, `Description`, `Location`, `Purpose`, `Revision`, `Status`, `CreationTime`, `LastRevisionTime`, `ElectronicFormat`
`IfcDocumentReference`	Uma referência ao documento, por exemplo, a uma página ou arquivo	`Location` (URI ou caminho), `Identification`, `Name`, `ReferencedDocument`
`IfcRelAssociatesDocument`	A ligação com os objetos	`RelatedObjects`, `RelatingDocument`

Tabela 5.15: Entidades de documentos.

O atributo `Revision` de `IfcDocumentInformation` guarda a revisão **do documento**, por exemplo, a folha de dados de uma bomba na revisão B.

5.9.2 Aprovações

`IfcApproval` registra uma aprovação (quem, quando, com que situação), ligada aos objetos por `IfcRelAssociatesApproval`. É pouco usada pelos exportadores, mas existe no esquema de todas as versões, com atributos diferentes no IFC2x3.

5.9.3 E a revisão do objeto?

O IFC não tem um atributo "revisão" nos elementos. O `OwnerHistory` (Capítulo 3) registra alterações, mas raramente é usado com esse fim, e é opcional desde o IFC4. Quando o BEP exige a revisão de cada objeto, a solução usual é uma **propriedade personalizada** (por exemplo, `ARV_Controle.Revisao`), preenchida no software de autoria ou por script (Capítulo 10).

5.10 Do BEP ao IFC: mapear requisitos de informação

5.10.1 O problema

Um plano de execução BIM (BEP) costuma listar a informação que cada objeto deve ter: identificação, tipo, situação, dimensões, material, classificação. Para que essa exigência possa ser cumprida e verificada, cada item precisa de um **endereço no IFC**: em que atributo, conjunto ou associação ele será gravado. Sem esse mapeamento, cada software exporta a informação num lugar diferente, e a verificação automática fica impossível.

5.10.2 Um mapeamento de referência

A tabela abaixo mostra um mapeamento possível para requisitos típicos de redes de tubulação e equipamentos, em IFC 4.3. Os conjuntos padronizados citados foram conferidos nos modelos da versão. Onde não há lugar padronizado, a tabela indica um conjunto personalizado, que deve ser definido no BEP.

REQUISITO DO BEP	ONDE GRAVAR (IFC 4.3)	OBSERVAÇÃO
TAG (identificação)	Atributo `Name` ou `Tag` da ocorrência; ou `Pset_ConstructionOccurence.TagNumber`	Definir um único lugar no BEP. Muitos exportadores usam `Tag` para o identificador interno do software
Tipo de objeto	Entidade (`IfcPipeSegment`, `IfcValve` ...) com `PredefinedType` no tipo; `ObjectType` quando `USERDEFINED`	Capítulo 3
Revisão do objeto	Conjunto personalizado	Não há lugar padronizado
Situação na obra (novo, existente, a demolir)	`Pset_...Common.Status`	Enumeração padronizada
Situação de aprovação	Conjunto personalizado ou `IfcApproval`	`Status` padronizado não é aprovação
Fase	`IfcProject.Phase` (fase do projeto); tarefas de cronograma para o planejamento; ou conjunto personalizado por objeto	Definir o significado de "fase" no BEP
Dimensões nominais	`Pset_PipeSegmentTypeCommon.NominalDiameter`, `InnerDiameter`, `OuterDiameter`; `Pset_ValveTypeCommon.Size`; `Pset_TankTypeCommon.NominalDepth`	No tipo
Dimensões medidas	`Qto_...BaseQuantities` (`Length`, `Width`, `Height` ...)	Calculadas pelo exportador
Coordenadas e cotas	Posicionamento e geometria (Capítulo 7)	Se o BEP exigir em propriedade, gravar como informação derivada, com unidade clara
Peso	`Qto_...BaseQuantities.GrossWeight` / `NetWeight` (`IfcQuantityWeight`); `Pset_TankTypeCommon.OperatingWeight`	Distinguir peso vazio e em operação

REQUISITO DO BEP	ONDE GRAVAR (IFC 4.3)	OBSERVAÇÃO
Fluido	`IfcDistributionSystem` com `PredefinedType` (categoria do serviço) e conjunto personalizado para o fluido específico	Não há propriedade padronizada de fluido nos conjuntos de tubo
Pressão	`Pset_PipeSegmentTypeCommon.WorkingPressure` e `PressureRange`; `Pset_ValveTypeCommon.TestPressure`	Pressão de projeto da linha: conjunto personalizado no sistema ou no elemento
Temperatura	`Pset_PipeSegmentTypeCommon.TemperatureRange` (faixa)	Temperatura de projeto da linha: conjunto personalizado
Material	`IfcRelAssociatesMaterial` (`IfcMaterial`, perfis ou camadas)	Categoria do material em `Category`
Pintura, revestimento	Conjunto personalizado	Não há conjunto padronizado de pintura para tubos e equipamentos
Classificação	`IfcRelAssociatesClassification` (por exemplo, NBR 15965, tabela 3E)	Seção 5.8
Fabricante e modelo	`Pset_ManufacturerTypeInformation`	No tipo

Tabela 5.16: Mapeamento de requisitos de um BEP para o IFC 4.3 (tubulação e equipamentos).

A mesma lógica vale para qualquer área. Numa edificação, a resistência ao fogo vai para `Pset_WallCommon.FireRating`; numa rodovia, a espessura nominal da camada vai para `Pset_CourseCommon.NominalThickness`; num ativo em operação, a data de inspeção vai para `Pset_Condition.AssessmentDate`.

> **DICA**
>
> O mapeamento é a base de três entregas: a configuração do exportador (Capítulo 13), o script de preenchimento em lote (Capítulo 10) e o IDS de verificação (Capítulo 12). Um mapeamento bem feito no início evita retrabalho nas três.

5.11 Leitura com um script

O script abaixo mostra, para um objeto, as unidades do projeto, os conjuntos de propriedades e de quantidades com a origem (ocorrência ou tipo), o tipo de cada propriedade e de cada valor, o material e as classificações do objeto, do seu tipo e do seu contêiner. Os comandos são explicados no Capítulo 9. Grave-o como `C:\IFC_Curso\exercicios\cap5\informacao.py`.

```
# -*- coding: utf-8 -*-
"""
informacao.py | Capítulo 5: informação não geométrica de um objeto IFC.

O que faz (somente leitura; nada é gravado):
```

```
  1. Mostra as unidades do projeto (IfcUnitAssignment).
  2. Mostra os conjuntos de propriedades e de quantidades do objeto, indicando a origem
     (ocorrência ou tipo), a entidade de cada propriedade, o valor e o tipo do valor.
  3. Mostra o material associado e as classificações do objeto.

Uso (ambiente virtual ativado):
    python informacao.py                        -> arquivo e instância padrão abaixo
    python informacao.py <arquivo.ifc> <id>     -> outro arquivo e outra instância

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.util.element as util_elem
import ifcopenshell.util.unit as util_unid

ARQUIVO_PADRAO = (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc"
)
ID_PADRAO = 53  # a instância #53 é uma IfcSlab (laje térrea)

arquivo = sys.argv[1] if len(sys.argv) > 1 else ARQUIVO_PADRAO
numero = int(sys.argv[2]) if len(sys.argv) > 2 else ID_PADRAO
modelo = ifcopenshell.open(arquivo)
obj = modelo.by_id(numero)
print(f"Objeto: #{obj.id()} {obj.is_a()} '{obj.Name}' | esquema {modelo.schema_identifier}")

# ------------------------------------------------------------ 1. Unidades
print("\n=== 1. UNIDADES DO PROJETO ===")
for unidade in modelo.by_type("IfcUnitAssignment")[0].Units:
    if unidade.is_a("IfcNamedUnit"):          # IfcSIUnit e IfcConversionBasedUnit
        print(f"   {unidade.UnitType:<22} {util_unid.get_full_unit_name(unidade)}")
    else:                                     # IfcDerivedUnit, IfcMonetaryUnit
        print(f"   {unidade.is_a():<22} {unidade}")
print("   Fator para metro (comprimento):", util_unid.calculate_unit_scale(modelo, "LENGTHUNIT"))


def valor_texto(prop):
    """Descreve uma propriedade ou quantidade: valor e tipo do valor."""
    if prop.is_a("IfcPropertySingleValue"):
        v = prop.NominalValue
        return f"{v.wrappedValue!r} ({v.is_a()})" if v is not None else "$ (sem valor)"
    if prop.is_a("IfcPropertyEnumeratedValue"):
        return "[" + ", ".join(str(v.wrappedValue) for v in (prop.EnumerationValues or [])) + "]"
    if prop.is_a("IfcPropertyBoundedValue"):
        inf = prop.LowerBoundValue.wrappedValue if prop.LowerBoundValue else None
        sup = prop.UpperBoundValue.wrappedValue if prop.UpperBoundValue else None
        return f"de {inf} a {sup}"
    if prop.is_a("IfcPhysicalSimpleQuantity"):
        return repr(prop[3])                  # 4º atributo: LengthValue, AreaValue, VolumeValue...
    return f"({prop.is_a()})"


def mostrar_conjunto(conjunto, origem):
    """Mostra um IfcPropertySet ou um IfcElementQuantity."""
    print(f"   [{origem}] {conjunto.Name} ({conjunto.is_a()}, #{conjunto.id()})")
    itens = conjunto.HasProperties if conjunto.is_a("IfcPropertySet") else conjunto.Quantities
    for item in itens:
        print(f"        {item.Name:<24} {item.is_a():<28} {valor_texto(item)}")


# ------------------------------------------------------------ 2. Propriedades e quantidades
print("\n=== 2. CONJUNTOS DE PROPRIEDADES E DE QUANTIDADES ===")
# 2a. Na ocorrência: relações IfcRelDefinesByProperties (inverso IsDefinedBy)
for rel in obj.IsDefinedBy:
    if rel.is_a("IfcRelDefinesByProperties"):
        mostrar_conjunto(rel.RelatingPropertyDefinition, "ocorrência")
# 2b. No tipo: atributo HasPropertySets do IfcTypeObject
tipo = util_elem.get_type(obj)
if tipo is not None:
    for conjunto in tipo.HasPropertySets or []:
        mostrar_conjunto(conjunto, f"tipo #{tipo.id()}")
```

```
# ------------------------------------------------------------ 3. Material e classificação
print("\n=== 3. MATERIAL E CLASSIFICAÇÃO ===")
material = util_elem.get_material(obj)       # procura na ocorrência e, se não houver, no tipo
print("   Material:", material if material else "nenhum")
for alvo in [obj, tipo, util_elem.get_container(obj)]:
    if alvo is None:
        continue
    for rel in alvo.HasAssociations:
        if rel.is_a("IfcRelAssociatesClassification"):
            ref = rel.RelatingClassification
            # no IFC2X3 o código fica em ItemReference; no IFC4 e no IFC 4.3, em Identification
            codigo = getattr(ref, "Identification", None) or getattr(ref, "ItemReference", None)
            print(f"   Classificação em #{alvo.id()} {alvo.is_a()}: {codigo} '{ref.Name}'"
                  f" | sistema: {ref.ReferencedSource.Name if ref.ReferencedSource else '-'}"
                  f" | URI: {ref.Location}")
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap5
python informacao.py
```

Saída obtida no teste:

```
Objeto: #53 IfcSlab 'floor' | esquema IFC4X3_ADD2

=== 1. UNIDADES DO PROJETO ===
   LENGTHUNIT             MILLIMETRE
   AREAUNIT               SQUARE_METRE
   VOLUMEUNIT             CUBIC_METRE
   Fator para metro (comprimento): 0.001

=== 2. CONJUNTOS DE PROPRIEDADES E DE QUANTIDADES ===
   [ocorrência] Pset_SlabCommon (IfcPropertySet, #58)
        IsExternal               IfcPropertySingleValue       True (IfcBoolean)
        LoadBearing              IfcPropertySingleValue       False (IfcBoolean)
        FireRating               IfcPropertySingleValue       'REI30' (IfcLabel)
        AcousticRating           IfcPropertySingleValue       '29dB Rw' (IfcLabel)
   [ocorrência] Qto_SlabBaseQuantities (IfcElementQuantity, #65)
        NetVolume                IfcQuantityVolume            6.437500000000378
        Depth                    IfcQuantityLength            250.0
        NetArea                  IfcQuantityArea              25.749999999991743
   [tipo #48] Pset_SlabCommon (IfcPropertySet, #52)
        FireRating               IfcPropertySingleValue       'REI60' (IfcLabel)
        SurfaceSpreadOfFlame     IfcPropertySingleValue       'A2 s1 d0' (IfcLabel)

=== 3. MATERIAL E CLASSIFICAÇÃO ===
   Material: #61=IfcMaterial('concrete_reinforced_in-situ',$,$)
```

A laje tem 250 mm de espessura (`Depth`, em milímetros), 25,75 m² de área e 6,44 m³ de volume (25,75 x 0,25 = 6,44). Para a edificação (`#35`), a última seção mostra a classificação da Figura 5.8:

```
=== 3. MATERIAL E CLASSIFICAÇÃO ===
   Material: nenhum
   Classificação em #35 IfcBuilding: E-AAA 'Single-family house' | sistema: CCI Construction | URI: https://
identifier.buildingsmart.org/uri/molio/cciconstruction/1.0/class/E-AAA
```

O script foi testado também na versão IFC2x3 da mesma amostra, com o mesmo resultado para a laje (instância `#44`).

5.12 Resumo do capítulo

- A informação não geométrica fica em atributos, propriedades, quantidades, materiais, classificações e documentos.
- `IfcPropertySet` agrupa propriedades; `IfcRelDefinesByProperties` o liga às ocorrências; o tipo guarda conjuntos em `HasPropertySets`. A ocorrência sobrescreve o valor do tipo.
- Há sete tipos de propriedade; os valores são tipados. Medidas devem ser gravadas como medidas, não como texto.
- `Pset_` e `Qto_` são reservados aos conjuntos padronizados, que mudam entre versões. Conjuntos personalizados usam prefixo próprio.
- As unidades padrão estão em `IfcProject.UnitsInContext`; sem `Unit`, vale a unidade do projeto para a grandeza.
- Materiais, classificações e documentos são associações (`IfcRelAssociates...`).
- Todo requisito do BEP precisa de um endereço no IFC; esse mapeamento orienta a exportação, o preenchimento em lote e a verificação.

5.13 Vocabulário técnico

INGLÊS	PORTUGUÊS
property set	conjunto de propriedades
quantity set / base quantities	conjunto de quantidades / quantidades básicas
single value / enumerated value / bounded value	valor único / valor enumerado / valor em faixa
nominal value	valor nominal
gross / net	bruto / líquido
unit assignment	atribuição de unidades
conversion based unit / derived unit	unidade por conversão / unidade derivada
material layer set / profile set / constituent set	conjunto de camadas / de perfis / de constituintes
classification / classification reference	sistema de classificação / item de classificação
data dictionary	dicionário de dados
document information / document reference	informação de documento / referência a documento
approval	aprovação
template	modelo
type driven override	orientado pelo tipo, com sobrescrita

Tabela 5.17: Vocabulário do capítulo.

5.14 Exercícios

Fixação

1. Por que o IFC usa conjuntos de propriedades, em vez de acrescentar atributos ao esquema?
2. Uma propriedade `FireRating` está no tipo com `EI60` e na ocorrência com `EI90`. Qual valor vale para a ocorrência?
3. Qual tipo de propriedade você usaria para: (a) faixa de temperatura de operação; (b) curva vazão x altura de uma bomba; (c) situação da obra (novo, existente, a demolir)?
4. Por que um conjunto personalizado não pode se chamar `Pset_Tubulacao`?
5. Num arquivo em milímetros, uma quantidade `Length` vale 3500. Quanto é em metros? E uma propriedade `IfcPressureMeasure(250000.)`, sem unidade e sem unidade de pressão declarada no projeto, quanto é em bar?

Aplicação

6. Execute `informacao.py` para a parede `#158` de `Building-Architecture.ifc`. Registre as quantidades e verifique a coerência entre `Length`, `Width`, `NetSideArea` e `NetVolume`. Qual é a altura aproximada da parede?
7. Em `Infra-Landscaping.ifc` (IFC 4.3), procure `SizeSet` no VS Code. A que objeto o conjunto está ligado, e por qual mecanismo (relação ou atributo)? Execute `informacao.py` para uma ocorrência ligada a esse objeto e identifique o conjunto, o material e a classificação. Os valores de `Height` e `Volume` são coerentes entre si, considerando as unidades?

Desafio

8. Escolha um tipo de objeto da sua área de atuação (por exemplo: válvula, luminária, viga metálica, camada de pavimento, porta corta-fogo). Liste oito requisitos de informação que um BEP poderia exigir para ele e monte o mapeamento para o IFC 4.3, como na Tabela 5.16, indicando: atributo, conjunto padronizado (confira o nome na documentação) ou conjunto personalizado; tipo de valor; e se a informação fica no tipo ou na ocorrência.

5.15 Respostas dos exercícios

1. Porque os atributos são fixos e iguais para todos os projetos, enquanto a informação necessária varia com a disciplina, a fase e o cliente. Os conjuntos de propriedades permitem acrescentar informação sem alterar o esquema, e a buildingSMART padroniza os mais comuns (`Pset_`).
2. `EI90`: a ocorrência sobrescreve o valor do tipo quando a propriedade tem o mesmo nome no conjunto de mesmo nome.
3. (a) `IfcPropertyBoundedValue` (limites inferior e superior); (b) `IfcPropertyTableValue` (pares de valores); (c) `IfcPropertyEnumeratedValue` (lista fechada; é o caso de `Status` nos conjuntos padronizados).
4. Porque o prefixo `Pset_` é reservado aos conjuntos da especificação. Um conjunto personalizado com esse prefixo seria confundido com um padronizado e reprovado por validadores. Use um prefixo próprio, como `ARV_Tubulacao`.

5. 3500 mm = 3,5 m. A pressão, sem unidade na propriedade e sem unidade de pressão no projeto, é interpretada em pascal: 250.000 Pa = 2,5 bar.
6. Saída obtida: `NetVolume` 1,269 m³, `Width` 200 (mm), `Length` 1800 (mm) e `NetSideArea` 6,346 m². Coerência: 6,346 m² x 0,200 m = 1,269 m³, igual ao volume. A altura aproximada é a área dividida pelo comprimento: 6,346 / 1,8 = 3,53 m (valor aproximado, porque a área líquida pode ter recortes).
7. A linha que cita `#85` é `#81=IFCGEOGRAPHICELEMENTTYPE(...,'apple',...,(#85),...,.VEGETATION.)`: o conjunto está no **tipo** (atributo `HasPropertySets`), e não ligado por relação. O tipo `#81` tem 48 ocorrências (`IfcGeographicElement` 'apple', as maçãs da árvore). Para a ocorrência `#89`, a saída obtida foi: `[tipo #81] SizeSet` com `Height` = 167.0 (`IfcLengthMeasure`) e `Volume` = 0.004 (`IfcVolumeMeasure`); material `vegetation_apple`; classificação `apple` ('Apple') do sistema "Fruit and vegetables", com URI de um dicionário de demonstração do bSDD. Nas unidades do projeto, a altura é 167 mm e o volume 0,004 m³ (4 litros); um volume de 4 litros corresponde a um cubo de cerca de 16 cm de aresta, compatível com a ordem de grandeza da altura. O exercício mostra que propriedades do tipo valem para todas as ocorrências e que a conferência de unidades é indispensável.
8. Resposta pessoal. Exemplo para uma válvula de bloqueio: TAG em `Name`; tipo `IfcValve` com `IfcValveType` `PredefinedType` = `ISOLATING`; mecanismo em `Pset_ValveTypeCommon.ValveMechanism` (enumerado, no tipo: `WEDGEGATE` para gaveta); acionamento em `Pset_ValveTypeCommon.ValveOperation` (enumerado, no tipo: `WHEEL` para volante); diâmetro em `Pset_ValveTypeCommon.Size` (`IfcPositiveLengthMeasure`, no tipo); pressão de teste em `Pset_ValveTypeCommon.TestPressure` (`IfcPressureMeasure`, no tipo); fabricante em `Pset_ManufacturerTypeInformation.Manufacturer` (no tipo); número de série em `Pset_ManufacturerOccurrence.SerialNumber` (na ocorrência); revisão em conjunto personalizado `ARV_Controle.Revisao` (`IfcLabel`, na ocorrência).

5.16 Referências

- Documentação do IFC 4.3, `IfcPropertySet`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcPropertySet.htm
- Documentação do IFC 4.3, `IfcPropertySingleValue`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcPropertySingleValue.htm
- Documentação do IFC 4.3, `IfcMaterial` e `IfcMaterialList`: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcMaterial.htm
- Documentação do IFC 4.3 (índice geral, com os conjuntos de propriedades e de quantidades padronizados): https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/
- bSDD, portal de busca: https://search.bsdd.buildingsmart.org/
- Recomendações para a implementação da interoperabilidade entre SINAPI e normas da série NBR 15965, *Ambiente Construído*: https://www.scielo.br/j/ac/a/zvZfvx3jCT6MkxrT6XtDjgx/?lang=pt
- CBIC, normas técnicas publicadas em 2022 (NBR 15965-5 e 15965-6): https://cbic.org.br/wp-content/uploads/2023/01/alteracoes-de-normas-tecnicas-setor-da-construcao-civil-2022.pdf

6 Domínios: instalações, estruturas, arquitetura e infraestrutura

Os capítulos anteriores trataram do que é comum a todo modelo IFC: a organização do esquema, a estrutura espacial e a informação não geométrica. Este capítulo percorre as **disciplinas**: que entidades o IFC oferece para instalações e processos industriais, estruturas de concreto e aço, arquitetura e obras de infraestrutura, e como os elementos de uma rede se conectam. O objetivo não é decorar entidades, e sim saber onde procurar e reconhecer as diferenças entre versões que mais afetam a troca de arquivos.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- localizar, na hierarquia do esquema, as entidades de cada disciplina;
- escolher a entidade e o `PredefinedType` adequados a um objeto real;
- representar a conectividade de uma rede com portas e relações de conexão;
- reconhecer como IFC2x3, IFC4 e IFC 4.3 representam elementos de instalações, estruturas e infraestrutura;
- identificar o uso indevido de `IfcBuildingElementProxy` e propor a correção;
- gerar e percorrer, por script, uma rede simples de tubulação.

6.1 Introdução: disciplinas dentro de um único esquema

6.1.1 Um esquema, várias disciplinas

Um mesmo arquivo IFC pode conter a arquitetura, a estrutura, as instalações e a infraestrutura de um empreendimento. Isso só é possível porque todas as disciplinas compartilham a mesma base (Capítulo 3): `IfcRoot`, `IfcProduct`, `IfcElement`, as relações e os conjuntos de propriedades. Sobre essa base, cada disciplina acrescenta entidades específicas.

Na documentação, o esquema é organizado em **quatro camadas**: a de **recursos** (geometria, unidades, materiais, datas, que não derivam de `IfcRoot`), a de **núcleo** (as entidades fundamentais, como `IfcRoot`, `IfcProduct`, `IfcRelationship`), a **compartilhada** (entidades usadas por várias disciplinas, como `IfcWall`, `IfcBeam`, `IfcFlowSegment`) e a de **domínios** (entidades próprias de uma disciplina, como `IfcPump` ou `IfcRail`). Uma entidade de uma camada só pode referenciar entidades da mesma camada ou de camadas inferiores; isso mantém o esquema organizado.

6.1.2 Como procurar a entidade certa

Diante de um objeto real (um filtro de linha, um guarda-corpo, uma defensa metálica), o caminho é:

1. identificar a **família** na hierarquia: é um elemento de distribuição (conduz ou controla fluxo), um elemento construído, um componente, um elemento geográfico?
2. procurar a **entidade** mais específica da família, na versão usada;
3. escolher o **`PredefinedType`** na enumeração da entidade; se nenhum servir, usar `USERDEFINED` com `ObjectType` (Capítulo 3);
4. só em último caso, usar `IfcBuildingElementProxy` (seção 6.7).

A documentação oficial tem um índice alfabético de entidades e, para cada uma, a lista de valores de `PredefinedType` com definições. Os scripts do Capítulo 3 também listam atributos e cadeias de herança de qualquer entidade.

6.2 Instalações e processos: elementos de distribuição

6.2.1 A família de distribuição

Tudo o que conduz, controla, armazena, transforma ou consome um fluxo (água, esgoto, ar, gás, vapor, óleo, produtos químicos, energia elétrica, dados) pertence à família `IfcDistributionElement`. Ela se divide em dois ramos:

- **`IfcDistributionFlowElement`**: elementos pelos quais o fluxo passa;
- **`IfcDistributionControlElement`**: elementos que medem ou comandam o fluxo (sensores, atuadores, controladores).

CATEGORIA (IFC 4.3)	FUNÇÃO	ENTIDADES (SELEÇÃO)
`IfcFlowSegment`	Trecho linear que conduz o fluxo	`IfcPipeSegment`, `IfcDuctSegment`, `IfcCableSegment`, `IfcCableCarrierSegment`, `IfcConveyorSegment`
`IfcFlowFitting`	Conexão que muda direção, divide ou une	`IfcPipeFitting`, `IfcDuctFitting`, `IfcCableFitting`, `IfcCableCarrierFitting`, `IfcJunctionBox`
`IfcFlowController`	Regula ou interrompe o fluxo	`IfcValve`, `IfcDamper`, `IfcFlowMeter`, `IfcSwitchingDevice`, `IfcProtectiveDevice`, `IfcElectricDistributionBoard`
`IfcFlowMovingDevice`	Movimenta o fluido	`IfcPump`, `IfcFan`, `IfcCompressor`
`IfcFlowStorageDevice`	Armazena	`IfcTank`, `IfcElectricFlowStorageDevice`
`IfcFlowTreatmentDevice`	Trata ou separa	`IfcFilter`, `IfcInterceptor` (caixa separadora), `IfcDuctSilencer`
`IfcEnergyConversionDevice`	Converte energia	`IfcHeatExchanger`, `IfcBoiler`, `IfcChiller`, `IfcCoolingTower`, `IfcElectricMotor`, `IfcElectricGenerator`, `IfcTransformer`, `IfcSolarDevice`
`IfcFlowTerminal`	Início ou fim do fluxo, uso final	`IfcSanitaryTerminal`, `IfcAirTerminal`, `IfcLightFixture`, `IfcFireSuppressionTerminal`, `IfcWasteTerminal`, `IfcOutlet`, `IfcSignal`
`IfcDistributionChamberElement`	Caixa ou câmara da rede	Poço de visita, caixa de inspeção, caixa de válvulas
`IfcDistributionControlElement`	Medição e comando	`IfcSensor`, `IfcActuator`, `IfcController`, `IfcAlarm`, `IfcFlowInstrument`

Tabela 6.1: Família de distribuição (IFC 4.3).

6.2.2 Enumerações úteis

As enumerações de `PredefinedType` detalham cada entidade. Seleção de valores do IFC 4.3, conferidos no esquema:

ENTIDADE	VALORES DE `PREDEFINEDTYPE`
`IfcPipeSegment`	`RIGIDSEGMENT`, `FLEXIBLESEGMENT`, `SPOOL`, `GUTTER`, `CULVERT`
`IfcPipeFitting`	`BEND` (curva), `JUNCTION` (tê, derivação), `TRANSITION` (redução), `CONNECTOR` (luva, união, flange), `OBSTRUCTION` (tampão), `ENTRY`, `EXIT`
`IfcValve`	`ISOLATING` (bloqueio), `REGULATING`, `CHECK` (retenção), `PRESSURERELIEF` (alívio), `PRESSUREREDUCING`, `SAFETYCUTOFF`, `DIVERTING`, `MIXING`, `DRAWOFFCOCK`, `FAUCET`, `STEAMTRAP`, entre outros
`IfcPump`	`ENDSUCTION`, `SPLITCASE`, `VERTICALINLINE`, `VERTICALTURBINE`, `SUBMERSIBLEPUMP`, `SUMPPUMP`, `CIRCULATOR`
`IfcTank`	`STORAGE`, `VESSEL`, `PRESSUREVESSEL`, `BASIN`, `BREAKPRESSURE`, `EXPANSION`, `FEEDANDEXPANSION`, `OILRETENTIONTRAY`
`IfcCompressor`	`RECIPROCATING`, `ROTARY`, `SCROLL`, `DYNAMIC`, `TWINSCREW`, entre outros
`IfcHeatExchanger`	`PLATE`, `SHELLANDTUBE` (casco e tubos), `TURNOUTHEATING`
`IfcFilter`	`STRAINER` (filtro de linha), `WATERFILTER`, `OILFILTER`, `COMPRESSEDAIRFILTER`, `AIRPARTICLEFILTER`, `ODORFILTER`
`IfcFlowMeter`	`WATERMETER`, `GASMETER`, `OILMETER`, `ENERGYMETER`
`IfcSensor`	`PRESSURESENSOR`, `TEMPERATURESENSOR`, `FLOWSENSOR`, `LEVELSENSOR`, `GASSENSOR`, `SMOKESENSOR`, entre muitos outros
`IfcActuator`	`ELECTRICACTUATOR`, `PNEUMATICACTUATOR`, `HYDRAULICACTUATOR`, `HANDOPERATEDACTUATOR`, `THERMOSTATICACTUATOR`
`IfcDistributionChamberElement`	`MANHOLE` (poço de visita), `INSPECTIONCHAMBER`, `INSPECTIONPIT`, `VALVECHAMBER`, `METERCHAMBER`, `SUMP`, `TRENCH`, `FORMEDDUCT`
`IfcInterceptor`	`GREASE` (caixa de gordura), `OIL`, `PETROL`, `CYCLONIC`

Tabela 6.2: Enumerações de elementos de distribuição (IFC 4.3).

Note que a enumeração de válvulas descreve a **função** (bloqueio, retenção, alívio), não o **mecanismo** (gaveta, esfera, borboleta, globo). O mecanismo fica na propriedade `ValveMechanism` de `Pset_ValveTypeCommon` (Capítulo 5), com valores como `WEDGEGATE`, `BALL`, `BUTTERFLY` e `GLOBE`.

6.2.3 A grande diferença do IFC2x3

No IFC2x3, as entidades específicas de ocorrência (`IfcPipeSegment`, `IfcValve`, `IfcPump`, `IfcTank`) **não existem**. A ocorrência é uma das categorias genéricas, e a especialização está só no tipo:

OBJETO	IFC2X3: OCORRÊNCIA + TIPO	IFC4 E IFC 4.3: OCORRÊNCIA + TIPO
Tubo	`IfcFlowSegment` + `IfcPipeSegmentType`	`IfcPipeSegment` + `IfcPipeSegmentType`
Conexão	`IfcFlowFitting` + `IfcPipeFittingType`	`IfcPipeFitting` + `IfcPipeFittingType`

OBJETO	IFC2X3: OCORRÊNCIA + TIPO	IFC4 E IFC 4.3: OCORRÊNCIA + TIPO
Válvula	`IfcFlowController` + `IfcValveType`	`IfcValve` + `IfcValveType`
Bomba	`IfcFlowMovingDevice` + `IfcPumpType`	`IfcPump` + `IfcPumpType`
Tanque	`IfcFlowStorageDevice` + `IfcTankType`	`IfcTank` + `IfcTankType`
Trocador de calor	`IfcEnergyConversionDevice` + `IfcHeatExchangerType`	`IfcHeatExchanger` + `IfcHeatExchangerType`
Sensor	`IfcDistributionControlElement` + `IfcSensorType`	`IfcSensor` + `IfcSensorType`

Tabela 6.3: Instalações em IFC2x3 e em IFC4/IFC 4.3.

Consequência prática: num arquivo IFC2x3 **sem tipos**, não há como saber se um `IfcFlowController` é uma válvula ou um registro de ar; a informação se perdeu na exportação. Ao receber modelos de instalações em IFC2x3, verifique sempre se os tipos foram exportados. Esse é mais um motivo para preferir IFC4 ou IFC 4.3 em redes de tubulação.

6.3 Conectividade: portas e conexões

6.3.1 Por que registrar conexões

Uma rede não é só um conjunto de peças: é uma sequência. Saber que o tubo TB-001 sai do tanque e chega à válvula permite traçar o caminho do fluido, localizar a válvula que isola um trecho, somar o comprimento de uma linha ou verificar se há peças soltas. Num modelo sem conexões, essa informação só pode ser deduzida pela proximidade geométrica, com erros.

O IFC registra conexões por meio de **portas**. Uma porta (`IfcDistributionPort`) é um ponto de conexão de um elemento, como um bocal de equipamento ou a extremidade de um tubo. Duas portas de elementos diferentes são ligadas por uma relação `IfcRelConnectsPorts`.

6.3.2 As entidades

ENTIDADE	PAPEL	ATRIBUTOS PRINCIPAIS
`IfcDistributionPort`	Ponto de conexão	`FlowDirection` (`SOURCE`: o fluxo sai; `SINK`: o fluxo entra; `SOURCEANDSINK`; `NOTDEFINED`); no IFC4 e no IFC 4.3, também `PredefinedType` (`PIPE`, `DUCT`, `CABLE`, `CABLECARRIER`, `WIRELESS`) e `SystemType` (a mesma enumeração de `IfcDistributionSystem`)
`IfcRelNests`	Liga as portas ao elemento que as possui (IFC4 e IFC 4.3)	`RelatingObject` (elemento), `RelatedObjects` (portas)
`IfcRelConnectsPortToElement`	Liga a porta ao elemento (IFC2x3)	`RelatingPort`, `RelatedElement`

ENTIDADE	PAPEL	ATRIBUTOS PRINCIPAIS
`IfcRelConnectsPorts`	Liga duas portas	`RelatingPort`, `RelatedPort`, `RealizingElement` (opcional: elemento que realiza a conexão)
`IfcRelFlowControlElements`	Liga sensores e atuadores ao elemento que controlam	`RelatedControlElements`, `RelatingFlowElement`

Tabela 6.4: Entidades de conectividade.

No IFC4 e no IFC 4.3, as portas são aninhadas no elemento com `IfcRelNests`; no IFC2x3, são ligadas com `IfcRelConnectsPortToElement`. A biblioteca IfcOpenShell segue essa regra: sua função de criação de portas usa `IfcRelNests` para IFC4 e IFC 4.3 e `IfcRelConnectsPortToElement` para IFC2x3.

6.3.3 Conectividade explícita e implícita

A documentação do IfcOpenShell distingue dois níveis de detalhe:

- **explícita**: cada peça é conectada à seguinte, exatamente como construída: tanque, tubo, curva, tubo, válvula, tubo, bomba. É a forma adequada ao projeto detalhado e à fabricação;
- **implícita**: só os elementos principais são conectados (por exemplo, o tanque à bomba), e o elemento que realiza a conexão pode ser indicado em `RealizingElement`. É adequada às fases iniciais.

6.3.4 Uma rede de estudo

As amostras da buildingSMART usadas até aqui não têm portas. Para estudar a conectividade, o script abaixo cria um arquivo novo, em IFC 4.3, com uma pequena linha: tanque, tubo, válvula de bloqueio, tubo e bomba, conectados em sequência e reunidos num sistema. Os comandos de criação são explicados no Capítulo 10; aqui, basta executar o script para obter o arquivo de estudo.

Grave-o como `C:\IFC_Curso\exercicios\cap6\rede_exemplo.py`.

```
# -*- coding: utf-8 -*-
"""
rede_exemplo.py | Capítulo 6: cria um arquivo IFC 4.3 de estudo com uma pequena rede de tubulação.

O que faz:
  Cria, do zero, um arquivo novo (nenhum arquivo existente é alterado) com:
  - estrutura espacial: projeto > terreno > edificação "Casa de bombas" > pavimento;
  - um tanque, dois trechos de tubo, uma válvula de bloqueio e uma bomba, com tipos;
  - portas de conexão (IfcDistributionPort) ligando os elementos em sequência;
  - um sistema de distribuição (IfcDistributionSystem) que reúne os elementos.
  Os elementos não têm geometria: o foco é a informação e a conectividade.
  A criação de objetos por código é explicada no Capítulo 10.

Uso (ambiente virtual ativado):
    python rede_exemplo.py                  -> grava rede_exemplo.ifc na pasta atual
    python rede_exemplo.py <saida.ifc>      -> grava em outro caminho

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.api.aggregate
import ifcopenshell.api.project
import ifcopenshell.api.pset
```

```
import ifcopenshell.api.root
import ifcopenshell.api.spatial
import ifcopenshell.api.system
import ifcopenshell.api.type
import ifcopenshell.api.unit

SAIDA = sys.argv[1] if len(sys.argv) > 1 else "rede_exemplo.ifc"

modelo = ifcopenshell.api.project.create_file(version="IFC4X3")
criar = ifcopenshell.api.root.create_entity

# Estrutura espacial
projeto = criar(modelo, ifc_class="IfcProject", name="Rede de estudo")
ifcopenshell.api.unit.assign_unit(modelo)            # unidades padrão: mm, m² e m³
terreno = criar(modelo, ifc_class="IfcSite", name="Unidade industrial")
edificacao = criar(modelo, ifc_class="IfcBuilding", name="Casa de bombas")
pavimento = criar(modelo, ifc_class="IfcBuildingStorey", name="Térreo")
ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)
ifcopenshell.api.aggregate.assign_object(modelo, products=[edificacao], relating_object=terreno)
ifcopenshell.api.aggregate.assign_object(modelo, products=[pavimento], relating_object=edificacao)

# Tipos (o que é comum) e ocorrências (o que é individual)
t_tanque = criar(modelo, ifc_class="IfcTankType", predefined_type="STORAGE", name="Tanque 20 m3")
t_tubo = criar(modelo, ifc_class="IfcPipeSegmentType", predefined_type="RIGIDSEGMENT",
               name="Tubo DN100 aço-carbono")
t_valvula = criar(modelo, ifc_class="IfcValveType", predefined_type="ISOLATING", name="Válvula gaveta DN100")
t_bomba = criar(modelo, ifc_class="IfcPumpType", predefined_type="ENDSUCTION", name="Bomba centrífuga")

tanque = criar(modelo, ifc_class="IfcTank", name="TQ-001")
tubo1 = criar(modelo, ifc_class="IfcPipeSegment", name="TB-001")
valvula = criar(modelo, ifc_class="IfcValve", name="VG-001")
tubo2 = criar(modelo, ifc_class="IfcPipeSegment", name="TB-002")
bomba = criar(modelo, ifc_class="IfcPump", name="B-001")
sequencia = [tanque, tubo1, valvula, tubo2, bomba]

for ocorrencia, tipo in [(tanque, t_tanque), (tubo1, t_tubo), (valvula, t_valvula),
                         (tubo2, t_tubo), (bomba, t_bomba)]:
    ifcopenshell.api.type.assign_type(modelo, related_objects=[ocorrencia], relating_type=tipo)
ifcopenshell.api.spatial.assign_container(modelo, products=sequencia, relating_structure=pavimento)

# Uma propriedade padronizada no tipo do tubo. O valor segue a unidade de comprimento
# do projeto, que é o milímetro: 100.0 significa 100 mm.
pset = ifcopenshell.api.pset.add_pset(modelo, product=t_tubo, name="Pset_PipeSegmentTypeCommon")
ifcopenshell.api.pset.edit_pset(modelo, pset=pset, properties={"NominalDiameter": 100.0})

# Portas: cada elemento recebe uma porta de entrada e uma de saída (o tanque só saída,
# a bomba só entrada, para simplificar). Depois, cada saída é conectada à entrada seguinte.
portas = {}
for elem in sequencia:
    entrada = ifcopenshell.api.system.add_port(modelo, element=elem) if elem is not tanque else None
    saida = ifcopenshell.api.system.add_port(modelo, element=elem) if elem is not bomba else None
    for porta in (entrada, saida):
        if porta is not None:
            porta.PredefinedType = "PIPE"
            porta.SystemType = "CHEMICAL"
    portas[elem] = (entrada, saida)

for anterior, seguinte in zip(sequencia[:-1], sequencia[1:]):
    saida_anterior = portas[anterior][1]
    entrada_seguinte = portas[seguinte][0]
    # direção SOURCE: o fluxo vai da primeira porta (saída) para a segunda (entrada)
    ifcopenshell.api.system.connect_port(modelo, port1=saida_anterior, port2=entrada_seguinte,
                                         direction="SOURCE")

# Sistema de distribuição que reúne os elementos
sistema = ifcopenshell.api.system.add_system(modelo, ifc_class="IfcDistributionSystem")
sistema.Name = "Linha de transferência de produto"
sistema.PredefinedType = "CHEMICAL"
ifcopenshell.api.system.assign_system(modelo, products=sequencia, system=sistema)

modelo.write(SAIDA)
print("Arquivo gravado:", SAIDA, "| esquema:", modelo.schema_identifier,
      "| instâncias:", len(list(modelo)))
```

Execução:

```
cd C:\IFC_Curso\exercicios\cap6
python rede_exemplo.py
```

Saída obtida no teste:

```
Arquivo gravado: rede_exemplo.ifc | esquema: IFC4X3_ADD2 | instâncias: 46
```

O arquivo gerado foi verificado com o validador do IfcOpenShell (`python -m ifcopenshell.validate --rules rede_exemplo.ifc`, Capítulo 12), sem nenhuma violação do esquema nem de suas regras. Trechos do arquivo:

```
#2=IFCSIUNIT(*,.LENGTHUNIT.,.MILLI.,.METRE.);
#12=IFCTANKTYPE('0Jlib2u2b4oww4NEYeLBrX',$,'Tanque 20 m3',$,$,$,$,$,.STORAGE.);
#13=IFCPIPESEGMENTTYPE('2iBt3PiB1Afxf1HfZpTxIQ',$,'Tubo DN100 a\X2\00E7\X0\o-carbono',$,$,(#26),$,$,
$,.RIGIDSEGMENT.);
#16=IFCTANK('06t9a9v0j4OOG69JK$1d7T',$,'TQ-001',$,$,$,$,$,$);
#17=IFCPIPESEGMENT('3iHL5xQvH6jQESlnlbSxJ6',$,'TB-001',$,$,$,$,$,$);
#26=IFCPROPERTYSET('0OGrR1G311pB8uahXwitYi',$,'Pset_PipeSegmentTypeCommon',$,(#27));
#27=IFCPROPERTYSINGLEVALUE('NominalDiameter',$,IFCPOSITIVELENGTHMEASURE(100.),$);
#28=IFCDISTRIBUTIONPORT('1Igg282zf9ze1TrS$xJgLI',$,$,$,$,$,$,.SOURCE.,.PIPE.,.CHEMICAL.);
#29=IFCRELNESTS('1G_2AC0AX8aAGiKNfPS1Ui',$,$,$,#16,(#28));
#30=IFCDISTRIBUTIONPORT('2jCMQRdQz5hhLXi55QbR22',$,$,$,$,$,$,.SINK.,.PIPE.,.CHEMICAL.);
#31=IFCRELNESTS('1gRQwZ_vnBqOkhlzgHpMHn',$,$,$,#17,(#32,#30));
#32=IFCDISTRIBUTIONPORT('2v0Znv8gP7auj2$vTZhiIl',$,$,$,$,$,$,.SOURCE.,.PIPE.,.CHEMICAL.);
#41=IFCRELCONNECTSPORTS('3eagC8Uy1EQf8VAEGl6Cgz',$,$,$,#28,#30,$);
#42=IFCRELCONNECTSPORTS('0U_cFOu7nBReAgkv__pawn',$,$,$,#32,#33,$);
#45=IFCDISTRIBUTIONSYSTEM('1fNM_e0oDB_RWb2FaqUkY4',$,'Linha de transfer\X2\00EA\X0\ncia de produto',$,$,
$,.CHEMICAL.);
#46=IFCRELASSIGNSTOGROUP('0Jnpf_xkbAcPgynt9yiB_Z',$,$,$,(#16,#17,#18,#19,#20),$,#45);
```

Figura 6.1: Tanque e primeiro tubo conectados por portas (os GlobalIds mudam a cada execução).

Leitura: o tanque `#16` aninha (`#29`) a porta de saída `#28` (`SOURCE`); o tubo `#17` aninha (`#31`) a porta de entrada `#30` (`SINK`) e a de saída `#32`; a relação `#41` liga a saída do tanque à entrada do tubo; a relação `#42` liga a saída do tubo à entrada da válvula (`#33`). O `OwnerHistory` vazio (`$`) é válido no IFC 4.3.

6.3.5 Percorrer a rede

O segundo script lê o arquivo, lista as portas de cada elemento e percorre a rede seguindo o fluxo: da porta de saída de um elemento para a porta de entrada do seguinte. Ele usa o módulo `ifcopenshell.util.system`, que traz funções prontas para portas (`get_connected_port`, `get_port_element`). Grave-o como `C:\IFC_Curso\exercicios\cap6\percorrer_rede.py`.

```
# -*- coding: utf-8 -*-
"""
percorrer_rede.py | Capítulo 6: percorre uma rede de distribuição pelas portas de conexão.

O que faz (somente leitura; nada é gravado):
  1. Lista, para cada elemento de distribuição, as portas (IfcDistributionPort) com
     a direção do fluxo e o elemento conectado do outro lado (IfcRelConnectsPorts).
  2. Começa por um elemento sem porta de entrada conectada e segue o fluxo
     (porta de saída -> porta de entrada do elemento seguinte), mostrando a sequência.

Uso (ambiente virtual ativado):
    python percorrer_rede.py                  -> usa rede_exemplo.ifc da pasta atual
    python percorrer_rede.py <arquivo.ifc>    -> outro arquivo (IFC4 ou IFC4X3)

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
```

```
"""
import sys
import ifcopenshell
import ifcopenshell.util.element as util_elem
import ifcopenshell.util.system as util_sistema

arquivo = sys.argv[1] if len(sys.argv) > 1 else "rede_exemplo.ifc"
modelo = ifcopenshell.open(arquivo)
print("Arquivo:", arquivo, "| esquema:", modelo.schema_identifier)


def rotulo(elem):
    tipo = util_elem.get_type(elem)
    especial = tipo.PredefinedType if tipo is not None else getattr(elem, "PredefinedType", None)
    return f"{elem.Name} ({elem.is_a()}, {especial})"


def portas_de(elem):
    """Portas aninhadas no elemento (IfcRelNests, IFC4 e IFC4.3)."""
    portas = [p for rel in elem.IsNestedBy for p in rel.RelatedObjects if p.is_a("IfcDistributionPort")]
    return sorted(portas, key=lambda p: p.id())          # ordem estável: pelo número da instância


print("\n=== 1. PORTAS E CONEXÕES ===")
elementos = sorted((e for e in modelo.by_type("IfcDistributionElement") if portas_de(e)),
                   key=lambda e: e.Name or "")
for elem in elementos:
    print(rotulo(elem))
    for porta in portas_de(elem):
        conectada = util_sistema.get_connected_port(porta)
        outro = util_sistema.get_port_element(conectada).Name if conectada else "nenhum"
        print(f"    porta #{porta.id()} {porta.FlowDirection:<6} -> conectada a: {outro}")

print("\n=== 2. SEQUÊNCIA DO FLUXO ===")


def seguinte(elem):
    """Elemento ligado à porta de saída (SOURCE) deste elemento."""
    for porta in portas_de(elem):
        if porta.FlowDirection == "SOURCE":
            conectada = util_sistema.get_connected_port(porta)
            if conectada:
                return util_sistema.get_port_element(conectada)
    return None


def tem_entrada(elem):
    return any(p.FlowDirection == "SINK" and util_sistema.get_connected_port(p) for p in portas_de(elem))


inicios = [e for e in elementos if not tem_entrada(e)]
for inicio in inicios:
    atual, passo = inicio, 1
    while atual is not None and passo <= 1000:      # limite de segurança contra ciclos
        print(f"   {passo}. {rotulo(atual)}")
        atual, passo = seguinte(atual), passo + 1
```

Execução e saída obtida no teste:

```
python percorrer_rede.py
```

```
Arquivo: rede_exemplo.ifc | esquema: IFC4X3_ADD2

=== 1. PORTAS E CONEXÕES ===
B-001 (IfcPump, ENDSUCTION)
    porta #39 SINK   -> conectada a: TB-002
TB-001 (IfcPipeSegment, RIGIDSEGMENT)
    porta #30 SINK   -> conectada a: TQ-001
    porta #32 SOURCE -> conectada a: VG-001
TB-002 (IfcPipeSegment, RIGIDSEGMENT)
```

```
    porta #36 SINK   -> conectada a: VG-001
    porta #38 SOURCE -> conectada a: B-001
TQ-001 (IfcTank, STORAGE)
    porta #28 SOURCE -> conectada a: TB-001
VG-001 (IfcValve, ISOLATING)
    porta #33 SINK   -> conectada a: TB-001
    porta #35 SOURCE -> conectada a: TB-002

=== 2. SEQUÊNCIA DO FLUXO ===
   1. TQ-001 (IfcTank, STORAGE)
   2. TB-001 (IfcPipeSegment, RIGIDSEGMENT)
   3. VG-001 (IfcValve, ISOLATING)
   4. TB-002 (IfcPipeSegment, RIGIDSEGMENT)
   5. B-001 (IfcPump, ENDSUCTION)
```

O mesmo raciocínio serve para redes de esgoto, de água, de gás, de dutos de ar ou de cabos elétricos. A qualidade do resultado depende do exportador: muitos programas não exportam portas, e outros exportam portas sem conexão. Verificar se as portas existem e estão conectadas é uma boa pergunta para um IDS (Capítulo 12).

6.4 Estruturas de concreto e aço

6.4.1 Elementos estruturais

ENTIDADE	USO	`PREDEFINEDTYPE` (SELEÇÃO, IFC 4.3)
`IfcBeam`	Viga	`BEAM`, `JOIST` (barrote), `LINTEL` (verga), `GIRDER_SEGMENT`, `EDGEBEAM`, `PIERCAP` (travessa de pilar), `T_BEAM`, `HOLLOWCORE`, `SPANDREL`, `DIAPHRAGM`
`IfcColumn`	Pilar	`COLUMN`, `PILASTER`, `PIERSTEM` (fuste de pilar de ponte), `STANDCOLUMN`
`IfcMember`	Barra estrutural secundária ou de treliça	`BRACE` (contraventamento), `CHORD` (banzo), `STRUT`, `TIEBAR` (tirante), `PURLIN` (terça), `RAFTER` (caibro), `POST`, `STUD`, `MULLION` (montante de fachada), `STRINGER`, `STAY_CABLE`, `SUSPENSION_CABLE`
`IfcPlate`	Chapa	`BASE_PLATE` (placa de base), `GUSSET_PLATE` (chapa de ligação), `STIFFENER_PLATE` (enrijecedor), `SPLICE_PLATE` (tala de emenda), `FLANGE_PLATE`, `WEB_PLATE`, `COVER_PLATE`, `SHEET`, `CURTAIN_PANEL`
`IfcSlab`, `IfcWall`	Laje, parede ou muro	Capítulos 3 e 5
`IfcFooting`	Fundação rasa e blocos	`PAD_FOOTING` (sapata isolada), `STRIP_FOOTING` (sapata corrida), `PILE_CAP` (bloco sobre estacas), `FOOTING_BEAM` (viga de fundação), `CAISSON_FOUNDATION` (tubulão)
`IfcPile`	Estaca	`DRIVEN` (cravada), `BORED` (escavada), `JETGROUTING`, `FRICTION`, `SUPPORT`, `COHESION`
`IfcReinforcingBar`, `IfcReinforcingMesh`	Armadura em barras e telas	

ENTIDADE	USO	PREDEFINEDTYPE (SELEÇÃO, IFC 4.3)
IfcTendon, IfcTendonAnchor	Cabos de protensão e ancoragens	
IfcMechanicalFastener	Elemento de fixação	BOLT (parafuso), ANCHORBOLT (chumbador), RIVET (rebite), SCREW, DOWEL, COUPLER (luva de emenda), SHEARCONNECTOR e STUDSHEARCONNECTOR (conectores de cisalhamento)
IfcDiscreteAccessory	Acessório	Peças de fixação, suportes, apoios
IfcElementAssembly	Conjunto montado	TRUSS (treliça), RIGID_FRAME (pórtico), BRACED_FRAME, BEAM_GRID, SUPPORTINGASSEMBLY (suporte)

Tabela 6.5: Elementos estruturais.

> **ATENÇÃO**
>
> A tabela mostra os valores do IFC 4.3. As enumerações mudam entre versões: CAISSON_FOUNDATION não existe em IfcFooting no IFC2x3, e IfcMechanicalFastener só tem PredefinedType a partir do IFC4. Peças sem valor próprio, como porcas e arruelas, usam USERDEFINED com ObjectType. Sempre confira os valores na versão usada, com o script do Capítulo 3.

6.4.2 Perfis e materiais

Em estruturas metálicas, a informação essencial de uma barra é o **perfil**: I, H, U, L, tubo, caixão. No IFC4 e no IFC 4.3, o perfil é associado como material, por IfcMaterialProfileSet, e a geometria é gerada por extrusão do mesmo perfil (Capítulo 7). O exemplo do pilar IPE 200 em aço S355JR (Figura 5.8) mostra o padrão: o conjunto de perfis no tipo, o uso com o ponto de referência na ocorrência. No IFC2x3, a associação por conjunto de perfis não existe; o perfil só aparece na geometria, e o nome do perfil costuma ser gravado como texto.

Os perfis paramétricos do IFC incluem IfcIShapeProfileDef (I e H), IfcUShapeProfileDef (U), IfcLShapeProfileDef (cantoneira), IfcTShapeProfileDef (T), IfcCShapeProfileDef (U enrijecido), IfcRectangleProfileDef, IfcRectangleHollowProfileDef (tubo retangular), IfcCircleProfileDef e IfcCircleHollowProfileDef (tubo circular).

6.4.3 Escadas, guarda-corpos e plataformas

Estruturas metálicas industriais têm escadas, guarda-corpos e plataformas:

- **IfcStair** representa a escada inteira e **agrega** os lances (IfcStairFlight), patamares (em geral IfcSlab com PredefinedType = LANDING) e guarda-corpos. IfcStairFlight tem atributos próprios: NumberOfRisers (número de espelhos), NumberOfTreads (número de pisos), RiserHeight e TreadLength. No IFC2x3, o primeiro chama-se NumberOfRiser, no singular;
- **IfcRailing** representa guarda-corpos e corrimãos, com PredefinedType GUARDRAIL, HANDRAIL, BALUSTRADE ou FENCE;
- **escadas de marinheiro** e **plataformas** não têm entidade específica; usam-se IfcStair ou IfcElementAssembly, com USERDEFINED e ObjectType descritivo, conforme combinado no BEP;
- **grades de piso** podem ser IfcPlate ou IfcSlab, conforme a função.

6.4.4 Versões

TEMA	IFC2X3	IFC4	IFC 4.3
`PredefinedType` em `IfcBeam`, `IfcColumn`, `IfcMember`, `IfcPlate`	Só no tipo	Na ocorrência e no tipo	Na ocorrência e no tipo, com valores de pontes
Entidades "*StandardCase*"	`IfcWallStandardCase`	`IfcWallStandardCase`, `IfcBeamStandardCase`, `IfcColumnStandardCase` e outras	`IfcBeamStandardCase` e `IfcColumnStandardCase` removidas; `IfcWallStandardCase` ainda consta do esquema
Perfil como material	Não	`IfcMaterialProfileSet`	`IfcMaterialProfileSet`
Fundações profundas	`IfcPile`	`IfcPile`	`IfcDeepFoundation` (supertipo), `IfcPile`, `IfcCaissonFoundation`

Tabela 6.6: Estruturas por versão.

As entidades *StandardCase* do IFC4 indicavam elementos com geometria padronizada (extrusão com conjunto de materiais). Para arquivos novos, prefira a entidade geral (`IfcWall`, `IfcBeam`, `IfcColumn`) com `PredefinedType`, que existe em todas as versões recentes.

6.5 Arquitetura

6.5.1 Elementos

ENTIDADE	USO
`IfcWall`	Paredes e muros (`SOLIDWALL`, `PARTITIONING`, `PARAPET`, `SHEAR`, `RETAININGWALL` no IFC 4.3)
`IfcSlab`	Lajes, pisos, coberturas planas, patamares (`FLOOR`, `ROOF`, `LANDING`, `BASESLAB`)
`IfcRoof`	Cobertura como conjunto; agrega lajes, vigas e placas
`IfcDoor`, `IfcWindow`	Portas e janelas
`IfcCovering`	Revestimentos, forros, pisos acabados, isolamentos (`CEILING`, `FLOORING`, `CLADDING`, `ROOFING`, `INSULATION`, `MEMBRANE`, `SKIRTINGBOARD`...)
`IfcCurtainWall`	Fachada-cortina; agrega montantes (`IfcMember` `MULLION`) e painéis (`IfcPlate` `CURTAIN_PANEL`)
`IfcStair`, `IfcRamp`, `IfcRailing`	Circulação vertical e proteção
`IfcFurniture`	Mobiliário
`IfcSpace`	Ambientes (Capítulo 4)

Tabela 6.7: Elementos de arquitetura.

6.5.2 Aberturas

Uma porta não "fura" a parede por si só. O IFC representa o furo como um elemento próprio, a **abertura** (`IfcOpeningElement`), com `PredefinedType` `OPENING` (atravessa o elemento) ou `RECESS` (rebaixo, não atravessa). Duas relações completam o padrão:

- `IfcRelVoidsElement` liga a abertura ao elemento que ela fura (`RelatingBuildingElement` , `RelatedOpeningElement`);
- `IfcRelFillsElement` liga a porta ou janela à abertura que ela preenche (`RelatingOpeningElement` , `RelatedBuildingElement`).

Exemplo real, uma laje com um furo e um rebaixo (modelos de especificação, IFC 4.3):

```
#303= IFCSLAB('1wAj$J2Az2V8wnBiVYd3bU',#56,$,$,$,#304,#315,$,$);
#325= IFCOPENINGELEMENT('15RSTHd8nFVQWMRE7og7sd',#56,'Opening',$,$,#327,#324,$,$);
#328= IFCRELVOIDSELEMENT('0gqEDsyEzFXvY$fc_rUxyO',#56,$,$,#303,#325);
#336= IFCOPENINGELEMENT('0w93HZ19H2D99zbAVNb4o2',#56,'Recess',$,$,#338,#335,$,.RECESS.);
#339= IFCRELVOIDSELEMENT('3iUkij4q1DmxlXuHzQVJaM',#56,$,$,#303,#336);
```

Figura 6.2: Aberturas numa laje (`slab-openings.ifc`).

O mesmo padrão serve para furos de passagem de tubulação em lajes e paredes, informação importante para a compatibilização entre disciplinas. No IFC4 e no IFC 4.3, a relação `IfcRelVoidsElement` é um subtipo de `IfcRelDecomposes` ; no IFC2x3, de `IfcRelConnects` .

6.6 Infraestrutura (IFC 4.3)

O IFC 4.3 acrescentou entidades para obras lineares e marítimas. As principais, conferidas no esquema:

ÁREA	ENTIDADES	OBSERVAÇÕES
Pavimentação	`IfcCourse` (camada: `PAVEMENT` , `CORE` , `FILTER` , `PROTECTION` , `ARMOUR` , `BALLASTBED`), `IfcPavement` (`FLEXIBLE` , `RIGID`), `IfcKerb` (meio-fio)	A camada é o elemento principal de quantificação
Terraplenagem	`IfcEarthworksCut` (corte, escavação: `CUT` , `EXCAVATION` , `TRENCH` , `TOPSOILREMOVAL` , `PAVEMENTMILLING` ...), `IfcEarthworksFill` (aterro)	O corte é um elemento de subtração, como uma abertura
Sinalização	`IfcSign` (placa: `MARKER` , `MIRROR` , `PICTORAL`), `IfcSignal` (sinal luminoso, ferroviário), `IfcSurfaceFeature` (marcações no pavimento: `LINEMARKING` , `SYMBOLMARKING` , `RUMBLESTRIP` ...)	
Ferrovia	`IfcRail` (trilho: `RAIL` , `CHECKRAIL` , `GUARDRAIL` , `BLADE` , `STOCKRAIL` ...), `IfcTrackElement` (`SLEEPER` dormente, `FROG` jacaré, `VEHICLESTOP` ...), `IfcCourse` `BALLASTBED` (lastro)	Ver Capítulo 4, exercício 7
Pontes	`IfcBearing` (aparelho de apoio: `ELASTOMERIC` , `POT` , `SPHERICAL` , `GUIDE` ...), `IfcBeam` `GIRDER_SEGMENT` e `PIERCAP` , `IfcColumn` `PIERSTEM` , `IfcDeepFoundation` , `IfcTendon`	Partes da ponte por `IfcBridgePart` (Capítulo 4)

ÁREA	ENTIDADES	OBSERVAÇÕES
Geotecnia	`IfcBorehole` (sondagem), `IfcGeotechnicalStratum` (camada de solo), `IfcGeomodel`, `IfcGeoslice`	
Terreno e paisagismo	`IfcGeographicElement` (`TERRAIN`, `VEGETATION`, `SOIL_BORING_POINT`)	Existe desde o IFC4
Drenagem	`IfcPipeSegment` `CULVERT` e `GUTTER`, `IfcDistributionChamberElement` `MANHOLE`, `IfcInterceptor`	Família de distribuição (seção 6.2)
Obras marítimas	`IfcMooringDevice` (dispositivos de amarração), `IfcNavigationElement`, `IfcImpactProtectionDevice` (defensas)	

Tabela 6.8: Entidades de infraestrutura (IFC 4.3).

Túneis não têm entidades próprias no IFC 4.3; a extensão para túneis está em desenvolvimento pela buildingSMART para versões futuras.

6.7 IfcBuildingElementProxy: o último recurso

`IfcBuildingElementProxy` é a entidade genérica para elementos que não se enquadram em nenhuma outra. É útil, por exemplo, para um objeto realmente sem classe (uma escultura) ou para reservas de espaço (`PredefinedType` `PROVISIONFORSPACE` e `PROVISIONFORVOID`, no IFC4 e no IFC 4.3). O problema é o uso **indevido**: exportadores mal configurados gravam como *proxy* tudo o que não sabem classificar, e o arquivo perde a semântica.

Todas as amostras de certificação da buildingSMART têm ao menos um *proxy*: o elemento que representa o entorno (terreno com vegetação estilizada). É um uso aceitável. Já um modelo industrial em que tubos, válvulas e bombas chegam como *proxies* não serve para nada além de visualização.

Como evitar:

- configurar o mapeamento de classes no exportador (Capítulo 13);
- quando o software não permite, corrigir a classe por script, preservando o GlobalId (Capítulo 10);
- verificar com IDS que nenhum elemento de uma disciplina foi exportado como *proxy* (Capítulo 12).

Exemplo resolvido 6.1: classificar objetos reais

Problema. Escolha entidade e `PredefinedType` (IFC 4.3) para: (a) filtro tipo Y numa linha de água; (b) curva de 90° de tubo; (c) redução concêntrica; (d) válvula de retenção; (e) poço de visita de esgoto; (f) chapa de base de pilar metálico; (g) chumbador; (h) guarda-corpo de plataforma; (i) defensa metálica de rodovia; (j) dormente.

Solução.

OBJETO	ENTIDADE	`PREDEFINEDTYPE`
(a) Filtro Y	`IfcFilter`	`STRAINER`
(b) Curva 90°	`IfcPipeFitting`	`BEND`
(c) Redução	`IfcPipeFitting`	`TRANSITION`

OBJETO	ENTIDADE	PREDEFINEDTYPE
(d) Válvula de retenção	`IfcValve`	`CHECK`
(e) Poço de visita	`IfcDistributionChamberElement`	`MANHOLE`
(f) Chapa de base	`IfcPlate`	`BASE_PLATE`
(g) Chumbador	`IfcMechanicalFastener`	`ANCHORBOLT`
(h) Guarda-corpo	`IfcRailing`	`GUARDRAIL`
(i) Defensa metálica	`IfcRailing`	`GUARDRAIL` (ou `USERDEFINED` com `ObjectType` "Defensa metálica", se o BEP exigir distinção)
(j) Dormente	`IfcTrackElement`	`SLEEPER`

Tabela 6.9: Classificação de objetos reais.

O ângulo da curva, o diâmetro da redução e a classe de pressão não vão no `PredefinedType`: vão em propriedades (Capítulo 5).

6.8 Resumo do capítulo

- Todas as disciplinas compartilham a base do esquema; cada uma acrescenta entidades próprias, organizadas em famílias.
- Instalações e processos usam a família `IfcDistributionElement`: segmentos, conexões, controladores, movimentadores, armazenadores, tratamento, conversão de energia, terminais, câmaras e controle.
- No IFC2x3, a ocorrência é genérica (`IfcFlowSegment`, `IfcFlowController`...) e a especialização está só no tipo. Sem tipos, a semântica se perde.
- A conectividade usa portas (`IfcDistributionPort`), aninhadas no elemento (`IfcRelNests`, IFC4+) e ligadas por `IfcRelConnectsPorts`.
- Estruturas usam `IfcBeam`, `IfcColumn`, `IfcMember`, `IfcPlate`, fundações, armaduras e fixações; perfis são materiais (`IfcMaterialProfileSet`) no IFC4+.
- Aberturas são elementos próprios (`IfcOpeningElement`) com `IfcRelVoidsElement` e `IfcRelFillsElement`.
- O IFC 4.3 trouxe pavimentação, terraplenagem, ferrovia, pontes, geotecnia e obras marítimas.
- `IfcBuildingElementProxy` é o último recurso; seu uso indevido destrói a utilidade do modelo.

6.9 Vocabulário técnico

INGLÊS	PORTUGUÊS
distribution element	elemento de distribuição
flow segment / fitting / controller	trecho / conexão / controlador de fluxo

INGLÊS	PORTUGUÊS
flow moving / storage / treatment device	dispositivo de movimentação / armazenamento / tratamento
energy conversion device	dispositivo de conversão de energia
flow terminal	terminal (ponto de consumo ou saída)
port / source / sink	porta / origem (saída) / destino (entrada)
strainer / check valve / isolating valve	filtro de linha / válvula de retenção / válvula de bloqueio
manhole / interceptor	poço de visita / caixa separadora
brace / chord / purlin / gusset plate / base plate	contraventamento / banzo / terça / chapa de ligação / chapa de base
anchor bolt / fastener	chumbador / elemento de fixação
stair flight / riser / tread / landing	lance de escada / espelho / piso / patamar
opening / recess	abertura / rebaixo
course / kerb / earthworks cut and fill	camada / meio-fio / corte e aterro
sleeper / rail / bearing / borehole	dormente / trilho / aparelho de apoio / sondagem
proxy	substituto genérico

Tabela 6.10: Vocabulário do capítulo.

6.10 Exercícios

Fixação

1. Qual é a diferença entre `IfcDistributionFlowElement` e `IfcDistributionControlElement`? Dê dois exemplos de cada.
2. Num arquivo IFC2x3, como se reconhece que um `IfcFlowController` é uma válvula?
3. Qual a diferença entre `SOURCE` e `SINK` numa porta?
4. Onde fica a informação de que uma válvula é do tipo gaveta: no `PredefinedType` ou numa propriedade? Por quê?
5. Quais são as duas relações que ligam uma porta a uma parede, passando pela abertura?

Aplicação

6. Execute `rede_exemplo.py` e `percorrer_rede.py`. Em seguida, abra `rede_exemplo.ifc` no VS Code, sem alterá-lo, e responda: quantas instâncias `IFCDISTRIBUTIONPORT` e quantas `IFCRELCONNECTSPORTS` existem? A contagem é coerente com a rede de cinco elementos?
7. Faça uma cópia de `rede_exemplo.py` com o nome `rede_exemplo_filtro.py` e altere a cópia para incluir um filtro de linha (`IfcFilter`, tipo `STRAINER`, nome "FT-001") entre a válvula e o segundo tubo. Execute e confirme a nova sequência com `percorrer_rede.py`.

Desafio

8. Um fornecedor entrega o modelo de uma estação de bombeamento em IFC2x3, exportado sem tipos. Todos os equipamentos chegam como `IfcFlowMovingDevice`, `IfcFlowController` e `IfcFlowSegment` genéricos, sem portas. Liste os problemas que isso causa para (a) a quantificação por tipo de equipamento, (b) a verificação de requisitos do BEP e (c) a operação e manutenção, e proponha o que exigir numa nova entrega.

6.11 Respostas dos exercícios

1. Os elementos de fluxo são percorridos pelo fluxo (tubo, válvula, bomba, tanque, luminária); os de controle medem ou comandam o fluxo sem que ele passe por eles (sensor de pressão, atuador de válvula, controlador).
2. Pelo tipo associado: a ocorrência `IfcFlowController` está ligada, por `IfcRelDefinesByType`, a um `IfcValveType`. Sem o tipo, não é possível saber.
3. `SOURCE` indica que o fluxo sai do elemento pela porta; `SINK`, que o fluxo entra. `SOURCEANDSINK` indica fluxo nos dois sentidos, e `NOTDEFINED`, direção ainda não definida.
4. Numa propriedade: `Pset_ValveTypeCommon.ValveMechanism` = `WEDGEGATE`. O `PredefinedType` de `IfcValve` descreve a função (bloqueio, retenção, alívio), e uma válvula gaveta e uma válvula esfera podem ter a mesma função (`ISOLATING`).
5. `IfcRelVoidsElement` (a abertura fura a parede) e `IfcRelFillsElement` (a porta preenche a abertura).
6. Há 8 portas (o tanque tem 1, a bomba tem 1, e os três elementos intermediários têm 2 cada) e 4 relações `IfcRelConnectsPorts`, uma para cada ligação entre elementos vizinhos (tanque-tubo, tubo-válvula, válvula-tubo, tubo-bomba). A contagem é coerente: cinco elementos em linha formam quatro ligações, e cada ligação usa duas portas.
7. Na cópia, criar o tipo `t_filtro = criar(modelo, ifc_class="IfcFilterType", predefined_type="STRAINER", name="Filtro Y DN100")`, a ocorrência `filtro = criar(modelo, ifc_class="IfcFilter", name="FT-001")`, incluir o par `(filtro, t_filtro)` na lista de tipos e mudar a sequência para `[tanque, tubo1, valvula, filtro, tubo2, bomba]`. O restante do script (portas, conexões e sistema) usa a lista e se ajusta sozinho. A saída de `percorrer_rede.py` passa a mostrar seis passos, com `FT-001 (IfcFilter, STRAINER)` na quarta posição. A alteração foi testada.
8. (a) Sem tipos, não se distingue bomba de ventilador ou compressor, nem válvula de registro de ar ou medidor: a quantificação por equipamento é impossível sem abrir cada objeto e interpretar nomes. (b) Requisitos do BEP associados a classes (por exemplo, "toda bomba deve ter vazão e potência") não podem ser verificados, porque não há como selecionar as bombas; a validação por IDS falha ou fica sem objetos aplicáveis. (c) Na operação, não há conectividade para isolar trechos, rastrear o fluxo ou ligar sensores a equipamentos. O que exigir: exportação em IFC4 ou IFC 4.3, com as entidades específicas (`IfcPump`, `IfcValve`, `IfcPipeSegment`); tipos exportados com `PredefinedType`; portas conectadas; sistemas por serviço; e um IDS anexado ao contrato, verificado antes do aceite.

6.12 Referências

- Documentação do IFC 4.3, índice de entidades: https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/

- Documentação do IfcOpenShell, `ifcopenshell.api.system` e `ifcopenshell.util.system`: https://docs.ifcopenshell.org/
- Modelos de especificação da buildingSMART (`IFC4.x-specification-models`), pasta `building-elements`

7 Geometria e posicionamento

Até aqui, os objetos foram tratados como informação. Este capítulo mostra como o IFC descreve **onde** cada objeto está e **qual forma** ele tem, e como o modelo se liga a coordenadas reais do terreno. São três assuntos que costumam gerar os problemas mais visíveis na troca de arquivos: modelos deslocados, girados, em escala errada ou com geometria pesada demais.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar a separação entre posicionamento e forma;
- ler e compor uma cadeia de posicionamentos locais;
- reconhecer contextos, representações e os principais tipos de geometria;
- escolher o tipo de geometria adequado a cada caso (extrusão, varredura, malha, geometria mapeada);
- entender como alinhamentos e posicionamento linear funcionam no IFC 4.3;
- interpretar e verificar o georreferenciamento de um arquivo, com sistemas de referência usados no Brasil.

7.1 Introdução: onde está e que forma tem

7.1.1 A lógica do bloco

Quem usa AutoCAD conhece a lógica do **bloco**: a peça é desenhada uma vez, em torno de um ponto base, e depois inserida no desenho com um ponto de inserção, uma rotação e uma escala. A forma fica na definição do bloco; a posição fica na inserção.

O IFC segue a mesma lógica para todo produto (`IfcProduct`), com dois atributos independentes (Capítulo 3):

- **`ObjectPlacement`**: a posição e a orientação do sistema de coordenadas do objeto;
- **`Representation`**: a forma, descrita no sistema de coordenadas do próprio objeto.

Separar as duas coisas traz vantagens: mover um objeto é alterar só o posicionamento; objetos iguais podem compartilhar a mesma forma; e a forma pode ter várias representações (um eixo, um volume, uma caixa envolvente) sem repetir a posição.

7.1.2 Três sistemas de coordenadas

SISTEMA	O QUE É	UNIDADE
Local do objeto	O sistema em que a forma é descrita	A unidade de comprimento do projeto (Capítulo 5)
Do projeto	O sistema de referência de todo o modelo, também chamado de sistema de engenharia ou WCS (*world coordinate system*)	A unidade de comprimento do projeto
De mapa	Um sistema de referência cartográfico, com coordenadas como UTM (este, norte, altitude)	A unidade do sistema cartográfico, em geral o metro

Tabela 7.1: Sistemas de coordenadas.

O posicionamento leva do sistema local ao do projeto; o georreferenciamento (seção 7.6) leva do sistema do projeto ao de mapa.

7.2 Posicionamento local

7.2.1 IfcAxis2Placement3D: origem e eixos

Um sistema de coordenadas em 3D fica definido por uma origem e pela direção de dois eixos. No IFC, isso é `IfcAxis2Placement3D`:

ATRIBUTO	CONTEÚDO	VALOR PADRÃO, SE `$`
`Location`	Origem (`IfcCartesianPoint`)	Obrigatório
`Axis`	Direção do eixo Z local (`IfcDirection`)	(0, 0, 1)
`RefDirection`	Direção do eixo X local	(1, 0, 0)

Tabela 7.2: Atributos de `IfcAxis2Placement3D`.

O eixo Y é deduzido dos outros dois (é perpendicular a ambos). Uma rotação em planta é expressa pelo eixo X: `RefDirection` = (0,5; 0,866; 0) significa que o eixo X local está girado 60° no sentido anti-horário em relação ao eixo X do sistema de referência (cos 60° = 0,5; sen 60° = 0,866).

7.2.2 IfcLocalPlacement: posição relativa

`IfcLocalPlacement` tem dois atributos: `PlacementRelTo` (o posicionamento em relação ao qual este é definido) e `RelativePlacement` (o `IfcAxis2Placement3D`). Se `PlacementRelTo` é `$`, a posição é relativa ao sistema do projeto.

Na prática, cada objeto é posicionado em relação ao objeto que o contém: o ambiente em relação ao pavimento, o pavimento em relação à edificação, a edificação em relação ao terreno. Mover a edificação inteira é alterar um único posicionamento.

Exemplo resolvido 7.1: compor uma cadeia de posicionamentos

Problema. Onde está, no sistema do projeto, a origem da sala `#85` de `Building-Architecture.ifc` (IFC 4.3, unidade milímetro)?

Solução. A cadeia, obtida com o script da seção 7.7:

```
#87 IfcLocalPlacement de IfcSpace 'living room': origem local (200.0, 2000.0, 0.0) | eixo X local (1.0, 0.0, 0.0)
  #47 IfcLocalPlacement de IfcBuildingStorey '00 groundfloor': origem local (0.0, 0.0, 0.0) | eixo X local (1.0,
0.0, 0.0)
    #40 IfcLocalPlacement de IfcBuilding 'Single-family house': origem local (-2800.0, -2800.0, 1300.0) | eixo X
local (1.0, 0.0, 0.0)
      #30 IfcLocalPlacement de IfcSite 'house - site': origem local (0.0, 40000.0, 0.0) | eixo X local (0.5,
0.866, 0.0)
        #23 IfcLocalPlacement de IfcSite 'environment - site': origem local (-28841.016, -14200.0, -1300.0) |
eixo X local (0.5, -0.866, 0.0)
```

Figura 7.1: Cadeia de posicionamentos da sala.

A composição é feita de cima para baixo (do terreno principal para a sala):

1. **Terreno principal (`#23`)**: origem em (-28841,016; -14200; -1300), relativa ao projeto. O eixo X está girado -60° (0,5; -0,866). O eixo Y, perpendicular, fica em (0,866; 0,5).
2. **Terreno da casa (`#30`)**: origem local (0; 40000; 0), isto é, 40.000 mm ao longo do eixo Y do terreno principal: (-28841,016 + 40000 x 0,866; -14200 + 40000 x 0,5; -1300) = (5800; 5800; -1300). O eixo X deste terreno está girado +60° em relação ao anterior, o que anula a rotação: a partir daqui, os eixos coincidem com os do projeto.
3. **Edificação (`#40`)**: (5800 - 2800; 5800 - 2800; -1300 + 1300) = (3000; 3000; 0).
4. **Pavimento (`#47`)**: sem deslocamento: (3000; 3000; 0).
5. **Sala (`#87`)**: (3000 + 200; 3000 + 2000; 0) = **(3200; 5000; 0)**, isto é, (3,2 m; 5,0 m; 0).

O script chega ao mesmo resultado multiplicando as matrizes de transformação de cada nível. A composição manual mostra por que um erro num nível superior (uma rotação indevida no terreno, por exemplo) desloca todo o modelo.

7.2.3 Outros posicionamentos

ENTIDADE	VERSÕES	USO
`IfcLocalPlacement`	Todas	Posição relativa, com origem e eixos
`IfcGridPlacement`	Todas	Posição definida pelo cruzamento de eixos de uma malha de projeto (`IfcGrid`), como os eixos de pilares de uma estrutura
`IfcLinearPlacement`	IFC 4.3	Posição definida ao longo de uma curva, por distância e afastamentos (seção 7.5)

Tabela 7.3: Tipos de posicionamento.

7.3 Contextos e representações

7.3.1 Contextos de representação

O `IfcProject` declara os **contextos de representação** (`RepresentationContexts`, Capítulo 4). Um contexto (`IfcGeometricRepresentationContext`) define o ambiente em que as geometrias são descritas:

ATRIBUTO	CONTEÚDO
`ContextType`	`'Model'` (3D) ou `'Plan'` (2D, desenho)
`CoordinateSpaceDimension`	2 ou 3
`Precision`	Precisão numérica do modelo
`WorldCoordinateSystem`	O sistema de coordenadas do projeto
`TrueNorth`	A direção do norte verdadeiro no plano XY (opcional)

Tabela 7.4: Atributos do contexto de representação.

Os **subcontextos** (`IfcGeometricRepresentationSubContext`) especializam o contexto por finalidade, com o identificador (`ContextIdentifier`) e a vista de destino (`TargetView` , por exemplo `MODEL_VIEW` ou `PLAN_VIEW`). Exemplo da amostra de arquitetura:

```
#11=IFCGEOMETRICREPRESENTATIONCONTEXT($,'Model',3,0.001,#7,$);
#12=IFCGEOMETRICREPRESENTATIONSUBCONTEXT('Body','Model',*,*,*,*,#11,$,.MODEL_VIEW.,$);
```

Figura 7.2: Contexto e subcontexto `Body` .

Os atributos com `*` no subcontexto são derivados do contexto pai (Capítulo 2).

7.3.2 Representações de um produto

O atributo `Representation` de um produto aponta para uma `IfcProductDefinitionShape` , que reúne uma ou mais `IfcShapeRepresentation` . Cada representação tem:

- `ContextOfItems` : o contexto ou subcontexto;
- `RepresentationIdentifier` : **para que** serve a representação;
- `RepresentationType` : **que tipo** de geometria ela contém;
- `Items` : os itens geométricos.

Valores do identificador, conforme a documentação do IFC 4.3:

IDENTIFICADOR	USO
`Body`	Representação 3D do corpo do elemento (a mais comum)
`Axis`	Eixo ou linha única (eixo de viga, de parede, de tubo, de alinhamento)
`FootPrint`	Projeção no plano (linha dupla, contorno em planta)
`Box`	Caixa envolvente simplificada
`Body-FallBack`	Corpo como malha ou superfície de contorno, alternativa ao `Body`
`Clearance`	Volume de afastamento (espaço de manutenção, gabarito)
`CoG`	Centro de gravidade
`Annotation` , `Profile` , `Surface` , `Reference` , `Lighting`	Anotações, perfil, superfície analítica, referência, fonte de luz

Tabela 7.5: Valores de `RepresentationIdentifier` .

Valores do tipo (seleção):

TIPO	CONTEÚDO
`SweptSolid`	Sólidos por extrusão ou revolução de um perfil
`AdvancedSweptSolid`	Sólidos por varredura de um perfil ao longo de uma curva (por exemplo, `IfcSweptDiskSolid`)
`SolidModel`	Sólidos em geral: varreduras, resultados booleanos, B-rep

TIPO	CONTEÚDO
`Brep`	Sólido de faces planas (`IfcFacetedBrep`)
`AdvancedBrep`	Sólido de faces curvas (NURBS)
`Tessellation`	Malhas de triângulos ou polígonos
`CSG`	Resultado de operações booleanas entre sólidos
`Clipping`	Diferença booleana entre sólidos de extrusão (recortes)
`SurfaceModel`	Modelos de superfície
`MappedRepresentation`	Geometria mapeada (seção 7.4.5)
`Curve2D`, `Curve3D`, `Segment`	Curvas; `Segment` para partes de curvas (usado em alinhamentos)
`BoundingBox`	Caixa envolvente

Tabela 7.6: Valores de `RepresentationType`.

7.4 Tipos de geometria

7.4.1 Extrusão e revolução

A **extrusão** (`IfcExtrudedAreaSolid`) é a forma mais simples e mais eficiente: um perfil plano deslocado ao longo de uma direção, por uma distância. Atributos: `SweptArea` (o perfil), `Position` (posição do perfil no sistema local), `ExtrudedDirection` e `Depth` (comprimento). É a representação típica de paredes, lajes, vigas, pilares e tubos retos. O Capítulo 2 reconstruiu uma viga IPE 200 extrudada por 1000 mm.

A **revolução** (`IfcRevolvedAreaSolid`) gira um perfil em torno de um eixo, por um ângulo (`Axis`, `Angle`): curvas de tubo, peças torneadas, vigas curvas.

O perfil pode ser **paramétrico** (`IfcRectangleProfileDef`, `IfcCircleProfileDef`, `IfcCircleHollowProfileDef`, `IfcIShapeProfileDef`, `IfcLShapeProfileDef`...), descrito por poucos números, ou **arbitrário** (`IfcArbitraryClosedProfileDef`), descrito por uma polilinha ou curva. O paramétrico é preferível: é menor, exato e legível (a largura e a altura do perfil estão nos atributos).

7.4.2 Varredura circular: o sólido de tubo

`IfcSweptDiskSolid` varre um **disco** ao longo de uma curva (`Directrix`), com raio externo (`Radius`) e, opcionalmente, raio interno (`InnerRadius`), gerando um tubo vazado. É a representação natural de tubos, barras de armadura, cabos e eletrodutos, em todas as versões. Exemplo de um estribo de armadura (modelos de especificação, IFC 4.3), com raio de 6 mm:

```
#205= IFCINDEXEDPOLYCURVE(#204,(IFCLINEINDEX((1,2)),IFCARCINDEX((2,3,4)),IFCLINEINDEX((4,5)), ... ),$);
#206= IFCSWEPTDISKSOLID(#205,6.0,$,$,$);
#209= IFCSHAPEREPRESENTATION(#5,'Body','SolidModel',(#206));
```

Figura 7.3: Estribo como varredura circular (trecho abreviado).

A diretriz `#205` é uma polilinha indexada (`IfcIndexedPolyCurve`, IFC4 e IFC 4.3) com trechos retos (`IFCLINEINDEX`) e arcos (`IFCARCINDEX`), definidos sobre uma lista de pontos. Uma linha de tubulação com curvas pode ser descrita da mesma forma, num único sólido por trecho.

7.4.3 Malhas e superfícies de contorno

Quando a forma é complexa (equipamentos com muitos detalhes, terreno, objetos importados de outros programas), a geometria costuma ser uma **malha**:

ENTIDADE	VERSÕES	DESCRIÇÃO
`IfcTriangulatedFaceSet`	IFC4, IFC 4.3	Malha de triângulos, com a lista de pontos (`IfcCartesianPointList3D`) e os índices dos vértices de cada triângulo (`CoordIndex`)
`IfcPolygonalFaceSet`	IFC4, IFC 4.3	Malha de polígonos quaisquer
`IfcFacetedBrep`	Todas	Sólido de faces planas, cada face com seus próprios pontos
`IfcAdvancedBrep`	IFC4, IFC 4.3	Sólido de faces curvas (superfícies B-spline)

Tabela 7.7: Malhas e superfícies de contorno.

Exemplo: um bloco de 1000 x 1000 x 2000 mm como malha de 12 triângulos sobre 8 pontos:

```
#1020= IFCSHAPEREPRESENTATION(#202,'Body','Tessellation',(#1021));
#1021= IFCTRIANGULATEDFACESET(#1022,$,.T.,((1,6,5),(1,2,6),(6,2,7),(7,2,3),(7,8,6),(6,8,5),(5,8,1),(1,8,4),
(4,2,1),(2,4,3),(4,8,7),(7,3,4)),$);
#1022= IFCCARTESIANPOINTLIST3D(((-500.,-500.,0.),(500.,-500.,0.),(500.,500.,0.),(-500.,500.,0.),
(-500.,-500.,2000.),(500.,-500.,2000.),(500.,500.,2000.),(-500.,500.,2000.)),$);
```

Figura 7.4: Malha triangular (`triangulated-item.ifc`).

O terceiro atributo, `.T.`, indica que a malha é fechada (delimita um volume). Cada trio de índices aponta para pontos da lista (o primeiro ponto é o índice 1).

A escolha da geometria muda entre versões. A amostra de drenagem da buildingSMART usa **malhas triangulares** (`IfcTriangulatedFaceSet`, 35 itens) na versão IFC 4.3 e **B-rep de faces planas** (`IfcFacetedBrep`, 41 itens) na versão IFC2x3, porque as malhas indexadas não existem no IFC2x3. Na malha indexada, os pontos ficam numa única lista e as faces são descritas por índices; no B-rep, cada face é um laço (`IfcPolyLoop`) que aponta para instâncias de pontos, o que exige mais linhas no arquivo para a mesma forma.

7.4.4 Operações booleanas

Recortes e furos podem ser descritos por operações entre sólidos: `IfcBooleanClippingResult` (diferença entre um sólido e um semiespaço, tipo `Clipping`, usada para cortar o topo de paredes sob telhados) e `IfcBooleanResult` (união, interseção, diferença, tipo `CSG`). As aberturas de portas e janelas, porém, não são booleanas explícitas na geometria: são elementos próprios (`IfcOpeningElement`, Capítulo 6), subtraídos pelo programa que lê o arquivo.

7.4.5 Geometria mapeada

A **geometria mapeada** é o bloco do IFC. Uma `IfcRepresentationMap` define a forma uma vez, com uma origem (`MappingOrigin`); cada uso é um `IfcMappedItem`, que aponta para o mapa (`MappingSource`) e para uma transformação (`MappingTarget`, um `IfcCartesianTransformationOperator3D`, com origem, eixos e escala). Os mapas costumam ficar no tipo (`RepresentationMaps` de `IfcTypeProduct`, Capítulo 3). Exemplo com transformação não uniforme (escala 0,5 em X e Y):

```
#5010= IFCREPRESENTATIONMAP(#5011,#5100);
#5100= IFCSHAPEREPRESENTATION(#202,'Body','SweptSolid',(#5021));
#5021= IFCEXTRUDEDAREASOLID(#5022,$,#904,2000.);
#5022= IFCRECTANGLEPROFILEDEF(.AREA.,'1m x 1m rectangle',$,1000.,1000.);
#1020= IFCSHAPEREPRESENTATION(#202,'Body','MappedRepresentation',(#1021));
#1021= IFCMAPPEDITEM(#5010,#1022);
#1022= IFCCARTESIANTRANSFORMATIONOPERATOR3DNONUNIFORM(#1023,#1024,#901,0.5,#904,0.5,1.);
```

Figura 7.5: Geometria mapeada (`mapped-shape-with-transformation.ifc`).

Válvulas, flanges, luminárias, parafusos e qualquer peça repetida se beneficiam da geometria mapeada: um arquivo com mil válvulas iguais guarda a forma uma única vez.

7.4.6 Qual geometria usar

SITUAÇÃO	GEOMETRIA RECOMENDADA	MOTIVO
Paredes, lajes, vigas, pilares, tubos retos	Extrusão de perfil paramétrico	Leve, exata, com dimensões legíveis
Tubos com curvas, armaduras, cabos	`IfcSweptDiskSolid`	Um sólido por trecho, com raio explícito
Peças repetidas (válvulas, conexões, luminárias)	Geometria mapeada no tipo	A forma é gravada uma vez
Equipamentos complexos, terreno, formas livres	Malha (`IfcTriangulatedFaceSet`)	Descreve qualquer forma, com custo de tamanho
Obras lineares com seção variável (IFC 4.3)	`IfcSectionedSolidHorizontal`, `IfcFixedReferenceSweptAreaSolid` ao longo do alinhamento	A forma acompanha o alinhamento

Tabela 7.8: Escolha da geometria.

7.5 Alinhamentos e posicionamento linear (IFC 4.3)

7.5.1 O alinhamento em duas camadas

Obras lineares (rodovias, ferrovias, adutoras) são projetadas a partir de um **alinhamento**: um eixo com traçado horizontal (retas, curvas circulares, curvas de transição), perfil vertical (rampas e curvas verticais) e, em ferrovias, superelevação. O IFC 4.3 descreve o alinhamento em duas camadas:

1. **Parâmetros de projeto**: o alinhamento (`IfcAlignment`) aninha os leiautes horizontal (`IfcAlignmentHorizontal`), vertical (`IfcAlignmentVertical`) e de superelevação (`IfcAlignmentCant`). Cada leiaute aninha, em ordem, os seus segmentos (`IfcAlignmentSegment`), e cada segmento

guarda os parâmetros de projeto no atributo `DesignParameters`, com o comprimento, os raios e o tipo;

2. **Geometria**: curvas que um programa pode desenhar, na representação do alinhamento: `IfcCompositeCurve` (só o traçado horizontal), `IfcGradientCurve` (horizontal e vertical) e `IfcSegmentedReferenceCurve` (acrescenta a superelevação).

Atributos de `IfcAlignmentHorizontalSegment`: `StartPoint`, `StartDirection`, `StartRadiusOfCurvature`, `EndRadiusOfCurvature`, `SegmentLength`, `GravityCenterLineHeight` e `PredefinedType`, com valores como `LINE` (reta), `CIRCULARARC` (curva circular), `CLOTHOID` (clotoide, a transição mais usada), `CUBIC`, `BLOSSCURVE`, `HELMERTCURVE`, `SINECURVE`, `COSINECURVE` e `VIENNESEBEND`. No vertical (`IfcAlignmentVerticalSegment`), os tipos são `CONSTANTGRADIENT` (rampa), `CIRCULARARC`, `PARABOLICARC` (curva vertical parabólica) e `CLOTHOID`.

Trecho da amostra ferroviária `linear-placement-of-signal.ifc` (unidade metro):

```
#2293 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #2294, 3.49924145684616E-1, 0., 0., 387.723276296965, $, .LINE.);
#2300 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #2301, 3.49924145666035E-1, 0., 1000.00000000019, 39.9999999999925,
$, .CLOTHOID.);
#2307 = IFCALIGNMENTHORIZONTALSEGMENT($, $, #2308, 3.69924145685261E-1, 1000.00000000019, 1000.00000000019,
193.4644708377, $, .CIRCULARARC.);
```

Figura 7.6: Segmentos horizontais: reta, clotoide e curva circular.

Leitura: uma reta de 387,72 m; uma clotoide de 40 m, com raio variando de infinito a 1000 m; uma curva circular de raio 1000 m e 193,46 m de desenvolvimento. A documentação define que um raio 0 "shall be interpreted as INFINITE" (deve ser interpretado como infinito) e que valores positivos indicam curva no sentido anti-horário. As direções iniciais (quarto atributo) estão na unidade de ângulo do arquivo, aqui o radiano: 0,3499 rad, cerca de 20°. Note, nesta amostra, os espaços entre os atributos, permitidos pelo formato STEP.

7.5.2 Posicionamento linear

Com o alinhamento, um objeto pode ser posicionado como os engenheiros de obras lineares o fazem: "na estaca tal, a tantos metros do eixo". No IFC 4.3:

- `IfcLinearPlacement` aponta para um `IfcAxis2PlacementLinear`;
- cuja origem é um `IfcPointByDistanceExpression`, com `DistanceAlong` (distância ao longo da curva), `OffsetLateral` (afastamento lateral), `OffsetVertical` (afastamento vertical), `OffsetLongitudinal` e `BasisCurve` (a curva de referência).

Na amostra ferroviária, o primeiro sinal está a 353,1 m ao longo da curva `#2543` (`IfcGradientCurve`), a 3,0 m do eixo e 2,5 m acima dele. O script da seção 7.7, usando o núcleo geométrico do IfcOpenShell, calcula a posição correspondente: (452600,861; 4539527,818; 7,5), com o eixo X local na direção (0,9394; 0,3428), isto é, alinhado à via. O `IfcLinearPlacement` também tem o atributo opcional `CartesianPosition`; a documentação do IFC 4.3 não o descreve, mas, nesta amostra, o valor gravado coincide com a posição calculada.

7.6 Georreferenciamento

7.6.1 Conceitos

Georreferenciar é ligar o sistema de coordenadas do projeto a um sistema de referência da superfície terrestre, para que o modelo possa ser sobreposto a mapas, levantamentos topográficos e outros modelos da mesma região. Conceitos básicos:

- **datum geodésico**: o modelo matemático da Terra (elipsoide e sua fixação), ao qual as coordenadas se referem;
- **projeção cartográfica**: a regra que transforma latitude e longitude em coordenadas planas; a mais usada em engenharia no Brasil é a **UTM** (Universal Transversa de Mercator), em fusos de 6° de longitude;
- **código EPSG**: um número que identifica, numa base pública de sistemas de referência, a combinação de datum, projeção e unidade. `EPSG:31983`, por exemplo, identifica SIRGAS 2000 / UTM zona 23S.

No Brasil, segundo o IBGE, "desde 25 de fevereiro de 2015, o SIRGAS2000 [...] é o único sistema geodésico de referência oficialmente adotado", após período de transição a partir dos sistemas SAD 69 e Córrego Alegre. Códigos EPSG do SIRGAS 2000, conferidos na base EPSG:

CÓDIGO	SISTEMA
4674	SIRGAS 2000 (coordenadas geográficas)
31978 a 31985	SIRGAS 2000 / UTM zonas 18S a 25S, em sequência (31982 = 22S; 31983 = 23S; 31984 = 24S)
5396	SIRGAS 2000 / UTM zona 26S

Tabela 7.9: Códigos EPSG do SIRGAS 2000.

No Brasil, os fusos "S" são usados também nas regiões ao norte do Equador, conforme a área de uso declarada na base EPSG.

7.6.2 Georreferenciamento no IFC4 e no IFC 4.3

A partir do IFC4, o georreferenciamento usa duas entidades:

- **`IfcProjectedCRS`**: o sistema de referência de mapa. Atributos: `Name` (o código, na forma `'EPSG:31983'`), `Description`, `GeodeticDatum`, `VerticalDatum`, `MapProjection`, `MapZone` e `MapUnit`;
- **`IfcMapConversion`**: a transformação do sistema do projeto (`SourceCRS`, o contexto de representação) para o de mapa (`TargetCRS`). Atributos: `Eastings`, `Northings`, `OrthogonalHeight`, `XAxisAbscissa`, `XAxisOrdinate` e `Scale`.

A documentação descreve a transformação em três passos: escala de todos os eixos pelo fator `Scale`; rotação anti-horária em torno de Z, pelo ângulo θ = arctan(`XAxisOrdinate` / `XAxisAbscissa`); e translação por `Eastings`, `Northings` e `OrthogonalHeight`. Em fórmulas, com s = `Scale`:

```
E = s·(x·cos θ − y·sen θ) + Eastings
N = s·(x·sen θ + y·cos θ) + Northings
H = s·z + OrthogonalHeight
```

Figura 7.7: Transformação de `IfcMapConversion`.

O `Scale` compensa a diferença de unidades: "Scale to be used, when the units of the CRS are not identical to the units of the engineering coordinate system. If omitted, the value of 1.0 is assumed". Um projeto em milímetros com mapa em metros exige `Scale` = 0,001.

Outras entidades: no IFC 4.3, `IfcMapConversionScaled` acrescenta fatores de escala por eixo (`FactorX`, `FactorY`, `FactorZ`), para incorporar o fator de escala da projeção, e `IfcRigidOperation` com `IfcGeographicCRS` permite referenciar o projeto a coordenadas geográficas. No IFC2x3, que não tem essas entidades, o guia de georreferenciamento da buildingSMART (*User Guide for Geo-referencing in IFC*, versão 2.0, 2020) recomenda os conjuntos de propriedades `ePSet_MapConversion` e `ePSet_ProjectedCRS`.

Duas informações relacionadas, mas insuficientes para georreferenciar: `RefLatitude` e `RefLongitude` de `IfcSite` (Capítulo 4), que dão uma posição aproximada, e o `TrueNorth` do contexto, que dá só a direção do norte.

Exemplo resolvido 7.2: calcular coordenadas de mapa

Problema. A amostra `Building-Architecture.ifc` (IFC 4.3) tem:

```
#18=IFCPROJECTEDCRS('EPSG:32760','EPSG:32760 - WGS 84 / UTM zone 60S','WGS 84',$,$,$,#19);
#19=IFCSIUNIT(*,.LENGTHUNIT.,$,.METRE.);
#20=IFCMAPCONVERSION(#11,#18,729013.3488297004,9063992.684697364,1.3000000000000018,0.4999999999999999,0.8660254037844387,0.001);
```

Quais são as coordenadas de mapa da origem da sala `#85`, que está em (3200; 5000; 0) mm no sistema do projeto (Exemplo resolvido 7.1)?

Solução. s = 0,001; θ = arctan(0,866 / 0,5) = 60°; cos θ = 0,5; sen θ = 0,866. Com x = 3200 e y = 5000:

- E = 0,001 x (3200 x 0,5 – 5000 x 0,866) + 729013,349 = 0,001 x (1600 – 4330,1) + 729013,349 = –2,730 + 729013,349 = **729010,619 m**;
- N = 0,001 x (3200 x 0,866 + 5000 x 0,5) + 9063992,685 = 0,001 x (2771,3 + 2500) + 9063992,685 = 5,271 + 9063992,685 = **9063997,956 m**;
- H = 0,001 x 0 + 1,300 = **1,300 m**.

O script da seção 7.7 obtém os mesmos valores. Note que a versão IFC2x3 da mesma amostra não tem `ePSet_MapConversion`: o georreferenciamento se perdeu na exportação para a versão antiga.

Exemplo resolvido 7.3: verificar a coerência do fator de escala

Problema. A amostra `geographic-referencing-utm.ifc`, dos modelos de especificação da buildingSMART, está em milímetros e tem:

```
#1= IFCPROJECTEDCRS('EPSG:25832','UTM in band 32','ETRS89',$,'UTM','UTM32',#3);
#2= IFCMAPCONVERSION(#100011,#1,458657.30,5438232.25,113.7,0.270600445976,0.962691746426,1000.);
#3= IFCSIUNIT(*,.LENGTHUNIT.,$,.METRE.);
```

O valor de `Scale` é coerente?

Solução. O projeto está em milímetros e o mapa em metros. Pela definição, o fator deveria converter milímetros em metros: 0,001. Com `Scale` = 1000, um ponto a 1000 mm (1 m) da origem, na direção X, seria levado a E = 1000 x 1000 x 0,2706 + 458657,30 = 729257,7 m e N = 1000 x 1000 x 0,9627 + 5438232,25 = 6400924,0 m, isto é, a cerca de 1.000 km da origem. O cálculo com a função `ifcopenshell.util.geolocation.auto_xyz2enh` dá o mesmo resultado. A origem do modelo cai no lugar certo, porque o fator não afeta o ponto (0, 0, 0), mas qualquer outro ponto fica deslocado. O valor coerente com a definição é 0,001. O exemplo mostra que, mesmo em arquivos de referência, o georreferenciamento deve ser verificado com pelo menos um ponto conhecido, diferente da origem.

7.7 Consulta com um script

O script abaixo mostra os contextos do arquivo, a cadeia de posicionamento de um produto, a posição e a orientação no sistema do projeto, as representações e, se houver, a conversão da origem para coordenadas de mapa. Para posicionamentos lineares, ele usa o núcleo geométrico do IfcOpenShell (`ifcopenshell.geom`, Capítulo 11). Grave-o como `C:\IFC_Curso\exercicios\cap7\posicao_geometria.py`.

```
# -*- coding: utf-8 -*-
"""
posicao_geometria.py | Capítulo 7: posicionamento, representações e georreferenciamento.

O que faz (somente leitura; nada é gravado):
  1. Mostra os contextos de representação geométrica do arquivo.
  2. Mostra a cadeia de posicionamento de um produto (ObjectPlacement e PlacementRelTo).
  3. Calcula a posição e a orientação do produto no sistema de coordenadas do projeto.
  4. Lista as representações do produto (identificador, tipo e itens geométricos).
  5. Se houver georreferenciamento (IfcMapConversion), converte a origem do produto
     para coordenadas de mapa (E, N, H).

Uso (ambiente virtual ativado):
    python posicao_geometria.py                        -> arquivo e instância padrão abaixo
    python posicao_geometria.py <arquivo.ifc> <id>     -> outro arquivo e outra instância

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import numpy as np
import ifcopenshell
import ifcopenshell.geom
import ifcopenshell.util.geolocation as util_geo
import ifcopenshell.util.placement as util_posic
import ifcopenshell.util.unit as util_unid

np.set_printoptions(suppress=True)          # números sem notação científica

ARQUIVO_PADRAO = (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc"
)
ID_PADRAO = 85  # a instância #85 é um IfcSpace (sala)

arquivo = sys.argv[1] if len(sys.argv) > 1 else ARQUIVO_PADRAO
numero = int(sys.argv[2]) if len(sys.argv) > 2 else ID_PADRAO
modelo = ifcopenshell.open(arquivo)
prod = modelo.by_id(numero)
fator_m = util_unid.calculate_unit_scale(modelo, "LENGTHUNIT")   # unidade do arquivo -> metro
print(f"Produto: #{prod.id()} {prod.is_a()} '{prod.Name}' | esquema {modelo.schema_identifier}"
      f" | 1 unidade de comprimento = {fator_m} m")

print("\n=== 1. CONTEXTOS DE REPRESENTAÇÃO ===")
for ctx in modelo.by_type("IfcGeometricRepresentationContext"):
    if ctx.is_a("IfcGeometricRepresentationSubContext"):
        print(f"   #{ctx.id()} subcontexto '{ctx.ContextIdentifier}' ({ctx.ContextType}, {ctx.TargetView})"
              f" -> pai #{ctx.ParentContext.id()}")
```

```
    else:
        print(f"   #{ctx.id()} contexto '{ctx.ContextType}' | dimensão {ctx.CoordinateSpaceDimension}"
              f" | precisão {ctx.Precision}")

print("\n=== 2. CADEIA DE POSICIONAMENTO ===")
posic = prod.ObjectPlacement
nivel = 0
while posic is not None:
    donos = [p for p in modelo.get_inverse(posic) if p.is_a("IfcProduct") and p.ObjectPlacement == posic]
    dono = f"{donos[0].is_a()} '{donos[0].Name}'" if donos else "-"
    if posic.is_a("IfcLocalPlacement"):
        eixos = posic.RelativePlacement
        local = eixos.Location.Coordinates
        eixo_x = eixos.RefDirection.DirectionRatios if eixos.RefDirection else (1.0, 0.0, 0.0)  # $ = padrão
        print(f"   {'  ' * nivel}#{posic.id()} IfcLocalPlacement de {dono}: origem local {tuple(local)}"
              f" | eixo X local {tuple(round(c, 4) for c in eixo_x)}")
    elif posic.is_a("IfcLinearPlacement"):
        expr = posic.RelativePlacement.Location           # IfcPointByDistanceExpression
        print(f"   {'  ' * nivel}#{posic.id()} IfcLinearPlacement de {dono}: distância ao longo "
              f"{expr.DistanceAlong.wrappedValue}, afastamento lateral {expr.OffsetLateral}, "
              f"vertical {expr.OffsetVertical}, curva #{expr.BasisCurve.id()} {expr.BasisCurve.is_a()}")
    else:
        print(f"   {'  ' * nivel}#{posic.id()} {posic.is_a()} de {dono}")
    posic, nivel = posic.PlacementRelTo, nivel + 1

print("\n=== 3. POSIÇÃO NO SISTEMA DO PROJETO ===")
cadeia_local = True
p = prod.ObjectPlacement
while p is not None:
    cadeia_local = cadeia_local and p.is_a("IfcLocalPlacement")
    p = p.PlacementRelTo
if prod.ObjectPlacement is None:
    matriz = None
    print("   O produto não tem posicionamento.")
elif cadeia_local:
    matriz = util_posic.get_local_placement(prod.ObjectPlacement)   # composição das matrizes
else:
    # Posicionamento linear: o cálculo exige o núcleo geométrico (Capítulo 11)
    config = ifcopenshell.geom.settings()
    forma = ifcopenshell.geom.create_shape(config, prod)
    matriz = np.array(forma.transformation.matrix).reshape(4, 4).T  # matriz em metros
    matriz[:3, 3] = matriz[:3, 3] / fator_m                         # volta para a unidade do arquivo
if matriz is not None:
    origem = matriz[:3, 3]
    print("   Origem (unidade do arquivo):", np.round(origem, 3))
    print("   Origem (metros).............:", np.round(origem * fator_m, 3))
    print("   Eixo X local................:", np.round(matriz[:3, 0], 4) + 0.0)   # + 0.0 evita "-0."
    print("   Eixo Z local................:", np.round(matriz[:3, 2], 4) + 0.0)

print("\n=== 4. REPRESENTAÇÕES ===")
if prod.Representation is None:
    print("   O produto não tem representação geométrica.")
else:
    for rep in prod.Representation.Representations:
        itens = []
        for item in rep.Items:
            if item.is_a("IfcMappedItem"):                          # geometria compartilhada
                origem_map = item.MappingSource.MappedRepresentation
                itens.append(f"IfcMappedItem -> {origem_map.RepresentationType}: "
                             + ", ".join(i.is_a() for i in origem_map.Items))
            else:
                itens.append(item.is_a())
        print(f"   #{rep.id()} '{rep.RepresentationIdentifier}' / '{rep.RepresentationType}': "
              f"{'; '.join(itens)}")

print("\n=== 5. GEORREFERENCIAMENTO ===")
if modelo.schema_identifier == "IFC2X3":
    # No IFC2X3 não há IfcMapConversion; o guia da buildingSMART usa o conjunto
    # ePSet_MapConversion no IfcProject, que a função abaixo também reconhece.
    conversoes = []
    parametros = util_geo.get_helmert_transformation_parameters(modelo)
    if parametros is None:
        print("   IFC2X3 sem ePSet_MapConversion: o arquivo não está georreferenciado.")
```

```
    elif matriz is not None:
        e, n, h = util_geo.auto_xyz2enh(modelo, *origem)
        print(f"   ePSet_MapConversion (IFC2X3): origem do produto em "
              f"E = {e:.3f} | N = {n:.3f} | H = {h:.3f}")
else:
    conversoes = modelo.by_type("IfcCoordinateOperation")
    if not conversoes:
        print("   Nenhuma operação de coordenadas (IfcMapConversion) no arquivo.")
for op in conversoes:
    print(f"   #{op.id()} {op.is_a()} -> CRS de destino: {op.TargetCRS.Name} ({op.TargetCRS.is_a()})")
    if op.is_a("IfcMapConversion") and matriz is not None:
        print(f"      Eastings {op.Eastings} | Northings {op.Northings} | "
              f"OrthogonalHeight {op.OrthogonalHeight}"
              f" | XAxisAbscissa {op.XAxisAbscissa} | XAxisOrdinate {op.XAxisOrdinate} | Scale {op.Scale}")
        e, n, h = util_geo.auto_xyz2enh(modelo, *origem)
        print(f"      Origem do produto em coordenadas de mapa: E = {e:.3f} | N = {n:.3f} | H = {h:.3f}")
```

Execução, com o ambiente ativo:

```
cd C:\IFC_Curso\exercicios\cap7
python posicao_geometria.py
```

Saída obtida no teste (a seção 2 aparece na Figura 7.1):

```
Produto: #85 IfcSpace 'living room' | esquema IFC4X3_ADD2 | 1 unidade de comprimento = 0.001 m

=== 1. CONTEXTOS DE REPRESENTAÇÃO ===
   #11 contexto 'Model' | dimensão 3 | precisão 0.001
   #12 subcontexto 'Body' (Model, MODEL_VIEW) -> pai #11

=== 3. POSIÇÃO NO SISTEMA DO PROJETO ===
   Origem (unidade do arquivo): [3200. 5000.    0.]
   Origem (metros).............: [3.2 5.  0. ]
   Eixo X local................: [1. 0. 0.]
   Eixo Z local................: [0. 0. 1.]

=== 4. REPRESENTAÇÕES ===
   #92 'Body' / 'SweptSolid': IfcExtrudedAreaSolid

=== 5. GEORREFERENCIAMENTO ===
   #20 IfcMapConversion -> CRS de destino: EPSG:32760 (IfcProjectedCRS)
      Eastings 729013.3488297004 | Northings 9063992.684697364 | OrthogonalHeight 1.3000000000000018 |
XAxisAbscissa 0.4999999999999999 | XAxisOrdinate 0.8660254037844387 | Scale 0.001
      Origem do produto em coordenadas de mapa: E = 729010.619 | N = 9063997.956 | H = 1.300
```

Com o sinal ferroviário (`#3020` de `linear-placement-of-signal.ifc`), trecho da saída obtida:

```
=== 2. CADEIA DE POSICIONAMENTO ===
   #3021 IfcLinearPlacement de IfcSignal 'Route Indicator_01': distância ao longo 353.1, afastamento lateral 3.0,
vertical 2.5, curva #2543 IfcGradientCurve
     #2279 IfcLocalPlacement de IfcAlignment 'Track alignment': origem local (0.0, 0.0, 0.0) | eixo X local (1.0,
0.0, 0.0)
       #2274 IfcLocalPlacement de IfcRailway 'Default Railway Name': origem local (0.0, 0.0, 0.0) | eixo X local
(1.0, 0.0, 0.0)
         #2269 IfcLocalPlacement de IfcSite 'Default Site Name': origem local (0.0, 0.0, 0.0) | eixo X local
(1.0, 0.0, 0.0)

=== 3. POSIÇÃO NO SISTEMA DO PROJETO ===
   Origem (unidade do arquivo): [ 452600.861 4539527.818       7.5  ]
   Origem (metros).............: [ 452600.861 4539527.818       7.5  ]
   Eixo X local................: [0.9394 0.3428 0.    ]
   Eixo Z local................: [0. 0. 1.]

=== 4. REPRESENTAÇÕES ===
   #3030 'Body' / 'MappedRepresentation': IfcMappedItem -> Tessellation: IfcPolygonalFaceSet,
IfcPolygonalFaceSet, IfcPolygonalFaceSet, IfcPolygonalFaceSet, IfcPolygonalFaceSet
```

O script foi executado também na versão IFC2x3 da amostra de arquitetura (mesma posição da sala, sem georreferenciamento), no estribo de armadura (geometria mapeada com `IfcSweptDiskSolid`) e na amostra UTM do Exemplo resolvido 7.3.

7.8 Resumo do capítulo

- A posição (`ObjectPlacement`) e a forma (`Representation`) são independentes, como a inserção e a definição de um bloco.
- `IfcLocalPlacement` posiciona cada objeto em relação ao que o contém; a posição final é a composição da cadeia. `IfcAxis2Placement3D` define origem, eixo Z e eixo X.
- Contextos e subcontextos organizam as representações; cada representação tem identificador (`Body`, `Axis`, `FootPrint` ...) e tipo (`SweptSolid`, `Tessellation`, `MappedRepresentation` ...).
- Extrusões de perfis paramétricos são leves e exatas; `IfcSweptDiskSolid` descreve tubos e barras; malhas descrevem formas livres; a geometria mapeada evita repetição.
- No IFC 4.3, alinhamentos têm parâmetros de projeto e geometria; objetos podem ser posicionados por distância ao longo do eixo e afastamentos.
- O georreferenciamento (IFC4+) usa `IfcProjectedCRS` e `IfcMapConversion`; o `Scale` converte unidades. No Brasil, o sistema oficial é o SIRGAS 2000; em UTM, os códigos EPSG vão de 31978 a 31985 (zonas 18S a 25S) e 5396 (26S).
- Verifique o georreferenciamento com um ponto conhecido, diferente da origem.

7.9 Vocabulário técnico

INGLÊS	PORTUGUÊS
placement / local placement	posicionamento / posicionamento local
world coordinate system	sistema de coordenadas do projeto
representation context / subcontext	contexto / subcontexto de representação
shape representation / representation item	representação de forma / item de representação
extruded area solid / revolved area solid	sólido extrudado / sólido de revolução
swept disk solid	sólido de varredura circular
profile definition	definição de perfil
triangulated face set / faceted brep	malha triangular / sólido de faces planas
mapped item / representation map	item mapeado / mapa de representação
alignment / segment / clothoid	alinhamento / segmento / clotoide
linear placement / distance along / lateral offset	posicionamento linear / distância ao longo / afastamento lateral
coordinate reference system (CRS)	sistema de referência de coordenadas
map conversion / eastings / northings	conversão para mapa / coordenada este / coordenada norte
geodetic datum / map projection	datum geodésico / projeção cartográfica

INGLÊS	PORTUGUÊS
true north / grid north	norte verdadeiro / norte da quadrícula

Tabela 7.10: Vocabulário do capítulo.

7.10 Exercícios

Fixação

1. Por que o IFC separa `ObjectPlacement` e `Representation`?
2. Num `IfcAxis2Placement3D`, o que significam `Axis` e `RefDirection` iguais a `$`?
3. Qual a diferença entre `RepresentationIdentifier` e `RepresentationType`? Dê um exemplo de cada.
4. Que geometria você usaria para: (a) uma laje; (b) um trecho de tubo com duas curvas; (c) 500 luminárias iguais; (d) o modelo de um equipamento importado de um fabricante?
5. Qual é o código EPSG de SIRGAS 2000 / UTM zona 23S? E de SIRGAS 2000 em coordenadas geográficas?

Aplicação

6. Execute `posicao_geometria.py` para uma das paredes de `Building-Architecture.ifc` (procure `IFCWALL(` no VS Code). Registre a origem no sistema do projeto, o eixo X local e as coordenadas de mapa. Verifique manualmente a coordenada E com a fórmula da Figura 7.7.
7. Abra `Infra-Plumbing.ifc` nas versões IFC 4.3 e IFC2x3 no VS Code, sem alterá-los, e conte as ocorrências de `IFCTRIANGULATEDFACESET(` e de `IFCFACETEDBREP(`. Explique a diferença e compare o tamanho dos dois arquivos.

Desafio

8. Um projeto em Civil 3D usa SIRGAS 2000 / UTM zona 22S, em metros. O modelo IFC será exportado com o sistema do projeto em milímetros, origem local no ponto de coordenadas E = 721.500,000 m, N = 7.905.300,000 m e cota 850,000 m, e eixo X local alinhado ao leste da quadrícula. Escreva as linhas `IFCPROJECTEDCRS` e `IFCMAPCONVERSION` corretas (use `#11` para o contexto e `#19` para a unidade metro) e calcule as coordenadas de mapa de um ponto em (12.000; 3.500; 2.000) mm.

7.11 Respostas dos exercícios

1. Para que a forma possa ser descrita uma vez, no sistema do próprio objeto, e a posição possa mudar sem alterar a forma; para permitir formas compartilhadas (geometria mapeada) e várias representações do mesmo objeto sem repetir a posição.
2. Que valem os padrões: eixo Z local = (0, 0, 1) e eixo X local = (1, 0, 0); isto é, o sistema local tem a mesma orientação do sistema de referência, só com a origem deslocada.
3. O identificador diz para que serve a representação (`Body`, corpo 3D; `Axis`, eixo); o tipo diz que geometria ela contém (`SweptSolid`, extrusão; `Tessellation`, malha). Uma viga costuma ter uma representação `Axis` / `Curve3D` e outra `Body` / `SweptSolid`.

4. (a) Extrusão de um perfil (`IfcExtrudedAreaSolid`); (b) `IfcSweptDiskSolid` com diretriz de retas e arcos (`IfcIndexedPolyCurve`); (c) geometria mapeada no tipo, com um `IfcMappedItem` por luminária; (d) malha (`IfcTriangulatedFaceSet`), de preferência como geometria mapeada se houver repetição.
5. 31983 (SIRGAS 2000 / UTM zona 23S) e 4674 (SIRGAS 2000, geográficas).
6. Com a parede `#158` , a saída obtida foi: origem (7100; 4800; 0) mm, isto é, (7,1 m; 4,8 m; 0); eixo X local (0; -1; 0), ou seja, a parede está girada -90° em relação ao eixo X do projeto; coordenadas de mapa E = 729012,742, N = 9064001,233 e H = 1,300. Verificação manual de E, com s = 0,001 e θ = 60°: E = 0,001 x (7100 x 0,5 – 4800 x 0,866) + 729013,349 = 0,001 x (3550 – 4156,9) + 729013,349 = −0,607 + 729013,349 = 729012,742 m. O resultado coincide com o do script.
7. Na versão IFC 4.3 há 35 `IFCTRIANGULATEDFACESET(` e nenhum `IFCFACETEDBREP(` ; na IFC2x3, 41 `IFCFACETEDBREP(` e nenhum `IFCTRIANGULATEDFACESET(` . As malhas indexadas foram introduzidas no IFC4; no IFC2x3, a opção para formas facetadas é o B-rep. Quanto ao tamanho, a versão IFC 4.3 tem cerca de 351 KB e a IFC2x3, cerca de 187 KB: o tamanho total depende de todo o conteúdo do arquivo (estrutura espacial, tipos, propriedades, estilos), e não só do tipo de geometria. Para comparar apenas a geometria, seria preciso isolar as instâncias geométricas de cada arquivo.
8. Linhas:

```
#1000=IFCPROJECTEDCRS('EPSG:31982','SIRGAS 2000 / UTM zona 22S','SIRGAS 2000',$,'UTM','22S',#19);
#1001=IFCMAPCONVERSION(#11,#1000,721500.,7905300.,850.,1.,0.,0.001);
```

`XAxisAbscissa` = 1 e `XAxisOrdinate` = 0 (θ = 0°, eixo X local alinhado ao leste); `Scale` = 0,001 (milímetro para metro). Para o ponto (12000; 3500; 2000) mm: E = 0,001 x 12000 + 721500 = 721.512,000 m; N = 0,001 x 3500 + 7905300 = 7.905.303,500 m; H = 0,001 x 2000 + 850 = 852,000 m. Em arquivos exportados pelo Civil 3D, esses valores são gerados pelo exportador a partir do sistema de coordenadas do desenho (Capítulo 13); o exercício serve para conferir o resultado.

7.12 Referências

- Documentação do IFC 4.3, `IfcShapeRepresentation` : https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcShapeRepresentation.htm
- Documentação do IFC 4.3, `IfcMapConversion` : https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcMapConversion.htm
- Documentação do IFC 4.3, `IfcLinearPlacement` : https://standards.buildingsmart.org/IFC/RELEASE/IFC4_3/HTML/lexical/IfcLinearPlacement.htm
- buildingSMART Australasia, *User Guide for Geo-referencing in IFC*, versão 2.0, 2020: https://www.buildingsmart.org/wp-content/uploads/2020/02/User-Guide-for-Geo-referencing-in-IFC-v2.0.pdf
- IBGE, Projeto Mudança do Referencial Geodésico (SIRGAS2000): https://www.ibge.gov.br/geociencias/informacoes-sobre-posicionamento-geodesico/sirgas/16691-projeto-mudanca-do-referencial-geodesico-pmrg.html
- Base EPSG (consulta de códigos): https://epsg.io/
- Documentação do IfcOpenShell, `ifcopenshell.util.geolocation` e `ifcopenshell.util.placement` : https://docs.ifcopenshell.org/

8 Python essencial para IFC

Os capítulos anteriores usaram scripts prontos para ler arquivos IFC. Este capítulo ensina a linguagem em que esses scripts são escritos, o **Python**, do zero, com exemplos tirados do trabalho com modelos: listas de elementos, conjuntos de propriedades, planilhas de metadados. Ao final, você será capaz de ler, entender, adaptar e escrever scripts como os dos Capítulos 9 a 14.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que é um programa e como o Python o executa;
- usar variáveis, tipos básicos, listas, dicionários e conjuntos;
- escrever decisões, repetições e funções;
- ler e gravar arquivos de texto, CSV, JSON e planilhas Excel;
- interpretar mensagens de erro e tratar erros previstos;
- entender classes e objetos e reconhecer que as instâncias IFC são objetos;
- organizar um script com argumentos, funções e boas práticas.

8.1 Introdução: o que é programar

8.1.1 Programa, linguagem e interpretador

Um **programa** é uma sequência de instruções que o computador executa em ordem. Uma **linguagem de programação** é um conjunto de regras para escrever essas instruções de forma precisa. O **Python** é uma linguagem de uso geral, conhecida pela leitura fácil, e o **interpretador** Python (instalado no Capítulo 0) é o programa que lê o texto do script e executa as instruções, uma a uma.

Quem já usou o Dynamo conhece a programação visual: nós ligados por fios, cada nó uma operação. Num script Python, as mesmas ideias aparecem como texto: cada linha é uma operação, as variáveis fazem o papel dos fios, e as funções fazem o papel dos nós personalizados. Quem já escreveu AutoLISP também reconhecerá variáveis, listas, repetições e funções, com outra sintaxe.

8.1.2 Por que Python para IFC

- A biblioteca **IfcOpenShell**, a principal ferramenta aberta para ler e editar arquivos IFC, é usada a partir do Python.
- O Python tem bibliotecas prontas para quase tudo o que um fluxo de metadados exige: planilhas (`openpyxl`, `pandas`), arquivos XML (usados pelo IDS), gráficos e relatórios.
- O código é texto: pode ser guardado, comparado, revisado e reutilizado.

8.1.3 Como executar os exemplos

Todos os exemplos deste capítulo são arquivos `.py`, gravados em `C:\IFC_Curso\exercicios\cap8\`. Para executar, com o ambiente virtual ativado (Capítulo 0):

```
cd C:\IFC_Curso\exercicios\cap8
python p01_primeiro.py
```

No VS Code, também é possível abrir o arquivo e usar o botão de execução, desde que o interpretador selecionado seja o do ambiente virtual `.venv` (Capítulo 0). Todos os exemplos foram testados com Python 3.11 e 3.13, com saídas idênticas nas duas versões.

8.2 O primeiro programa

```
# p01_primeiro.py | Capítulo 8: o primeiro programa
# Tudo o que vem depois do sinal # é comentário: o Python ignora.
print("Olá, IFC!")                # print mostra um texto no terminal
print("Versões do esquema:", "IFC2X3", "IFC4", "IFC4X3_ADD2")
print(2 + 3 * 4)                  # operações seguem a precedência da matemática
```

Saída obtida:

```
Olá, IFC!
Versões do esquema: IFC2X3 IFC4 IFC4X3_ADD2
14
```

Três conceitos aparecem aqui:

- **comentário**: o texto depois de `#` é ignorado pelo Python; serve para explicar o código a quem o lê;
- **função**: `print(...)` é uma função, isto é, um comando com nome que recebe valores entre parênteses (os **argumentos**) e faz algo com eles; `print` mostra os valores no terminal, separados por espaço;
- **expressão**: `2 + 3 * 4` é calculada antes de ser mostrada, respeitando a precedência (multiplicação antes da soma).

8.3 Variáveis e tipos básicos

8.3.1 Variáveis

Uma **variável** é um nome que guarda um valor. O sinal `=` significa "guarde o valor da direita no nome da esquerda"; não é uma igualdade matemática. Regras para nomes:

- letras, números e sublinhado (`_`), sem começar por número;
- maiúsculas e minúsculas são diferentes (`Tag` e `tag` são variáveis distintas);
- por convenção, nomes de variáveis em minúsculas, com palavras separadas por sublinhado (`diametro_mm`, `comprimento_total`). Evite acentos em nomes, embora o Python os aceite.

8.3.2 Tipos básicos

TIPO	NOME NO PYTHON	EXEMPLOS	EQUIVALENTE NO IFC
Texto	`str`	`"TB-001"`, `'Água bruta'`	`IfcLabel`, `IfcText`, `IfcIdentifier`
Número inteiro	`int`	`100`, `-3`	`IfcInteger`, `IfcCountMeasure`
Número real	`float`	`6.25`, `1e-5`	`IfcReal`, medidas (`IfcLengthMeasure` ...)
Lógico	`bool`	`True`, `False`	`IfcBoolean`

TIPO	NOME NO PYTHON	EXEMPLOS	EQUIVALENTE NO IFC
Ausência de valor	`None`	`None`	`$` no arquivo

Tabela 8.1: Tipos básicos do Python.

```
# p02_variaveis.py | Capítulo 8: variáveis e tipos básicos
tag = "TB-001"                 # str (texto)
diametro_mm = 100              # int (número inteiro)
comprimento_m = 6.25           # float (número real; o separador decimal é o ponto)
isolado = False                # bool (verdadeiro ou falso)
fabricante = None              # None (ausência de valor, como o $ do arquivo IFC)

print(type(tag), type(diametro_mm), type(comprimento_m), type(isolado), type(fabricante))

# Operações com números
area_m2 = 3.1416 * (diametro_mm / 1000) ** 2 / 4    # ** é potência
print("Área da seção:", area_m2, "m²")
print("Área arredondada:", round(area_m2, 5))

# Operações com textos
print(tag.lower(), tag.upper(), len(tag))            # minúsculas, maiúsculas, tamanho
print(tag.startswith("TB"), tag.replace("-", "_"))   # começa com? substituir
print(tag.split("-"))                                # separa em partes: ['TB', '001']

# f-string: texto com valores embutidos entre chaves
print(f"O tubo {tag} tem DN {diametro_mm} e {comprimento_m:.1f} m")
print(f"Comprimento em mm: {comprimento_m * 1000:,.0f}")

# Conversões entre tipos
print(int("25") + 1, float("2.5") * 2, str(100) + " mm")
```

Saída obtida:

```
<class 'str'> <class 'int'> <class 'float'> <class 'bool'> <class 'NoneType'>
Área da seção: 0.007854000000000002 m²
Área arredondada: 0.00785
tb-001 TB-001 6
True TB_001
['TB', '001']
O tubo TB-001 tem DN 100 e 6.2 m
Comprimento em mm: 6,250
26 5.0 100 mm
```

Observações:

- no Python, o separador decimal é o **ponto** (`6.25`), independentemente da configuração regional do Windows;
- os números reais são guardados em binário, com precisão limitada. Por isso, a área aparece como `0.007854000000000002`, e não `0.007854`. Use `round(valor, casas)` para mostrar resultados, e nunca compare números reais com `==`; compare a diferença com uma tolerância (`abs(a - b) < 1e-9`);
- na f-string, `{comprimento_m:.1f}` mostra o valor com uma casa decimal. O valor 6,25 aparece como `6.2`, e não `6.3`: quando o número está exatamente no meio, o Python arredonda para o dígito par. Em relatórios em que isso importa, arredonde de forma explícita, com a regra desejada;
- `{valor:,.0f}` mostra o número com separador de milhar (no padrão inglês, a vírgula) e sem casas decimais.

8.4 Coleções: listas, tuplas, dicionários e conjuntos

Coleções guardam vários valores num único nome. São a base de qualquer script de IFC: a lista de elementos de um modelo, o dicionário de propriedades de um elemento, o conjunto de classes presentes num arquivo.

COLEÇÃO	SINTAXE	CARACTERÍSTICAS	USO TÍPICO EM IFC
Lista (`list`)	`[a, b, c]`	Ordenada, alterável, aceita repetidos	Elementos de uma classe; linhas de uma planilha
Tupla (`tuple`)	`(a, b, c)`	Ordenada, não alterável	Coordenadas (x, y, z)
Dicionário (`dict`)	`{chave: valor}`	Pares chave e valor; chaves únicas	Propriedades de um conjunto; um registro de planilha
Conjunto (`set`)	`{a, b, c}`	Sem ordem, sem repetidos	Classes distintas; GlobalIds já processados

Tabela 8.2: Coleções do Python.

```
# p03_colecoes.py | Capítulo 8: listas, tuplas, dicionários e conjuntos
# Lista: sequência ordenada e alterável
tags = ["TB-001", "VG-001", "TB-002"]
tags.append("B-001")                    # acrescenta no fim
print(tags, len(tags))
print(tags[0], tags[-1], tags[1:3])     # primeiro, último, fatia (índices 1 e 2)
print(sorted(tags))                     # cópia em ordem alfabética

# Tupla: sequência que não se altera (coordenadas, por exemplo)
origem = (3200.0, 5000.0, 0.0)
x, y, z = origem                        # "desempacotar" em três variáveis
print("x =", x, "| y =", y, "| z =", z)

# Dicionário: pares chave -> valor (como um conjunto de propriedades)
pset = {"NominalDiameter": 100.0, "WorkingPressure": 1000000.0, "Status": "NEW"}
print(pset["NominalDiameter"])          # acesso pela chave
pset["Reference"] = "DN100-A106"        # inclui ou altera uma chave
print(pset.get("Colour", "sem valor"))  # get: valor padrão se a chave não existir
for nome, valor in pset.items():        # percorre os pares
    print(f"   {nome} = {valor}")

# Dicionário dentro de dicionário (é assim que o IfcOpenShell devolve os Psets)
psets = {"Pset_PipeSegmentTypeCommon": {"NominalDiameter": 100.0},
         "Qto_PipeSegmentBaseQuantities": {"Length": 6250.0}}
print(psets["Qto_PipeSegmentBaseQuantities"]["Length"])

# Conjunto (set): elementos únicos, sem ordem
classes = {"IfcPipeSegment", "IfcValve", "IfcPipeSegment"}
print(sorted(classes), len(classes))    # o repetido aparece uma vez só (sorted: em ordem)
```

Saída obtida:

```
['TB-001', 'VG-001', 'TB-002', 'B-001'] 4
TB-001 B-001 ['VG-001', 'TB-002']
['B-001', 'TB-001', 'TB-002', 'VG-001']
x = 3200.0 | y = 5000.0 | z = 0.0
100.0
sem valor
   NominalDiameter = 100.0
   WorkingPressure = 1000000.0
   Status = NEW
   Reference = DN100-A106
6250.0
['IfcPipeSegment', 'IfcValve'] 2
```

Pontos importantes:

- os **índices** começam em 0: `tags[0]` é o primeiro elemento; `tags[-1]`, o último. A fatia `tags[1:3]` pega os índices 1 e 2 (o limite final não entra);
- o dicionário é a estrutura mais usada com IFC: a função `ifcopenshell.util.element.get_psets` (Capítulo 9) devolve exatamente um dicionário de dicionários, como `psets` no exemplo;
- acessar uma chave inexistente com colchetes (`pset["Colour"]`) gera erro; com `get`, devolve o valor padrão informado.

8.5 Decisões e repetições

8.5.1 Blocos e indentação

No Python, os blocos de código (o que está "dentro" de um `if`, de um `for` ou de uma função) são definidos pela **indentação**: o recuo das linhas. A linha que abre o bloco termina com dois-pontos (`:`), e as linhas do bloco ficam recuadas com **4 espaços**. Não misture tabulações e espaços; o VS Code converte a tecla Tab em 4 espaços automaticamente em arquivos Python.

8.5.2 Comparações e operadores lógicos

OPERADOR	SIGNIFICADO	EXEMPLO
`==`, `!=`	Igual, diferente	`classe == "IfcValve"`
`<`, `<=`, `>`, `>=`	Menor, maior	`dn >= 100`
`in`, `not in`	Está contido	`"Pset_" in nome`; `tag in lista`
`is None`, `is not None`	É (ou não é) ausência de valor	`valor is None`
`and`, `or`, `not`	E, ou, não	`e_tubo and dn >= 100`

Tabela 8.3: Operadores de comparação e lógicos.

8.5.3 Exemplo

```
# p04_decisoes_repeticoes.py | Capítulo 8: decisões e repetições
elementos = [
    {"tag": "TB-001", "classe": "IfcPipeSegment", "dn": 100, "comprimento": 6.0},
    {"tag": "VG-001", "classe": "IfcValve", "dn": 100, "comprimento": None},
    {"tag": "TB-002", "classe": "IfcPipeSegment", "dn": 80, "comprimento": 4.5},
    {"tag": "B-001", "classe": "IfcPump", "dn": None, "comprimento": None},
]

# if / elif / else: decisões
for elem in elementos:
    if elem["classe"] == "IfcPipeSegment" and elem["dn"] >= 100:
        situacao = "tubo principal"
    elif elem["classe"] == "IfcPipeSegment":
        situacao = "tubo secundário"
    else:
        situacao = "não é tubo"
    print(f"{elem['tag']:<8} {situacao}")

# Somar comprimentos só dos tubos (ignorando os que não têm valor)
total = 0.0
for elem in elementos:
    if elem["comprimento"] is None:
```

```
        continue                        # pula para o próximo elemento
    total += elem["comprimento"]        # o mesmo que total = total + ...
print("Comprimento total de tubos:", total, "m")

# range e enumerate: contadores
for i in range(3):
    print("volta", i)                   # 0, 1, 2
for numero, elem in enumerate(elementos, start=1):
    print(numero, elem["tag"])

# zip: percorre duas listas ao mesmo tempo
tags = ["TB-001", "TB-002"]
revisoes = ["A", "B"]
for tag, rev in zip(tags, revisoes):
    print(f"{tag} -> revisão {rev}")

# while: repete enquanto a condição for verdadeira
nivel = 0
while nivel < 3:
    print("nível", nivel)
    nivel += 1

# Compreensão de lista: filtra e transforma numa única linha
tubos = [e["tag"] for e in elementos if e["classe"] == "IfcPipeSegment"]
print(tubos)
```

Saída obtida:

```
TB-001   tubo principal
VG-001   não é tubo
TB-002   tubo secundário
B-001    não é tubo
Comprimento total de tubos: 10.5 m
volta 0
volta 1
volta 2
1 TB-001
2 VG-001
3 TB-002
4 B-001
TB-001 -> revisão A
TB-002 -> revisão B
nível 0
nível 1
nível 2
['TB-001', 'TB-002']
```

Resumo das estruturas:

- `if`, `elif` (senão, se) e `else` (senão) escolhem um caminho;
- `for item in colecao:` repete o bloco para cada item; `range(n)` gera os números de 0 a n-1; `enumerate` acrescenta um contador; `zip` percorre duas coleções em paralelo;
- `continue` pula para a próxima volta; `break` interrompe a repetição;
- `while condicao:` repete enquanto a condição for verdadeira. Cuidado: se a condição nunca ficar falsa, o programa não termina (interrompa com `Ctrl+C`);
- a **compreensão de lista** (`[e["tag"] for e in elementos if ...]`) cria uma lista nova a partir de outra, filtrando e transformando.

8.6 Funções

Uma **função** agrupa instruções sob um nome, recebe valores (**parâmetros**) e devolve um resultado (`return`). Funções evitam repetição, deixam o código legível e permitem testar partes isoladas.

```
# p05_funcoes.py | Capítulo 8: funções
def area_secao(diametro_mm):
    """Retorna a área da seção circular, em m², a partir do diâmetro em mm."""
    diametro_m = diametro_mm / 1000
    return 3.141592653589793 * diametro_m ** 2 / 4


def pressao_bar(valor, unidade="Pa"):
    """Converte uma pressão para bar. Unidades aceitas: 'Pa', 'kPa', 'MPa'."""
    fatores = {"Pa": 1e-5, "kPa": 1e-2, "MPa": 10.0}
    if unidade not in fatores:
        raise ValueError(f"Unidade não prevista: {unidade}")
    return valor * fatores[unidade]


print(round(area_secao(100), 6))              # argumento por posição
print(pressao_bar(1000000))                   # usa o valor padrão 'Pa'
print(pressao_bar(1.6, unidade="MPa"))        # argumento por nome
print(area_secao.__doc__)                     # a documentação da função


def resumo(*tags, separador=", "):
    """Aceita qualquer quantidade de TAGs e devolve um único texto."""
    return separador.join(tags)


print(resumo("TB-001", "VG-001", "B-001"))
```

Saída obtida:

```
0.007854
10.0
16.0
Retorna a área da seção circular, em m², a partir do diâmetro em mm.
TB-001, VG-001, B-001
```

Elementos de uma função:

- `def nome(parametros):` abre a definição; o bloco recuado é o corpo;
- o texto entre aspas triplas logo abaixo é a **docstring**, a documentação da função;
- parâmetros com valor padrão (`unidade="Pa"`) podem ser omitidos na chamada; argumentos podem ser passados por posição ou por nome;
- `return` devolve o resultado; uma função sem `return` devolve `None`;
- `raise ValueError(...)` interrompe a função com um erro, quando o valor recebido não faz sentido (seção 8.9);
- `*tags` recebe qualquer quantidade de argumentos numa tupla.

8.7 Módulos e bibliotecas

Um **módulo** é um arquivo `.py` com funções e outras definições prontas. Uma **biblioteca** (ou pacote) é um conjunto de módulos. Há três origens:

ORIGEM	EXEMPLOS	INSTALAÇÃO
Biblioteca padrão (vem com o Python)	`csv`, `json`, `pathlib`, `sys`, `math`, `datetime`, `argparse`	Nenhuma
Bibliotecas de terceiros	`ifcopenshell`, `ifctester`, `pandas`, `openpyxl`	`pip install` no ambiente virtual (Capítulo 0)
Seus próprios módulos	Um arquivo `utilitarios.py` na mesma pasta	Nenhuma: basta estar na pasta do script

Tabela 8.4: Origens dos módulos.

Formas de importar:

```
import csv                                  # usa-se csv.DictWriter(...)
import ifcopenshell.util.element as util_elem  # apelido curto: util_elem.get_psets(...)
from pathlib import Path                    # traz só Path: usa-se Path(...)
```

Muitos scripts terminam com o bloco `if __name__ == "__main__": main()`. Ele significa: "execute a função principal só quando este arquivo for rodado diretamente". Assim, o mesmo arquivo pode ser importado por outro script sem executar nada automaticamente (seção 8.13).

8.8 Arquivos e caminhos

8.8.1 Caminhos com pathlib

O módulo `pathlib` trata caminhos de arquivos como objetos. O operador `/` monta caminhos e funciona no Windows e no Linux. Um caminho **relativo** (como `saida_cap8`) parte da pasta atual do terminal; um caminho **absoluto** (como `C:\IFC_Curso\amostras`) indica o local completo.

Em textos Python, a barra invertida `\` tem significado especial. Para escrever caminhos do Windows, use o prefixo `r` antes das aspas (texto "cru"), como nos scripts deste livro: `r"C:\IFC_Curso\amostras"`.

8.8.2 Texto, CSV e JSON

```
# p06_arquivos.py | Capítulo 8: módulos, caminhos e arquivos de texto, CSV e JSON
import csv                                  # módulo da biblioteca padrão
import json
from pathlib import Path                    # importa só o que interessa do módulo

pasta = Path("saida_cap8")                  # caminho relativo à pasta atual
pasta.mkdir(exist_ok=True)                  # cria a pasta, se ainda não existir

# Montar caminhos com / (funciona no Windows e no Linux)
arquivo_txt = pasta / "relatorio.txt"
print("Arquivo:", arquivo_txt, "| nome:", arquivo_txt.name, "| extensão:", arquivo_txt.suffix)

# Escrever e ler texto (sempre informe a codificação UTF-8)
arquivo_txt.write_text("Relatório de verificação\nTB-001: ok\n", encoding="utf-8")
print(arquivo_txt.read_text(encoding="utf-8"))

# CSV: tabela em texto, uma linha por registro
linhas = [
    {"tag": "TB-001", "classe": "IfcPipeSegment", "dn": 100},
    {"tag": "VG-001", "classe": "IfcValve", "dn": 100},
]
arquivo_csv = pasta / "elementos.csv"
with open(arquivo_csv, "w", newline="", encoding="utf-8-sig") as f:   # utf-8-sig: acentos no Excel
```

```
    escritor = csv.DictWriter(f, fieldnames=["tag", "classe", "dn"], delimiter=";")
    escritor.writeheader()
    escritor.writerows(linhas)
with open(arquivo_csv, encoding="utf-8-sig") as f:
    for registro in csv.DictReader(f, delimiter=";"):
        print(registro["tag"], registro["classe"], registro["dn"])

# JSON: dicionários e listas gravados como texto estruturado
arquivo_json = pasta / "elementos.json"
arquivo_json.write_text(json.dumps(linhas, indent=2, ensure_ascii=False), encoding="utf-8")
print(json.loads(arquivo_json.read_text(encoding="utf-8"))[1]["classe"])

# Listar arquivos de uma pasta por padrão de nome
print(sorted(p.name for p in pasta.glob("*.*")))
```

Saída obtida (no teste, feito em Linux, o caminho aparece com barra normal; no Windows, aparece como `saida_cap8\relatorio.txt`):

```
Arquivo: saida_cap8/relatorio.txt | nome: relatorio.txt | extensão: .txt
Relatório de verificação
TB-001: ok

TB-001 IfcPipeSegment 100
VG-001 IfcValve 100
IfcValve
['elementos.csv', 'elementos.json', 'relatorio.txt']
```

Pontos importantes:

- informe sempre `encoding="utf-8"` ao ler e gravar texto; sem isso, acentos podem ser corrompidos, porque o Windows usa outra codificação por padrão;
- o bloco `with open(...) as f:` abre o arquivo e o fecha automaticamente ao final, mesmo que ocorra um erro;
- para CSV que será aberto no Excel com configuração brasileira, use `;` como separador (a vírgula é o separador decimal) e a codificação `utf-8-sig`, que o Excel reconhece com acentos;
- o JSON guarda dicionários e listas como texto estruturado, e é o formato de relatórios de várias ferramentas (o IfcTester, no Capítulo 12, e o `ifcdiff`).

8.9 Erros: ler e tratar

8.9.1 Ler a mensagem de erro

Quando algo dá errado, o Python interrompe o programa e mostra um **traceback**: a sequência de chamadas até o ponto do erro, terminando com o **tipo** e a **mensagem** do erro. Leia de baixo para cima: a última linha diz o que aconteceu; as linhas acima dizem onde (arquivo e número da linha).

ERRO	CAUSA TÍPICA	EXEMPLO
`SyntaxError`, `IndentationError`	Código mal escrito: parêntese, aspas ou dois-pontos faltando; recuo errado	`if x == 1` sem `:`
`NameError`	Nome não definido (erro de digitação, variável criada depois)	`print(tga)`
`TypeError`	Operação com tipos incompatíveis	`"DN" + 100`

ERRO	CAUSA TÍPICA	EXEMPLO
`ValueError`	Valor de tipo certo, mas conteúdo inválido	`int("DN100")`
`KeyError`	Chave inexistente no dicionário	`pset["Colour"]`
`IndexError`	Índice fora da lista	`lista[10]` numa lista de 3 itens
`AttributeError`	Atributo inexistente no objeto	`parede.PredefinedType` num `IfcWall` de IFC2x3
`FileNotFoundError`	Arquivo ou pasta inexistente	Caminho digitado errado
`ModuleNotFoundError`	Biblioteca não instalada no ambiente ativo	`import ifcopenshell` fora do `.venv`

Tabela 8.5: Erros mais comuns.

8.9.2 Tratar erros previstos

```
# p07_erros.py | Capítulo 8: erros e como tratá-los
pset = {"NominalDiameter": 100.0}

# try / except: tenta executar; se der erro, trata sem interromper o programa
try:
    print(pset["WorkingPressure"])
except KeyError as erro:
    print("Propriedade ausente:", erro)

for texto in ["100", "DN100", ""]:
    try:
        print(int(texto) + 1)
    except ValueError:
        print(f"'{texto}' não é um número inteiro")

try:
    open("arquivo_que_nao_existe.ifc")
except FileNotFoundError as erro:
    print("Arquivo não encontrado:", erro.filename)

# Erros comuns, provocados de propósito, com a mensagem que o Python mostra
testes = [
    ("NameError", "print(variavel_inexistente)"),
    ("TypeError", "'DN' + 100"),
    ("IndexError", "[1, 2, 3][5]"),
    ("AttributeError", "'TB-001'.Name"),
    ("ZeroDivisionError", "10 / 0"),
]
for nome, codigo in testes:
    try:
        eval(codigo)                        # eval executa um texto como código (só para demonstração)
    except Exception as erro:
        print(f"{type(erro).__name__}: {erro}")
```

Saída obtida:

```
Propriedade ausente: 'WorkingPressure'
101
'DN100' não é um número inteiro
'' não é um número inteiro
Arquivo não encontrado: arquivo_que_nao_existe.ifc
NameError: name 'variavel_inexistente' is not defined
TypeError: can only concatenate str (not "int") to str
IndexError: list index out of range
AttributeError: 'str' object has no attribute 'Name'
ZeroDivisionError: division by zero
```

Use `try` / `except` para erros **previstos** e com tratamento claro (um arquivo que pode não existir, uma célula de planilha com texto no lugar de número). Não use `except` genérico para esconder erros que você não entende: o programa seguiria com dados errados, o que é pior do que parar.

> **ATENÇÃO**
> O exemplo usa `eval` apenas para provocar os erros de forma compacta. Não use `eval` em scripts reais: executar texto como código é inseguro e dificulta encontrar erros.

8.10 Classes e objetos

8.10.1 O conceito

No Capítulo 3, o esquema IFC foi apresentado como um conjunto de entidades (classes) e o arquivo como um conjunto de instâncias (objetos). O Python usa as mesmas ideias:

- uma **classe** é o modelo (o "formulário");
- um **objeto** é uma instância da classe, com seus próprios dados;
- os **atributos** são os dados do objeto (`t1.tag`);
- os **métodos** são funções que pertencem ao objeto (`t1.volume_interno_m3()`).

```
# p08_objetos.py | Capítulo 8: classes, objetos, atributos e métodos
class Tubo:
    """Um trecho de tubo simplificado, para estudo."""

    def __init__(self, tag, dn_mm, comprimento_m):
        self.tag = tag                      # atributos: dados do objeto
        self.dn_mm = dn_mm
        self.comprimento_m = comprimento_m

    def volume_interno_m3(self):            # método: função que pertence ao objeto
        area = 3.141592653589793 * (self.dn_mm / 1000) ** 2 / 4
        return area * self.comprimento_m

    def __repr__(self):                     # como o objeto aparece no print
        return f"Tubo({self.tag}, DN{self.dn_mm}, {self.comprimento_m} m)"


t1 = Tubo("TB-001", 100, 6.0)               # cria um objeto (uma instância da classe)
t2 = Tubo("TB-002", 80, 4.5)
print(t1, t2)
print(t1.tag, round(t1.volume_interno_m3(), 4), "m³")
print(isinstance(t1, Tubo), type(t2).__name__)
```

Saída obtida:

```
Tubo(TB-001, DN100, 6.0 m) Tubo(TB-002, DN80, 4.5 m)
TB-001 0.0471 m³
True Tubo
```

O parâmetro `self` representa o próprio objeto dentro dos métodos. O método especial `__init__` é executado quando o objeto é criado; `__repr__` define como o objeto aparece no `print`.

8.10.2 As instâncias IFC são objetos

O IfcOpenShell transforma o arquivo e cada linha do arquivo em objetos Python. Os atributos têm os mesmos nomes do esquema; os métodos oferecem operações úteis:

```
# p09_objetos_ifc.py | Capítulo 8: as instâncias IFC são objetos Python
import sys
import ifcopenshell

ARQUIVO = sys.argv[1] if len(sys.argv) > 1 else (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Infra-Plumbing.ifc")

modelo = ifcopenshell.open(ARQUIVO)         # o arquivo inteiro vira um objeto
tubo = modelo.by_id(35)                     # uma instância: também um objeto

print(type(modelo).__name__, type(tubo).__name__)
print(tubo.is_a(), tubo.id())               # métodos: is_a() e id()
print(tubo.Name, "|", tubo.GlobalId)        # atributos: os mesmos nomes do esquema
print(tubo.is_a("IfcFlowSegment"))         # é subtipo de IfcFlowSegment? (herança)
print(tubo.get_info()["type"], len(tubo.get_info()))   # get_info: dicionário com os atributos

tubos = modelo.by_type("IfcPipeSegment")    # uma lista de objetos
print(len(tubos), "tubos; o primeiro é", tubos[0].Name)
nomes = sorted({t.Name for t in tubos})     # conjunto de nomes distintos, em ordem
print(nomes)
```

Saída obtida, com `Infra-Plumbing.ifc` (IFC 4.3):

```
file entity_instance
IfcPipeSegment 35
sewer pipe | 1F4KLUGJf1aAa0YXxTvOR5
True
IfcPipeSegment 11
24 tubos; o primeiro é sewer pipe
['sewer pipe']
```

`modelo` é um objeto da classe `file`; `tubo` é um `entity_instance`. `tubo.Name` lê o atributo `Name` (terceira posição da linha STEP); `tubo.is_a("IfcFlowSegment")` responde se o tubo é daquela entidade ou de um subtipo dela (herança, Capítulo 3); `get_info()` devolve um dicionário com o número (`id`), a entidade (`type`) e os nove atributos, 11 chaves no total. O Capítulo 9 aprofunda essas operações.

8.11 Planilhas Excel

Planilhas são a forma mais comum de trocar listas de metadados. Duas bibliotecas são usadas neste livro:

BIBLIOTECA	PONTO FORTE	USO TÍPICO
`openpyxl`	Controle de cada célula, formatação, fórmulas	Gerar planilhas formatadas; ler células específicas
`pandas`	Tabelas inteiras (`DataFrame`): filtrar, agrupar, cruzar, exportar	Ler a planilha de metadados; gerar relatórios

Tabela 8.6: Bibliotecas de planilhas.

```
# p10_planilhas.py | Capítulo 8: planilhas Excel com openpyxl e pandas
from pathlib import Path
import openpyxl
```

```
import pandas as pd

pasta = Path("saida_cap8")
pasta.mkdir(exist_ok=True)
arquivo = pasta / "metadados_exemplo.xlsx"

# 1. Criar uma planilha com openpyxl (controle célula a célula)
livro = openpyxl.Workbook()
aba = livro.active
aba.title = "Metadados"
aba.append(["TAG", "Classe", "DN_mm", "Fluido", "Revisao"])        # cabeçalho
aba.append(["TB-001", "IfcPipeSegment", 100, "Água bruta", "A"])
aba.append(["VG-001", "IfcValve", 100, "Água bruta", "A"])
aba.append(["B-001", "IfcPump", None, "Água bruta", "B"])
livro.save(arquivo)
print("Planilha gravada:", arquivo.name)

# 2. Ler com openpyxl
livro = openpyxl.load_workbook(arquivo)
aba = livro["Metadados"]
for linha in aba.iter_rows(min_row=2, values_only=True):          # pula o cabeçalho
    print(linha)

# 3. Ler com pandas: a planilha vira uma tabela (DataFrame)
tabela = pd.read_excel(arquivo, sheet_name="Metadados")
print(tabela)
print("Colunas:", list(tabela.columns))
print("Linhas com revisão A:", len(tabela[tabela["Revisao"] == "A"]))

# Percorrer as linhas como dicionários (forma usada no preenchimento em lote)
for registro in tabela.to_dict(orient="records"):
    dn = registro["DN_mm"]
    texto_dn = "sem DN" if pd.isna(dn) else f"DN {dn:.0f}"       # célula vazia vira NaN
    print(f"{registro['TAG']}: {texto_dn}, {registro['Fluido']}")
```

Saída obtida:

```
Planilha gravada: metadados_exemplo.xlsx
('TB-001', 'IfcPipeSegment', 100, 'Água bruta', 'A')
('VG-001', 'IfcValve', 100, 'Água bruta', 'A')
('B-001', 'IfcPump', None, 'Água bruta', 'B')
      TAG          Classe  DN_mm      Fluido Revisao
0  TB-001  IfcPipeSegment  100.0  Água bruta       A
1  VG-001        IfcValve  100.0  Água bruta       A
2   B-001         IfcPump    NaN  Água bruta       B
Colunas: ['TAG', 'Classe', 'DN_mm', 'Fluido', 'Revisao']
Linhas com revisão A: 2
TB-001: DN 100, Água bruta
VG-001: DN 100, Água bruta
B-001: sem DN, Água bruta
```

Observações:

- no `pandas`, uma célula vazia vira `NaN` (*not a number*, "não é um número"), e a coluna numérica passa a ser de números reais (`100.0`). Teste células vazias com `pd.isna(valor)`;
- `to_dict(orient="records")` transforma a tabela numa lista de dicionários, um por linha, com os nomes das colunas como chaves. É a forma usada no preenchimento em lote do Capítulo 10;
- o nome da coluna precisa ser escrito exatamente como está na planilha, inclusive maiúsculas e acentos. Por isso, planilhas de metadados devem ter cabeçalhos simples e estáveis.

8.12 Argumentos de linha de comando

Os scripts deste livro recebem o arquivo e outras opções na linha de comando. A forma mais simples é `sys.argv`, a lista de textos digitados após `python` (o índice 0 é o nome do script). A forma mais completa é o módulo `argparse`, que gera ajuda automática e trata opções nomeadas:

```
# p11_argumentos.py | Capítulo 8: argumentos de linha de comando com argparse
import argparse

leitor = argparse.ArgumentParser(description="Exemplo de script com argumentos.")
leitor.add_argument("arquivo", help="caminho do arquivo IFC de entrada")
leitor.add_argument("--classe", default="IfcElement", help="classe a consultar (padrão: IfcElement)")
leitor.add_argument("--saida", help="arquivo de saída (opcional)")
args = leitor.parse_args()

print("Arquivo:", args.arquivo)
print("Classe..:", args.classe)
print("Saída...:", args.saida if args.saida else "(não informada)")
```

Execução e saída obtida:

```
python p11_argumentos.py modelo.ifc --classe IfcValve
```

```
Arquivo: modelo.ifc
Classe..: IfcValve
Saída...: (não informada)
```

Com `python p11_argumentos.py --help`, o `argparse` mostra a ajuda: a sintaxe, a descrição e cada opção.

8.13 Exemplo integrador: do IFC para o Excel

O script abaixo reúne os recursos do capítulo: argumentos, funções, dicionários, listas, tratamento de erros e `pandas`. Ele lê um IFC (somente leitura) e grava uma planilha **nova** com os elementos de uma classe: GlobalId, classe, nome, tipo, `PredefinedType` do tipo, contêiner e quantidade de conjuntos de propriedades.

```
# -*- coding: utf-8 -*-
"""
p12_ifc_para_planilha.py | Capítulo 8: exemplo integrador (IFC -> Excel).

Reúne os recursos do capítulo: argumentos, funções, dicionários, listas,
tratamento de erros e pandas. Lê um arquivo IFC (somente leitura) e grava
uma planilha NOVA com os elementos de uma classe.

Uso:
    python p12_ifc_para_planilha.py <arquivo.ifc> [--classe IfcElement] [--saida elementos.xlsx]

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; pandas 3.0; openpyxl 3.1.
"""
import argparse
from pathlib import Path
import pandas as pd
import ifcopenshell
import ifcopenshell.util.element as util_elem


def linha_do_elemento(elem):
    """Monta um dicionário (uma linha da planilha) para um elemento."""
    tipo = util_elem.get_type(elem)
```

```
    conteiner = util_elem.get_container(elem)
    return {
        "GlobalId": elem.GlobalId,
        "Classe": elem.is_a(),
        "Nome": elem.Name,
        "Tipo": tipo.Name if tipo else None,
        "PredefinedType (tipo)": getattr(tipo, "PredefinedType", None) if tipo else None,
        "Contêiner": conteiner.Name if conteiner else None,
        "Qtd. de conjuntos": len(util_elem.get_psets(elem)),
    }


def main():
    leitor = argparse.ArgumentParser(description="Exporta elementos de um IFC para Excel.")
    leitor.add_argument("arquivo")
    leitor.add_argument("--classe", default="IfcElement")
    leitor.add_argument("--saida", default="elementos.xlsx")
    args = leitor.parse_args()

    saida = Path(args.saida)
    if saida.resolve() == Path(args.arquivo).resolve():
        raise SystemExit("A saída não pode ser o próprio arquivo de entrada.")

    try:
        modelo = ifcopenshell.open(args.arquivo)
    except (OSError, RuntimeError) as erro:            # arquivo inexistente ou inválido
        raise SystemExit(f"Não foi possível abrir o arquivo: {erro}")

    try:
        elementos = modelo.by_type(args.classe)
    except RuntimeError:                               # classe inexistente nesta versão
        raise SystemExit(f"A classe {args.classe} não existe no esquema {modelo.schema_identifier}.")

    tabela = pd.DataFrame([linha_do_elemento(e) for e in elementos])
    tabela.to_excel(saida, index=False, sheet_name="Elementos")
    print(f"{len(tabela)} elementos da classe {args.classe} gravados em {saida}")
    if not tabela.empty:
        print(tabela.groupby("Classe").size().to_string())


if __name__ == "__main__":       # executa main() só quando o arquivo é rodado como script
    main()
```

Execução, com a amostra de drenagem:

```
python p12_ifc_para_planilha.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Infra-Plumbing.ifc" --saida drenagem.xlsx
```

Saída obtida no teste:

```
29 elementos da classe IfcElement gravados em drenagem.xlsx
Classe
IfcBuildingElementProxy           1
IfcDistributionChamberElement     2
IfcElementAssembly                2
IfcPipeSegment                   24
```

Também foram testadas as situações de erro: com `--classe IfcPipe` (entidade inexistente), o script responde "A classe IfcPipe não existe no esquema IFC4X3_ADD2."; com um arquivo inexistente, "Não foi possível abrir o arquivo: Path does not exist: ...". O teste que impede gravar a saída sobre o arquivo de entrada protege os arquivos originais.

8.14 Boas práticas

1. **Nunca altere o arquivo de entrada.** Grave sempre resultados em arquivos novos, com nomes que indiquem o conteúdo e a data ou a revisão.
2. **Trabalhe em cópias** das amostras e dos modelos reais, numa pasta de exercícios.
3. **Dê nomes claros** a variáveis e funções: `comprimento_total_m` diz mais que `x`.
4. **Escreva funções pequenas**, cada uma com uma tarefa, e documente-as com docstrings.
5. **Comente o porquê**, não o óbvio: `# IFC2X3 não tem IsTypedBy` é útil; `# soma 1` não é.
6. **Indique unidades** nos nomes ou nos comentários (`_mm`, `_m`, `_pa`).
7. **Verifique a versão do esquema** (`modelo.schema_identifier`) quando o comportamento mudar entre versões.
8. **Teste com arquivos pequenos** antes de rodar em modelos grandes, e confira manualmente alguns resultados.
9. **Registre as versões** do Python e das bibliotecas usadas (`pip freeze > requisitos.txt`), para que o script possa ser repetido no futuro.
10. **Use o depurador** do VS Code quando o `print` não bastar: um clique à esquerda do número da linha cria um ponto de parada, e o comando Executar e Depurar (`F5`) roda o script até ali, mostrando o valor de cada variável.

8.15 Resumo do capítulo

- Um script Python é um texto com instruções executadas em ordem pelo interpretador do ambiente virtual.
- Tipos básicos: `str`, `int`, `float`, `bool` e `None`; o separador decimal é o ponto; números reais têm precisão limitada.
- Coleções: listas (ordem), tuplas (fixas), dicionários (chave e valor) e conjuntos (únicos). Dicionários representam bem conjuntos de propriedades e linhas de planilha.
- Blocos são definidos pela indentação de 4 espaços; `if`, `for`, `while` e compreensões controlam o fluxo.
- Funções organizam o código; módulos e bibliotecas o ampliam.
- Arquivos: `pathlib`, `encoding="utf-8"`, `csv` com `;` para o Excel brasileiro, `json`, `openpyxl` e `pandas`.
- Erros são lidos de baixo para cima; `try`/`except` trata só os previstos.
- As instâncias IFC são objetos Python com os atributos do esquema.

8.16 Vocabulário técnico

INGLÊS	PORTUGUÊS
script / interpreter	roteiro (programa curto) / interpretador
variable / assignment	variável / atribuição
string / integer / float / boolean	texto / inteiro / real / lógico

INGLÊS	PORTUGUÊS
list / tuple / dictionary / set	lista / tupla / dicionário / conjunto
index / slice	índice / fatia
loop / condition	repetição / condição
function / parameter / argument / return value	função / parâmetro / argumento / valor de retorno
module / package / library / import	módulo / pacote / biblioteca / importar
exception / traceback	exceção (erro) / rastreamento de chamadas
class / object / attribute / method	classe / objeto / atributo / método
DataFrame	tabela do pandas
encoding	codificação de caracteres
debugger / breakpoint	depurador / ponto de parada

Tabela 8.7: Vocabulário do capítulo.

8.17 Exercícios

Fixação

1. Qual é o tipo Python de cada valor: `"IfcWall"`, `3`, `3.0`, `True`, `None`, `[1, 2]`, `{"a": 1}`?
2. Dada a lista `tags = ["TB-001", "TB-002", "VG-001", "B-001"]`, o que retornam `tags[1]`, `tags[-2]` e `tags[1:3]`?
3. Por que `0.1 + 0.2 == 0.3` resulta em `False`? Como comparar corretamente?
4. Qual a diferença entre `pset["Status"]` e `pset.get("Status")` quando a chave não existe?
5. O que acontece se um bloco dentro de um `for` não estiver recuado?

Aplicação

6. Escreva uma função `peso_tubo(dn_mm, espessura_mm, comprimento_m, densidade=7850)` que retorne o peso, em kg, de um tubo de aço (área da coroa circular x comprimento x densidade). Teste com DN externo 114,3 mm, espessura 6,02 mm e 6 m.
7. Modifique uma cópia de `p10_planilhas.py` para, além de ler, gravar uma nova planilha `metadados_filtrados.xlsx` só com as linhas de revisão A.

Desafio

8. Usando `p12_ifc_para_planilha.py` como base (numa cópia), acrescente à planilha uma coluna com o valor da propriedade `NominalDiameter` do conjunto `Pset_PipeSegmentTypeCommon`, quando existir. Dica: `util_elem.get_psets(elem)` devolve um dicionário de dicionários; use `.get` para evitar erros quando o conjunto ou a propriedade não existirem. Teste com o arquivo `rede_exemplo.ifc` do Capítulo 6.

8.18 Respostas dos exercícios

1. `str`, `int`, `float`, `bool`, `NoneType` (o tipo de `None`), `list`, `dict`.
2. `tags[1]` = `"TB-002"`; `tags[-2]` = `"VG-001"`; `tags[1:3]` = `["TB-002", "VG-001"]` (índices 1 e 2).
3. Porque 0,1 e 0,2 não têm representação exata em binário, e a soma resulta em 0,30000000000000004. Compare pela diferença com uma tolerância: `abs((0.1 + 0.2) - 0.3) < 1e-9`, que resulta em `True`.
4. `pset["Status"]` gera `KeyError`; `pset.get("Status")` devolve `None` (ou o valor padrão informado no segundo argumento).
5. O Python gera `IndentationError` (esperava um bloco recuado), e o programa nem começa a ser executado.
6. Uma solução:

```
def peso_tubo(dn_mm, espessura_mm, comprimento_m, densidade=7850):
    """Peso de um tubo, em kg: área da coroa (m²) x comprimento (m) x densidade (kg/m³)."""
    externo = dn_mm / 1000
    interno = (dn_mm - 2 * espessura_mm) / 1000
    area = 3.141592653589793 * (externo ** 2 - interno ** 2) / 4
    return area * comprimento_m * densidade


print(round(peso_tubo(114.3, 6.02, 6.0), 1))    # resultado obtido no teste: 96.5
```

O resultado, 96,5 kg (16,08 kg/m), é coerente com o peso tabelado de um tubo de 4" SCH 40 (cerca de 16,1 kg/m).

7. Após ler a tabela, acrescentar: `tabela[tabela["Revisao"] == "A"].to_excel(pasta / "metadados_filtrados.xlsx", index=False)`. A planilha nova terá as linhas de TB-001 e VG-001. Testado.
8. Na função `linha_do_elemento`, acrescentar ao dicionário a chave `"NominalDiameter": util_elem.get_psets(elem).get("Pset_PipeSegmentTypeCommon", {}).get("NominalDiameter")`. Como `get_psets` considera também os conjuntos do tipo (Capítulo 5), os dois tubos de `rede_exemplo.ifc` recebem 100.0 (mm), valor gravado no tipo; os demais elementos ficam vazios. Testado.

8.19 Referências

- Tutorial oficial do Python, em português: https://docs.python.org/pt-br/3/tutorial/
- Documentação do pandas: https://pandas.pydata.org/docs/
- Documentação do openpyxl: https://openpyxl.readthedocs.io/
- Documentação do IfcOpenShell: https://docs.ifcopenshell.org/

9 IfcOpenShell I: leitura e consulta

Com o esquema entendido (Capítulos 3 a 7) e o Python básico dominado (Capítulo 8), é hora de usar a principal ferramenta aberta para trabalhar com IFC por programa: o **IfcOpenShell**. Este capítulo trata da leitura: abrir arquivos, encontrar objetos, ler atributos, percorrer relações, obter propriedades e exportar resultados para planilhas. O Capítulo 10 trata da edição.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que é o IfcOpenShell e como ele se organiza em módulos;
- abrir um arquivo e localizar instâncias por entidade, número e GlobalId;
- ler atributos por nome e por posição e interpretar os valores devolvidos;
- percorrer relações por atributos inversos, `get_inverse` e `traverse`;
- usar as funções prontas de `ifcopenshell.util` para tipo, contêiner, propriedades, materiais e classificação;
- selecionar elementos com consultas em texto e exportar os resultados para Excel ou CSV;
- escrever consultas que funcionem em IFC2x3, IFC4 e IFC 4.3.

9.1 Introdução: o que é o IfcOpenShell

9.1.1 Uma biblioteca aberta

O **IfcOpenShell** é uma biblioteca de código aberto (licença LGPL) para ler, escrever, validar e processar arquivos IFC. O núcleo é escrito em C++, o que o torna rápido, e o uso é feito, em geral, pelo Python. Ele dá suporte às versões IFC2x3, IFC4 e IFC 4.3 e é a base de várias ferramentas, entre elas o **Bonsai**, a extensão de BIM do Blender instalada no Capítulo 0: tudo o que o Bonsai faz com o modelo passa pelo IfcOpenShell.

Este livro usa a versão **0.8.5**. A documentação oficial está em https://docs.ifcopenshell.org/.

9.1.2 Os módulos

MÓDULO	FUNÇÃO	CAPÍTULO
`ifcopenshell`	Abrir, criar e gravar arquivos; acessar instâncias e atributos	9 e 10
`ifcopenshell.util`	Funções prontas de consulta: elementos, propriedades, unidades, posicionamento, georreferenciamento, seletor	9
`ifcopenshell.api`	Funções de edição de alto nível, que mantêm o modelo coerente	10
`ifcopenshell.geom`	Núcleo geométrico: gera malhas, calcula volumes e caixas envolventes	11
`ifcopenshell.validate`	Validação contra o esquema e suas regras	12
`ifctester` (pacote separado)	Verificação de requisitos com IDS	12
`ifcdiff` (pacote separado)	Comparação entre versões de um modelo	12

Tabela 9.1: Módulos do IfcOpenShell usados no livro.

9.1.3 Leitura em camadas

Há dois níveis de acesso, e ambos são usados neste capítulo:

- **nível baixo**: o arquivo e suas instâncias, exatamente como estão no texto (Capítulo 2). Dá controle total, mas exige conhecer o esquema e as diferenças entre versões;
- **nível alto**: as funções de `ifcopenshell.util`, que respondem perguntas comuns ("qual o tipo deste elemento?", "quais são as propriedades efetivas?") e já tratam as diferenças entre versões.

A regra prática: use as funções de `util` sempre que existirem; desça ao nível baixo quando precisar de algo que elas não oferecem, ou para entender o que está acontecendo.

9.2 Abrir o arquivo e encontrar instâncias

O primeiro script percorre as operações básicas. Grave-o como `C:\IFC_Curso\exercicios\cap9\c9_01_abrir_consultar.py`.

```
# -*- coding: utf-8 -*-
"""
c9_01_abrir_consultar.py | Capítulo 9: abrir um arquivo e consultar instâncias.

Mostra: cabeçalho, contagem por entidade, by_type (com e sem subtipos), by_id,
by_guid, leitura de atributos, get_info, get_inverse e traverse.
Somente leitura: nada é gravado.

Uso: python c9_01_abrir_consultar.py [arquivo.ifc]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
from collections import Counter
import ifcopenshell

ARQUIVO = sys.argv[1] if len(sys.argv) > 1 else (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Infra-Plumbing.ifc")

modelo = ifcopenshell.open(ARQUIVO)

print("=== 1. ARQUIVO ===")
print("Esquema...........:", modelo.schema_identifier)
print("Sistema de origem.:", modelo.header.file_name.originating_system)
print("Vista de modelo...:", modelo.header.file_description.description)
print("Total de instâncias:", len(list(modelo)))
print("Instâncias de IfcRoot (com GlobalId):", len(modelo.by_type("IfcRoot")))

print("\n=== 2. AS 5 ENTIDADES MAIS FREQUENTES ===")
for entidade, qtd in Counter(inst.is_a() for inst in modelo).most_common(5):
    print(f"   {entidade:<28} {qtd}")

print("\n=== 3. by_type ===")
print("IfcFlowSegment com subtipos....:", len(modelo.by_type("IfcFlowSegment")))
print("IfcFlowSegment sem subtipos....:", len(modelo.by_type("IfcFlowSegment", include_subtypes=False)))
print("IfcDistributionElement.........:", len(modelo.by_type("IfcDistributionElement")))

print("\n=== 4. by_id E by_guid ===")
tubo = modelo.by_id(35)
mesmo = modelo.by_guid(tubo.GlobalId)
print(tubo)
print("by_guid encontra a mesma instância?", mesmo == tubo)

print("\n=== 5. ATRIBUTOS ===")
print("Por nome.....:", tubo.Name, "|", tubo.ObjectType, "|", tubo.PredefinedType)
```

```
print("Por posição..:", tubo[2], "(posição 3 no arquivo = índice 2 no Python)")
print("Nome do 9º...:", tubo.attribute_name(8))
info = tubo.get_info()                          # dicionário com os atributos
print("get_info.....:", {k: info[k] for k in ["id", "type", "GlobalId", "Name"]})

print("\n=== 6. QUEM APONTA PARA O TUBO (get_inverse) ===")
for rel in sorted(modelo.get_inverse(tubo), key=lambda r: r.id()):
    print(f"   #{rel.id()} {rel.is_a()}")

print("\n=== 7. DO QUE O TUBO DEPENDE (traverse) ===")
dependencias = modelo.traverse(tubo)            # tudo o que é referenciado, direta ou indiretamente
print("Instâncias alcançadas:", len(dependencias))
print("Primeiro nível.......:", [d.is_a() for d in modelo.traverse(tubo, max_levels=1)][1:])
```

Execução e saída obtida com `Infra-Plumbing.ifc` (IFC 4.3):

```
cd C:\IFC_Curso\exercicios\cap9
python c9_01_abrir_consultar.py
```

```
=== 1. ARQUIVO ===
Esquema...........: IFC4X3_ADD2
Sistema de origem.: BIM_Tools - Sketchup_IFC_manager - 5.6.0
Vista de modelo...: ('ViewDefinition [ReferenceView]',)
Total de instâncias: 443
Instâncias de IfcRoot (com GlobalId): 62

=== 2. AS 5 ENTIDADES MAIS FREQUENTES ===
   IfcDirection                74
   IfcLocalPlacement           38
   IfcAxis2Placement3D         37
   IfcCartesianPoint           37
   IfcTriangulatedFaceSet      35

=== 3. by_type ===
IfcFlowSegment com subtipos....: 24
IfcFlowSegment sem subtipos....: 0
IfcDistributionElement.........: 26

=== 4. by_id E by_guid ===
#35=IfcPipeSegment('1F4KLUGJf1aAa0YXxTvOR5',#1,'sewer pipe','A sturdy sewer pipe, ensuring efficient waste
management.','culvert',#39,#51,'454425.1027892.1880634.1093597',$)
by_guid encontra a mesma instância? True

=== 5. ATRIBUTOS ===
Por nome.....: sewer pipe | culvert | None
Por posição..: sewer pipe (posição 3 no arquivo = índice 2 no Python)
Nome do 9º...: PredefinedType
get_info.....: {'id': 35, 'type': 'IfcPipeSegment', 'GlobalId': '1F4KLUGJf1aAa0YXxTvOR5', 'Name': 'sewer pipe'}

=== 6. QUEM APONTA PARA O TUBO (get_inverse) ===
   #32 IfcRelAssignsToGroup
   #34 IfcRelDefinesByType
   #36 IfcRelAssociatesMaterial
   #38 IfcRelContainedInSpatialStructure

=== 7. DO QUE O TUBO DEPENDE (traverse) ===
Instâncias alcançadas: 30
Primeiro nível.......: ['IfcOwnerHistory', 'IfcLocalPlacement', 'IfcProductDefinitionShape']
```

9.2.1 O arquivo

`ifcopenshell.open(caminho)` lê o arquivo inteiro e devolve um objeto `file`. A partir dele:

OPERAÇÃO	RESULTADO
`modelo.schema_identifier`	Versão exata do esquema (`IFC2X3` , `IFC4` , `IFC4X3_ADD2`)

OPERAÇÃO	RESULTADO
`modelo.schema`	Versão resumida (`IFC2X3`, `IFC4`, `IFC4X3`)
`modelo.header`	Cabeçalho: `file_description`, `file_name`, `file_schema` (Capítulo 2)
`list(modelo)`	Todas as instâncias
`modelo.by_type("IfcRoot")`	Instâncias com GlobalId

Tabela 9.2: Operações sobre o arquivo.

A amostra tem 443 instâncias, das quais só 62 derivam de `IfcRoot`; as mais frequentes são de geometria (`IfcDirection`, `IfcCartesianPoint`) e posicionamento. É o padrão da maioria dos arquivos: a informação "de negócio" é uma fração pequena do conteúdo.

9.2.2 by_type, by_id e by_guid

MÉTODO	RETORNA	OBSERVAÇÃO
`by_type("IfcFlowSegment")`	Lista de instâncias da entidade **e de seus subtipos**	É o método mais usado. Aceita entidades abstratas
`by_type("IfcFlowSegment", include_subtypes=False)`	Só instâncias da própria entidade	Útil para detectar ocorrências genéricas (IFC2x3)
`by_id(35)`	A instância `#35`	O número só vale naquele arquivo (Capítulo 2)
`by_guid("1F4KLUGJf1aAa0YXxTvOR5")`	A instância com aquele GlobalId	É a forma estável de localizar um objeto entre revisões

Tabela 9.3: Métodos de busca.

Na saída, `by_type("IfcFlowSegment")` encontra 24 instâncias (os `IfcPipeSegment`), e a mesma busca sem subtipos encontra 0: no IFC 4.3, os tubos não são gravados como `IfcFlowSegment` genérico. Num arquivo IFC2x3, o resultado seria o inverso (Capítulo 6).

`by_type` com uma entidade inexistente na versão do arquivo gera erro (`RuntimeError`). `by_id` e `by_guid` com um número ou GlobalId inexistente também geram erro; trate com `try` / `except` quando o valor vier de fora do script (por exemplo, de uma planilha).

9.3 Ler atributos

9.3.1 Por nome e por posição

Os atributos podem ser lidos pelo **nome** do esquema (`tubo.Name`) ou pela **posição**, com índice começando em 0 (`tubo[2]` é o terceiro atributo). O método `attribute_name(i)` informa o nome do atributo na posição i, e `get_info()` devolve um dicionário com todos os atributos, mais o número (`id`) e a entidade (`type`).

9.3.2 O que cada atributo devolve

NO ARQUIVO	NO PYTHON	EXEMPLO
Texto `'sewer pipe'`	`str`	`tubo.Name` = `'sewer pipe'`
Número `1000.`	`float` ou `int`	`extrusao.Depth` = `1000.0`
Enumeração `.CULVERT.`	`str`, sem os pontos	`tipo.PredefinedType` = `'CULVERT'`
Lógico `.T.` / `.F.`	`bool`	`True` / `False`
Vazio `$`	`None`	`tubo.PredefinedType` = `None`
Referência `#39`	`entity_instance`	`tubo.ObjectPlacement`
Lista `(#35,#58)`	`tuple`	`rel.RelatedObjects`
Valor tipado `IFCLABEL('REI30')`	`entity_instance` com `wrappedValue`	`prop.NominalValue.wrappedValue` = `'REI30'`

Tabela 9.4: Valores devolvidos pelo IfcOpenShell.

Acessar um atributo que não existe na entidade (ou na versão) gera `AttributeError`. Para código que deve funcionar em várias versões, use `getattr(objeto, "Atributo", None)`, que devolve `None` em vez de gerar erro (seção 9.8).

9.4 Navegar pelas relações

Há três formas de ir de um objeto aos objetos ligados a ele:

1. **atributos diretos**, que o objeto guarda: `tubo.ObjectPlacement`, `rel.RelatingType`;
2. **atributos inversos**, deduzidos na leitura (Capítulo 3): `tubo.IsTypedBy`, `tubo.ContainedInStructure`, `tubo.HasAssociations`, `tubo.IsDefinedBy`;
3. **métodos do arquivo**: `modelo.get_inverse(inst)` devolve **todas** as instâncias que apontam para `inst`, de qualquer tipo; `modelo.traverse(inst)` devolve tudo o que `inst` referencia, direta ou indiretamente.

Na saída do script, `get_inverse` mostrou as quatro relações que citam o tubo (grupo, tipo, material e contêiner), e `traverse` alcançou 30 instâncias: o tubo, o registro de autoria, o posicionamento, a forma e toda a geometria que eles usam. `get_inverse` é útil para descobrir relações que você não sabia existir; `traverse` é útil para copiar ou medir tudo o que um objeto usa.

9.5 Funções prontas: ifcopenshell.util

9.5.1 As funções mais usadas

O módulo `ifcopenshell.util.element` resolve as perguntas mais comuns sobre um elemento, sem que seja preciso percorrer as relações à mão:

FUNÇÃO	PERGUNTA QUE RESPONDE
`get_type(elem)`	Qual é o tipo do elemento?

FUNÇÃO	PERGUNTA QUE RESPONDE
`get_types(tipo)`	Quais ocorrências usam este tipo?
`get_container(elem)`	Em que elemento espacial ele está contido?
`get_aggregate(elem)`	De que todo ele faz parte (agregação)?
`get_decomposition(elem)`	Quais são todas as suas partes, recursivamente?
`get_psets(elem)`	Quais são os conjuntos de propriedades e de quantidades efetivos (tipo + ocorrência)?
`get_pset(elem, "Pset_...", "Prop")`	Qual é o valor efetivo de uma propriedade?
`get_material(elem)` / `get_materials(elem)`	Qual é o material (ou a lista de materiais)?
`get_groups(elem)`	A que grupos e sistemas ele pertence?

Tabela 9.5: Funções de `ifcopenshell.util.element`.

Outros módulos de `util` usados no livro: `util.classification.get_references` (classificações), `util.unit` (unidades, Capítulo 5), `util.placement` e `util.geolocation` (Capítulo 7), `util.system` (portas, Capítulo 6) e `util.selector` (seção 9.6).

9.5.2 Exemplo

```
# -*- coding: utf-8 -*-
"""
c9_02_util_element.py | Capítulo 9: funções prontas de ifcopenshell.util.

Para um elemento: tipo, ocorrências do tipo, contêiner, agregação, conjuntos de
propriedades e quantidades, uma propriedade específica, material, classificação e grupos.
Somente leitura: nada é gravado.

Uso: python c9_02_util_element.py [arquivo.ifc] [id]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.util.classification as util_class
import ifcopenshell.util.element as util_elem

ARQUIVO = sys.argv[1] if len(sys.argv) > 1 else (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc")
NUMERO = int(sys.argv[2]) if len(sys.argv) > 2 else 53   # #53: a laje térrea

modelo = ifcopenshell.open(ARQUIVO)
elem = modelo.by_id(NUMERO)
print(f"Elemento: #{elem.id()} {elem.is_a()} '{elem.Name}'")

tipo = util_elem.get_type(elem)
print("\nget_type.............:", tipo.is_a() if tipo else None, "|", tipo.Name if tipo else "")
if tipo:
    print("get_types (do tipo)..:", len(util_elem.get_types(tipo)), "ocorrência(s)")
conteiner = util_elem.get_container(elem)
print("get_container........:", conteiner.is_a() if conteiner else None, "|",
      conteiner.Name if conteiner else "")
todo = util_elem.get_aggregate(elem)
print("get_aggregate........:", todo.is_a() if todo else None)

print("\nget_psets (efetivo, ocorrência + tipo):")
for nome, props in util_elem.get_psets(elem).items():
```

```
    print(f"   {nome}: {props}")
print("get_psets(psets_only=True):", list(util_elem.get_psets(elem, psets_only=True)))
print("get_psets(qtos_only=True).:", list(util_elem.get_psets(elem, qtos_only=True)))
print("get_pset(..., 'FireRating'):", util_elem.get_pset(elem, "Pset_SlabCommon", "FireRating"))
print("Só na ocorrência (should_inherit=False):",
      util_elem.get_pset(elem, "Pset_SlabCommon", "SurfaceSpreadOfFlame", should_inherit=False))

material = util_elem.get_material(elem)
print("\nget_material.........:", material.is_a() if material else None, "|",
      getattr(material, "Name", None) if material else "")
print("get_references (classificação):",
      [r.Identification for r in util_class.get_references(elem)] or "nenhuma")
print("get_groups...........:", [g.Name for g in util_elem.get_groups(elem)] or "nenhum")
```

Saída obtida com a laje `#53` de `Building-Architecture.ifc`:

```
Elemento: #53 IfcSlab 'floor'

get_type.............: IfcSlabType | house - groundfloor
get_types (do tipo)..: 1 ocorrência(s)
get_container........: IfcBuildingStorey | 00 groundfloor
get_aggregate........: None

get_psets (efetivo, ocorrência + tipo):
   Pset_SlabCommon: {'FireRating': 'REI30', 'SurfaceSpreadOfFlame': 'A2 s1 d0', 'id': 58, 'IsExternal': True,
'LoadBearing': False, 'AcousticRating': '29dB Rw'}
   Qto_SlabBaseQuantities: {'NetVolume': 6.437500000000378, 'Depth': 250.0, 'NetArea': 25.749999999991743, 'id':
65}
get_psets(psets_only=True): ['Pset_SlabCommon']
get_psets(qtos_only=True).: ['Qto_SlabBaseQuantities']
get_pset(..., 'FireRating'): REI30
Só na ocorrência (should_inherit=False): None

get_material.........: IfcMaterial | concrete_reinforced_in-situ
get_references (classificação): nenhuma
get_groups...........: nenhum
```

Observe:

- `get_psets` devolve o conjunto **efetivo**, com a regra do Capítulo 5: `FireRating` = `REI30` (da ocorrência, que sobrescreve o `REI60` do tipo) e `SurfaceSpreadOfFlame` (só do tipo). A chave `id` informa o número da instância do conjunto;
- `should_inherit=False` ignora o tipo: `SurfaceSpreadOfFlame` não existe na ocorrência, e o resultado é `None`;
- `psets_only` e `qtos_only` separam propriedades (`Pset_...`) de quantidades (`Qto_...`);
- as funções devolvem `None` ou listas vazias quando não há o que retornar, em vez de gerar erro.

9.6 Consultas em texto: o seletor

9.6.1 filter_elements

O módulo `ifcopenshell.util.selector` permite selecionar elementos com uma **consulta em texto**, sem escrever repetições e condições. A mesma sintaxe é usada no Bonsai, para filtrar elementos na interface.

```
# -*- coding: utf-8 -*-
"""
c9_03_seletor.py | Capítulo 9: consultas com ifcopenshell.util.selector.

filter_elements: seleciona elementos com uma consulta em texto.
```

```
get_element_value: lê um valor de um elemento com um "caminho" em texto.
Somente leitura: nada é gravado.

Uso: python c9_03_seletor.py [arquivo.ifc]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.util.selector as seletor

ARQUIVO = sys.argv[1] if len(sys.argv) > 1 else (
    r"C:\IFC_Curso\amostras\bSI\Certification-datasets-main"
    r"\IFC 4.3.2.0 (IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc")
modelo = ifcopenshell.open(ARQUIVO)

consultas = [
    "IfcWall",                                          # uma classe
    "IfcWall + IfcSlab",                                # união (+)
    "IfcElement, !IfcWall",                             # exclusão (!)
    "IfcSlab, Pset_SlabCommon.FireRating=REI30",        # valor de propriedade
    "IfcSlab, Pset_SlabCommon.FireRating=/REI.*/",      # expressão regular entre barras
    "IfcWall, Qto_WallBaseQuantities.Length>1500",      # comparação numérica
    "IfcElement, material=stone_sand-lime",             # material
    'IfcSpace, location="00 groundfloor"',              # localização (contêiner)
    'IfcSlab, type="house - groundfloor"',              # nome do tipo
    "IfcBuilding, classification=E-AAA",                # código de classificação
]
print("=== filter_elements ===")
for consulta in consultas:
    encontrados = seletor.filter_elements(modelo, consulta)
    ids = sorted(e.id() for e in encontrados)
    print(f"{consulta:<46} -> {len(encontrados)}: {ids}")

print("\n=== get_element_value (parede #158) ===")
parede = modelo.by_id(158)
for caminho in ["Name", "class", "type.Name", "container.Name", "material.Name",
                "Qto_WallBaseQuantities.Length", "Qto_WallBaseQuantities.NetSideArea"]:
    print(f"{caminho:<36} = {seletor.get_element_value(parede, caminho)}")
```

Saída obtida:

```
=== filter_elements ===
IfcWall                                     -> 4: [158, 182, 201, 228]
IfcWall + IfcSlab                           -> 7: [53, 158, 182, 201, 228, 261, 285]
IfcElement, !IfcWall                        -> 9: [53, 116, 220, 252, 261, 285, 317, 335, 353]
IfcSlab, Pset_SlabCommon.FireRating=REI30   -> 1: [53]
IfcSlab, Pset_SlabCommon.FireRating=/REI.*/ -> 1: [53]
IfcWall, Qto_WallBaseQuantities.Length>1500 -> 4: [158, 182, 201, 228]
IfcElement, material=stone_sand-lime        -> 4: [158, 182, 201, 220]
IfcSpace, location="00 groundfloor"         -> 2: [85, 135]
IfcSlab, type="house - groundfloor"         -> 1: [53]
IfcBuilding, classification=E-AAA           -> 1: [35]

=== get_element_value (parede #158) ===
Name                                = house - outer wall - house right front
class                               = IfcWall
type.Name                           = house - outer wall - house right front
container.Name                      = 00 groundfloor
material.Name                       = stone_sand-lime
Qto_WallBaseQuantities.Length       = 1800.0
Qto_WallBaseQuantities.NetSideArea  = 6.346324676317878
```

Sintaxe das consultas, verificada na versão 0.8.5:

ELEMENTO DA CONSULTA	EXEMPLO	SIGNIFICADO
Entidade	`IfcWall`	Elementos da entidade e de seus subtipos

ELEMENTO DA CONSULTA	EXEMPLO	SIGNIFICADO
Vírgula	`IfcSlab, Pset_SlabCommon.FireRating=REI30`	"E": todas as condições devem valer
`+`	`IfcWall + IfcSlab`	"Ou": une os resultados de duas consultas
`!`	`IfcElement, !IfcWall`	Exclui
`Atributo=valor`	`Name=/.*wall.*/`	Atributo com valor exato ou expressão regular entre barras
`Pset.Propriedade`	`Qto_WallBaseQuantities.Length>1500`	Propriedade ou quantidade, com `=`, `!=`, `>`, `>=`, `<`, `<=` ou `*=` (contém)
`material=`	`material=stone_sand-lime`	Nome do material
`location=`	`location="00 groundfloor"`	Nome do contêiner espacial
`type=`	`type="house - groundfloor"`	Nome do tipo
`classification=`	`classification=E-AAA`	Código de classificação
`group=`	`group="road wastewater system"`	Nome de grupo ou sistema
`parent=`	`parent="house - roof"`	Nome do todo (agregação ou contenção)

Tabela 9.6: Sintaxe do seletor.

Textos com espaços devem ficar entre aspas duplas. Na linha de comando do PowerShell, envolva a consulta inteira em aspas simples para preservar as aspas internas, ou use o script de exportação da seção 9.7, que recebe a consulta como argumento.

9.6.2 get_element_value

`get_element_value(elem, caminho)` lê um valor seguindo um caminho de nomes separados por ponto. Além dos atributos (`Name`, `Tag`) e das propriedades (`Pset_...Prop`), o caminho aceita palavras especiais, verificadas na versão 0.8.5:

PALAVRA	VALOR
`class`, `id`, `predefined_type`	Entidade, número, `PredefinedType` efetivo (da ocorrência ou do tipo)
`type`, `types`	O tipo; as ocorrências de um tipo
`container`, `space`, `storey`, `building`, `site`, `parent`	Elementos espaciais e o todo
`material`, `materials`, `profiles`, `styles`	Materiais, perfis e estilos
`classification`, `group`, `system`, `zone`	Associações e grupos
`x`, `y`, `z`	Coordenadas da origem no sistema do projeto, na unidade do arquivo
`easting`, `northing`, `elevation`	Coordenadas de mapa (se houver georreferenciamento)

Tabela 9.7: Palavras especiais de `get_element_value`.

Para a parede `#158` de `Building-Architecture.ifc`, por exemplo, `x` e `y` devolvem aproximadamente 7100 e 4800 (mm), e `easting` e `northing` devolvem 729012,742 e 9064001,233, os mesmos valores calculados no Capítulo 7. Quando há mais de um valor possível (como `system.Name` de um elemento em vários sistemas), o resultado é uma lista.

9.7 Exportar para planilha

O script abaixo junta o seletor e o `pandas` numa ferramenta genérica: você informa a consulta e as colunas (caminhos de `get_element_value`), e ele grava uma planilha nova em Excel ou CSV. É o ponto de partida para listas de verificação, quantitativos e conferência de metadados.

```
# -*- coding: utf-8 -*-
"""
c9_04_exportar.py | Capítulo 9: exporta elementos de um IFC para Excel ou CSV.

Seleciona elementos com uma consulta do ifcopenshell.util.selector e grava uma
planilha NOVA, com uma coluna para cada caminho de valor informado.
O arquivo IFC é aberto somente para leitura.

Uso (ambiente virtual ativado):
    python c9_04_exportar.py <arquivo.ifc> [--consulta "IfcElement"]
                             [--colunas GlobalId class Name ...] [--saida exportacao.xlsx]

Exemplos de colunas: GlobalId, class, Name, Tag, type.Name, container.Name,
material.Name, Pset_WallCommon.FireRating, Qto_WallBaseQuantities.Length

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; pandas 3.0; openpyxl 3.1.
"""
import argparse
from pathlib import Path
import pandas as pd
import ifcopenshell
import ifcopenshell.util.selector as seletor

COLUNAS_PADRAO = ["GlobalId", "class", "Name", "type.Name", "container.Name", "material.Name"]


def valor(elem, caminho):
    """Lê um valor com o seletor; listas viram texto separado por ' | '."""
    v = seletor.get_element_value(elem, caminho)
    if isinstance(v, (list, tuple)):
        return " | ".join(str(x) for x in v) if v else None
    return v


def main():
    leitor = argparse.ArgumentParser(description="Exporta elementos de um IFC para Excel ou CSV.")
    leitor.add_argument("arquivo", help="arquivo IFC de entrada (somente leitura)")
    leitor.add_argument("--consulta", default="IfcElement", help="consulta do seletor")
    leitor.add_argument("--colunas", nargs="+", default=COLUNAS_PADRAO, help="caminhos de valor")
    leitor.add_argument("--saida", default="exportacao.xlsx", help="arquivo .xlsx ou .csv a criar")
    args = leitor.parse_args()

    saida = Path(args.saida)
    if saida.suffix.lower() not in (".xlsx", ".csv"):
        raise SystemExit("A saída deve terminar em .xlsx ou .csv")
    if saida.resolve() == Path(args.arquivo).resolve():
        raise SystemExit("A saída não pode ser o próprio arquivo de entrada.")

    modelo = ifcopenshell.open(args.arquivo)
    elementos = sorted(seletor.filter_elements(modelo, args.consulta), key=lambda e: e.id())
    linhas = [{c: valor(e, c) for c in args.colunas} for e in elementos]
    tabela = pd.DataFrame(linhas, columns=args.colunas)
```

```
    if saida.suffix.lower() == ".xlsx":
        tabela.to_excel(saida, index=False, sheet_name="Elementos")
    else:
        tabela.to_csv(saida, index=False, sep=";", encoding="utf-8-sig")   # CSV para o Excel brasileiro
    print(f"Consulta: {args.consulta} | {len(tabela)} elementos | {len(args.colunas)} colunas -> {saida}")
    print(tabela.head(5).to_string(index=False))


if __name__ == "__main__":
    main()
```

Exemplo 1: paredes e lajes, com uma propriedade e uma quantidade.

```
python c9_04_exportar.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Building-Architecture.ifc" --consulta "IfcWall + IfcSlab" --colunas GlobalId class Name
container.Name Pset_SlabCommon.FireRating Qto_WallBaseQuantities.Length --saida paredes_lajes.xlsx
```

```
Consulta: IfcWall + IfcSlab | 7 elementos | 6 colunas -> paredes_lajes.xlsx
             GlobalId   class                                  Name container.Name Pset_SlabCommon.FireRating
Qto_WallBaseQuantities.Length
3zR0BOEcLADRKln4HYporH IfcSlab                                floor 00 groundfloor
REI30                         NaN
1AQAupaRP1txwK1AGiN61V IfcWall house - outer wall - house right front 00 groundfloor
NaN                           1800.0
3wdauVJT5Fx9drrREiDqA$ IfcWall  house - outer wall - house right back 00 groundfloor
NaN                           4200.0
0OfZwWc8j9QP5uX8xPTxDH IfcWall        house - outer wall - house left 00 groundfloor
NaN                           6000.0
0Q5LjisRjDwhFSOc_WQjlF IfcWall                          plumbing wall 00 groundfloor
NaN                           3800.0
```

Exemplo 2: tubos da drenagem, em CSV.

```
python c9_04_exportar.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Infra-Plumbing.ifc" --consulta "IfcPipeSegment" --colunas GlobalId Name type.PredefinedType
container.Name material.Name --saida tubos.csv
```

```
Consulta: IfcPipeSegment | 24 elementos | 5 colunas -> tubos.csv
             GlobalId       Name type.PredefinedType      container.Name              material.Name
1F4KLUGJf1aAa0YXxTvOR5 sewer pipe             CULVERT road parking - site concrete_reinforced_prefab
0uGYOC9Nz33PfGM5uuVHhW sewer pipe             CULVERT road parking - site concrete_reinforced_prefab
259jK4QKD3OPEdfCl4GH_l sewer pipe             CULVERT road parking - site concrete_reinforced_prefab
0_AaoqH0fA_wWg2hQ6QoLY sewer pipe             CULVERT road parking - site concrete_reinforced_prefab
0X_fsZ0Kj9a8olHcZQNbJM sewer pipe             CULVERT road parking - site concrete_reinforced_prefab
```

Colunas sem valor para um elemento (a resistência ao fogo de uma parede, que não tem `Pset_SlabCommon`) aparecem como `NaN` na tela e como células vazias na planilha.

9.8 Consultas que funcionam em todas as versões

As diferenças entre versões (Capítulos 3 a 7) afetam diretamente os scripts. As mais frequentes, e como contorná-las:

DIFERENÇA	CONSEQUÊNCIA	COMO ESCREVER O CÓDIGO
`IsTypedBy` não existe no IFC2x3	`elem.IsTypedBy` gera erro	Use `util.element.get_type(elem)`
Ocorrências sem `PredefinedType` no IFC2x3	`elem.PredefinedType` gera erro	`getattr(elem, "PredefinedType", None)` ou `get_element_value(elem, "predefined_type")`

DIFERENÇA	CONSEQUÊNCIA	COMO ESCREVER O CÓDIGO
`IfcBuiltElement` (IFC 4.3) x `IfcBuildingElement` (IFC2x3, IFC4)	`by_type` gera erro na versão errada	Consulte `IfcElement` e filtre, ou escolha a entidade conforme `modelo.schema`
Entidades específicas de instalações só no IFC4+	`by_type("IfcValve")` gera erro no IFC2x3	No IFC2x3, consulte a categoria (`IfcFlowController`) e o tipo associado
`Description` renomeado `Specification` nas propriedades (IFC 4.3)	Acesso por nome falha	Acesse pela posição (`prop[1]`) ou use `util.element.get_psets`
`ItemReference` (IFC2x3) x `Identification` (IFC4+) na classificação	Acesso por nome falha	`getattr(ref, "Identification", None) or getattr(ref, "ItemReference", None)`

Tabela 9.8: Diferenças entre versões e código robusto.

Um padrão útil para escolher a entidade conforme a versão:

```
elementos_construidos = "IfcBuiltElement" if modelo.schema == "IFC4X3" else "IfcBuildingElement"
paredes_e_afins = modelo.by_type(elementos_construidos)
```

9.9 Resumo do capítulo

- O IfcOpenShell (versão 0.8.5 neste livro) lê, escreve, valida e processa IFC2x3, IFC4 e IFC 4.3; o Bonsai é construído sobre ele.
- `ifcopenshell.open` lê o arquivo; `by_type` (com subtipos), `by_id` e `by_guid` localizam instâncias; o GlobalId é a busca estável.
- Atributos são lidos por nome ou posição; enumerações viram texto, `$` vira `None`, referências viram objetos e valores tipados têm `wrappedValue`.
- Relações: atributos inversos, `get_inverse` (quem aponta para mim) e `traverse` (para onde eu aponto).
- `ifcopenshell.util.element` responde às perguntas comuns e aplica as regras do esquema (propriedades efetivas, tipo, contêiner).
- O seletor filtra elementos com consultas em texto e lê valores por caminhos; com `pandas`, vira uma ferramenta de exportação.
- Código robusto usa as funções de `util`, `getattr` com valor padrão e testes de versão.

9.10 Vocabulário técnico

INGLÊS	PORTUGUÊS
open / write	abrir / gravar
by type / by id / by guid	por entidade / por número / por GlobalId
include subtypes	incluir subtipos
entity instance	instância de entidade

INGLÊS	PORTUGUÊS
wrapped value	valor encapsulado (de um valor tipado)
inverse / traverse	inverso / percorrer as referências
utility functions	funções utilitárias
selector / query / filter	seletor / consulta / filtro
regular expression (regex)	expressão regular
inherit	herdar (do tipo)
export	exportar

Tabela 9.9: Vocabulário do capítulo.

9.11 Exercícios

Fixação

1. Qual a diferença entre `by_type("IfcElement")` e `by_type("IfcElement", include_subtypes=False)`? Por que a segunda devolve uma lista vazia em qualquer arquivo válido?
2. Por que `by_guid` é preferível a `by_id` para localizar um objeto de uma planilha preparada numa revisão anterior do modelo?
3. O que o IfcOpenShell devolve para: `.NOTDEFINED.`, `$`, `#12` e `IFCBOOLEAN(.T.)`?
4. Qual a diferença entre `modelo.get_inverse(elem)` e `modelo.traverse(elem)`?
5. Qual consulta do seletor encontra as paredes com resistência ao fogo começando por "EI" ou "REI"?

Aplicação

6. Com `c9_04_exportar.py`, exporte para Excel todos os elementos de `Infra-Bridge.ifc` (IFC 4.3), com as colunas `GlobalId`, `class`, `Name`, `container.Name` e `predefined_type`. Quantos elementos há, e quantas classes distintas?
7. Escreva um script que liste, para cada tipo (`IfcTypeObject`) de `Building-Structural.ifc`, o nome, a entidade, o `PredefinedType` e a quantidade de ocorrências (use `util.element.get_types`).

Desafio

8. Escreva um script que compare dois arquivos da mesma amostra em versões diferentes (por exemplo, `Infra-Plumbing.ifc` em IFC2x3 e em IFC 4.3) pelo GlobalId: quantos GlobalIds existem nos dois, quantos só em um e quantos só no outro? Explique o resultado.

9.12 Respostas dos exercícios

1. A primeira devolve todas as instâncias de `IfcElement` e de seus subtipos; a segunda, só as instâncias da própria entidade. Como `IfcElement` é abstrata (Capítulo 3), não pode ter instâncias próprias, e a lista é sempre vazia num arquivo válido.

2. Porque o número da instância pode mudar a cada exportação, enquanto o GlobalId é o identificador permanente (Capítulo 3), desde que o exportador o preserve.
3. `'NOTDEFINED'` (texto); `None`; um `entity_instance` (a instância `#12`); um `entity_instance` do tipo `IfcBoolean`, cujo `wrappedValue` é `True`. Dentro de conjuntos de propriedades, `get_psets` já devolve o valor desembrulhado (`True`).
4. `get_inverse` devolve as instâncias que apontam para o elemento (relações, em geral); `traverse` devolve as instâncias para as quais o elemento aponta, direta e indiretamente (posicionamento, geometria, autoria).
5. `IfcWall, Pset_WallCommon.FireRating=/R?EI.*/`. A expressão `R?` torna o "R" opcional. Na amostra de arquitetura, as paredes não têm `Pset_WallCommon`, e o resultado é vazio; teste também com `IfcSlab, Pset_SlabCommon.FireRating=/R?EI.*/`, que encontra a laje `#53`.
6. Comando: `python c9_04_exportar.py "<caminho>\Infra-Bridge.ifc" --colunas GlobalId class Name container.Name predefined_type --saida ponte.xlsx`. Resultado obtido no teste: 50 elementos, em 11 classes distintas (8 vigas, 8 barras, 7 pilares, 7 sapatas, 4 aterros, 4 placas de sinalização, 4 muros, 3 lajes, 2 guarda-corpos, 2 conjuntos e 1 *proxy*). A contagem de classes pode ser obtida no próprio Excel, com uma tabela dinâmica, ou no `pandas`, com `tabela["class"].nunique()`.
7. Uma solução:

```
import sys
import ifcopenshell
import ifcopenshell.util.element as util_elem

modelo = ifcopenshell.open(sys.argv[1])
for tipo in sorted(modelo.by_type("IfcTypeObject"), key=lambda t: t.id()):
    ocorrencias = util_elem.get_types(tipo)
    print(f"#{tipo.id()} {tipo.is_a():<28} {tipo.Name!s:<34} "
          f"{getattr(tipo, 'PredefinedType', None)!s:<16} {len(ocorrencias)}")
```

Na amostra estrutural, a viga "house - girder" (`IfcBeamType`, `GIRDER_SEGMENT`) aparece com 6 ocorrências, como no Capítulo 3.

8. Uma solução: abrir os dois arquivos, montar dois conjuntos com `{e.GlobalId for e in modelo.by_type("IfcRoot")}` e usar as operações de conjunto `&` (interseção) e `-` (diferença). Resultado obtido no teste com `Infra-Plumbing.ifc`: 48 GlobalIds na versão IFC2x3 e 62 na IFC 4.3; 44 em comum; 4 só na IFC2x3 (três relações de agregação e contenção e uma de grupo); 18 só na IFC 4.3 (5 terrenos parciais, com suas 5 relações de agregação e 5 de contenção, um tipo de conjunto montado, uma relação de tipo e uma de grupo). Interpretação: os elementos (tubos, caixas) e os tipos mantêm o GlobalId entre as versões; a diferença está na estrutura espacial, mais detalhada na versão IFC 4.3 (os terrenos parciais do Capítulo 4), e nas relações, que o exportador recria quando a estrutura muda.

9.13 Referências

- Documentação do IfcOpenShell: https://docs.ifcopenshell.org/
- Documentação do IfcOpenShell, módulo `ifcopenshell.util.selector`: https://docs.ifcopenshell.org/autoapi/ifcopenshell/util/selector/index.html

- Documentação do IfcOpenShell, módulo `ifcopenshell.util.element`: https://docs.ifcopenshell.org/autoapi/ifcopenshell/util/element/index.html
- Bonsai (extensão de BIM para o Blender): https://bonsaibim.org/

10 IfcOpenShell II: edição e metadados em lote

Ler um modelo responde perguntas; editar um modelo resolve problemas. Este capítulo mostra como alterar arquivos IFC por programa com a API do IfcOpenShell: editar atributos, criar e preencher conjuntos de propriedades com os tipos de valor corretos, associar materiais e classificações, corrigir classes de elementos exportados de forma genérica e, principalmente, **preencher metadados em lote a partir de uma planilha**, o fluxo que substitui, no próprio IFC, rotinas hoje feitas por Dynamo dentro do software de autoria.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- usar a API de edição (`ifcopenshell.api.<módulo>.<função>`) e reconhecer a sintaxe antiga;
- editar atributos e conjuntos de propriedades, escolhendo o tipo de valor correto;
- associar materiais e classificações e remover propriedades;
- corrigir a classe de elementos preservando o GlobalId;
- preencher metadados em lote a partir de uma planilha, com mapeamento, conversão de unidades e registro de resultados;
- aplicar as regras de segurança: nunca alterar o original, validar e conferir o resultado.

10.1 Introdução: por que editar por código

10.1.1 Situações típicas

- **Metadados do BEP**: o contrato exige TAG, fluido, pressão, revisão e classificação em cada objeto, e esses dados estão numa planilha.
- **Correção de exportação**: o exportador gravou tubos como `IfcBuildingElementProxy`, deixou o `PredefinedType` vazio ou usou o nome errado de um conjunto de propriedades.
- **Padronização**: vários fornecedores entregam modelos com convenções diferentes, e é preciso uniformizá-los antes da federação.
- **Criação**: gerar modelos simples, bibliotecas de tipos ou arquivos de teste (como a rede do Capítulo 6).

Em todos os casos, o trabalho é repetitivo, sujeito a erro manual e precisa ser refeito a cada revisão do modelo. Um script faz o mesmo trabalho em segundos, sempre do mesmo jeito, e deixa registro do que fez.

10.1.2 Regras de segurança

1. **Nunca altere o arquivo original.** Abra o original, edite na memória e grave um arquivo **novo**. Todos os scripts deste capítulo recusam gravar sobre a entrada.
2. **Preserve o GlobalId.** Os vínculos entre revisões, apontamentos BCF e planilhas dependem dele (Capítulo 3). As funções da API usadas aqui o preservam.
3. **Valide e confira.** Depois de gravar, valide o arquivo contra o esquema (`python -m ifcopenshell.validate`) e confira alguns valores com um script de leitura (Capítulo 9).

4. **Registre o que foi feito.** Um registro por linha de planilha mostra o que foi gravado, o que não foi encontrado e o que deu erro.

10.2 A API de edição

10.2.1 Sintaxe

A API de edição reúne funções de alto nível, organizadas em módulos por assunto. Na versão 0.8, a forma de chamada é:

```
import ifcopenshell.api.pset
pset = ifcopenshell.api.pset.add_pset(modelo, product=elemento, name="ARV_Controle")
```

O primeiro argumento é sempre o arquivo; os demais são nomeados. As funções cuidam dos detalhes do esquema: criam as relações, reaproveitam as existentes, geram GlobalIds e, no IFC2x3, registram o `OwnerHistory` obrigatório.

> **ATENÇÃO**
> Muitos exemplos na internet usam a sintaxe antiga: `ifcopenshell.api.run("pset.add_pset", modelo, product=elemento, name="...")`. Na versão 0.8.5, a função `run` ainda existe, mas sua documentação diz: "This is deprecated and will be removed in a future version. Do not use this function." Ao adaptar um exemplo antigo, troque `ifcopenshell.api.run("modulo.funcao", modelo, ...)` por `ifcopenshell.api.modulo.funcao(modelo, ...)`.

10.2.2 Módulos mais usados

MÓDULO	FUNÇÕES (SELEÇÃO)	USO
`project`	`create_file`	Criar um arquivo vazio de uma versão
`root`	`create_entity`, `remove_product`, `reassign_class`, `copy_class`	Criar, remover, trocar a classe e copiar elementos
`attribute`	`edit_attributes`	Editar atributos (`Name`, `Description`, `Tag`, `ObjectType`...)
`pset`	`add_pset`, `edit_pset`, `remove_pset`, `add_qto`, `edit_qto`	Conjuntos de propriedades e de quantidades
`material`	`add_material`, `assign_material`, `add_material_set`, `add_layer`, `add_profile`	Materiais e conjuntos de materiais
`classification`	`add_classification`, `add_reference`	Sistemas e itens de classificação
`spatial`, `aggregate`, `nest`	`assign_container`, `reference_structure`, `assign_object`	Estrutura espacial e composição (Capítulo 4)
`type`	`assign_type`	Ligar ocorrências a tipos
`system`, `group`	`add_system`, `assign_system`, `add_port`, `connect_port`, `add_group`, `assign_group`	Sistemas, portas e grupos (Capítulo 6)

MÓDULO	FUNÇÕES (SELEÇÃO)	USO
`unit`	`assign_unit`, `add_si_unit`, `add_conversion_based_unit`	Unidades do projeto
`document`	`add_information`, `add_reference`, `assign_document`	Documentos
`geometry`, `context`, `georeference`	`edit_object_placement`, `add_context`, `add_georeferencing`, `edit_georeferencing`	Posicionamento, contextos e georreferenciamento (Capítulos 7 e 11)

Tabela 10.1: Módulos da API de edição (versão 0.8.5).

A documentação de cada função está em https://docs.ifcopenshell.org/ e também no próprio Python: `help(ifcopenshell.api.pset.edit_pset)`.

10.2.3 API ou atribuição direta?

Também é possível alterar um atributo diretamente, como no Capítulo 6 (`porta.PredefinedType = "PIPE"`), ou criar instâncias com `modelo.create_entity(...)`. Para atributos simples, isso funciona. Para qualquer operação que envolva relações (tipos, contêineres, propriedades, classes, remoções), use a API: ela mantém o modelo coerente. Remover um elemento com `modelo.remove(elemento)`, por exemplo, deixa as relações apontando para o vazio; `ifcopenshell.api.root.remove_product` remove também o que deixou de ter sentido.

10.3 Edição básica

O script a seguir aplica as operações mais comuns à amostra de drenagem e grava o resultado num arquivo novo. Grave-o como `C:\IFC_Curso\exercicios\cap10\c10_01_edicao_basica.py`.

```
# -*- coding: utf-8 -*-
"""
c10_01_edicao_basica.py | Capítulo 10: operações básicas de edição com ifcopenshell.api.

Lê um arquivo IFC (o original nunca é alterado), faz as edições abaixo na memória
e grava o resultado num arquivo NOVO:
  1. atributos: Tag e Description dos tubos;
  2. conjunto personalizado (ARV_Controle) com valores de texto;
  3. conjunto padronizado no TIPO (Pset_PipeSegmentTypeCommon), com tipos de medida
     definidos pelo modelo do conjunto e um valor enumerado (Status);
  4. conjunto personalizado com um valor de medida explícito (IfcPressureMeasure);
  5. material e classificação;
  6. remoção de uma propriedade.

Uso (ambiente virtual ativado):
    python c10_01_edicao_basica.py <entrada.ifc> <saida.ifc>

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
from pathlib import Path
import ifcopenshell
import ifcopenshell.api.attribute
import ifcopenshell.api.classification
import ifcopenshell.api.material
import ifcopenshell.api.pset
import ifcopenshell.util.element as util_elem

entrada, saida = Path(sys.argv[1]), Path(sys.argv[2])
```

```
if saida.resolve() == entrada.resolve():
    raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")

modelo = ifcopenshell.open(entrada)
tubos = sorted(modelo.by_type("IfcPipeSegment"), key=lambda t: t.id())
print(f"{len(tubos)} tubos no arquivo {entrada.name} ({modelo.schema_identifier})")

# 1. Atributos da ocorrência
for n, tubo in enumerate(tubos, start=1):
    ifcopenshell.api.attribute.edit_attributes(
        modelo, product=tubo,
        attributes={"Tag": f"TB-{n:03d}", "Description": "Tubo de esgoto (galeria)"})

# 2. Conjunto personalizado em cada ocorrência (valores de texto -> IfcLabel)
for tubo in tubos:
    pset = ifcopenshell.api.pset.add_pset(modelo, product=tubo, name="ARV_Controle")
    ifcopenshell.api.pset.edit_pset(modelo, pset=pset,
                                    properties={"Revisao": "A", "SituacaoAprovacao": "Em análise"})

# 3. Conjunto padronizado no tipo: o modelo do conjunto define os tipos de medida.
#    Status é uma propriedade enumerada: o valor é passado numa lista.
tipo = util_elem.get_type(tubos[0])
pset_tipo = ifcopenshell.api.pset.add_pset(modelo, product=tipo, name="Pset_PipeSegmentTypeCommon")
ifcopenshell.api.pset.edit_pset(modelo, pset=pset_tipo, properties={
    "NominalDiameter": 600.0,          # mm (unidade de comprimento do arquivo)
    "WorkingPressure": 100000.0,       # Pa (o arquivo não declara unidade de pressão: vale o SI)
    "Status": ["NEW"],
})

# 4. Conjunto personalizado com medida explícita: sem isso, um número vira IfcReal
pset_proc = ifcopenshell.api.pset.add_pset(modelo, product=tubos[0], name="ARV_Processo")
ifcopenshell.api.pset.edit_pset(modelo, pset=pset_proc, properties={
    "Fluido": "Esgoto sanitário",
    "PressaoProjeto_sem_tipo": 150000.0,                                    # vira IfcReal
    "PressaoProjeto": modelo.create_entity("IfcPressureMeasure", 150000.0),     # medida de pressão
})

# 5. Material (substitui a associação anterior) e classificação
concreto = ifcopenshell.api.material.add_material(modelo, name="Concreto armado pré-moldado",
                                                  category="concrete")
ifcopenshell.api.material.assign_material(modelo, products=tubos, type="IfcMaterial", material=concreto)
sistema = ifcopenshell.api.classification.add_classification(modelo, classification="ABNT NBR 15965")
ifcopenshell.api.classification.add_reference(modelo, products=[tipo], identification="3E.EXEMPLO",
                                              name="Código ilustrativo", classification=sistema)

# 6. Remover uma propriedade: valor None
ifcopenshell.api.pset.edit_pset(modelo, pset=pset_proc, properties={"PressaoProjeto_sem_tipo": None})

modelo.write(saida)
print("Arquivo gravado:", saida.name)

# Conferência: reabrir o arquivo gravado e ler o primeiro tubo
conferencia = ifcopenshell.open(saida)
t = conferencia.by_guid(tubos[0].GlobalId)          # o GlobalId foi preservado
print("Tag:", t.Tag, "| Description:", t.Description)
for nome, props in util_elem.get_psets(t).items():
    print(f"   {nome}: {props}")
print("Material:", util_elem.get_material(t).Name)
pv = [p for p in conferencia.by_type("IfcPropertySingleValue") if p.Name == "PressaoProjeto"][0]
print("Tipo do valor de PressaoProjeto:", pv.NominalValue.is_a())
```

Execução:

```
cd C:\IFC_Curso\exercicios\cap10
python c10_01_edicao_basica.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Infra-Plumbing.ifc" Infra-Plumbing_editado.ifc
```

Saída obtida no teste:

```
24 tubos no arquivo Infra-Plumbing.ifc (IFC4X3_ADD2)
Arquivo gravado: Infra-Plumbing_editado.ifc
Tag: TB-001 | Description: Tubo de esgoto (galeria)
   Pset_PipeSegmentTypeCommon: {'NominalDiameter': 600.0, 'WorkingPressure': 100000.0, 'Status': ['NEW'], 'id':
613}
   ARV_Controle: {'Revisao': 'A', 'SituacaoAprovacao': 'Em análise', 'id': 469}
   ARV_Processo: {'Fluido': 'Esgoto sanitário', 'PressaoProjeto': 150000.0, 'id': 619}
Material: Concreto armado pré-moldado
Tipo do valor de PressaoProjeto: IfcPressureMeasure
```

O arquivo gravado foi validado com `python -m ifcopenshell.validate --rules`, sem violações, e o original permaneceu inalterado (conferido por soma de verificação antes e depois dos testes).

10.3.1 O tipo de cada valor

Este é o ponto mais importante da edição de propriedades. A função `edit_pset` escolhe o tipo IFC de cada valor assim:

SITUAÇÃO	TIPO GRAVADO	EXEMPLO DO TESTE
Conjunto **padronizado** (`Pset_...`)	O tipo definido no modelo do conjunto (Capítulo 5)	`WorkingPressure` → `IfcPressureMeasure`; `NominalDiameter` → `IfcPositiveLengthMeasure`
Conjunto personalizado, valor de texto	`IfcLabel`	`Revisao` = `IfcLabel('A')`
Conjunto personalizado, número real	`IfcReal` (número puro, **sem grandeza**)	`PressaoProjeto_sem_tipo` = `IfcReal(150000.)`
Conjunto personalizado, inteiro	`IfcInteger`	
Conjunto personalizado, lógico	`IfcBoolean`	
Valor criado com `modelo.create_entity("IfcPressureMeasure", 150000.0)`	O tipo informado	`PressaoProjeto` = `IfcPressureMeasure(150000.)`

Tabela 10.2: Tipos de valor em `edit_pset` (verificado na versão 0.8.5).

Consequência: em conjuntos personalizados, **grandezas físicas devem ser criadas com o tipo de medida explícito**. Um `IfcReal` não tem unidade: um programa não sabe se 150000 são pascal, quilopascal ou outra coisa, e um IDS que exija `IFCPRESSUREMEASURE` (Capítulo 12) reprova a propriedade.

Outros detalhes verificados:

- **propriedades enumeradas** (como `Status`): passe o valor numa lista (`["NEW"]`); a função cria `IfcPropertyEnumeratedValue` com a lista de valores permitidos do modelo. Um texto simples (`"NEW"`) seria gravado como `IfcLabel` num valor único, fora do padrão do conjunto;
- **remover uma propriedade**: passe `None` como valor;
- **unidades**: o valor numérico é gravado como informado, na unidade do projeto para aquela grandeza, ou no SI se o projeto não declarar unidade (Capítulo 5). Converta antes de gravar;
- **reaproveitamento**: se o elemento já tem um conjunto com o nome informado, `add_pset` devolve o conjunto existente, em vez de criar outro (verificado no teste). O script de lote da seção 10.5 também

localiza o conjunto existente com `util.element.get_pset(elem, nome, should_inherit=False)` e guarda os conjuntos numa memória temporária, para não repetir buscas.

10.3.2 Material, classificação e cabeçalho

- `assign_material` **substitui** a associação de material anterior do elemento (verificado no teste);
- `add_classification` com um texto cria um sistema de classificação próprio (`IfcClassification` com o nome informado); `add_reference` cria o item (`IfcClassificationReference`) e o associa. Os códigos da ABNT NBR 15965 devem ser conferidos na norma; os códigos usados nos exemplos (`3E.EXEMPLO`, `3E.EX.01`...) são ilustrativos;
- ao gravar, o IfcOpenShell mantém o **cabeçalho** do arquivo original, inclusive o nome e a data. O script de lote da seção 10.5 atualiza o nome, a data e o campo de pré-processador, e mantém o sistema de origem, de onde os dados vieram.

10.3.3 Editar arquivos IFC2x3

No IFC2x3, todo objeto derivado de `IfcRoot` precisa de `OwnerHistory` (Capítulo 3). A API cria o registro automaticamente; no teste com a amostra `Infra-Plumbing.ifc` em IFC2x3, o novo conjunto de propriedades recebeu um `IfcOwnerHistory` que reaproveita a pessoa e o aplicativo já existentes no arquivo. Os nomes e as enumerações dos conjuntos padronizados, porém, são os do IFC2x3 (Capítulo 5), e as entidades específicas de instalações não existem (Capítulo 6).

10.4 Corrigir a classe de elementos

Quando um exportador sem mapeamento grava tudo como `IfcBuildingElementProxy`, a informação de "o que é cada objeto" se perde (Capítulo 6). Se o nome dos objetos seguir um padrão (TAG com prefixo por tipo de componente), um script pode restaurar a classe. A função `ifcopenshell.api.root.reassign_class` troca a entidade de uma instância, mantendo o GlobalId, os atributos comuns e as relações.

Para estudar a correção, o primeiro script cria um arquivo que simula uma exportação ruim. Grave-os como `c10_00_rede_com_proxies.py` e `c10_02_reclassificar.py`.

```
# -*- coding: utf-8 -*-
"""
c10_00_rede_com_proxies.py | Capítulo 10: cria um arquivo de estudo que simula uma
exportação ruim: todos os elementos da rede como IfcBuildingElementProxy.

Uso: python c10_00_rede_com_proxies.py [saida.ifc]      (padrão: rede_proxies.ifc)
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import ifcopenshell
import ifcopenshell.api.aggregate
import ifcopenshell.api.project
import ifcopenshell.api.root
import ifcopenshell.api.spatial
import ifcopenshell.api.unit

saida = sys.argv[1] if len(sys.argv) > 1 else "rede_proxies.ifc"
modelo = ifcopenshell.api.project.create_file(version="IFC4X3")
criar = ifcopenshell.api.root.create_entity
projeto = criar(modelo, ifc_class="IfcProject", name="Exportação sem mapeamento")
ifcopenshell.api.unit.assign_unit(modelo)
terreno = criar(modelo, ifc_class="IfcSite", name="Unidade industrial")
```

```
ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)
nomes = ["TQ-001", "TB-001", "VG-001", "FT-001", "TB-002", "VR-001", "B-001", "SUPORTE-01"]
proxies = [criar(modelo, ifc_class="IfcBuildingElementProxy", name=n) for n in nomes]
ifcopenshell.api.spatial.assign_container(modelo, products=proxies, relating_structure=terreno)
modelo.write(saida)
print("Arquivo gravado:", saida, "| proxies:", len(proxies))
```

```
# -*- coding: utf-8 -*-
"""
c10_02_reclassificar.py | Capítulo 10: corrige a classe de elementos exportados como proxy.

Situação típica: um exportador sem mapeamento de classes grava tubos, válvulas e
bombas como IfcBuildingElementProxy. Este script aplica regras "prefixo do nome ->
classe e PredefinedType" e troca a classe com ifcopenshell.api.root.reassign_class,
que preserva o GlobalId e as relações. O arquivo original não é alterado.

Uso (ambiente virtual ativado):
    python c10_02_reclassificar.py <entrada.ifc> <saida.ifc>

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5 (arquivos IFC4 e IFC4X3).
"""
import re
import sys
from pathlib import Path
import ifcopenshell
import ifcopenshell.api.root

# Regras: expressão regular aplicada ao Name -> (classe, PredefinedType)
REGRAS = [
    (r"^TB-", "IfcPipeSegment", "RIGIDSEGMENT"),
    (r"^VG-", "IfcValve", "ISOLATING"),
    (r"^VR-", "IfcValve", "CHECK"),
    (r"^FT-", "IfcFilter", "STRAINER"),
    (r"^B-", "IfcPump", "ENDSUCTION"),
    (r"^TQ-", "IfcTank", "STORAGE"),
]

entrada, saida = Path(sys.argv[1]), Path(sys.argv[2])
if saida.resolve() == entrada.resolve():
    raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")
modelo = ifcopenshell.open(entrada)

alterados, sem_regra = [], []
for proxy in sorted(modelo.by_type("IfcBuildingElementProxy"), key=lambda e: e.id()):
    regra = next((r for r in REGRAS if re.match(r[0], proxy.Name or "")), None)
    if regra is None:
        sem_regra.append(proxy.Name)
        continue
    _, classe, tipo_predef = regra
    guid_antes = proxy.GlobalId
    novo = ifcopenshell.api.root.reassign_class(modelo, product=proxy, ifc_class=classe,
                                                predefined_type=tipo_predef)
    alterados.append((novo.Name, classe, tipo_predef, novo.GlobalId == guid_antes))

modelo.write(saida)
print(f"Reclassificados: {len(alterados)} | sem regra: {len(sem_regra)} -> {saida.name}")
for nome, classe, tipo_predef, guid_ok in alterados:
    print(f"   {nome:<8} -> {classe} ({tipo_predef}) | GlobalId preservado: {guid_ok}")
for nome in sem_regra:
    print(f"   {nome!s:<8} -> sem regra; mantido como IfcBuildingElementProxy")
```

Execução e saídas obtidas:

```
python c10_00_rede_com_proxies.py rede_proxies.ifc
python c10_02_reclassificar.py rede_proxies.ifc rede_reclassificada.ifc
```

```
Arquivo gravado: rede_proxies.ifc | proxies: 8
Reclassificados: 7 | sem regra: 1 -> rede_reclassificada.ifc
```

```
TQ-001   -> IfcTank (STORAGE) | GlobalId preservado: True
TB-001   -> IfcPipeSegment (RIGIDSEGMENT) | GlobalId preservado: True
VG-001   -> IfcValve (ISOLATING) | GlobalId preservado: True
FT-001   -> IfcFilter (STRAINER) | GlobalId preservado: True
TB-002   -> IfcPipeSegment (RIGIDSEGMENT) | GlobalId preservado: True
VR-001   -> IfcValve (CHECK) | GlobalId preservado: True
B-001    -> IfcPump (ENDSUCTION) | GlobalId preservado: True
SUPORTE-01 -> sem regra; mantido como IfcBuildingElementProxy
```

Os sete elementos com regra trocaram de classe e mantiveram o GlobalId; o suporte, sem regra, continuou como *proxy*, e o script informou. O arquivo foi validado sem violações, e a estrutura espacial foi preservada: `estrutura_espacial.py` (Capítulo 4) mostra o terreno contendo 1 *proxy*, 1 filtro, 2 tubos, 1 bomba, 1 tanque e 2 válvulas.

A correção por regras de nome é tão boa quanto o padrão de nomes do modelo. Sempre revise a lista de "sem regra" e confira uma amostra de elementos reclassificados.

10.5 Metadados em lote a partir de planilha

10.5.1 O desenho da solução

O preenchimento em lote tem três partes:

1. **a planilha de dados**: uma linha por elemento, com uma coluna-chave que o identifica no modelo (a TAG, o GlobalId ou outro atributo) e uma coluna por informação;
2. **o mapeamento**: para cada coluna, onde gravar no IFC (Capítulo 5, Tabela 5.16): atributo, conjunto de propriedades, material ou classificação; em que conjunto e propriedade; com que tipo IFC; na ocorrência ou no tipo; e com que conversão de unidade;
3. **o script**: lê as duas abas, localiza cada elemento, converte e grava os valores, e produz um arquivo novo e um registro.

Separar o mapeamento dos dados tem uma vantagem: o mesmo script serve para qualquer projeto. Muda-se a planilha, não o código.

Estrutura da aba **Mapeamento**:

COLUNA	DESTINO	CONJUNTO	PROPRIEDADE	TIPOIFC	APLICAREM	FATOR	SOMA
Nome da coluna da aba Dados	`Atributo`, `Pset`, `Material` ou `Classificacao`	Nome do conjunto (`Pset`) ou do sistema de classificação	Nome da propriedade (`Pset`) ou do atributo (`Atributo`)	Tipo IFC do valor (`Pset`)	`ocorrencia` ou `tipo`	Multiplicador de conversão	Parcela somada após a multiplicação

Tabela 10.3: Estrutura do mapeamento.

10.5.2 Conversões de unidade

O IFC guarda valores sem unidade explícita na unidade do projeto ou, se o projeto não declarar unidade para a grandeza, no SI (Capítulo 5). As planilhas de engenharia, porém, usam as unidades do dia a dia. As colunas `Fator` e `Soma` convertem: valor gravado = valor da planilha x `Fator` + `Soma`.

UNIDADE NA PLANILHA	UNIDADE GRAVADA (SI)	FATOR	SOMA
kPa	Pa	1000	0
bar	Pa	100000	0
MPa	Pa	1000000	0
°C	K (kelvin)	1	273,15
t (tonelada)	kg	1000	0

Tabela 10.4: Conversões para o SI.

A temperatura merece atenção: a unidade SI de temperatura termodinâmica é o **kelvin**. Um valor 40 gravado como `IfcThermodynamicTemperatureMeasure`, num projeto sem unidade de temperatura declarada, significa 40 K (-233 °C). Por isso, o exemplo converte °C em K (40 °C = 313,15 K). A alternativa é declarar o grau Celsius como unidade de temperatura do projeto; nesse caso, `Soma` fica 0. O importante é que a unidade seja explícita e coerente com o IDS de verificação.

10.5.3 A planilha de exemplo

O script abaixo cria uma planilha de exemplo para a rede do Capítulo 6 (`rede_exemplo.ifc`), com seis linhas de dados, incluindo duas situações de erro propositais: uma pressão digitada como texto ("alta") e uma TAG que não existe no modelo. Grave-o como `c10_03a_criar_planilha_exemplo.py`.

```
# -*- coding: utf-8 -*-
"""
c10_03a_criar_planilha_exemplo.py | Capítulo 10: cria a planilha de metadados de exemplo.

A planilha tem duas abas:
  Dados       -> uma linha por elemento (chave: TAG, igual ao Name do elemento)
  Mapeamento  -> para cada coluna de Dados, onde gravar no IFC

Uso: python c10_03a_criar_planilha_exemplo.py [saida.xlsx]   (padrão: metadados_rede.xlsx)
Testado com: Python 3.11 e 3.13; pandas 3.0; openpyxl 3.1.
"""
import sys
import pandas as pd

saida = sys.argv[1] if len(sys.argv) > 1 else "metadados_rede.xlsx"

dados = pd.DataFrame([
    # TAG, Descrição, Fluido, pressão (kPa), temperatura (°C), revisão, situação,
    # material, classificação, fabricante
    ["TQ-001", "Tanque de produto", "Produto A", None, 40, "B", "Aprovado",
     "Aço-carbono ASTM A283 Gr. C", "3E.EX.01", "Fabricante X"],
    ["TB-001", "Tubo de sucção", "Produto A", 600, 40, "B", "Aprovado",
     "Aço-carbono ASTM A106 Gr. B", "3E.EX.02", None],
    ["VG-001", "Válvula de bloqueio", "Produto A", 600, 40, "A", "Em análise",
     "Aço-carbono ASTM A216 WCB", "3E.EX.03", "Fabricante Y"],
    ["TB-002", "Tubo de recalque", "Produto A", "alta", 40, "B", "Aprovado",
     "Aço-carbono ASTM A106 Gr. B", "3E.EX.02", None],
    ["B-001", "Bomba de transferência", "Produto A", 1000, 40, "A", "Em análise",
     "Ferro fundido", "3E.EX.04", "Fabricante Z"],
    ["XX-999", "Linha sem elemento no modelo", "Produto A", 600, 40, "A", "Em análise",
     None, None, None],
], columns=["TAG", "Descricao", "Fluido", "PressaoProjeto_kPa", "TemperaturaProjeto_C", "Revisao",
            "SituacaoAprovacao", "Material", "Classificacao", "Fabricante"])

mapeamento = pd.DataFrame([
    # Coluna, Destino, Conjunto (ou sistema), Propriedade (ou atributo), TipoIFC, AplicarEm, Fator, Soma
    ["Descricao", "Atributo", None, "Description", None, "ocorrencia", None, None],
    ["Fluido", "Pset", "ARV_Processo", "Fluido", "IfcLabel", "ocorrencia", None, None],
```

```
    ["PressaoProjeto_kPa", "Pset", "ARV_Processo", "PressaoProjeto", "IfcPressureMeasure",
     "ocorrencia", 1000, 0],
    ["TemperaturaProjeto_C", "Pset", "ARV_Processo", "TemperaturaProjeto",
     "IfcThermodynamicTemperatureMeasure", "ocorrencia", 1, 273.15],
    ["Revisao", "Pset", "ARV_Controle", "Revisao", "IfcLabel", "ocorrencia", None, None],
    ["SituacaoAprovacao", "Pset", "ARV_Controle", "SituacaoAprovacao", "IfcLabel", "ocorrencia", None, None],
    ["Material", "Material", None, None, None, "ocorrencia", None, None],
    ["Classificacao", "Classificacao", "ABNT NBR 15965", None, None, "ocorrencia", None, None],
    ["Fabricante", "Pset", "Pset_ManufacturerTypeInformation", "Manufacturer", "IfcLabel",
     "tipo", None, None],
], columns=["Coluna", "Destino", "Conjunto", "Propriedade", "TipoIFC", "AplicarEm", "Fator", "Soma"])

with pd.ExcelWriter(saida) as livro:
    dados.to_excel(livro, sheet_name="Dados", index=False)
    mapeamento.to_excel(livro, sheet_name="Mapeamento", index=False)
print("Planilha gravada:", saida, "| linhas de dados:", len(dados), "| regras:", len(mapeamento))
```

```
Planilha gravada: metadados_rede.xlsx | linhas de dados: 6 | regras: 9
```

10.5.4 O script de preenchimento

Grave-o como `C:\IFC_Curso\exercicios\cap10\c10_03_preencher_metadados.py`.

```
# -*- coding: utf-8 -*-
"""
c10_03_preencher_metadados.py | Capítulo 10: preenchimento de metadados em lote a partir de planilha.

Entradas (nenhuma delas é alterada):
  - um arquivo IFC (IFC4 ou IFC4X3);
  - uma planilha Excel com as abas:
      Dados       -> uma linha por elemento; a coluna-chave identifica o elemento
      Mapeamento  -> Coluna | Destino | Conjunto | Propriedade | TipoIFC | AplicarEm | Fator | Soma
        Destino:   Atributo, Pset, Material ou Classificacao
        AplicarEm: ocorrencia (padrão) ou tipo
        Fator/Soma: conversão numérica opcional: valor_gravado = valor * Fator + Soma

Saídas (arquivos novos):
  - o IFC com os metadados (padrão: <entrada>_metadados.ifc);
  - um registro em Excel com o resultado de cada linha (padrão: <saida>_registro.xlsx).

Uso (ambiente virtual ativado):
    python c10_03_preencher_metadados.py <modelo.ifc> <planilha.xlsx>
           [--saida novo.ifc] [--chave Name|Tag|GlobalId] [--coluna-chave TAG]

Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; pandas 3.0; openpyxl 3.1.
"""
import argparse
from collections import defaultdict
from datetime import datetime
from pathlib import Path

import pandas as pd
import ifcopenshell
import ifcopenshell.api.attribute
import ifcopenshell.api.classification
import ifcopenshell.api.material
import ifcopenshell.api.pset
import ifcopenshell.ifcopenshell_wrapper
import ifcopenshell.util.element as util_elem

TIPOS_TEXTO = {"IfcLabel", "IfcText", "IfcIdentifier"}
TIPOS_INTEIROS = {"IfcInteger", "IfcCountMeasure"}


def vazio(valor):
    """Célula vazia da planilha (None, NaN ou texto em branco)."""
    return valor is None or (isinstance(valor, float) and pd.isna(valor)) or str(valor).strip() == ""
```

```
def converter(valor, tipo_ifc, fator, soma):
    """Converte o valor da planilha para o tipo Python adequado ao tipo IFC."""
    if tipo_ifc in TIPOS_TEXTO:
        return str(valor).strip()
    if tipo_ifc == "IfcBoolean":
        texto = str(valor).strip().lower()
        if texto in ("sim", "s", "true", "verdadeiro", "1"):
            return True
        if texto in ("não", "nao", "n", "false", "falso", "0"):
            return False
        raise ValueError(f"valor lógico não reconhecido: {valor!r}")
    try:
        numero = float(str(valor).replace(",", "."))      # aceita vírgula decimal digitada como texto
    except ValueError:
        raise ValueError(f"valor numérico inválido: {valor!r}") from None
    numero = numero * (1 if vazio(fator) else float(fator)) + (0 if vazio(soma) else float(soma))
    return int(round(numero)) if tipo_ifc in TIPOS_INTEIROS else numero


class Preenchedor:
    """Aplica as regras de mapeamento a um modelo, reaproveitando conjuntos, materiais e sistemas."""

    def __init__(self, modelo):
        self.modelo = modelo
        self.psets = {}                  # (id do alvo, nome do conjunto) -> IfcPropertySet
        self.materiais = {m.Name: m for m in modelo.by_type("IfcMaterial")}
        self.sistemas = {c.Name: c for c in modelo.by_type("IfcClassification")}
        self.valores_tipo = {}           # (id do tipo, conjunto, propriedade) -> valor já gravado

    def alvo(self, elem, aplicar_em):
        if str(aplicar_em).strip().lower() == "tipo":
            tipo = util_elem.get_type(elem)
            if tipo is None:
                raise ValueError("o elemento não tem tipo associado")
            return tipo
        return elem

    def pset(self, alvo, nome):
        chave = (alvo.id(), nome)
        if chave not in self.psets:
            existente = util_elem.get_pset(alvo, nome, should_inherit=False)   # reaproveita o que já existe
            if existente:
                self.psets[chave] = self.modelo.by_id(existente["id"])
            else:
                self.psets[chave] = ifcopenshell.api.pset.add_pset(self.modelo, product=alvo, name=nome)
        return self.psets[chave]

    def aplicar(self, elem, regra, valor):
        destino = regra["Destino"]
        alvo = self.alvo(elem, regra["AplicarEm"])
        if destino == "Atributo":
            ifcopenshell.api.attribute.edit_attributes(
                self.modelo, product=alvo, attributes={regra["Propriedade"]: str(valor).strip()})
        elif destino == "Pset":
            tipo_ifc = regra["TipoIFC"]
            convertido = converter(valor, tipo_ifc, regra["Fator"], regra["Soma"])
            if alvo.is_a("IfcTypeObject"):                    # aviso: tipos são compartilhados
                chave = (alvo.id(), regra["Conjunto"], regra["Propriedade"])
                anterior = self.valores_tipo.get(chave)
                if anterior is not None and anterior != convertido:
                    raise ValueError(f"o tipo '{alvo.Name}' já recebeu {anterior!r} nesta execução")
                self.valores_tipo[chave] = convertido
            ifcopenshell.api.pset.edit_pset(
                self.modelo, pset=self.pset(alvo, regra["Conjunto"]),
                properties={regra["Propriedade"]: self.modelo.create_entity(tipo_ifc, convertido)})
        elif destino == "Material":
            nome = str(valor).strip()
            if nome not in self.materiais:
                self.materiais[nome] = ifcopenshell.api.material.add_material(self.modelo, name=nome)
            ifcopenshell.api.material.assign_material(
                self.modelo, products=[alvo], type="IfcMaterial", material=self.materiais[nome])
        elif destino == "Classificacao":
            nome_sistema = regra["Conjunto"]
```

```
            if nome_sistema not in self.sistemas:
                self.sistemas[nome_sistema] = ifcopenshell.api.classification.add_classification(
                    self.modelo, classification=nome_sistema)
            ifcopenshell.api.classification.add_reference(
                self.modelo, products=[alvo], identification=str(valor).strip(),
                classification=self.sistemas[nome_sistema])
        else:
            raise ValueError(f"destino desconhecido: {destino}")


def main():
    leitor = argparse.ArgumentParser(description="Preenche metadados de um IFC a partir de planilha.")
    leitor.add_argument("modelo")
    leitor.add_argument("planilha")
    leitor.add_argument("--saida")
    leitor.add_argument("--chave", default="Name", choices=["Name", "Tag", "GlobalId"])
    leitor.add_argument("--coluna-chave", default="TAG")
    args = leitor.parse_args()

    entrada = Path(args.modelo)
    saida = Path(args.saida) if args.saida else entrada.with_name(entrada.stem + "_metadados.ifc")
    registro = saida.with_name(saida.stem + "_registro.xlsx")
    if saida.resolve() == entrada.resolve():
        raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")

    modelo = ifcopenshell.open(entrada)
    dados = pd.read_excel(args.planilha, sheet_name="Dados", dtype=object)
    regras = pd.read_excel(args.planilha, sheet_name="Mapeamento", dtype=object).to_dict(orient="records")

    # Conferência do mapeamento antes de alterar qualquer coisa
    esquema = ifcopenshell.ifcopenshell_wrapper.schema_by_name(modelo.schema_identifier)
    for regra in regras:
        if regra["Coluna"] not in dados.columns:
            raise SystemExit(f"Mapeamento: a coluna '{regra['Coluna']}' não existe na aba Dados.")
        if regra["Destino"] not in ("Atributo", "Pset", "Material", "Classificacao"):
            raise SystemExit(f"Mapeamento: destino desconhecido '{regra['Destino']}'.")
        if regra["Destino"] == "Pset":
            try:
                esquema.declaration_by_name(regra["TipoIFC"])       # o tipo IFC existe nesta versão?
            except Exception:
                raise SystemExit(f"Mapeamento: o tipo '{regra['TipoIFC']}' não existe no esquema "
                                 f"{modelo.schema_identifier}.")

    # Índice: valor da chave -> elementos
    indice = defaultdict(list)
    for elem in modelo.by_type("IfcElement"):
        valor_chave = getattr(elem, args.chave, None)
        if valor_chave:
            indice[str(valor_chave).strip()].append(elem)

    preenchedor = Preenchedor(modelo)
    linhas_registro = []
    for posicao, linha in enumerate(dados.to_dict(orient="records")):
        linha_excel = posicao + 2                          # linha 1 é o cabeçalho
        chave = linha.get(args.coluna_chave)
        if vazio(chave):
            linhas_registro.append([linha_excel, None, "SEM_CHAVE", "coluna-chave vazia"])
            continue
        chave = str(chave).strip()
        encontrados = indice.get(chave, [])
        if not encontrados:
            linhas_registro.append([linha_excel, chave, "NAO_ENCONTRADO",
                                    f"nenhum elemento com {args.chave} = {chave}"])
            continue
        if len(encontrados) > 1:
            linhas_registro.append([linha_excel, chave, "DUPLICADO",
                                    f"{len(encontrados)} elementos com a mesma chave"])
            continue
        elem, gravados, erros = encontrados[0], 0, []
        for regra in regras:
            valor = linha.get(regra["Coluna"])
            if vazio(valor):
                continue                                   # célula vazia: nada é alterado
```

```
            try:
                preenchedor.aplicar(elem, regra, valor)
                gravados += 1
            except (ValueError, TypeError) as erro:
                erros.append(f"{regra['Coluna']}: {erro}")
        situacao = "OK" if not erros else "OK_COM_ERROS"
        detalhe = f"{gravados} valor(es) gravado(s)" + (" | " + "; ".join(erros) if erros else "")
        linhas_registro.append([linha_excel, chave, situacao, detalhe])

    # Cabeçalho: registra o nome do novo arquivo, a data e a ferramenta que o gravou.
    # O sistema de origem (originating_system) é mantido: os dados vieram do software de autoria.
    cabecalho = modelo.header.file_name
    cabecalho.name = saida.name
    cabecalho.time_stamp = datetime.now().replace(microsecond=0).isoformat()
    cabecalho.preprocessor_version = f"IfcOpenShell {ifcopenshell.version} (c10_03_preencher_metadados.py)"

    modelo.write(saida)
    tabela = pd.DataFrame(linhas_registro, columns=["LinhaPlanilha", "Chave", "Situacao", "Detalhe"])
    tabela.to_excel(registro, index=False)
    print(f"Modelo gravado: {saida.name} | registro: {registro.name}")
    print(tabela.to_string(index=False))


if __name__ == "__main__":
    main()
```

Como o script trata cada situação:

SITUAÇÃO	TRATAMENTO	SITUAÇÃO NO REGISTRO
Elemento encontrado, todos os valores gravados	Grava	`OK`
Alguns valores inválidos	Grava os válidos; registra os inválidos	`OK_COM_ERROS`
Chave não encontrada no modelo	Nada é gravado	`NAO_ENCONTRADO`
Mais de um elemento com a mesma chave	Nada é gravado, para não aplicar dados ao elemento errado	`DUPLICADO`
Coluna-chave vazia	Nada é gravado	`SEM_CHAVE`
Célula de dado vazia	O valor existente no modelo **não** é alterado	(não conta como gravado)
Duas linhas gravam valores diferentes no mesmo **tipo**	O segundo valor é recusado, porque o tipo é compartilhado	Erro na linha
Mapeamento com coluna ou tipo IFC inexistente	O script para antes de alterar qualquer coisa	(mensagem na tela)

Tabela 10.5: Tratamento das situações.

Execução, depois de copiar para a pasta deste capítulo a rede gerada no Capítulo 6:

```
copy ..\cap6\rede_exemplo.ifc .
python c10_03_preencher_metadados.py rede_exemplo.ifc metadados_rede.xlsx
```

Saída obtida no teste:

```
Modelo gravado: rede_exemplo_metadados.ifc | registro: rede_exemplo_metadados_registro.xlsx
 LinhaPlanilha  Chave       Situacao                                                    Detalhe
```

```
             2 TQ-001             OK                                                          8 valor(es) gravado(s)
             3 TB-001             OK                                                          8 valor(es) gravado(s)
             4 VG-001             OK                                                          9 valor(es) gravado(s)
             5 TB-002   OK_COM_ERROS 7 valor(es) gravado(s) | PressaoProjeto_kPa: valor numérico inválido: 'alta'
             6  B-001             OK                                                          9 valor(es) gravado(s)
             7 XX-999 NAO_ENCONTRADO                                        nenhum elemento com Name = XX-999
```

As linhas 2 a 6 foram gravadas; a linha 5 teve a pressão recusada ("alta" não é número), mas os demais valores foram gravados; a linha 7 não encontrou elemento. O registro em Excel (`rede_exemplo_metadados_registro.xlsx`) tem o mesmo conteúdo, para arquivamento.

10.5.5 Conferência

A conferência usa o exportador do Capítulo 9 sobre o arquivo gerado:

```
python ..\cap9\c9_04_exportar.py rede_exemplo_metadados.ifc --consulta IfcElement --colunas Name
ARV_Processo.PressaoProjeto ARV_Processo.TemperaturaProjeto material.Name classification.Identification
Pset_ManufacturerTypeInformation.Manufacturer --saida conferencia.xlsx
```

```
Consulta: IfcElement | 5 elementos | 6 colunas -> conferencia.xlsx
  Name  ARV_Processo.PressaoProjeto  ARV_Processo.TemperaturaProjeto               material.Name
classification.Identification Pset_ManufacturerTypeInformation.Manufacturer
TQ-001                           NaN                           313.15 Aço-carbono ASTM A283 Gr.
C                     3E.EX.01                                    Fabricante X
TB-001                    600000.0                           313.15 Aço-carbono ASTM A106 Gr.
B                     3E.EX.02                                         NaN
VG-001                    600000.0                           313.15   Aço-carbono ASTM A216
WCB                     3E.EX.03                                 Fabricante Y
TB-002                           NaN                           313.15 Aço-carbono ASTM A106 Gr.
B                     3E.EX.02                                         NaN
 B-001                   1000000.0                           313.15               Ferro
fundido                    3E.EX.04                                   Fabricante Z
```

Leitura: pressões em pascal (600 kPa = 600.000 Pa); temperaturas em kelvin (40 °C = 313,15 K); a pressão do tanque (vazia na planilha) e a do tubo TB-002 (valor inválido) não foram gravadas; o fabricante foi gravado nos tipos do tanque, da válvula e da bomba. A validação do arquivo:

```
python -m ifcopenshell.validate --rules rede_exemplo_metadados.ifc
```

```
Validating rede_exemplo_metadados.ifc
No validation issues found.
```

O script também foi executado uma segunda vez, sobre o arquivo já preenchido: os conjuntos existentes foram atualizados, sem duplicação, e o número de materiais e de itens de classificação não mudou. Essa propriedade (poder rodar de novo sem efeitos colaterais) permite atualizar o modelo a cada nova revisão da planilha.

10.5.6 Comparação com o fluxo no software de autoria

ASPECTO	DYNAMO NO SOFTWARE DE AUTORIA	SCRIPT SOBRE O IFC
Onde a informação é gravada	No modelo nativo; chega ao IFC pela exportação	Diretamente no IFC entregue
Dependência do exportador	Total: o mapeamento de parâmetros para conjuntos precisa estar configurado	Nenhuma: o script decide conjunto, propriedade e tipo

ASPECTO	DYNAMO NO SOFTWARE DE AUTORIA	SCRIPT SOBRE O IFC
Repetição a cada revisão	Rodar o Dynamo e exportar de novo	Exportar e rodar o script de novo
Rastreabilidade	Depende do registro feito à mão	Registro por linha, gravado automaticamente
Quando preferir	Quando o modelo nativo é o entregável principal, ou a informação precisa existir nele	Quando o IFC é o entregável, ou o exportador não grava a informação como exigido

Tabela 10.6: Dynamo no software de autoria e script sobre o IFC.

Os dois fluxos não são excludentes: é comum manter a informação no modelo nativo e usar o script para completar ou padronizar o IFC antes da entrega.

10.6 Outras operações

OPERAÇÃO	FUNÇÃO	OBSERVAÇÃO
Remover um elemento	`root.remove_product(modelo, product=elem)`	Remove também relações e geometria que só ele usava
Copiar um elemento	`root.copy_class(modelo, product=elem)`	A cópia recebe um GlobalId novo
Mudar o contêiner	`spatial.assign_container(modelo, products=[...], relating_structure=pav)`	Substitui a contenção anterior
Ligar a um tipo	`type.assign_type(modelo, related_objects=[...], relating_type=tipo)`	
Mover	`geometry.edit_object_placement(modelo, product=elem, matrix=m)`	Matriz 4x4, em metros por padrão (`is_si=True`)
Georreferenciar	`georeference.add_georeferencing` e `edit_georeferencing`	Cria e edita `IfcMapConversion` e `IfcProjectedCRS` (Capítulo 7)
Quantidades	`pset.add_qto` e `pset.edit_qto`	Mesma lógica dos conjuntos de propriedades
Documentos	`document.add_information`, `add_reference`, `assign_document`	Folhas de dados, desenhos

Tabela 10.7: Outras operações da API.

10.7 Resumo do capítulo

- A API de edição é chamada como `ifcopenshell.api.modulo.funcao(modelo, ...)`; a sintaxe `ifcopenshell.api.run(...)` está obsoleta.
- Nunca altere o original: edite na memória, grave um arquivo novo, valide e confira.
- `edit_pset` usa os tipos do modelo nos conjuntos padronizados; nos personalizados, números viram `IfcReal`. Crie medidas com `modelo.create_entity("IfcPressureMeasure", valor)`.

- Enumeradas recebem listas; `None` remove a propriedade; `assign_material` substitui o material; `reassign_class` troca a classe preservando o GlobalId.
- O preenchimento em lote separa dados e mapeamento, converte unidades para o SI, trata chaves ausentes e duplicadas, registra cada linha e pode ser executado de novo sem duplicar informação.

10.8 Vocabulário técnico

INGLÊS	PORTUGUÊS
API (application programming interface)	interface de programação
use case / function	caso de uso / função
deprecated	obsoleto, desaconselhado
edit / add / assign / remove	editar / acrescentar / associar / remover
reassign class	trocar a classe
batch	em lote
mapping	mapeamento
key / lookup	chave / busca
log	registro
idempotent	que pode ser repetido sem efeito adicional
header	cabeçalho

Tabela 10.8: Vocabulário do capítulo.

10.9 Exercícios

Fixação

1. Reescreva na sintaxe atual: `ifcopenshell.api.run("attribute.edit_attributes", modelo, product=tubo, attributes={"Tag": "TB-001"})`.
2. Num conjunto personalizado, `edit_pset(..., properties={"Pressao": 600000.0})` grava que tipo? Como gravar como pressão?
3. Como remover a propriedade `Revisao` do conjunto `ARV_Controle` com a API?
4. Por que usar `reassign_class` em vez de criar um elemento novo da classe certa e apagar o antigo?
5. Um valor de 25 °C deve ser gravado como `IfcThermodynamicTemperatureMeasure` num projeto sem unidade de temperatura declarada. Que valor gravar?

Aplicação

6. Acrescente à planilha de exemplo (numa cópia) a coluna `Isolamento`, com "Sim" ou "Não", e uma regra que a grave em `ARV_Processo.Isolado`, tipo `IfcBoolean`. Execute o preenchimento e confira os valores com o exportador do Capítulo 9.

7. Use o exportador do Capítulo 9 para gerar uma planilha com o `GlobalId` e o `Name` dos elementos de `rede_exemplo.ifc`. Monte, a partir dela, uma planilha de dados cuja chave seja o GlobalId, com uma coluna `Revisao` = "C", e execute o preenchimento com `--chave GlobalId --coluna-chave GlobalId`.

Desafio

8. Acrescente ao script de preenchimento (numa cópia) a opção `--simular`, que executa todas as verificações e produz o registro, mas não grava o IFC. Qual é a utilidade dessa opção no fluxo de trabalho?

10.10 Respostas dos exercícios

1. `ifcopenshell.api.attribute.edit_attributes(modelo, product=tubo, attributes={"Tag": "TB-001"})`, com `import ifcopenshell.api.attribute` no início do script.
2. `IfcReal`, um número sem grandeza. Para gravar como pressão: `properties={"Pressao": modelo.create_entity("IfcPressureMeasure", 600000.0)}`.
3. `ifcopenshell.api.pset.edit_pset(modelo, pset=conjunto, properties={"Revisao": None})`, em que `conjunto` é o `IfcPropertySet` `ARV_Controle` do elemento (obtido, por exemplo, com `modelo.by_id(util.element.get_pset(elem, "ARV_Controle", should_inherit=False)["id"])`).
4. Porque um elemento novo teria outro GlobalId, e todos os vínculos com revisões anteriores, apontamentos e planilhas se perderiam; além disso, seria preciso recriar à mão as relações (contêiner, tipo, propriedades, material). `reassign_class` preserva o GlobalId e as relações.
5. 298,15 (kelvin): 25 + 273,15. Com o grau Celsius declarado como unidade do projeto, o valor seria 25.
6. Na aba Dados, a coluna `Isolamento`; na aba Mapeamento, a linha `Isolamento | Pset | ARV_Processo | Isolado | IfcBoolean | ocorrencia | (vazio) | (vazio)`. O script converte "Sim" em verdadeiro e "Não" em falso. No teste, com os valores Sim, Não, Não, Sim, Não e Sim para as seis linhas, o exportador mostrou `True` para TQ-001 e TB-002 e `False` para TB-001, VG-001 e B-001, e o arquivo trouxe linhas como `IFCPROPERTYSINGLEVALUE('Isolado',$,IFCBOOLEAN(.T.),$)`. Um tipo digitado errado no mapeamento (por exemplo, `IfcBooleano`) é detectado antes de qualquer alteração: "Mapeamento: o tipo 'IfcBooleano' não existe no esquema IFC4X3_ADD2."
7. Com a aba Dados contendo `GlobalId` e `Revisao`, e o mapeamento `Revisao | Pset | ARV_Controle | Revisao | IfcLabel | ocorrencia`, a execução com `--chave GlobalId --coluna-chave GlobalId` gravou um valor em cada um dos cinco elementos (situação `OK`). A chave por GlobalId é a mais segura entre revisões, desde que o exportador preserve os identificadores (Capítulo 13).
8. Acrescentar o argumento `leitor.add_argument("--simular", action="store_true")` e condicionar a gravação: `if not args.simular: modelo.write(saida)`; ajustar a mensagem final. No teste, a simulação produziu apenas o registro, sem o IFC. Utilidade: revisar a planilha antes da gravação, corrigir TAGs não encontradas, duplicadas e valores inválidos, e só então gravar o modelo, evitando versões intermediárias com dados incompletos.

10.11 Referências

- Documentação do IfcOpenShell, API: https://docs.ifcopenshell.org/autoapi/ifcopenshell/api/index.html
- Documentação do IfcOpenShell, `ifcopenshell.api.pset.edit_pset`: https://docs.ifcopenshell.org/autoapi/ifcopenshell/api/pset/edit_pset/index.html
- Documentação do IfcOpenShell (geral): https://docs.ifcopenshell.org/

11 Geometria por código

O Capítulo 7 mostrou como a geometria é **descrita** no arquivo: extrusões, varreduras, malhas, posicionamentos. Este capítulo mostra como **processá-la** por programa: gerar a forma final de cada elemento, medir volumes, áreas e cotas, conferir as quantidades declaradas pelo exportador, criar geometria simples, exportar malhas para outros programas e detectar interferências.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar o que faz um núcleo geométrico e por que ele é necessário;
- gerar a geometria de elementos com `ifcopenshell.geom` e medir volume, área, dimensões e cotas;
- conferir quantidades declaradas contra a geometria;
- criar paredes, perfis extrudados e tubos por código, com as unidades corretas;
- exportar a geometria para OBJ e glTF;
- detectar interferências entre grupos de elementos.

11.1 Introdução: da descrição à forma

11.1.1 O núcleo geométrico

No arquivo IFC, uma parede pode ser "um retângulo de 5000 x 200 mm extrudado 3000 mm, posicionado em relação ao pavimento, com uma abertura de porta". Para saber o volume dessa parede, desenhá-la na tela ou verificar se um tubo a atravessa, é preciso **calcular** a forma final: aplicar a extrusão, compor os posicionamentos, subtrair a abertura. Esse cálculo é feito por um **núcleo geométrico** (*geometry kernel*).

O IfcOpenShell tem um núcleo geométrico próprio, acessado pelo módulo `ifcopenshell.geom`, que usa a biblioteca Open CASCADE. Ele converte cada elemento numa **malha de triângulos**: uma lista de vértices (pontos) e uma lista de faces (trios de índices de vértices), a mesma estrutura do `IfcTriangulatedFaceSet` do Capítulo 7. Sobre essa malha, as funções de `ifcopenshell.util.shape` calculam volumes, áreas e dimensões.

11.1.2 Configurações importantes

CONFIGURAÇÃO	EFEITO
`use-world-coords`	Se verdadeira, os vértices vêm no sistema do projeto; se falsa (padrão), no sistema local do elemento, e a posição vem numa matriz separada
`disable-opening-subtractions`	Se verdadeira, não subtrai as aberturas (mais rápido, mas o volume fica bruto)
`mesher-linear-deflection`, `mesher-angular-deflection`	Precisão da aproximação de superfícies curvas por triângulos
`context-identifiers`	Quais representações usar (por exemplo, só `Body`)

Tabela 11.1: Configurações do núcleo geométrico (seleção).

As coordenadas devolvidas pelo núcleo estão em **metros**, qualquer que seja a unidade do arquivo: o núcleo converte as unidades automaticamente.

11.1.3 Duas formas de gerar geometria

- `ifcopenshell.geom.create_shape(config, elemento)` : gera a forma de um elemento. Simples e adequada a poucos elementos;
- `ifcopenshell.geom.iterator(config, modelo, num_threads, include=[...])` : percorre muitos elementos, com processamento em paralelo e reaproveitamento de geometrias repetidas. É a forma adequada a modelos inteiros.

> ATENÇÃO
>
> Guarde a forma numa variável antes de usar a geometria: `forma = ifcopenshell.geom.create_shape(config, elem)` e, depois, `forma.geometry`. Encadear `create_shape(config, elem).geometry` descarta a forma logo em seguida, e a geometria passa a apontar para dados já liberados da memória. Nos testes, esse encadeamento produziu caixas envolventes erradas e a função `get_volume` falhou com `IndexError`. Todos os scripts deste capítulo guardam a forma.

11.2 Medir elementos

O script abaixo gera a geometria dos elementos selecionados por uma consulta (Capítulo 9) e calcula, para cada um, volume, área de superfície, dimensões da caixa envolvente e cotas inferior e superior, comparando o volume calculado com o declarado nos conjuntos de quantidades. Grave-o como `C:\IFC_Curso\exercicios\cap11\c11_01_medir_elementos.py`.

```
# -*- coding: utf-8 -*-
"""
c11_01_medir_elementos.py | Capítulo 11: medições a partir da geometria.

Gera a geometria dos elementos selecionados com o núcleo geométrico do IfcOpenShell
(ifcopenshell.geom) e calcula, para cada um: volume, área de superfície, dimensões da
caixa envolvente e cotas inferior e superior, no sistema do projeto, em metros.
Compara o volume calculado com a quantidade NetVolume ou GrossVolume gravada no
arquivo (Qto_...BaseQuantities), quando existir. Grava uma planilha NOVA.
O arquivo IFC é aberto somente para leitura.

Uso: python c11_01_medir_elementos.py <arquivo.ifc> [--consulta IfcElement] [--saida medicoes.xlsx]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; pandas 3.0.
"""
import argparse
import multiprocessing
import pandas as pd
import ifcopenshell
import ifcopenshell.geom
import ifcopenshell.util.element as util_elem
import ifcopenshell.util.selector as seletor
import ifcopenshell.util.shape as util_forma


def volume_declarado(elem):
    """Volume gravado nos conjuntos de quantidades (m³, se a unidade de volume do arquivo for m³)."""
    for nome, qtos in util_elem.get_psets(elem, qtos_only=True).items():
        for chave in ("NetVolume", "GrossVolume"):
            if chave in qtos:
                return qtos[chave]
    return None
```

```
def main():
    leitor = argparse.ArgumentParser(description="Medições a partir da geometria IFC.")
    leitor.add_argument("arquivo")
    leitor.add_argument("--consulta", default="IfcElement")
    leitor.add_argument("--saida", default="medicoes.xlsx")
    args = leitor.parse_args()

    modelo = ifcopenshell.open(args.arquivo)
    elementos = list(seletor.filter_elements(modelo, args.consulta))
    config = ifcopenshell.geom.settings()
    config.set("use-world-coords", True)          # vértices no sistema do projeto, em metros

    linhas = []
    iterador = ifcopenshell.geom.iterator(config, modelo, multiprocessing.cpu_count(), include=elementos)
    if iterador.initialize():
        while True:
            forma = iterador.get()
            elem = modelo.by_id(forma.id)
            geo = forma.geometry
            minimo, maximo = util_forma.get_bbox(util_forma.get_vertices(geo))
            vol = util_forma.get_volume(geo)
            declarado = volume_declarado(elem)
            linhas.append({
                "id": elem.id(), "Classe": elem.is_a(), "Nome": elem.Name,
                "Volume_m3": round(vol, 4),
                "Volume_declarado_m3": None if declarado is None else round(declarado, 4),
                "Area_superficie_m2": round(util_forma.get_area(geo), 3),
                "DX_m": round(maximo[0] - minimo[0], 3), "DY_m": round(maximo[1] - minimo[1], 3),
                "DZ_m": round(maximo[2] - minimo[2], 3),
                "Cota_inferior_m": round(util_forma.get_bottom_elevation(geo), 3),
                "Cota_superior_m": round(util_forma.get_top_elevation(geo), 3),
            })
            if not iterador.next():
                break

    tabela = pd.DataFrame(linhas).sort_values("id")
    tabela.to_excel(args.saida, index=False)
    sem_geometria = len(elementos) - len(tabela)
    print(f"{len(tabela)} elementos medidos ({sem_geometria} sem geometria) -> {args.saida}")
    print(tabela.drop(columns=["Area_superficie_m2"]).to_string(index=False))


if __name__ == "__main__":
    main()
```

Execução com as paredes e lajes da amostra de arquitetura:

```
cd C:\IFC_Curso\exercicios\cap11
python c11_01_medir_elementos.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Building-Architecture.ifc" --consulta "IfcWall + IfcSlab" --saida medicoes.xlsx
```

Saída obtida no teste:

```
7 elementos medidos (0 sem geometria) -> medicoes.xlsx
 id  Classe                                   Nome  Volume_m3  Volume_declarado_m3  DX_m  DY_m  DZ_m
Cota_inferior_m  Cota_superior_m
 53 IfcSlab                                 floor     6.4375               6.4375 5.200   5.8 0.250
-0.250            0.000
158 IfcWall house - outer wall - house right front     1.2693               1.2693 0.200   1.8 3.626
-0.250            3.376
182 IfcWall  house - outer wall - house right back     1.7856               1.7856 0.200   4.2 2.226
-0.250            1.976
201 IfcWall        house - outer wall - house left     4.2309               4.2309 0.200   6.0 3.626
-0.250            3.376
228 IfcWall                         plumbing wall     0.1647               0.1647 0.024   3.8 2.226
0.000             2.226
261 IfcSlab               house - roof - slab left     6.7203               6.7203 2.400   6.6 2.824
2.876             5.700
```

```
285 IfcSlab              house - roof - slab right     9.3635              9.3635 3.800   6.6 4.224
1.476           5.700
```

Leitura:

- os volumes calculados pela geometria coincidem com os declarados pelo exportador em todas as sete peças (por exemplo, 1,2693 m³ na parede `#158`, o mesmo valor do Capítulo 5). Quando não coincidem, há três suspeitos: quantidades calculadas numa revisão anterior da geometria; regras de medição diferentes (bruto x líquido, Capítulo 5); ou geometria com erro;
- as dimensões da caixa envolvente (`DX`, `DY`, `DZ`) estão alinhadas aos eixos do projeto. Para elementos girados, a caixa é maior que a peça;
- as cotas inferior e superior permitem verificar, por exemplo, se todas as paredes de um pavimento começam na mesma cota. Aqui, a parede `plumbing wall` (`#228`) começa em 0,000, enquanto as externas começam em -0,250.

11.3 Criar geometria

11.3.1 Unidades: o cuidado principal

Ao criar geometria por código, cada função trabalha numa unidade, e confundi-las é o erro mais comum. Na versão 0.8.5, conforme verificado nos testes deste capítulo:

RECURSO	UNIDADE DOS VALORES INFORMADOS
`geometry.add_wall_representation` (comprimento, altura, espessura)	Metros; a função converte para a unidade do arquivo
`geometry.add_profile_representation` (`depth`, a extensão)	Metros; a função converte
`geometry.edit_object_placement` (matriz)	Metros, com `is_si=True` (padrão)
Atributos de perfis paramétricos (`OverallWidth`, `OverallDepth`...)	Unidade do arquivo
`ShapeBuilder` (pontos, raios)	Unidade do arquivo
Atributos gravados diretamente (`Depth`, `Radius`)	Unidade do arquivo

Tabela 11.2: Unidades nas funções de criação de geometria.

No exemplo a seguir, o arquivo usa milímetros (padrão de `unit.assign_unit`): a parede é criada com `length=5.0` (metros) e gravada com 5000 mm; o perfil IPE 200 recebe 100 e 200 (milímetros).

11.3.2 Exemplo: parede, pilar e tubo

O script cria um arquivo novo com três elementos, cada um com um tipo de geometria do Capítulo 7, e confere os volumes. O tubo foi posicionado de propósito para que um de seus trechos atravesse a parede (seção 11.5). Grave-o como `c11_02_criar_geometria.py`.

```
# -*- coding: utf-8 -*-
"""
c11_02_criar_geometria.py | Capítulo 11: cria um arquivo IFC 4.3 com geometria simples.

Cria, num arquivo NOVO:
  - uma parede de 5,0 x 3,0 x 0,2 m (função pronta da API);
```

```
  - um pilar metálico IPE 200 de 3,0 m (perfil paramétrico extrudado);
  - um tubo DN100 (114,3 x 6,02 mm) com três trechos retos (IfcSweptDiskSolid).
Depois, reabre o arquivo e mede os volumes com o núcleo geométrico.

Uso: python c11_02_criar_geometria.py [saida.ifc]     (padrão: geometria_exemplo.ifc)
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import math
import sys
import numpy as np
import ifcopenshell
import ifcopenshell.api.aggregate
import ifcopenshell.api.context
import ifcopenshell.api.geometry
import ifcopenshell.api.profile
import ifcopenshell.api.project
import ifcopenshell.api.root
import ifcopenshell.api.spatial
import ifcopenshell.api.unit
import ifcopenshell.geom
import ifcopenshell.util.shape as util_forma
import ifcopenshell.util.shape_builder as construtor

saida = sys.argv[1] if len(sys.argv) > 1 else "geometria_exemplo.ifc"
modelo = ifcopenshell.api.project.create_file(version="IFC4X3")
criar = ifcopenshell.api.root.create_entity

projeto = criar(modelo, ifc_class="IfcProject", name="Geometria de estudo")
ifcopenshell.api.unit.assign_unit(modelo)                      # comprimentos em milímetros
modelo3d = ifcopenshell.api.context.add_context(modelo, context_type="Model")
corpo = ifcopenshell.api.context.add_context(modelo, context_type="Model", context_identifier="Body",
                                             target_view="MODEL_VIEW", parent=modelo3d)
terreno = criar(modelo, ifc_class="IfcSite", name="Terreno")
ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)


def posicionar(produto, x_m, y_m, z_m):
    """Posiciona o produto; a matriz é informada em metros (is_si=True, padrão)."""
    matriz = np.eye(4)
    matriz[:3, 3] = (x_m, y_m, z_m)
    ifcopenshell.api.geometry.edit_object_placement(modelo, product=produto, matrix=matriz)


# 1. Parede: a função recebe as dimensões em metros e converte para a unidade do arquivo
parede = criar(modelo, ifc_class="IfcWall", predefined_type="SOLIDWALL", name="Parede P-01")
rep = ifcopenshell.api.geometry.add_wall_representation(modelo, context=corpo, length=5.0,
                                                        height=3.0, thickness=0.2)
ifcopenshell.api.geometry.assign_representation(modelo, product=parede, representation=rep)
posicionar(parede, 0.0, 0.0, 0.0)

# 2. Pilar IPE 200: perfil paramétrico (dimensões na unidade do arquivo, mm) extrudado 3,0 m
pilar = criar(modelo, ifc_class="IfcColumn", predefined_type="COLUMN", name="Pilar PL-01")
perfil = ifcopenshell.api.profile.add_parameterized_profile(modelo, ifc_class="IfcIShapeProfileDef")
perfil.ProfileName = "IPE200"
perfil.OverallWidth, perfil.OverallDepth = 100.0, 200.0
perfil.WebThickness, perfil.FlangeThickness, perfil.FilletRadius = 5.6, 8.5, 12.0
rep = ifcopenshell.api.geometry.add_profile_representation(modelo, context=corpo, profile=perfil,
                                                           depth=3.0)       # profundidade em metros
ifcopenshell.api.geometry.assign_representation(modelo, product=pilar, representation=rep)
posicionar(pilar, 7.0, 0.0, 0.0)

# 3. Tubo DN100: disco varrido ao longo de uma polilinha (coordenadas em mm)
tubo = criar(modelo, ifc_class="IfcPipeSegment", predefined_type="RIGIDSEGMENT", name="TB-001")
cb = construtor.ShapeBuilder(modelo)
caminho = cb.polyline([(0.0, 0.0, 0.0), (4000.0, 0.0, 0.0), (4000.0, 3000.0, 0.0), (4000.0, 3000.0, 2000.0)])
solido = cb.create_swept_disk_solid(caminho, radius=114.3 / 2)
solido.InnerRadius = 114.3 / 2 - 6.02                          # tubo vazado
rep = cb.get_representation(context=corpo, items=[solido])
ifcopenshell.api.geometry.assign_representation(modelo, product=tubo, representation=rep)
posicionar(tubo, 0.0, -1.0, 0.5)                               # o 2º trecho atravessa a parede

ifcopenshell.api.spatial.assign_container(modelo, products=[parede, pilar, tubo], relating_structure=terreno)
modelo.write(saida)
```

```
print("Arquivo gravado:", saida)

# Conferência: volumes calculados pelo núcleo geométrico (resultados em metros)
config = ifcopenshell.geom.settings()
reaberto = ifcopenshell.open(saida)
esperado = {
    "Parede P-01": 5.0 * 3.0 * 0.2,
    "Pilar PL-01": (2 * 100 * 8.5 + (200 - 2 * 8.5) * 5.6 + (4 - math.pi) * 12.0 ** 2) * 1e-6 * 3.0,
    "TB-001": math.pi * ((0.1143 / 2) ** 2 - (0.1143 / 2 - 0.00602) ** 2) * 9.0,
}
for elem in sorted(reaberto.by_type("IfcElement"), key=lambda e: e.Name):
    forma = ifcopenshell.geom.create_shape(config, elem)
    volume = util_forma.get_volume(forma.geometry)
    print(f"{elem.Name:<12} {elem.Representation.Representations[0].RepresentationType:<12}"
          f" volume calculado {volume:.6f} m³ | esperado {esperado[elem.Name]:.6f} m³")
```

Execução e saída obtida:

```
python c11_02_criar_geometria.py geometria_exemplo.ifc
```

```
Arquivo gravado: geometria_exemplo.ifc
Parede P-01  SweptSolid   volume calculado 3.000000 m³ | esperado 3.000000 m³
Pilar PL-01  SweptSolid   volume calculado 0.008557 m³ | esperado 0.008545 m³
TB-001       SolidModel   volume calculado 0.018323 m³ | esperado 0.018431 m³
```

A validação do arquivo gerado:

```
python -m ifcopenshell.validate --rules geometria_exemplo.ifc
```

```
Validating geometria_exemplo.ifc
No validation issues found.
```

Os resultados mostram três situações:

- **parede**: volume calculado idêntico ao esperado (5,0 x 3,0 x 0,2 = 3,0 m³);
- **pilar**: diferença de 0,14% em relação à área teórica do perfil (28,48 cm², com as concordâncias). O núcleo aproxima os arcos das concordâncias por segmentos de reta, o que altera ligeiramente a área. Para peças com superfícies curvas, o volume da malha é uma aproximação, controlada pelas configurações de precisão (Tabela 11.1);
- **tubo**: volume 0,6% menor que o produto da seção pelo comprimento do eixo (9,0 m). Nos cantos vivos da polilinha, os trechos são cortados em esquadria e não se sobrepõem, e o cálculo simples pelo eixo conta duas vezes a região dos cantos. Em redes reais, as curvas são peças próprias (`IfcPipeFitting`, `BEND`), e o comprimento de cada trecho reto é medido entre as conexões.

Trechos do arquivo gravado, que confirmam as unidades:

```
#23=IFCEXTRUDEDAREASOLID(#18,#22,#17,3000.);
#32=IFCISHAPEPROFILEDEF(.AREA.,'IPE200',$,100.,200.,5.5999999999999996,8.5,12.,$,$);
#38=IFCEXTRUDEDAREASOLID(#32,#36,#37,3000.);
#47=IFCCARTESIANPOINTLIST3D(((0.,0.,0.),(4000.,0.,0.),(4000.,3000.,0.),(4000.,3000.,2000.)),$);
#49=IFCSWEPTDISKSOLID(#48,57.149999999999999,51.129999999999995,$,$);
```

Figura 11.1: Geometria gravada em milímetros.

O valor `5.5999999999999996` é a representação binária de 5,6 (Capítulo 8); não é erro.

11.4 Exportar malhas

Outros programas (visualizadores web, ferramentas de renderização, realidade virtual, plataformas de gêmeos digitais) costumam aceitar malhas em formatos como OBJ e glTF. O núcleo geométrico do IfcOpenShell tem **serializadores** para esses formatos. Grave o script como `c11_03_exportar_malha.py`.

```
# -*- coding: utf-8 -*-
"""
c11_03_exportar_malha.py | Capítulo 11: exporta a geometria de um IFC para OBJ ou glTF (.glb).

Usa os serializadores do núcleo geométrico do IfcOpenShell. O formato é escolhido pela
extensão da saída: .obj (gera também o .mtl, com as cores) ou .glb (glTF binário).
O arquivo IFC é aberto somente para leitura.

Uso: python c11_03_exportar_malha.py <arquivo.ifc> <saida.obj|saida.glb>
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import multiprocessing
import sys
from pathlib import Path
import ifcopenshell
import ifcopenshell.geom

entrada, saida = Path(sys.argv[1]), Path(sys.argv[2])
modelo = ifcopenshell.open(entrada)
config = ifcopenshell.geom.settings()
config.set("use-world-coords", True)
config_serial = ifcopenshell.geom.serializer_settings()

if saida.suffix.lower() == ".obj":
    serializador = ifcopenshell.geom.serializers.obj(str(saida), str(saida.with_suffix(".mtl")),
                                                     config, config_serial)
elif saida.suffix.lower() == ".glb":
    serializador = ifcopenshell.geom.serializers.gltf(str(saida), config, config_serial)
else:
    raise SystemExit("A saída deve terminar em .obj ou .glb")

serializador.setFile(modelo)
serializador.setUnitNameAndMagnitude("METER", 1.0)
serializador.writeHeader()
iterador = ifcopenshell.geom.iterator(config, modelo, multiprocessing.cpu_count())
quantidade = 0
if iterador.initialize():
    while True:
        serializador.write(iterador.get())
        quantidade += 1
        if not iterador.next():
            break
serializador.finalize()
del serializador

for temporario in saida.parent.glob(saida.name + ".*.tmp"):      # arquivos auxiliares do glTF
    temporario.unlink()
print(f"{quantidade} elementos exportados -> {saida.name} ({saida.stat().st_size / 1024:.0f} KB)")
```

Execução, com a amostra de arquitetura, nos dois formatos:

```
python c11_03_exportar_malha.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Building-Architecture.ifc" casa.obj
python c11_03_exportar_malha.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC 4.3
ADD2)\Simple-Scene\Building-Architecture.ifc" casa.glb
```

Saídas obtidas:

```
14 elementos exportados -> casa.obj (42 KB)
14 elementos exportados -> casa.glb (32 KB)
```

A exportação leva só a geometria e as cores. As propriedades não vão para o OBJ; no glTF, apenas identificadores básicos. Por isso, essas malhas servem para visualização, e o IFC continua sendo o arquivo de informação.

Para conversões sem programar, o projeto IfcOpenShell distribui o **IfcConvert**, um programa de linha de comando baixado da página de versões do projeto (não é instalado pelo `pip`). A sintaxe básica é `IfcConvert entrada.ifc saida.glb`, e o formato é escolhido pela extensão: `.obj`, `.dae`, `.glb`, `.stp`, `.igs`, `.xml`, `.json`, `.svg` (plantas), `.h5`, `.ttl` e `.ifc`, segundo a documentação da versão 0.8.5.

11.5 Detectar interferências

11.5.1 O conceito

A **detecção de interferências** (*clash detection*) procura elementos que se sobrepõem indevidamente: um tubo atravessando uma viga, um duto passando por uma parede sem abertura, uma bandeja elétrica encostada numa tubulação quente. É uma das principais utilidades de modelos federados, feita em geral no Navisworks ou em plataformas de coordenação (Capítulo 13). Com o IfcOpenShell, é possível automatizar verificações específicas.

O núcleo geométrico oferece uma **árvore espacial** (`ifcopenshell.geom.tree`), que organiza as formas por região e permite consultas rápidas. A função `clash_intersection_many` procura interseções entre dois grupos e classifica cada ocorrência:

TIPO	SIGNIFICADO
Saliência (*protrusion*)	Um elemento penetra parcialmente no outro
Atravessamento (*pierce*)	Um elemento atravessa o outro de lado a lado
Colisão (*collision*)	Contato ou sobreposição
Afastamento (*clearance*)	Distância menor que a exigida (usado por outra função, `clash_clearance_many`)

Tabela 11.3: Tipos de interferência do IfcOpenShell.

11.5.2 Exemplo

Grave o script como `c11_04_interferencias.py`.

```
# -*- coding: utf-8 -*-
"""
c11_04_interferencias.py | Capítulo 11: detecção de interferências (colisões) entre dois grupos.

Gera a geometria dos elementos, monta a árvore espacial do IfcOpenShell (ifcopenshell.geom.tree)
e procura interseções entre os elementos do grupo A e os do grupo B, selecionados com
consultas do seletor (Capítulo 9). Grava uma planilha NOVA com as interferências.
O arquivo IFC é aberto somente para leitura.

Uso: python c11_04_interferencias.py <arquivo.ifc> --grupo-a "IfcWall" --grupo-b "IfcPipeSegment"
                                     [--tolerancia 0.002] [--saida interferencias.xlsx]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; pandas 3.0.
"""
import argparse
import multiprocessing
import pandas as pd
import ifcopenshell
```

```
import ifcopenshell.geom
import ifcopenshell.util.selector as seletor

TIPOS = {0: "saliência (protrusion)", 1: "atravessamento (pierce)", 2: "colisão (collision)",
         3: "afastamento (clearance)"}


def main():
    leitor = argparse.ArgumentParser(description="Detecção de interferências entre dois grupos.")
    leitor.add_argument("arquivo")
    leitor.add_argument("--grupo-a", required=True)
    leitor.add_argument("--grupo-b", required=True)
    leitor.add_argument("--tolerancia", type=float, default=0.002, help="em metros (padrão: 0,002)")
    leitor.add_argument("--saida", default="interferencias.xlsx")
    args = leitor.parse_args()

    modelo = ifcopenshell.open(args.arquivo)
    grupo_a = list(seletor.filter_elements(modelo, args.grupo_a))
    grupo_b = list(seletor.filter_elements(modelo, args.grupo_b))

    arvore = ifcopenshell.geom.tree()
    config = ifcopenshell.geom.settings()
    iterador = ifcopenshell.geom.iterator(config, modelo, multiprocessing.cpu_count(),
                                          include=grupo_a + grupo_b)
    if iterador.initialize():
        while True:
            arvore.add_element(iterador.get())
            if not iterador.next():
                break

    choques = arvore.clash_intersection_many(grupo_a, grupo_b, tolerance=args.tolerancia, check_all=True)
    linhas = []
    for c in choques:
        a, b = modelo.by_id(c.a.id()), modelo.by_id(c.b.id())   # instâncias do arquivo
        linhas.append({
            "Elemento_1": f"{a.is_a()} {a.Name}", "GlobalId_1": a.GlobalId,
            "Elemento_2": f"{b.is_a()} {b.Name}", "GlobalId_2": b.GlobalId,
            "Tipo": TIPOS.get(c.clash_type, c.clash_type),
            "Distancia_m": round(c.distance, 4),
            "Ponto_x_m": round(c.p1[0], 3), "Ponto_y_m": round(c.p1[1], 3), "Ponto_z_m": round(c.p1[2], 3),
        })
    tabela = pd.DataFrame(linhas, columns=["Elemento_1", "GlobalId_1", "Elemento_2", "GlobalId_2", "Tipo",
                                           "Distancia_m", "Ponto_x_m", "Ponto_y_m", "Ponto_z_m"])
    tabela.to_excel(args.saida, index=False)
    print(f"Grupo A: {len(grupo_a)} | grupo B: {len(grupo_b)} | interferências: {len(tabela)} "
          f"-> {args.saida}")
    if not tabela.empty:
        print(tabela.drop(columns=["GlobalId_1", "GlobalId_2"]).to_string(index=False))


if __name__ == "__main__":
    main()
```

Execução com o arquivo criado na seção 11.3:

```
python c11_04_interferencias.py geometria_exemplo.ifc --grupo-a "IfcWall + IfcColumn" --grupo-b IfcPipeSegment --
saida interferencias.xlsx
```

```
Grupo A: 2 | grupo B: 1 | interferências: 1 -> interferencias.xlsx
           Elemento_1          Elemento_2                   Tipo  Distancia_m  Ponto_x_m  Ponto_y_m  Ponto_z_m
IfcPipeSegment TB-001 IfcWall Parede P-01 atravessamento (pierce)          0.2      3.959        0.0      0.531
```

O tubo atravessa a parede: a distância informada (0,2 m) é a espessura da parede, e o ponto indica a face onde o tubo entra. Na prática, essa informação alimenta um apontamento BCF (Capítulo 12), com o GlobalId dos dois elementos, para que o projetista crie a abertura ou desvie o tubo.

Para verificações maiores, o projeto IfcOpenShell mantém o **IfcClash** (pacote `ifcclash` no PyPI, versão 0.8.5), que organiza conjuntos de testes, grupos e tolerâncias e exporta os resultados em BCF.

11.6 Resumo do capítulo

- O núcleo geométrico calcula a forma final de cada elemento como malha de triângulos, com coordenadas em metros.
- `create_shape` serve para poucos elementos; `iterator` para modelos inteiros, em paralelo.
- `ifcopenshell.util.shape` mede volume, área, caixa envolvente e cotas; compare com as quantidades declaradas.
- Ao criar geometria, confira a unidade de cada função: algumas recebem metros e convertem, outras usam a unidade do arquivo.
- Superfícies curvas são aproximadas por triângulos; cantos de varreduras não somam volume como o eixo.
- Os serializadores exportam OBJ e glTF; o IfcConvert faz conversões pela linha de comando.
- A árvore espacial detecta interferências entre grupos de elementos.

11.7 Vocabulário técnico

INGLÊS	PORTUGUÊS
geometry kernel	núcleo geométrico
mesh / vertex / face	malha / vértice / face
tessellation / deflection	triangulação / desvio (precisão da aproximação)
world coordinates	coordenadas do projeto
bounding box	caixa envolvente
iterator / multiprocessing	iterador / processamento em paralelo
serializer	serializador (exportador de formato)
clash detection	detecção de interferências
protrusion / pierce / collision / clearance	saliência / atravessamento / colisão / afastamento
spatial tree	árvore espacial

Tabela 11.4: Vocabulário do capítulo.

11.8 Exercícios

Fixação

1. Por que é preciso um núcleo geométrico para saber o volume de uma parede com uma porta?
2. Qual a diferença entre gerar a geometria com `use-world-coords` verdadeiro e falso?

3. Num arquivo em milímetros, `add_wall_representation(..., length=6.0)` cria uma parede de quanto? E um `IfcIShapeProfileDef` com `OverallDepth = 300.0`?
4. Por que o volume calculado de um pilar com perfil I pode diferir levemente do volume teórico?
5. Para que serve a árvore espacial na detecção de interferências?

Aplicação

6. Execute `c11_01_medir_elementos.py` com `Infra-Bridge.ifc` (IFC 4.3) e a consulta `IfcColumn + IfcFooting`. Os volumes calculados coincidem com os declarados? Qual pilar tem a maior cota superior?
7. Altere uma cópia de `c11_02_criar_geometria.py` para posicionar o tubo em (0,0; 2,0; 0,5) e execute a detecção de interferências de novo. O que muda?

Desafio

8. Escreva um script que, para cada pavimento (`IfcBuildingStorey`) de um modelo, liste os elementos contidos cuja cota inferior (calculada pela geometria) difere da cota do pavimento (`Elevation`, convertida para metros) em mais de 5 cm. Teste com `Building-Architecture.ifc`.

11.9 Respostas dos exercícios

1. Porque o arquivo descreve a parede e a abertura separadamente (extrusão da parede, elemento de abertura e relação `IfcRelVoidsElement`); o volume líquido só existe depois que a abertura é subtraída da forma, cálculo feito pelo núcleo geométrico.
2. Com `use-world-coords` verdadeiro, os vértices vêm no sistema do projeto, já posicionados; com falso, vêm no sistema local do elemento, e a posição é informada numa matriz de transformação separada (útil para reaproveitar formas repetidas).
3. 6000 mm (a função recebe metros e converte). O perfil terá 300 mm de altura, porque os atributos do perfil estão na unidade do arquivo.
4. Porque as concordâncias (arcos) entre alma e mesas são aproximadas por segmentos de reta na malha; a precisão depende das configurações de desvio do núcleo.
5. Para organizar as formas por região e só comparar pares de elementos próximos, em vez de testar todos contra todos; isso torna a verificação rápida mesmo em modelos grandes.
6. A saída obtida foi:

```
14 elementos medidos (0 sem geometria) -> ponte.xlsx
 id     Classe                      Nome  Volume_m3  Volume_declarado_m3  DX_m  DY_m  DZ_m  Cota_inferior_m
Cota_superior_m
335  IfcColumn road river bridge pierstem     4.5269               4.5269 2.276 3.393 2.286
-2.80           -0.514
357 IfcFooting   foundation - bridge road     7.3500                  NaN 4.319 5.380 0.700
-3.50           -2.800
392  IfcColumn road river bridge pierstem     4.5269               4.5269 2.276 3.393 2.286
-2.80           -0.514
407 IfcFooting   foundation - bridge road     7.3500                  NaN 4.319 5.380 0.700
-3.50           -2.800
440  IfcColumn road river bridge pierstem     4.5269               4.5269 2.276 3.393 2.286
-2.80           -0.514
452 IfcFooting   foundation - bridge road     7.3500                  NaN 4.319 5.380 0.700
-3.50           -2.800
502  IfcColumn     rail bridge - pierstem    24.9503              24.9503 3.499 4.561 3.780
```

```
-0.49            3.290
519 IfcFooting   foundation - bridge rail    22.4000                 NaN 6.231 7.293 1.000
-1.49           -0.490
536  IfcColumn     rail bridge - pierstem    24.9503             24.9503 3.499 4.561 3.780
-0.49            3.290
548 IfcFooting   foundation - bridge rail    22.4000                 NaN 6.231 7.293 1.000
-1.49           -0.490
707  IfcColumn     rail bridge - pierstem    24.9503             24.9503 3.499 4.561 3.780
-0.49            3.290
722 IfcFooting   foundation - bridge rail    22.4000                 NaN 6.231 7.293 1.000
-1.49           -0.490
739  IfcColumn     rail bridge - pierstem    24.9503             24.9503 3.499 4.561 3.780
-0.49            3.290
751 IfcFooting   foundation - bridge rail    22.4000                 NaN 6.231 7.293 1.000
-1.49           -0.490
```

Os volumes calculados dos 7 pilares coincidem com os declarados (4,5269 m³ nos pilares da ponte rodoviária e 24,9503 m³ nos da ponte ferroviária). As sapatas (`IfcFooting`) não têm quantidade de volume gravada, e a coluna mostra `NaN` (vazio): nesse caso, só a geometria fornece o volume (7,35 m³ e 22,40 m³). A maior cota superior é a dos pilares da ponte ferroviária (3,290 m). Observe ainda que a cota inferior de cada pilar é igual à cota superior da sapata correspondente (-2,80 m e -0,49 m): os elementos se tocam sem sobreposição, o que é o esperado.

7. Basta trocar `posicionar(tubo, 0.0, -1.0, 0.5)` por `posicionar(tubo, 0.0, 2.0, 0.5)`. Com o tubo em y = 2,0 m, o segundo trecho vai de y = 2,0 a y = 5,0 m, sem cruzar a parede (que ocupa y de 0 a 0,2 m). A detecção informa `Grupo A: 2 | grupo B: 1 | interferências: 0`. Os volumes impressos por `c11_02` não mudam, porque a posição não altera a forma.
8. Uma solução comentada, testada com a amostra:

```
# -*- coding: utf-8 -*-
"""
c11_05_cotas_pavimentos.py | Capítulo 11, exercício 8 (solução comentada).

Para cada pavimento (IfcBuildingStorey), lista os elementos contidos cuja cota inferior,
calculada pela geometria, difere da cota do pavimento em mais de uma tolerância.
O arquivo IFC é aberto somente para leitura.

Uso: python c11_05_cotas_pavimentos.py <arquivo.ifc> [--tolerancia 0.05]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import argparse
import ifcopenshell
import ifcopenshell.geom
import ifcopenshell.util.placement as util_pos
import ifcopenshell.util.shape as util_forma
import ifcopenshell.util.unit as util_unid


def main():
    leitor = argparse.ArgumentParser(description="Compara a cota inferior dos elementos com a do pavimento.")
    leitor.add_argument("arquivo")
    leitor.add_argument("--tolerancia", type=float, default=0.05, help="diferença admitida, em metros")
    args = leitor.parse_args()

    modelo = ifcopenshell.open(args.arquivo)
    escala = util_unid.calculate_unit_scale(modelo)       # unidade de comprimento do arquivo -> metro
    config = ifcopenshell.geom.settings()
    config.set("use-world-coords", True)                   # geometria no sistema do projeto, em metros

    for pavimento in sorted(modelo.by_type("IfcBuildingStorey"), key=util_pos.get_storey_elevation):
        cota_pav = util_pos.get_storey_elevation(pavimento) * escala
        contidos = [e for rel in pavimento.ContainsElements for e in rel.RelatedElements]
        divergentes = []
        for elem in contidos:
            if not elem.Representation:
```

```
                continue                                    # sem forma, sem cota calculável
            forma = ifcopenshell.geom.create_shape(config, elem)
            cota_elem = util_forma.get_bottom_elevation(forma.geometry)
            if abs(cota_elem - cota_pav) > args.tolerancia:
                divergentes.append((elem, cota_elem))
        print(f"{pavimento.Name}: cota {cota_pav:.3f} m | {len(contidos)} elementos contidos | "
              f"{len(divergentes)} fora da tolerância de {args.tolerancia:.3f} m")
        for elem, cota_elem in sorted(divergentes, key=lambda par: (par[0].is_a(), par[0].id())):
            print(f"  #{elem.id()} {elem.is_a()} {elem.Name!r}: cota inferior {cota_elem:.3f} m "
                  f"(diferença {cota_elem - cota_pav:+.3f} m)")


if __name__ == "__main__":
    main()
```

Saída com `Building-Architecture.ifc` (IFC 4.3):

```
00 groundfloor: cota 0.000 m | 6 elementos contidos | 4 fora da tolerância de 0.050 m
  #53 IfcSlab 'floor': cota inferior -0.250 m (diferença -0.250 m)
  #158 IfcWall 'house - outer wall - house right front': cota inferior -0.250 m (diferença -0.250 m)
  #182 IfcWall 'house - outer wall - house right back': cota inferior -0.250 m (diferença -0.250 m)
  #201 IfcWall 'house - outer wall - house left': cota inferior -0.250 m (diferença -0.250 m)
```

A cota do pavimento foi obtida com `ifcopenshell.util.placement.get_storey_elevation`, que lê a posição do pavimento e, na falta dela, o atributo `Elevation`; o resultado vem na unidade do arquivo e é convertido para metros com `calculate_unit_scale`. Os quatro elementos listados não estão errados: a laje tem a face superior na cota 0,000 e 25 cm de espessura, e as paredes externas descem até a face inferior da laje. O exercício mostra que a verificação automática aponta candidatos, e a análise continua sendo do projetista. Uma variante útil compara a cota superior da laje, e não a inferior, com a cota do pavimento.

11.10 Referências

- Documentação do IfcOpenShell, geometria (`ifcopenshell.geom`): https://docs.ifcopenshell.org/ifcopenshell-python/geometry_processing.html
- Documentação do IfcOpenShell, IfcConvert: https://docs.ifcopenshell.org/ifcconvert/usage.html
- Documentação do IfcOpenShell (geral): https://docs.ifcopenshell.org/

12 Requisitos e validação: IDS, IfcTester e BCF

Um arquivo IFC pode abrir sem erro em qualquer visualizador e, ainda assim, não servir ao projeto: faltam as propriedades exigidas pelo BEP, a TAG está fora do padrão, a pressão foi gravada na unidade errada. Este capítulo trata da verificação de modelos em quatro níveis, da sintaxe do arquivo aos requisitos do contrato, e das ferramentas abertas para cada um: o validador do IfcOpenShell e o serviço de validação da buildingSMART, para o esquema; o IDS e o IfcTester, para os requisitos de informação; o ifcdiff, para comparar revisões; e o BCF, para comunicar os problemas a quem precisa corrigi-los.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- distinguir a validação do esquema da verificação de requisitos de informação;
- validar arquivos com `ifcopenshell.validate` e interpretar as mensagens;
- ler e escrever arquivos IDS 1.0: especificações, facetas, restrições e cardinalidade;
- verificar modelos com o IfcTester, pela linha de comando e por programa, e gerar relatórios em HTML, BCF e planilha;
- comparar duas revisões de um modelo e detalhar o que mudou;
- explicar o formato BCF e ler apontamentos por programa.

12.1 Introdução: o que significa validar

12.1.1 Quatro níveis de verificação

A pergunta "o modelo está válido?" pode significar quatro coisas diferentes, respondidas por ferramentas diferentes:

NÍVEL	PERGUNTA	EXEMPLO DE FALHA	FERRAMENTAS
1. Sintaxe	O arquivo segue o formato STEP e pode ser lido?	Parêntese não fechado; referência a uma instância `#n` inexistente	Qualquer leitor IFC; `ifcopenshell.validate`; serviço de validação da buildingSMART
2. Esquema	As entidades, os atributos e as regras da versão do IFC são respeitados?	Atributo obrigatório vazio; lista vazia onde o esquema exige ao menos um item; GlobalId repetido	`ifcopenshell.validate`; serviço de validação da buildingSMART
3. Regras normativas e boas práticas	O arquivo segue as convenções de uso publicadas pela buildingSMART, além das regras do esquema?	Uso de entidades em desacordo com os acordos de implementação	Serviço de validação da buildingSMART
4. Requisitos do projeto	O modelo contém a informação que o contrato (EIR, BEP) exige?	Tubo sem pressão de projeto; TAG fora do padrão; parede sem indicação de externa	IDS com IfcTester, ou outro programa compatível com IDS

Tabela 12.1: Níveis de verificação de um modelo IFC.

Passar em um nível não garante os outros. Um arquivo sem nenhuma violação do esquema pode não ter nenhuma das propriedades exigidas pelo BEP; e um arquivo com todas as propriedades pode ter erros de esquema que fazem outro software lê-lo de forma diferente. Na prática, a verificação segue de baixo para cima: não adianta conferir requisitos num arquivo que não pode ser lido corretamente.

12.1.2 Onde a verificação entra no fluxo

O Capítulo 1 apresentou a ISO 19650: o contratante define os requisitos de troca de informação (EIR), e o plano de execução BIM (BEP) descreve como a equipe os cumprirá. O IDS é a forma **legível por máquina** de parte desses requisitos: aquela que diz quais informações cada tipo de objeto deve ter.

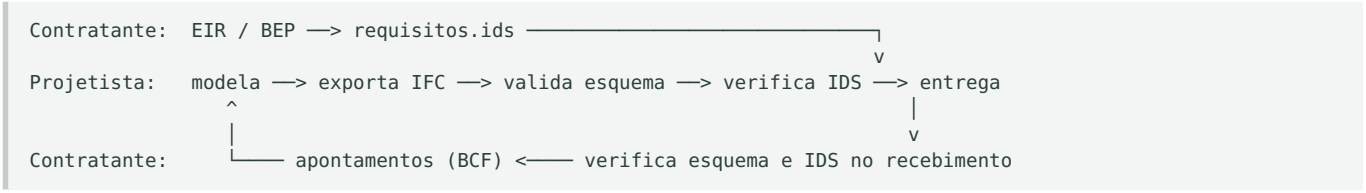


Figura 12.1: A verificação no ciclo de entrega.

O mesmo arquivo IDS é usado em dois momentos: no **autocontrole**, pelo projetista, antes da entrega; e no **recebimento**, pelo contratante. Com critérios objetivos e conhecidos por ambos os lados, a entrega deixa de depender de conferências manuais por amostragem.

12.2 Validação do esquema

12.2.1 O validador do IfcOpenShell

O módulo `ifcopenshell.validate` confere o arquivo contra o esquema da versão declarada no cabeçalho: tipos dos atributos, atributos obrigatórios, tamanho das listas, formato e unicidade do GlobalId (regra `UNIQUE` de `IfcRoot`) e atributos inversos. Com a opção `--rules`, executa também as demais regras escritas em EXPRESS (Capítulo 3): as regras locais das entidades (`WHERE`) e as regras globais. A opção `--rules` usa internamente a biblioteca `pytest`, instalada no Capítulo 0. A opção `--json` produz o resultado em JSON, para outros programas.

Para ver as mensagens de erro, o script a seguir cria uma **cópia** do modelo preenchido no Capítulo 10 com quatro erros típicos. Grave-o como `C:\IFC_Curso\exercicios\cap12\c12_01_modelo_com_erros.py`.

```
# -*- coding: utf-8 -*-
"""
c12_01_modelo_com_erros.py | Capítulo 12: cria uma cópia de um modelo com erros de esquema, para treino.

Lê um modelo válido (não o altera) e grava uma CÓPIA com quatro erros típicos:
  1. GlobalId com formato inválido;
  2. GlobalId repetido em dois elementos;
  3. relacionamento de contenção com a lista de elementos vazia;
  4. propriedade sem nome (atributo obrigatório), simulando um exportador defeituoso.
O quarto erro é inserido no texto do arquivo, porque a API do IfcOpenShell recusa
atributos obrigatórios vazios.

Uso: python c12_01_modelo_com_erros.py <modelo_valido.ifc> <copia_com_erros.ifc>
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import uuid
from pathlib import Path
```

```
import ifcopenshell
import ifcopenshell.guid


def main():
    entrada, saida = Path(sys.argv[1]), Path(sys.argv[2])
    if entrada.resolve() == saida.resolve():
        raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")
    modelo = ifcopenshell.open(entrada)

    tanque = modelo.by_type("IfcTank")[0]
    tanque.GlobalId = "ABC"                                          # erro 1: formato inválido

    valvula = modelo.by_type("IfcValve")[0]
    tubo = modelo.by_type("IfcPipeSegment")[0]
    valvula.GlobalId = tubo.GlobalId                                 # erro 2: GlobalId repetido

    pavimento = modelo.by_type("IfcBuildingStorey")[0]
    gid_fixo = ifcopenshell.guid.compress(uuid.UUID(int=12).hex)    # GlobalId fixo: saída reprodutível
    modelo.create_entity("IfcRelContainedInSpatialStructure", GlobalId=gid_fixo,
                         RelatedElements=[], RelatingStructure=pavimento)  # erro 3: lista vazia

    texto = modelo.to_string()
    texto = texto.replace("IFCPROPERTYSINGLEVALUE('Fluido'", "IFCPROPERTYSINGLEVALUE($", 1)  # erro 4
    saida.write_text(texto, encoding="utf-8")
    print(f"Cópia com erros gravada: {saida} (original intacto: {entrada})")


if __name__ == "__main__":
    main()
```

Antes de executar, copie para a pasta do capítulo os dois arquivos da rede usados neste capítulo: a versão sem metadados (gerada no Capítulo 6 e copiada para a pasta do Capítulo 10) e a versão preenchida no Capítulo 10. Os novos nomes indicam a revisão:

```
cd C:\IFC_Curso\exercicios\cap12
copy ..\cap10\rede_exemplo.ifc rede_rev0.ifc
copy ..\cap10\rede_exemplo_metadados.ifc rede_rev1.ifc
python c12_01_modelo_com_erros.py rede_rev1.ifc rede_com_erros.ifc
python -m ifcopenshell.validate --rules rede_com_erros.ifc
```

Saídas obtidas:

```
Cópia com erros gravada: rede_com_erros.ifc (original intacto: rede_rev1.ifc)
```

```
Validating rede_com_erros.ifc
On instance:
    #16=IfcTank('ABC',$,'TQ-001','Tanque de produto',$,$,$,$,$)
                ^^^^^
IfcGloballyUniqueId base64 validation:
    The attribute GlobalId should be valid base64 encoded 128-bit number.
Violated by:
    Guid length should be 22 characters.

On instance:
    #18=IfcValve('3iHL5xQvH6jQESlnlbSxJ6',$,'VG-001','Válvula de bloqueio',$,$,$,$,$)
                 ^^^^^^^^^^^^^^^^^^^^^^^^
Rule IfcRoot.UR1:
    The attribute GlobalId should be unique
Violated by:
    #17=IfcPipeSegment('3iHL5xQvH6jQESlnlbSxJ6',$,'TB-001','Tubo de sucção',$,$,$,$,$)
                       ^^^^^^^^^^^^^^^^^^^^^^^^
For instance:
    #49=IfcPropertySingleValue($,$,IfcLabel('Produto A'),$)
                                ^
With attribute:
    <attribute Name: <type IfcIdentifier: <string>>>
```

```
Not optional

For instance:
    #114=IfcRelContainedInSpatialStructure('0000000000000000000C',$,$,$,(),#8)
                                                                    ^^
With attribute:
    <attribute RelatedElements: <set [1:?] of <entity IfcProduct>>>
Value:
    ()
Not valid

On instance:
    #47=IfcPropertySet('2TLKojNy5ChO7kpPqy_5Wi',$,'ARV_Processo',$,(#49,#50))
Rule IfcPropertySet.UniquePropertyNames:
    IfcUniquePropertyName(hasproperties)
Violated by:
    unhashable type: 'indeterminate_type'
```

A mesma validação no arquivo original, `python -m ifcopenshell.validate --rules rede_rev1.ifc`, resulta em:

```
Validating rede_rev1.ifc
No validation issues found.
```

MENSAGEM	SIGNIFICADO	CAUSA PROVÁVEL
`IfcGloballyUniqueId base64 validation` … `Guid length should be 22 characters`	GlobalId fora do formato: deve ter 22 caracteres da base 64 do IFC (Capítulo 3)	Edição manual; exportador que gera identificadores próprios
`Rule IfcRoot.UR1: The attribute GlobalId should be unique`	Dois objetos com o mesmo GlobalId; `UR1` é a regra de unicidade (`UNIQUE`) de `IfcRoot`	Cópia de elementos sem novo identificador; junção de arquivos
`With attribute: <attribute Name ...> Not optional`	Atributo obrigatório vazio (`$`)	Exportador defeituoso; edição do texto do arquivo
`<attribute RelatedElements: <set [1:?] ...>>` … `Value: ()` … `Not valid`	Conjunto vazio onde o esquema exige ao menos um item (`[1:?]`)	Relação criada sem elementos, por exportador ou script
`Rule IfcPropertySet.UniquePropertyNames` … `Violated by: unhashable type`	A regra que exige nomes de propriedade únicos no conjunto não pôde ser avaliada	Consequência do erro anterior (propriedade sem nome)

Tabela 12.2: Interpretação das mensagens do validador.

Três observações práticas:

- **Erros em cascata.** A última mensagem não é um quinto erro: é efeito da propriedade sem nome. Corrija primeiro os erros básicos e valide de novo.
- **Código de saída.** O validador termina com código 1 quando encontra problemas e com código 0 quando não encontra, o que permite usá-lo em rotinas automáticas (exercício 10).
- **Proteção da API.** A API do IfcOpenShell recusa deixar vazio um atributo obrigatório (`TypeError: attribute 'Name' is not optional`); por isso, o script inseriu o quarto erro diretamente no texto.

Scripts que usam a API ficam protegidos contra essa categoria de erro, mas não contra todas: o GlobalId repetido e a lista vazia foram aceitos sem aviso.

12.2.2 O serviço de validação da buildingSMART

A buildingSMART mantém um serviço gratuito na internet, em `validate.buildingsmart.org`, que exige cadastro de usuário. Segundo a documentação do serviço, ele verifica:

- a sintaxe STEP;
- o esquema: atributos, atributos inversos, regras locais (`WHERE`) e regras globais;
- as regras normativas: acordos de implementação (*Implementer Agreements*) e proposições informais (*Informal Propositions*);
- as boas práticas do setor (*Industry Practices*), relatadas apenas como avisos.

O serviço aceita arquivos IFC2X3, IFC4 e IFC4X3_ADD2, com limite de tamanho informado na página do serviço (da ordem de 250 MB na data de consulta). A própria documentação declara que ele não verifica requisitos específicos de cada projeto, papel atribuído ao IDS.

ASPECTO	`IFCOPENSHELL.VALIDATE`	SERVIÇO DA BUILDINGSMART
Onde roda	No seu computador	Na internet, com cadastro
Verificações	Sintaxe, esquema e, com `--rules`, regras EXPRESS	Sintaxe, esquema, regras normativas e boas práticas
Automação	Linha de comando e Python	Interface web
Confidencialidade	O arquivo não sai do computador	O arquivo é enviado ao servidor do serviço
Uso típico	Conferência rotineira de cada arquivo gerado	Referência oficial; teste das configurações de exportação no início do projeto (Capítulo 13)

Tabela 12.3: Validador local e serviço da buildingSMART.

> **ATENÇÃO**
> Antes de enviar modelos de clientes a qualquer serviço na internet, confira as cláusulas de confidencialidade do contrato e os termos de uso do serviço.

12.3 IDS: requisitos de informação legíveis por máquina

12.3.1 O conceito

O **IDS** (*Information Delivery Specification*, especificação de entrega de informação) é o padrão da buildingSMART para descrever, num arquivo XML de extensão `.ids`, os requisitos de informação de um modelo IFC. A versão 1.0 tornou-se padrão oficial da buildingSMART em junho de 2024; as versões anteriores eram rascunhos e não têm valor de padrão.

Cada requisito do IDS é uma **especificação** com a forma:

> Todo **[conjunto de elementos]** deve ter **[informação]**.

Por exemplo: "todo `IfcPipeSegment`, `IfcValve` e `IfcPump` deve ter, no conjunto `ARV_Processo`, a propriedade `PressaoProjeto`, do tipo `IfcPressureMeasure`, entre 0 e 2 MPa". Escrito uma vez, o arquivo é lido por qualquer programa compatível, que aponta os elementos que não cumprem cada especificação.

O IDS verifica **informação**: classe e tipo predefinido, atributos, propriedades e quantidades, materiais, classificações e relações de composição, agrupamento e contenção. Não verifica geometria (forma, dimensões desenhadas, interferências), qualidade de representação nem a coerência entre elementos (por exemplo, se o diâmetro de um tubo é compatível com o bocal da bomba). Para esses casos, usam-se scripts como os dos Capítulos 9 a 11 ou programas de verificação especializados.

12.3.2 Estrutura do arquivo

```
ids                                raiz (espaço de nomes http://standards.buildingsmart.org/IDS)
├── info                           dados do documento
│   ├── title                      obrigatório
│   └── copyright, version, description, author (e-mail), date (AAAA-MM-DD), purpose, milestone
└── specifications
    └── specification (uma ou mais)  atributos obrigatórios: name, ifcVersion
        │                            atributos opcionais: identifier, description, instructions
        ├── applicability            A QUAIS elementos se aplica (atributos minOccurs e maxOccurs)
        └── requirements (opcional)  O QUE esses elementos devem ter
```

Figura 12.2: Estrutura de um arquivo IDS 1.0.

Detalhes que o esquema XSD do IDS impõe:

- `ifcVersion` é uma lista, separada por espaços, com um ou mais dos valores `IFC2X3`, `IFC4` e `IFC4X3_ADD2`;
- `author` deve ser um endereço de e-mail;
- `date` segue o formato AAAA-MM-DD;
- `instructions` é um texto para o autor do modelo, que os programas compatíveis podem exibir ao lado do requisito.

12.3.3 As seis facetas

A aplicabilidade e os requisitos são descritos por **facetas**, cada uma dedicada a um tipo de informação:

FACETA	DESCREVE	PARÂMETROS	EXEMPLO
`entity`	Classe IFC e tipo predefinido	`name`, `predefinedType`	`IFCPIPESEGMENT`, `RIGIDSEGMENT`
`partOf`	Relação com outro elemento	`entity` (com `name` e `predefinedType`); atributo `relation`	Contido em `IFCBUILDINGSTOREY`
`classification`	Referência de classificação	`value`, `system`	`ABNT NBR 15965`, `3E.EX.02`
`attribute`	Atributo da entidade	`name`, `value`	`Name` no padrão `TB-001`
`property`	Propriedade ou quantidade de um conjunto	`propertySet`, `baseName`, `value`; atributo `dataType`	`Pset_WallCommon.IsExternal`, `IFCBOOLEAN`

FACETA	DESCREVE	PARÂMETROS	EXEMPLO
`material`	Material associado	`value`	`Aço-carbono ASTM A106 Gr. B`

Tabela 12.4: Facetas do IDS 1.0.

Regras de escrita:

- as facetas aparecem sempre nesta ordem: `entity`, `partOf`, `classification`, `attribute`, `property`, `material`; a aplicabilidade admite no máximo uma faceta `entity`;
- nomes de entidades e de tipos predefinidos são escritos em **maiúsculas** (`IFCWALL`, `RIGIDSEGMENT`);
- `dataType` é o nome de um tipo definido do IFC, também em maiúsculas (`IFCLABEL`, `IFCBOOLEAN`, `IFCLENGTHMEASURE`, `IFCPRESSUREMEASURE`);
- nomes de conjuntos e de propriedades são escritos exatamente como no arquivo, com as mesmas maiúsculas e minúsculas;
- segundo a documentação do IDS, uma faceta `property` com valor (`value`) exige também o `dataType`;
- a faceta `property` também se aplica às quantidades (`Qto_`), como no exemplo da seção 12.3.6.

O atributo `relation` da faceta `partOf` aceita cinco valores:

VALOR DE `RELATION`	O ELEMENTO...	CAPÍTULO
`IFCRELAGGREGATES`	é parte de um conjunto (agregação)	4
`IFCRELASSIGNSTOGROUP`	pertence a um grupo ou sistema	4 e 6
`IFCRELCONTAINEDINSPATIALSTRUCTURE`	está contido numa estrutura espacial	4
`IFCRELNESTS`	está aninhado em outro (por exemplo, portas de conexão num equipamento)	4 e 6
`IFCRELVOIDSELEMENT` `IFCRELFILLSELEMENT`	preenche uma abertura (porta ou janela numa parede); é um único valor, com espaço	7

Tabela 12.5: Relações da faceta `partOf`.

Sem o atributo `relation`, todas as relações são consideradas.

12.3.4 Valores simples e restrições

Um valor pode ser **simples** (`<simpleValue>`), que exige igualdade exata, ou uma **restrição** (`<xs:restriction>`), que descreve os valores aceitos:

RESTRIÇÃO	ELEMENTOS XML	USO	EXEMPLO
Enumeração	`xs:enumeration`	Lista de valores aceitos	`REI30`, `REI60`, `REI90`, `REI120`
Padrão	`xs:pattern`	Expressão regular	`[A-Z]{1,3}-[0-9]{3}`
Limites	`xs:minInclusive`, `xs:maxInclusive`, `xs:minExclusive`, `xs:maxExclusive`	Faixa numérica	de 0,10 a 0,40 (m)

RESTRIÇÃO	ELEMENTOS XML	USO	EXEMPLO
Comprimento	`xs:length`, `xs:minLength`, `xs:maxLength`	Número de caracteres	exatamente 22 caracteres

Tabela 12.6: Tipos de restrição.

O atributo `base` da restrição indica o tipo do valor: `xs:string` para textos e `xs:double` (ou `xs:decimal`) para números.

O **padrão** usa a sintaxe de expressões regulares do XML Schema, mais simples que a do Python (Capítulo 9), e precisa descrever o valor **inteiro**: `[A-Z]{1,3}-[0-9]{3}` aceita `TB-001` e recusa `TB-001A` e `Linha TB-001`. Caracteres especiais, como o ponto, são escapados com barra invertida: `3E\.[A-Z]{2}\.[0-9]{2}`.

Os **valores numéricos** do IDS são expressos nas **unidades padrão do SI**, conforme a documentação do padrão referenciada no próprio esquema XSD. O programa de verificação converte o valor gravado no arquivo para o SI antes de comparar:

GRANDEZA	VALOR NO MODELO	VALOR NO IDS
Diâmetro nominal (arquivo em milímetros)	100	0.1 (m)
Espessura de parede (arquivo em milímetros)	24	0.024 (m)
Pressão de projeto de 600 kPa (gravada em pascal)	600000	600000 (Pa)
Temperatura de 40 °C (gravada em kelvin)	313,15	313.15 (K)

Tabela 12.7: Valores no modelo e no IDS.

12.3.5 Cardinalidade

A cardinalidade existe em dois níveis. No nível da **especificação**, os atributos `minOccurs` e `maxOccurs` da aplicabilidade dizem se o modelo deve, pode ou não pode conter elementos aplicáveis:

MINOCCURS	MAXOCCURS	ESPECIFICAÇÃO	SIGNIFICADO
1	`unbounded`	Obrigatória	Deve haver ao menos um elemento aplicável, e todos devem atender aos requisitos
0	`unbounded`	Opcional	Se houver elementos aplicáveis, todos devem atender aos requisitos; se não houver, a especificação é considerada atendida
0	0	Proibida	Não pode haver nenhum elemento aplicável

Tabela 12.8: Cardinalidade da especificação.

Declare sempre os dois atributos: é a prática dos exemplos oficiais e evita dúvidas de interpretação.

No nível do **requisito**, o atributo `cardinality` de cada faceta indica:

CARDINALITY	SIGNIFICADO
`required` (padrão)	A informação deve existir e, se houver valor ou tipo indicados, cumpri-los

CARDINALITY	SIGNIFICADO
optional	A informação pode faltar; se existir, deve cumprir o valor e o tipo indicados
prohibited	A informação não pode existir (com o valor indicado, se houver)

Tabela 12.9: Cardinalidade dos requisitos.

A faceta `partOf` admite apenas `required` e `prohibited`, e a faceta `entity`, usada como requisito, não tem cardinalidade: é sempre obrigatória. Nos testes com o IfcTester, uma propriedade opcional ausente foi aceita, e a mesma propriedade presente com tipo diferente do exigido foi recusada: "opcional" não significa "sem regras".

12.3.6 Exemplo escrito à mão: edificação

O arquivo abaixo reúne seis especificações para um modelo de arquitetura. Grave-o como `requisitos_edificacao.ids` na pasta do capítulo. Os comentários (`<!-- ... -->`) explicam cada especificação e são ignorados pelos programas.

```
<?xml version="1.0" encoding="UTF-8"?>
<ids:ids xmlns:ids="http://standards.buildingsmart.org/IDS"
         xmlns:xs="http://www.w3.org/2001/XMLSchema"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://standards.buildingsmart.org/IDS http://standards.buildingsmart.org/IDS/1.0/
ids.xsd">
  <ids:info>
    <ids:title>Requisitos de informação: edificação (exemplo do Capítulo 12)</ids:title>
    <ids:version>00</ids:version>
    <ids:description>Exemplo didático para verificação de um modelo de arquitetura.</ids:description>
    <ids:author>bim@exemplo.com.br</ids:author>
    <ids:date>2026-09-18</ids:date>
    <ids:purpose>Verificação do modelo antes da entrega</ids:purpose>
    <ids:milestone>Projeto executivo</ids:milestone>
  </ids:info>
  <ids:specifications>

    <!-- E1: toda parede deve informar se é externa (propriedade padrão, valor lógico) -->
    <ids:specification name="Paredes: indicação de parede externa" ifcVersion="IFC4 IFC4X3_ADD2" identifier="E1">
      <ids:applicability minOccurs="1" maxOccurs="unbounded">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCWALL</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:applicability>
      <ids:requirements>
        <ids:property dataType="IFCBOOLEAN" cardinality="required">
          <ids:propertySet><ids:simpleValue>Pset_WallCommon</ids:simpleValue></ids:propertySet>
          <ids:baseName><ids:simpleValue>IsExternal</ids:simpleValue></ids:baseName>
        </ids:property>
      </ids:requirements>
    </ids:specification>

    <!-- E2: espessura (quantidade Width) entre 0,10 m e 0,40 m; valores em unidades SI -->
    <ids:specification name="Paredes: espessura entre 0,10 m e 0,40 m" ifcVersion="IFC4 IFC4X3_ADD2"
identifier="E2">
      <ids:applicability minOccurs="0" maxOccurs="unbounded">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCWALL</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:applicability>
      <ids:requirements>
        <ids:property dataType="IFCLENGTHMEASURE">
          <ids:propertySet><ids:simpleValue>Qto_WallBaseQuantities</ids:simpleValue></ids:propertySet>
          <ids:baseName><ids:simpleValue>Width</ids:simpleValue></ids:baseName>
          <ids:value>
            <xs:restriction base="xs:double">
```

```
            <xs:minInclusive value="0.10"/>
            <xs:maxInclusive value="0.40"/>
          </xs:restriction>
        </ids:value>
      </ids:property>
    </ids:requirements>
  </ids:specification>

  <!-- E3: resistência ao fogo das lajes no formato REI seguido de minutos -->
  <ids:specification name="Lajes: resistência ao fogo" ifcVersion="IFC4 IFC4X3_ADD2" identifier="E3"
                     instructions="Informar FireRating no formato REI30, REI60, REI90 ou REI120.">
    <ids:applicability minOccurs="0" maxOccurs="unbounded">
      <ids:entity>
        <ids:name><ids:simpleValue>IFCSLAB</ids:simpleValue></ids:name>
      </ids:entity>
    </ids:applicability>
    <ids:requirements>
      <ids:property dataType="IFCLABEL">
        <ids:propertySet><ids:simpleValue>Pset_SlabCommon</ids:simpleValue></ids:propertySet>
        <ids:baseName><ids:simpleValue>FireRating</ids:simpleValue></ids:baseName>
        <ids:value>
          <xs:restriction base="xs:string">
            <xs:enumeration value="REI30"/>
            <xs:enumeration value="REI60"/>
            <xs:enumeration value="REI90"/>
            <xs:enumeration value="REI120"/>
          </xs:restriction>
        </ids:value>
      </ids:property>
    </ids:requirements>
  </ids:specification>

  <!-- E4: espaços com nome curto (Name) e nome por extenso (LongName) -->
  <ids:specification name="Espaços: identificação" ifcVersion="IFC4 IFC4X3_ADD2" identifier="E4">
    <ids:applicability minOccurs="0" maxOccurs="unbounded">
      <ids:entity>
        <ids:name><ids:simpleValue>IFCSPACE</ids:simpleValue></ids:name>
      </ids:entity>
    </ids:applicability>
    <ids:requirements>
      <ids:attribute>
        <ids:name><ids:simpleValue>Name</ids:simpleValue></ids:name>
      </ids:attribute>
      <ids:attribute>
        <ids:name><ids:simpleValue>LongName</ids:simpleValue></ids:name>
      </ids:attribute>
    </ids:requirements>
  </ids:specification>

  <!-- E5: elementos contidos num pavimento -->
  <ids:specification name="Paredes e lajes: contidas em pavimento" ifcVersion="IFC4 IFC4X3_ADD2"
identifier="E5">
    <ids:applicability minOccurs="0" maxOccurs="unbounded">
      <ids:entity>
        <ids:name>
          <xs:restriction base="xs:string">
            <xs:enumeration value="IFCWALL"/>
            <xs:enumeration value="IFCSLAB"/>
          </xs:restriction>
        </ids:name>
      </ids:entity>
    </ids:applicability>
    <ids:requirements>
      <ids:partOf relation="IFCRELCONTAINEDINSPATIALSTRUCTURE">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCBUILDINGSTOREY</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:partOf>
    </ids:requirements>
  </ids:specification>

  <!-- E6: proibição de elementos genéricos (applicability com minOccurs=0 e maxOccurs=0) -->
  <ids:specification name="Elementos genéricos proibidos" ifcVersion="IFC4 IFC4X3_ADD2" identifier="E6">
```

```
      <ids:applicability minOccurs="0" maxOccurs="0">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCBUILDINGELEMENTPROXY</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:applicability>
    </ids:specification>

  </ids:specifications>
</ids:ids>
```

ESPECIFICAÇÃO	APLICA-SE A	EXIGE	RECURSO DO IDS EXERCITADO
E1	Paredes (obrigatória)	`Pset_WallCommon.IsExternal`, lógico	Propriedade padrão com `dataType`
E2	Paredes	`Qto_WallBaseQuantities.Width` entre 0,10 e 0,40 m	Quantidade; limites; unidades SI
E3	Lajes	`Pset_SlabCommon.FireRating` em lista	Enumeração; `instructions`
E4	Espaços	`Name` e `LongName` preenchidos	Faceta `attribute`
E5	Paredes e lajes	Contidas num pavimento	Enumeração no nome da entidade; `partOf`
E6	*Proxies*	Não podem existir	Especificação proibida

Tabela 12.10: Especificações do exemplo de edificação.

12.4 Verificação com o IfcTester

12.4.1 Pela linha de comando

O IfcTester, instalado no Capítulo 0, verifica um modelo contra um IDS com um único comando:

```
python -m ifctester requisitos_edificacao.ids "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0
(IFC 4.3 ADD2)\Simple-Scene\Building-Architecture.ifc" --no-color
```

Saída obtida (as duas primeiras linhas mostram o tempo de leitura e de verificação, em segundos, que varia a cada execução e foi substituído por `[tempo]`):

```
Finished loading: [tempo]
Finished validating: [tempo]
Requisitos de informação: edificação (exemplo do Capítulo 12)
[FAIL] (0/4) *Paredes: indicação de parede externa
    Applies to:
        All IFCWALL data
    Requirements:
        IsExternal data shall be provided in the dataset Pset_WallCommon
            The required property set does not exist - #158=IfcWall('1AQAupaRP1txwK1AGiN61V',#1,'house - outer
wall - house right front','A solid outer wall, forming the right front side of the
house.','solidwall',#167,#179,'454425.1027891.979946.932083.920023',$)
            The required property set does not exist - #182=IfcWall('3wdauVJT5Fx9drrREiDqA$',#1,'house - outer
wall - house right back','A solid outer wall, forming the right back side of the
house.','solidwall',#189,#198,'454425.1027891.979946.932083.920031',$)
            The required property set does not exist - #201=IfcWall('0OfZwWc8j9QP5uX8xPTxDH',#1,'house - outer
wall - house left','A solid outer wall, forming the left side of the
house.','solidwall',#208,#217,'454425.1027891.979946.932083.920032',$)
            The required property set does not exist - #228=IfcWall('0Q5LjisRjDwhFSOc_WQjlF',#1,'plumbing
wall','A wall designed to house and protect plumbing systems, keeping the pipes snug and
secure.','plumbingwall',#237,#249,'454425.1027891.979946.932083.2037918.920027',$)
```

```
[FAIL] (3/4) Paredes: espessura entre 0,10 m e 0,40 m
    Applies to:
        All IFCWALL data
    Requirements:
        Width data shall be {'minInclusive': '0.10', 'maxInclusive': '0.40'} and in the dataset
Qto_WallBaseQuantities
            The property value 0.024 does not match the requirements -
#228=IfcWall('0Q5LjisRjDwhFSOc_WQjlF',#1,'plumbing wall','A wall designed to house and protect plumbing systems,
keeping the pipes snug and secure.','plumbingwall',#237,#249,'454425.1027891.979946.932083.2037918.920027',$)
[FAIL] (1/3) Lajes: resistência ao fogo
    Applies to:
        All IFCSLAB data
    Requirements:
        FireRating data shall be {'enumeration': ['REI30', 'REI60', 'REI90', 'REI120']} and in the dataset
Pset_SlabCommon
            The required property set does not exist - #261=IfcSlab('0ZTBBPo6f6bxqV2K7Oelrq',#1,'house - roof -
slab left','A roof slab that's got it all covered','roof',#270,#282,'454425.1027891.979946.932084.902510',$)
            The required property set does not exist - #285=IfcSlab('12UVOn4wvAJPMUExKdZLb8',#1,'house - roof -
slab right','A roof slab that's got it all covered','roof',#291,#300,'454425.1027891.979946.932084.902511',$)
[FAIL] (0/2) Espaços: identificação
    Applies to:
        All IFCSPACE data
    Requirements:
        The Name shall be provided
        The LongName shall be provided
            The attribute value None is empty - #85=IfcSpace('0xY$LvXaDEswJDk_VU74C_',#1,'living room','A cozy
space, perfect for relaxation and family gatherings.','living area',#87,#113,$,.ELEMENT.,$,0.)
            The attribute value None is empty - #135=IfcSpace('18QhMtUIXBvQktPHXXxs7H',#1,'entry hall','A
welcoming entry hall, the first impression of the home.','hallway',#136,#155,$,.ELEMENT.,$,0.)
[FAIL] (5/7) Paredes e lajes: contidas em pavimento
    Applies to:
        All {'enumeration': ['IFCWALL', 'IFCSLAB']} data
    Requirements:
        An element must have an IFCRELCONTAINEDINSPATIALSTRUCTURE relationship with an IFCBUILDINGSTOREY
            The entity has a relationship with incorrect entities: IFCSITE -
#261=IfcSlab('0ZTBBPo6f6bxqV2K7Oelrq',#1,'house - roof - slab left','A roof slab that's got it all
covered','roof',#270,#282,'454425.1027891.979946.932084.902510',$)
            The entity has a relationship with incorrect entities: IFCSITE -
#285=IfcSlab('12UVOn4wvAJPMUExKdZLb8',#1,'house - roof - slab right','A roof slab that's got it all
covered','roof',#291,#300,'454425.1027891.979946.932084.902511',$)
[FAIL] (2/2) Elementos genéricos proibidos
    Applies to:
        All IFCBUILDINGELEMENTPROXY data
    Requirements:
```

Como ler o relatório:

- `[PASS]` ou `[FAIL]` indica se a especificação foi atendida;
- `(a/b)`: `b` é o número de elementos aplicáveis, e `a`, o dos que atenderam a todos os requisitos;
- o asterisco antes do nome marca uma especificação obrigatória (`minOccurs` igual a 1);
- nas especificações proibidas, `b` é o número de elementos encontrados, que é justamente a causa da falha, e a contagem `a` não tem significado; `[PASS] (0/0)` numa especificação opcional indica que nenhum elemento se aplica (armadilha 4 da seção 12.4.4);
- sob cada requisito, aparecem as falhas (até dez), com o motivo e a linha STEP do elemento.

Análise dos resultados:

- **E1**: nenhuma das quatro paredes tem `Pset_WallCommon`. O exportador não gravou o conjunto padrão; a correção é feita no mapeamento de exportação (Capítulo 13) ou em lote (Capítulo 10);

- **E2**: três paredes atendem. A `plumbing wall` tem espessura de 24 mm: o IfcTester converteu o valor para 0,024 m antes de comparar. A questão é de projeto: trata-se de um erro de modelagem ou de um painel fino legítimo, que o requisito deveria prever?
- **E3**: a laje térrea tem `REI30` e atende; as duas lajes do telhado não têm `Pset_SlabCommon`;
- **E4**: `Name` está preenchido; `LongName`, não;
- **E5**: as lajes do telhado são partes do `IfcRoof` (agregação), e o telhado está contido no `IfcSite`, não no pavimento. O IfcTester considerou a contenção do conjunto e informou a relação com `IFCSITE`. Se o telhado deve ou não pertencer ao pavimento é uma convenção de modelagem que o BEP precisa definir (Capítulo 4);
- **E6**: a amostra contém dois *proxies*, `origin` e `geo-reference`, usados de propósito como marcos de coordenação (Capítulo 7). A proibição é ampla demais para esse projeto. O IDS 1.0 não tem um recurso direto de "exceto", e a sintaxe de padrões do XML Schema não oferece um modo de negar um padrão inteiro (por exemplo, "qualquer nome, menos este"). Uma alternativa é substituir a proibição por uma especificação que exija dos *proxies* um `ObjectType` pertencente a uma lista aprovada no BEP.

A opção `-r` escolhe o formato do relatório, e `-o` o arquivo de saída. Por exemplo: `python -m ifctester requisitos.ids modelo.ifc -r Html -o relatorio.html`. O programa confere as extensões e recusa arquivos que não terminem em `.ids` e `.ifc` (inclusive `.ifczip` e `.ifcxml`).

OPÇÃO `-R`	RESULTADO	OBSERVAÇÃO
`Console` (padrão)	Texto na tela	`--no-color` remove os códigos de cor
`Txt`	Texto em arquivo	Na versão 0.8.5, perde as marcações `[PASS]` / `[FAIL]` e as contagens; prefira `Console` ou `Html`
`Json`	Arquivo JSON	Para leitura por outros programas
`Html`	Página HTML	Para leitura e arquivamento no projeto
`Ods`, `OdsSummary`	Planilha OpenDocument, detalhada ou resumida	Abre no Excel e no LibreOffice; `--excel-safe` ajusta a compatibilidade com o Excel
`Bcf`	Arquivo BCF 2.1	Um tópico por falha (seção 12.6)

Tabela 12.11: Formatos de relatório do IfcTester.

12.4.2 Criar um IDS por programa

Escrever XML à mão funciona para poucos requisitos, mas um BEP industrial tem dezenas de propriedades por classe. Gerar o IDS por programa evita erros de digitação e permite, mais adiante, gerá-lo a partir da própria planilha de requisitos (Capítulo 14). O script abaixo converte os requisitos do exemplo de tubulação e equipamentos (Capítulo 10) em oito especificações. Grave-o como `c12_02_criar_ids.py`.

```
# -*- coding: utf-8 -*-
"""
c12_02_criar_ids.py | Capítulo 12: criação de um arquivo IDS por programa (IfcTester).

Converte requisitos de informação de um BEP de tubulação e equipamentos em um arquivo
IDS 1.0. Depois de gravar, reabre o arquivo com verificação contra o esquema XSD do IDS.
```

```
Uso: python c12_02_criar_ids.py [--saida requisitos_bep.ids]
Testado com: Python 3.11 e 3.13; ifctester 0.8.5; ifcopenshell 0.8.5.
"""
import argparse
from ifctester import ids

VERSOES = ["IFC4X3_ADD2"]                                  # versões do IFC às quais o requisito se aplica
COMPONENTES = ["IFCPIPESEGMENT", "IFCPIPEFITTING", "IFCVALVE", "IFCPUMP", "IFCTANK"]


def enumeracao(*valores):
    """Restrição 'enumeration': o valor deve ser um dos listados."""
    return ids.Restriction(options={"enumeration": list(valores)})


def especificacao(identificador, nome, entidades, instrucoes=None, minimo=0):
    """Cria uma especificação aplicável a uma ou mais classes (facet entity na aplicabilidade)."""
    espec = ids.Specification(name=nome, identifier=identificador, ifcVersion=VERSOES,
                              minOccurs=minimo, maxOccurs="unbounded", instructions=instrucoes)
    nome_entidade = entidades[0] if len(entidades) == 1 else enumeracao(*entidades)
    espec.applicability.append(ids.Entity(name=nome_entidade))
    return espec


def main():
    leitor = argparse.ArgumentParser(description="Cria o IDS do exemplo de tubulação e equipamentos.")
    leitor.add_argument("--saida", default="requisitos_bep.ids")
    args = leitor.parse_args()

    requisitos = ids.Ids(title="Requisitos de informação: tubulação e equipamentos (exemplo)",
                         version="00", author="bim@exemplo.com.br", date="2026-09-18",
                         description="Requisitos do BEP convertidos em IDS 1.0 (exemplo do Capítulo 12).",
                         purpose="Verificação dos metadados antes da entrega",
                         milestone="Projeto executivo")

    # R1: TAG no atributo Name, no formato de 1 a 3 letras, hífen e 3 algarismos (ex.: TB-001)
    r1 = especificacao("R1", "Componentes: TAG no formato do BEP", COMPONENTES, minimo=1,
                       instrucoes="Preencher Name com a TAG do BEP, por exemplo TB-001.")
    r1.requirements.append(ids.Attribute(name="Name",
                                         value=ids.Restriction(options={"pattern": "[A-Z]{1,3}-[0-9]{3}"})))

    # R2: dados de processo em conjunto customizado; valores em unidades SI (Pa e K)
    r2 = especificacao("R2", "Tubos, válvulas e bombas: dados de processo",
                       ["IFCPIPESEGMENT", "IFCVALVE", "IFCPUMP"])
    r2.requirements.append(ids.Property(propertySet="ARV_Processo", baseName="Fluido", dataType="IFCLABEL"))
    r2.requirements.append(ids.Property(
        propertySet="ARV_Processo", baseName="PressaoProjeto", dataType="IFCPRESSUREMEASURE",
        value=ids.Restriction(options={"minExclusive": 0, "maxInclusive": 2.0e6}, base="double"),
        instructions="Pressão de projeto manométrica, em pascal (0 < p <= 2 MPa)."))
    r2.requirements.append(ids.Property(
        propertySet="ARV_Processo", baseName="TemperaturaProjeto",
        dataType="IFCTHERMODYNAMICTEMPERATUREMEASURE",
        value=ids.Restriction(options={"minInclusive": 273.15, "maxInclusive": 473.15}, base="double")))

    # R3: controle de revisão e situação de aprovação
    r3 = especificacao("R3", "Componentes: revisão e situação", COMPONENTES)
    r3.requirements.append(ids.Property(propertySet="ARV_Controle", baseName="Revisao", dataType="IFCLABEL",
                                        value=ids.Restriction(options={"pattern": "[A-Z]|[0-9]{2}"})))
    r3.requirements.append(ids.Property(propertySet="ARV_Controle", baseName="SituacaoAprovacao",
                                        dataType="IFCLABEL",
                                        value=enumeracao("Em análise", "Aprovado", "Reprovado")))

    # R4: material e classificação obrigatórios
    r4 = especificacao("R4", "Componentes: material e classificação", COMPONENTES)
    r4.requirements.append(ids.Material())                          # qualquer material, mas obrigatório
    codigo_nbr = ids.Restriction(options={"pattern": r"3E\.[A-Z]{2}\.[0-9]{2}"})
    r4.requirements.append(ids.Classification(system="ABNT NBR 15965", value=codigo_nbr))

    # R5: diâmetro nominal dos tubos (propriedade do tipo, herdada pela ocorrência); valor em metros
    r5 = especificacao("R5", "Tubos: diâmetro nominal DN100", ["IFCPIPESEGMENT"])
    r5.requirements.append(ids.Property(propertySet="Pset_PipeSegmentTypeCommon", baseName="NominalDiameter",
                                        dataType="IFCPOSITIVELENGTHMEASURE", value="0.1"))
```

```
    # R6: localização espacial: contido em um pavimento
    r6 = especificacao("R6", "Componentes: contidos em pavimento", COMPONENTES)
    r6.requirements.append(ids.PartOf(name="IFCBUILDINGSTOREY", relation="IFCRELCONTAINEDINSPATIALSTRUCTURE"))

    # R7: fabricante informado nos equipamentos e válvulas
    r7 = especificacao("R7", "Equipamentos e válvulas: fabricante", ["IFCVALVE", "IFCPUMP", "IFCTANK"])
    r7.requirements.append(ids.Property(propertySet="Pset_ManufacturerTypeInformation",
                                        baseName="Manufacturer", dataType="IFCLABEL"))

    # R8: proibição de elementos genéricos (aplicabilidade com minOccurs = 0 e maxOccurs = 0)
    r8 = ids.Specification(name="Elementos genéricos proibidos", identifier="R8", ifcVersion=VERSOES,
                           minOccurs=0, maxOccurs=0,
                           instructions="Classificar cada elemento na classe IFC adequada.")
    r8.applicability.append(ids.Entity(name="IFCBUILDINGELEMENTPROXY"))

    requisitos.specifications.extend([r1, r2, r3, r4, r5, r6, r7, r8])
    valido = requisitos.to_xml(args.saida)                  # grava e confere contra o XSD do IDS 1.0
    if not valido:
        raise SystemExit(f"O arquivo {args.saida} foi gravado, mas não é válido segundo o XSD do IDS.")

    reaberto = ids.open(args.saida, validate=True)          # segunda conferência: leitura do arquivo gravado
    print(f"IDS gravado e válido: {args.saida} ({len(reaberto.specifications)} especificações)")
    for espec in requisitos.specifications:
        uso = "proibida" if espec.maxOccurs == 0 else ("obrigatória" if espec.minOccurs else "opcional")
        print(f"  {espec.identifier}  {espec.name} | {len(espec.requirements)} requisito(s) | {uso}")


if __name__ == "__main__":
    main()
```

Execução e saída obtida:

```
python c12_02_criar_ids.py
```

```
IDS gravado e válido: requisitos_bep.ids (8 especificações)
  R1  Componentes: TAG no formato do BEP | 1 requisito(s) | obrigatória
  R2  Tubos, válvulas e bombas: dados de processo | 3 requisito(s) | opcional
  R3  Componentes: revisão e situação | 2 requisito(s) | opcional
  R4  Componentes: material e classificação | 2 requisito(s) | opcional
  R5  Tubos: diâmetro nominal DN100 | 1 requisito(s) | opcional
  R6  Componentes: contidos em pavimento | 1 requisito(s) | opcional
  R7  Equipamentos e válvulas: fabricante | 1 requisito(s) | opcional
  R8  Elementos genéricos proibidos | 0 requisito(s) | proibida
```

Trecho do arquivo gerado, com a primeira especificação:

```
       <specification name="Componentes: TAG no formato do BEP" ifcVersion="IFC4X3_ADD2" identifier="R1"
instructions="Preencher Name com a TAG do BEP, por exemplo TB-001.">
           <applicability minOccurs="1" maxOccurs="unbounded">
               <entity>
                   <name>
                       <xs:restriction base="xs:string">
                           <xs:enumeration value="IFCPIPESEGMENT" />
                           <xs:enumeration value="IFCPIPEFITTING" />
                           <xs:enumeration value="IFCVALVE" />
                           <xs:enumeration value="IFCPUMP" />
                           <xs:enumeration value="IFCTANK" />
                       </xs:restriction>
                   </name>
               </entity>
           </applicability>
           <requirements>
               <attribute cardinality="required">
                   <name>
                       <simpleValue>Name</simpleValue>
                   </name>
                   <value>
```

```
                    <xs:restriction base="xs:string">
                        <xs:pattern value="[A-Z]{1,3}-[0-9]{3}" />
                    </xs:restriction>
                </value>
            </attribute>
        </requirements>
    </specification>
```

ELEMENTO	CLASSE OU FUNÇÃO (IFCTESTER 0.8.5)
Documento	`ids.Ids(title=..., version=..., author=..., date=...)`
Especificação	`ids.Specification(name, ifcVersion, minOccurs, maxOccurs, identifier, instructions)`, com as listas `applicability` e `requirements`
Facetas	`ids.Entity`, `ids.PartOf`, `ids.Classification`, `ids.Attribute`, `ids.Property`, `ids.Material`
Restrição	`ids.Restriction(options={...}, base="string")`, com as chaves `enumeration`, `pattern`, `minInclusive`, `maxInclusive`, `minExclusive`, `maxExclusive`, `length`, `minLength`, `maxLength`
Gravar	`documento.to_xml(caminho)`, que devolve `True` se o arquivo for válido segundo o XSD
Ler	`ids.open(caminho, validate=True)`
Verificar	`documento.validate(modelo)`

Tabela 12.12: API do IfcTester para arquivos IDS.

> **OBSERVAÇÃO**
>
> Três comportamentos da versão 0.8.5, verificados nos testes: na API, o valor padrão de `minOccurs` é 0 (especificação opcional), por isso o script informa `minimo=1` onde a especificação é obrigatória; o arquivo é gravado com o espaço de nomes padrão, sem o prefixo `ids:`, o que é equivalente em XML; e o atributo `identifier` é gravado, mas não é recuperado por `ids.open`. Se o relatório precisar do identificador, inclua-o também no nome da especificação.

Além da conferência contra o XSD, a buildingSMART disponibiliza a **IDS Audit Tool**, ferramenta oficial que verifica se os nomes de entidades, propriedades e tipos predefinidos usados no IDS existem nas versões do IFC declaradas.

12.4.3 Verificar por programa e gerar relatórios

A linha de comando basta para uma conferência pontual. Para integrar a verificação a um fluxo de trabalho, o script abaixo executa a verificação, grava os relatórios HTML e BCF do IfcTester e acrescenta uma **planilha de não conformidades**, uma linha por falha, com a especificação, o requisito, a classe, o GlobalId, o nome do elemento e o motivo. Grave-o como `c12_03_validar_ids.py`.

```
# -*- coding: utf-8 -*-
"""
c12_03_validar_ids.py | Capítulo 12: verificação de um modelo IFC contra um IDS (IfcTester).

Lê um arquivo IDS e um modelo IFC (nenhum dos dois é alterado), executa a verificação e grava:
  - <base>.html   relatório HTML do IfcTester;
  - <base>.bcf    um apontamento BCF por falha, para abrir no visualizador;
  - <base>.xlsx   planilha de não conformidades, uma linha por falha.
Também imprime no terminal o relatório em texto (sem cores).

Uso: python c12_03_validar_ids.py <requisitos.ids> <modelo.ifc> [--base relatorio]
Testado com: Python 3.11 e 3.13; ifctester 0.8.5; ifcopenshell 0.8.5; pandas 3.0.
```

```
"""
import argparse
import pandas as pd
import ifcopenshell
from ifctester import ids, reporter


def rotulo(req):
    """Descrição curta de um requisito (faceta), em português."""
    tipo = type(req).__name__
    if tipo == "Property":
        return f"propriedade {req.propertySet}.{req.baseName}"
    if tipo == "Attribute":
        return f"atributo {req.name}"
    if tipo == "Classification":
        return f"classificação {req.system}"
    if tipo == "PartOf":
        return f"parte de {req.name}"
    return tipo.lower()                                  # material, entity


def linha(espec, requisito, elemento, motivo, descricao=None):
    """Uma não conformidade, com os dados que identificam o elemento no modelo."""
    return {
        "Especificacao": espec.name,
        "Requisito": requisito,
        "Descricao": descricao,
        "Classe": elemento.is_a() if elemento else None,
        "GlobalId": getattr(elemento, "GlobalId", None) if elemento else None,
        "Nome": getattr(elemento, "Name", None) if elemento else None,
        "Motivo": motivo,
    }


def nao_conformidades(requisitos):
    """Percorre os resultados e devolve uma lista de falhas (uma por elemento e requisito)."""
    falhas = []
    for espec in requisitos.specifications:
        if espec.status is not False:
            continue
        if espec.maxOccurs == 0:                     # especificação proibida: cada elemento é uma falha
            for elem in espec.applicable_entities:
                falhas.append(linha(espec, "elemento proibido", elem, "o elemento não deveria existir"))
        elif espec.minOccurs != 0 and not espec.applicable_entities:
            falhas.append(linha(espec, "especificação obrigatória", None,
                                "nenhum elemento aplicável encontrado"))
        for req in espec.requirements:
            texto = req.to_string("requirement", espec, req)       # descrição completa, em inglês
            for falha in req.failures:
                falhas.append(linha(espec, rotulo(req), falha["element"], falha["reason"], texto))
    return falhas


def main():
    leitor = argparse.ArgumentParser(description="Verifica um modelo IFC contra um IDS.")
    leitor.add_argument("ids")
    leitor.add_argument("modelo")
    leitor.add_argument("--base", default="relatorio", help="nome-base dos relatórios gravados")
    args = leitor.parse_args()

    try:
        requisitos = ids.open(args.ids, validate=True)       # confere o IDS contra o XSD antes de usar
    except ids.IdsXmlValidationError as erro:
        raise SystemExit(f"IDS inválido: {erro.xml_error.reason} (em {erro.xml_error.path})")
    modelo = ifcopenshell.open(args.modelo)

    for espec in requisitos.specifications:                # aviso: versão do IFC fora do escopo do requisito
        if modelo.schema_identifier not in espec.ifcVersion:
            print(f"Aviso: '{espec.name}' vale para {espec.ifcVersion}; "
                  f"o modelo é {modelo.schema_identifier}.")

    requisitos.validate(modelo)
```

```
    relatorio_texto = reporter.Console(requisitos, use_colour=False)
    relatorio_texto.report()                                # imprime o relatório em texto

    html = reporter.Html(requisitos)
    html.report()
    html.to_file(f"{args.base}.html")

    bcf = reporter.Bcf(requisitos)
    bcf.report()
    bcf.to_file(f"{args.base}.bcf")

    falhas = nao_conformidades(requisitos)
    colunas = ["Especificacao", "Requisito", "Descricao", "Classe", "GlobalId", "Nome", "Motivo"]
    tabela = pd.DataFrame(falhas, columns=colunas)
    tabela.to_excel(f"{args.base}.xlsx", index=False)

    total = len(requisitos.specifications)
    aprovadas = sum(1 for espec in requisitos.specifications if espec.status)
    print(f"\nEspecificações atendidas: {aprovadas} de {total} | não conformidades: {len(falhas)}")
    print(f"Relatórios: {args.base}.html, {args.base}.bcf, {args.base}.xlsx")
    espec_atual = None
    for falha in falhas:                                    # resumo da planilha, por especificação
        if falha["Especificacao"] != espec_atual:
            espec_atual = falha["Especificacao"]
            print(f"  {espec_atual}")
        if falha["Classe"] is None:
            nome = "(nenhum elemento)"
        else:
            nome = falha["Nome"] or "(sem nome)"
        print(f"    {nome} | {falha['Requisito']}: {falha['Motivo']}")


if __name__ == "__main__":
    main()
```

Execução com o IDS gerado na seção anterior e o modelo preenchido no Capítulo 10:

```
python c12_03_validar_ids.py requisitos_bep.ids rede_rev1.ifc --base relatorio_rede
```

```
Requisitos de informação: tubulação e equipamentos (exemplo)
[PASS] (5/5) *Componentes: TAG no formato do BEP
    Applies to:
        All {'enumeration': ['IFCPIPESEGMENT', 'IFCPIPEFITTING', 'IFCVALVE', 'IFCPUMP', 'IFCTANK']} data
    Requirements:
        The Name shall be {'pattern': '[A-Z]{1,3}-[0-9]{3}'}
[FAIL] (3/4) Tubos, válvulas e bombas: dados de processo
    Applies to:
        All {'enumeration': ['IFCPIPESEGMENT', 'IFCVALVE', 'IFCPUMP']} data
    Requirements:
        Fluido data shall be provided in the dataset ARV_Processo
        PressaoProjeto data shall be {'minExclusive': '0', 'maxInclusive': '2000000.0'} and in the dataset
ARV_Processo
            The property set does not contain the required property -
#19=IfcPipeSegment('11xx5V1$r1t95xY4T4G5fp',$,'TB-002','Tubo de recalque',$,$,$,$,$)
        TemperaturaProjeto data shall be {'minInclusive': '273.15', 'maxInclusive': '473.15'} and in the dataset
ARV_Processo
[PASS] (5/5) Componentes: revisão e situação
    Applies to:
        All {'enumeration': ['IFCPIPESEGMENT', 'IFCPIPEFITTING', 'IFCVALVE', 'IFCPUMP', 'IFCTANK']} data
    Requirements:
        Revisao data shall be {'pattern': '[A-Z]|[0-9]{2}'} and in the dataset ARV_Controle
        SituacaoAprovacao data shall be {'enumeration': ['Em análise', 'Aprovado', 'Reprovado']} and in the
dataset ARV_Controle
[PASS] (5/5) Componentes: material e classificação
    Applies to:
        All {'enumeration': ['IFCPIPESEGMENT', 'IFCPIPEFITTING', 'IFCVALVE', 'IFCPUMP', 'IFCTANK']} data
    Requirements:
        Shall have a ABNT NBR 15965 reference of {'pattern': '3E\\.[A-Z]{2}\\.[0-9]{2}'}
        Shall have a material
[PASS] (2/2) Tubos: diâmetro nominal DN100
```

```
    Applies to:
        All IFCPIPESEGMENT data
    Requirements:
        NominalDiameter data shall be 0.1 and in the dataset Pset_PipeSegmentTypeCommon
[PASS] (5/5) Componentes: contidos em pavimento
    Applies to:
        All {'enumeration': ['IFCPIPESEGMENT', 'IFCPIPEFITTING', 'IFCVALVE', 'IFCPUMP', 'IFCTANK']} data
    Requirements:
        An element must have an IFCRELCONTAINEDINSPATIALSTRUCTURE relationship with an IFCBUILDINGSTOREY
[PASS] (3/3) Equipamentos e válvulas: fabricante
    Applies to:
        All {'enumeration': ['IFCVALVE', 'IFCPUMP', 'IFCTANK']} data
    Requirements:
        Manufacturer data shall be provided in the dataset Pset_ManufacturerTypeInformation
[PASS] (0/0) Elementos genéricos proibidos
    Applies to:
        All IFCBUILDINGELEMENTPROXY data
    Requirements:

Especificações atendidas: 7 de 8 | não conformidades: 1
Relatórios: relatorio_rede.html, relatorio_rede.bcf, relatorio_rede.xlsx
  Tubos, válvulas e bombas: dados de processo
    TB-002 | propriedade ARV_Processo.PressaoProjeto: The property set does not contain the required property
```

Das oito especificações, sete foram atendidas. A única não conformidade é a do tubo TB-002, cuja pressão foi recusada no Capítulo 10 por conter o texto "alta" na planilha: o registro do preenchimento em lote apontou o problema, e agora a verificação independente o confirma. Note que a especificação R5 foi atendida com o valor 0.1, embora o arquivo registre 100 (mm), e que a propriedade `NominalDiameter` está no tipo do tubo: o IfcTester considera as propriedades herdadas do tipo.

Com o IDS de edificação e a amostra de arquitetura, `python c12_03_validar_ids.py requisitos_edificacao.ids "<caminho da amostra>\Building-Architecture.ifc" --base relatorio_edificacao`, o resumo final fica:

```
Especificações atendidas: 0 de 6 | não conformidades: 13
Relatórios: relatorio_edificacao.html, relatorio_edificacao.bcf, relatorio_edificacao.xlsx
  Paredes: indicação de parede externa
    house - outer wall - house right front | propriedade Pset_WallCommon.IsExternal: The required property set
does not exist
    house - outer wall - house right back | propriedade Pset_WallCommon.IsExternal: The required property set
does not exist
    house - outer wall - house left | propriedade Pset_WallCommon.IsExternal: The required property set does not
exist
    plumbing wall | propriedade Pset_WallCommon.IsExternal: The required property set does not exist
  Paredes: espessura entre 0,10 m e 0,40 m
    plumbing wall | propriedade Qto_WallBaseQuantities.Width: The property value "0.024" does not match the
requirements
  Lajes: resistência ao fogo
    house - roof - slab left | propriedade Pset_SlabCommon.FireRating: The required property set does not exist
    house - roof - slab right | propriedade Pset_SlabCommon.FireRating: The required property set does not exist
  Espaços: identificação
    living room | atributo LongName: The attribute value "None" is empty
    entry hall | atributo LongName: The attribute value "None" is empty
  Paredes e lajes: contidas em pavimento
    house - roof - slab left | parte de IFCBUILDINGSTOREY: The entity has a relationship with incorrect entities:
"IFCSITE"
    house - roof - slab right | parte de IFCBUILDINGSTOREY: The entity has a relationship with incorrect
entities: "IFCSITE"
  Elementos genéricos proibidos
    origin | elemento proibido: o elemento não deveria existir
    geo-reference | elemento proibido: o elemento não deveria existir
```

Os resultados da verificação ficam nos próprios objetos do IfcTester, que o script percorre:

- `especificacao.status`: `True` (atendida), `False` (não atendida) ou `None` (não verificada);

- `especificacao.applicable_entities` e `especificacao.failed_entities`: elementos aplicáveis e elementos com falha;
- `requisito.failures`: lista de falhas, cada uma com o elemento (`"element"`) e o motivo (`"reason"`).

MOTIVO INFORMADO	SIGNIFICADO	AÇÃO TÍPICA
`The required property set does not exist`	O conjunto exigido não existe no elemento nem no tipo	Mapear o conjunto na exportação ou inserir em lote
`The property set does not contain the required property`	O conjunto existe, mas falta a propriedade	Preencher a propriedade
`The property value "X" does not match the requirements`	O valor, já convertido para o SI, não atende à restrição	Corrigir o valor ou revisar o requisito
`The property's data type "A" does not match the required data type of "B"`	O tipo de valor gravado difere do exigido	Gravar com o tipo correto (Capítulo 10)
`The attribute value "None" is empty`	Atributo vazio	Preencher o atributo
`The attribute value "X" does not match the requirement`	Atributo fora da restrição	Corrigir o valor
`The entity has no classification` / `The systems "[...]" do not match the requirements`	Sem classificação / sistema de classificação diferente	Associar a classificação correta
`The entity has no material`	Sem material	Associar o material
`The entity has a relationship with incorrect entities: "X"`	A relação existe, mas com outra classe	Revisar a estrutura espacial ou o requisito
`The property should not have met the requirement`	Informação proibida encontrada	Remover a informação

Tabela 12.13: Motivos de falha mais frequentes no IfcTester 0.8.5.

No relatório de tela, o IfcTester usa as aspas para colorir o texto e as omite na impressão; na planilha, elas aparecem.

12.4.4 Armadilhas frequentes

1. **Unidades.** O valor do IDS está no SI. Exigir `NominalDiameter = 100` num modelo em milímetros falha, porque o valor comparado é 0,1 (verificado nos testes).
2. **Tipo de dado exato.** `dataType="IFCLENGTHMEASURE"` falha para uma propriedade gravada como `IfcPositiveLengthMeasure`, embora ambas sejam comprimentos. Use o tipo previsto no modelo do conjunto (Capítulo 5).
3. **Valores gravados como `IfcReal`.** Uma pressão gravada sem o tipo de medida (Capítulo 10) falha num requisito com `IFCPRESSUREMEASURE`.
4. **Especificação sem elementos.** Uma especificação opcional sem nenhum elemento aplicável é considerada atendida. Um modelo sem nenhum tubo "passa" em todas as exigências sobre tubos. Declare como obrigatória (`minOccurs="1"`) ao menos uma especificação por classe essencial.
5. **Maiúsculas e minúsculas.** `tb-001` não é igual a `TB-001`, e `pset_wallcommon` não é `Pset_WallCommon`.

6. **Padrão parcial.** O padrão descreve o valor inteiro; para aceitar qualquer texto que comece com `TB-`, escreva `TB-.*`.
7. **Versão do IFC.** Se o esquema do modelo não constar de `ifcVersion`, o IfcTester verifica assim mesmo, sem avisar; o script `c12_03` emite um aviso nesse caso. Na API, `documento.validate(modelo, should_filter_version=True)` deixa essas especificações sem verificar.

12.5 Comparar revisões com o ifcdiff

12.5.1 O conceito

Comparar duas revisões de um modelo responde a perguntas como "o que o projetista alterou desde a última entrega?" e "a correção pedida foi feita?". A correspondência entre revisões é feita pelo **GlobalId** (Capítulo 3): o mesmo GlobalId nas duas revisões indica o mesmo objeto; um GlobalId que só existe na nova indica um objeto adicionado; um que só existe na antiga, um objeto removido. Se o exportador gerar novos GlobalId a cada exportação (Capítulo 13), a comparação perde o sentido: todos os objetos aparecem como removidos e adicionados.

O **ifcdiff**, instalado no Capítulo 0, é uma ferramenta do projeto IfcOpenShell, usada como programa de linha de comando ou como biblioteca. Ele verifica, conforme a lista de relações escolhida, a geometria (padrão), os atributos, o tipo, as propriedades, o contêiner, a agregação e a classificação. Pela linha de comando:

```
python -m ifcdiff rede_rev0.ifc rede_rev1.ifc -o diferencas.json -r "property classification"
```

A opção `-r` lista as relações a verificar; quando ela é informada, a geometria só é comparada se `geometry` constar da lista. Na versão testada, esse comando imprime o resumo e termina com erro ao gravar o JSON, como explica a seção seguinte.

12.5.2 Limitações verificadas na versão 0.8.5

Os testes deste capítulo revelaram quatro comportamentos que orientam o uso da ferramenta:

- o exemplo da documentação passa caminhos de arquivo à classe `IfcDiff`, mas a classe espera arquivos já abertos com `ifcopenshell.open`;
- para cada elemento, a verificação das relações para na primeira diferença encontrada: na comparação desta seção, o registro indica mudança de propriedades, mas não de classificação, embora a classificação também tenha mudado;
- as verificações `container` e `aggregate` comparam objetos de arquivos diferentes e acusam mudança até entre cópias idênticas (teste: `rede_proxies.ifc`, do Capítulo 10, comparado com uma cópia idêntica, apresentou oito "mudanças de contêiner" e uma "mudança de agregação");
- com o `deepdiff` 9.1, a gravação do JSON pela linha de comando falha quando há diferenças de propriedades (`TypeError: Object of type SetOrdered is not JSON serializable`), depois de imprimir o resumo.

Por isso, o script a seguir usa o ifcdiff como biblioteca, para localizar elementos adicionados, removidos e com geometria alterada, e detalha as diferenças com código próprio, comparando classe, atributos, propriedades, contêiner, material e classificação por texto. Grave-o como `c12_04_comparar_revisoes.py`.

```
# -*- coding: utf-8 -*-
"""
c12_04_comparar_revisoes.py | Capítulo 12: comparação entre duas revisões de um modelo IFC.

Usa a biblioteca ifcdiff para localizar elementos adicionados, removidos e alterados
(a correspondência entre revisões é feita pelo GlobalId) e detalha, com código próprio,
o que mudou em cada elemento comum: classe, atributos, propriedades, contêiner, material
e classificação. Os dois arquivos são abertos somente para leitura.

Uso: python c12_04_comparar_revisoes.py <antigo.ifc> <novo.ifc> [--saida comparacao.xlsx]
Testado com: Python 3.11 e 3.13; ifcdiff 0.8.5; ifcopenshell 0.8.5; pandas 3.0.
"""
import argparse
import pandas as pd
import ifcopenshell
import ifcopenshell.util.classification as util_class
import ifcopenshell.util.element as util_elem
from ifcdiff import IfcDiff


def atributos(elem):
    """Atributos com valor simples (texto, número, enumeração); referências a entidades são ignoradas."""
    return {nome: valor for nome, valor in elem.get_info(recursive=False).items()
            if nome not in ("id", "type") and not isinstance(valor, (ifcopenshell.entity_instance, tuple))}


def propriedades(elem):
    """Propriedades e quantidades no formato 'Conjunto.Propriedade' (inclui as herdadas do tipo)."""
    return {f"{conjunto}.{nome}": valor for conjunto, props in util_elem.get_psets(elem).items()
            for nome, valor in props.items() if nome != "id"}


def relacoes(elem):
    """Contêiner, material e classificação, descritos por texto (comparáveis entre arquivos)."""
    contem = util_elem.get_container(elem)
    material = util_elem.get_material(elem)
    refs = sorted((getattr(r, "Identification", None) or getattr(r, "ItemReference", None) or "")
                  for r in util_class.get_references(elem))        # ItemReference: nome no IFC2x3
    return {"contêiner": contem.Name if contem else None,
            "material": (getattr(material, "Name", None) or material.is_a()) if material else None,
            "classificação": ", ".join(refs) or None}


def diferencas(antes, depois):
    """Compara dois dicionários e devolve (chave, valor antigo, valor novo) para cada diferença."""
    return [(chave, antes.get(chave), depois.get(chave)) for chave in sorted(set(antes) | set(depois))
            if antes.get(chave) != depois.get(chave)]


def main():
    leitor = argparse.ArgumentParser(description="Compara duas revisões de um modelo IFC.")
    leitor.add_argument("antigo")
    leitor.add_argument("novo")
    leitor.add_argument("--saida", default="comparacao.xlsx")
    args = leitor.parse_args()

    antigo, novo = ifcopenshell.open(args.antigo), ifcopenshell.open(args.novo)
    # 'container' e 'aggregate' ficam de fora: na versão 0.8.5 acusam mudança até entre cópias idênticas
    comparacao = IfcDiff(antigo, novo, is_shallow=False,
                         relationships=["attributes", "type", "property", "classification", "geometry"])
    comparacao.diff()                                       # imprime o resumo do ifcdiff

    linhas = []
    for gid in sorted(comparacao.added_elements):
        linhas.append([gid, novo.by_guid(gid).Name, "adicionado", None, None, None])
    for gid in sorted(comparacao.deleted_elements):
        linhas.append([gid, antigo.by_guid(gid).Name, "removido", None, None, None])

    comuns = ({e.GlobalId for e in antigo.by_type("IfcElement")}
              & {e.GlobalId for e in novo.by_type("IfcElement")})
    for gid in sorted(comuns):                              # detalhamento próprio de todos os elementos comuns
        velho, atual = antigo.by_guid(gid), novo.by_guid(gid)
```

```
        if velho.is_a() != atual.is_a():
            linhas.append([gid, atual.Name, "classe", "classe IFC", velho.is_a(), atual.is_a()])
        for tipo, antes, depois in (("atributo", atributos(velho), atributos(atual)),
                                    ("propriedade", propriedades(velho), propriedades(atual)),
                                    ("relação", relacoes(velho), relacoes(atual))):
            for item, valor_antigo, valor_novo in diferencas(antes, depois):
                linhas.append([gid, atual.Name, tipo, item, valor_antigo, valor_novo])
        if comparacao.change_register.get(gid, {}).get("geometry_changed"):
            linhas.append([gid, atual.Name, "geometria", "forma ou posição", None, None])

    linhas.sort(key=lambda lin: (lin[1] or "", lin[0], lin[2], lin[3] or ""))  # nome, GlobalId, mudança
    colunas = ["GlobalId", "Nome", "Mudanca", "Item", "Antes", "Depois"]
    pd.DataFrame(linhas, columns=colunas).to_excel(args.saida, index=False)

    print(f"\nElementos alterados segundo o ifcdiff: {len(comparacao.change_register)}")
    print(f"Detalhamento: {len(linhas)} diferença(s) -> {args.saida}")
    for _, nome, mudanca, item, antes, depois in linhas:
        vazio = "(vazio)"
        print(f"  {nome} | {mudanca} | {item}: {vazio if antes is None else antes} -> "
              f"{vazio if depois is None else depois}")


if __name__ == "__main__":
    main()
```

Execução com as duas revisões da rede, antes e depois do preenchimento em lote do Capítulo 10:

```
python c12_04_comparar_revisoes.py rede_rev0.ifc rede_rev1.ifc
```

Saída obtida (as primeiras 22 linhas; as demais repetem o padrão para TB-001, TB-002, TQ-001 e VG-001):

```
 - 8 item(s) are in the old model
 - 8 item(s) are in the new model
 - 0 item(s) were added
 - 0 item(s) were deleted
 - 8 item(s) are common to both models
 - 5 item(s) had simple changes
 - 5 item(s) were changed

Elementos alterados segundo o ifcdiff: 5
Detalhamento: 41 diferença(s) -> comparacao.xlsx
  B-001 | atributo | Description: (vazio) -> Bomba de transferência
  B-001 | propriedade | ARV_Controle.Revisao: (vazio) -> A
  B-001 | propriedade | ARV_Controle.SituacaoAprovacao: (vazio) -> Em análise
  B-001 | propriedade | ARV_Processo.Fluido: (vazio) -> Produto A
  B-001 | propriedade | ARV_Processo.PressaoProjeto: (vazio) -> 1000000.0
  B-001 | propriedade | ARV_Processo.TemperaturaProjeto: (vazio) -> 313.15
  B-001 | propriedade | Pset_ManufacturerTypeInformation.Manufacturer: (vazio) -> Fabricante Z
  B-001 | relação | classificação: (vazio) -> 3E.EX.04
  B-001 | relação | material: (vazio) -> Ferro fundido
  TB-001 | atributo | Description: (vazio) -> Tubo de sucção
  TB-001 | propriedade | ARV_Controle.Revisao: (vazio) -> B
  TB-001 | propriedade | ARV_Controle.SituacaoAprovacao: (vazio) -> Aprovado
```

As linhas iniciadas por hífen são o resumo impresso pelo próprio ifcdiff, que conta elementos e elementos espaciais: os 8 itens são os 5 componentes da rede, o terreno, a edificação e o pavimento. O detalhamento lista 41 diferenças: a descrição, as propriedades de processo e de controle, o fabricante, o material e a classificação de cada elemento. É o registro exato do que o preenchimento em lote fez, útil para o controle de revisões exigido pelo BEP.

12.6 BCF: comunicar os problemas

12.6.1 O conceito

O **BCF** (*BIM Collaboration Format*, formato de colaboração BIM) é o padrão aberto da buildingSMART para trocar **apontamentos** sobre modelos sem trocar os modelos. Cada apontamento, chamado **tópico**, tem título, descrição, tipo, situação, autor, data e comentários, e pode ter **pontos de vista**: a posição da câmera, os componentes selecionados ou ocultos, identificados pelo GlobalId (`IfcGuid`), e uma imagem da tela. Quem recebe o BCF, com o mesmo modelo aberto, vai direto ao elemento apontado.

O BCF existe em duas formas:

- **BCF-XML**: um arquivo compactado com extensão `.bcf` (a extensão `.bcfzip` era usada nas versões 1.0 e 2.0), trocado entre as partes;
- **BCF-API**: um serviço web (REST, com dados em JSON sobre HTTPS), pelo qual plataformas de coordenação sincronizam tópicos diretamente.

As versões em uso são a 2.1 e a 3.0 (esta apresentada pela buildingSMART em 2021). Confira qual versão os programas da equipe leem antes de padronizar o fluxo.

12.6.2 Estrutura do arquivo

O relatório BCF gravado pelo IfcTester na seção 12.4.3 é um arquivo ZIP com esta estrutura (os identificadores e a data variam a cada execução):

```
relatorio_rede.bcf (ZIP)
├── bcf.version                                  versão do formato (2.1)
├── project.bcfp                                 dados do projeto (opcional)
└── 53f24714-f86d-43a7-a6b4-634fc5d139aa/        uma pasta por tópico (nome = GUID do tópico)
    ├── markup.bcf                               título, descrição, autor, data, comentários
    └── 66317ac6-137d-4b54-b330-435ef53bb457.bcfv   ponto de vista: câmera e componentes
```

Figura 12.3: Estrutura do arquivo BCF gerado.

Conteúdo do `markup.bcf`:

```
<?xml version="1.0" encoding="UTF-8"?>
<Markup xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <Topic Guid="53f24714-f86d-43a7-a6b4-634fc5d139aa" TopicType="" TopicStatus="">
    <Title>IfcPipeSegment - TB-002 - The property set does not contain the required property -
11xx5V1$r1t95xY4T4G5fp</Title>
    <CreationDate>2026-09-19T00:46:31.507735</CreationDate>
    <CreationAuthor>IfcTester</CreationAuthor>
    <Description>Tubos, válvulas e bombas: dados de processo - PressaoProjeto data shall be {'minExclusive': '0',
'maxInclusive': '2000000.0'} and in the dataset ARV_Processo</Description>
  </Topic>
  <Viewpoints Guid="66317ac6-137d-4b54-b330-435ef53bb457">
    <Viewpoint>66317ac6-137d-4b54-b330-435ef53bb457.bcfv</Viewpoint>
  </Viewpoints>
</Markup>
```

Início do ponto de vista (`.bcfv`), com o componente selecionado pelo GlobalId do tubo:

```
<?xml version="1.0" encoding="UTF-8"?>
<VisualizationInfo xmlns:xs="http://www.w3.org/2001/XMLSchema" Guid="66317ac6-137d-4b54-b330-435ef53bb457">
  <Components>
```

```
    <Selection>
      <Component IfcGuid="11xx5V1$r1t95xY4T4G5fp"/>
    </Selection>
    <Visibility DefaultVisibility="true"/>
  </Components>
```

O IfcTester nomeia os pontos de vista com um GUID e não grava imagens. O BCF 2.1 prevê nomes padrão para o ponto de vista e a imagem principais de cada tópico (`viewpoint.bcfv` e `snapshot.png`); se o programa de coordenação da equipe não abrir o arquivo, teste a importação antes de adotar o fluxo.

O vínculo entre o apontamento e o modelo é o `IfcGuid`. Por isso, o BCF também depende de GlobalId estáveis: se o projetista reexportar o modelo e os identificadores mudarem, os apontamentos deixam de encontrar os elementos.

12.6.3 Ler um BCF por programa

A biblioteca `bcf-client`, instalada junto com o IfcTester, lê e grava arquivos BCF. O script abaixo lista os tópicos e os elementos selecionados. Grave-o como `c12_05_ler_bcf.py`.

```
# -*- coding: utf-8 -*-
"""
c12_05_ler_bcf.py | Capítulo 12: leitura de um arquivo BCF (versão 2.1) com a biblioteca bcf-client.

Lista os tópicos (apontamentos) de um arquivo .bcf e os elementos IFC (IfcGuid) selecionados
nos pontos de vista de cada tópico. O arquivo BCF é aberto somente para leitura.

Uso: python c12_05_ler_bcf.py <arquivo.bcf>
Testado com: Python 3.11 e 3.13; bcf-client 0.8.5 (instalado junto com o ifctester).
"""
import sys
from bcf.v2.bcfxml import BcfXml


def main():
    bcf = BcfXml.load(sys.argv[1])
    topicos = sorted(bcf.topics.values(), key=lambda t: t.topic.title)
    print(f"{len(topicos)} tópico(s) em {sys.argv[1]}")
    for topico in topicos:
        guids = set()
        for ponto_de_vista in topico.viewpoints.values():           # arquivos .bcfv do tópico
            componentes = ponto_de_vista.visualization_info.components
            if componentes and componentes.selection:
                guids.update(c.ifc_guid for c in componentes.selection.component)
        print(f"- {topico.topic.title}")
        print(f"  autor: {topico.topic.creation_author} | pontos de vista: {len(topico.viewpoints)}"
              f" | elementos selecionados: {', '.join(sorted(guids))}")
    bcf.close()


if __name__ == "__main__":
    main()
```

Execução com os dois relatórios gerados:

```
python c12_05_ler_bcf.py relatorio_rede.bcf
python c12_05_ler_bcf.py relatorio_edificacao.bcf
```

```
1 tópico(s) em relatorio_rede.bcf
- IfcPipeSegment - TB-002 - The property set does not contain the required property - 11xx5V1$r1t95xY4T4G5fp
  autor: IfcTester | pontos de vista: 1 | elementos selecionados: 11xx5V1$r1t95xY4T4G5fp
```

```
11 tópico(s) em relatorio_edificacao.bcf
- IfcSlab - house - roof - slab left - The entity has a relationship with incorrect entities: "IFCSITE" -
0ZTBBPo6f6bxqV2K7Oelrq - 454425.1027891.979946.932084.902510
  autor: IfcTester | pontos de vista: 2 | elementos selecionados: 0ZTBBPo6f6bxqV2K7Oelrq
- IfcSlab - house - roof - slab left - The required property set does not exist - 0ZTBBPo6f6bxqV2K7Oelrq -
454425.1027891.979946.932084.902510
  autor: IfcTester | pontos de vista: 2 | elementos selecionados: 0ZTBBPo6f6bxqV2K7Oelrq
- IfcSlab - house - roof - slab right - The entity has a relationship with incorrect entities: "IFCSITE" -
12UVOn4wvAJPMUExKdZLb8 - 454425.1027891.979946.932084.902511
  autor: IfcTester | pontos de vista: 2 | elementos selecionados: 12UVOn4wvAJPMUExKdZLb8
```

(Trecho: 3 dos 11 tópicos.)

O relatório de edificação tem 11 tópicos, e não 13 como a planilha: o BCF do IfcTester é gerado a partir das falhas de requisitos, e a especificação proibida (E6) não tem requisitos. Nos elementos construtivos (`IfcElement`) com posicionamento, o IfcTester grava dois pontos de vista por tópico; nos espaços, um só.

12.6.4 O ciclo de coordenação

1. O projetista exporta o IFC, valida o esquema e verifica o IDS (autocontrole).
2. O contratante recebe e repete as verificações; as falhas viram tópicos BCF.
3. A coordenação abre o BCF num visualizador ou plataforma compatível, junto com o IFC, revisa os tópicos, comenta, define responsáveis e prazos.
4. O projetista abre os tópicos no seu ambiente, localiza os elementos pelo GlobalId e corrige no software de autoria.
5. Na nova revisão, a comparação (seção 12.5) mostra o que mudou, e a verificação IDS confirma a correção.
6. Os tópicos resolvidos são fechados, e o histórico fica registrado.

12.7 Resumo do capítulo

- A verificação de um modelo tem quatro níveis: sintaxe, esquema, regras normativas e requisitos do projeto. Passar em um não garante os outros.
- `python -m ifcopenshell.validate --rules` valida sintaxe, esquema e regras EXPRESS localmente; o serviço da buildingSMART acrescenta regras normativas e boas práticas.
- O IDS 1.0 descreve requisitos na forma "todo [aplicabilidade] deve ter [requisitos]", com seis facetas, valores simples ou restrições, e cardinalidade em dois níveis. Os valores numéricos ficam em unidades SI.
- O IfcTester verifica modelos pela linha de comando ou por programa e gera relatórios em vários formatos, inclusive BCF.
- O ifcdiff localiza mudanças entre revisões pelo GlobalId; na versão 0.8.5, o detalhamento confiável exige código próprio.
- O BCF comunica apontamentos vinculados aos elementos pelo GlobalId; a estabilidade dos identificadores é condição para todo o ciclo.

12.8 Vocabulário técnico

TERMO EM INGLÊS	TERMO EM PORTUGUÊS
validation	validação
schema / syntax	esquema / sintaxe
Implementer Agreement	acordo de implementação
Industry Practice	boa prática do setor
Information Delivery Specification (IDS)	especificação de entrega de informação
specification	especificação
applicability / requirement	aplicabilidade / requisito
facet	faceta
restriction / enumeration / pattern / bounds	restrição / enumeração / padrão / limites
cardinality: required, optional, prohibited	cardinalidade: obrigatório, opcional, proibido
reporter	gerador de relatório
diff / change register	comparação / registro de mudanças
BIM Collaboration Format (BCF)	formato de colaboração BIM
topic / viewpoint / snapshot	tópico (apontamento) / ponto de vista / imagem
markup	marcação

Tabela 12.14: Vocabulário do capítulo.

12.9 Exercícios

Fixação

1. Um arquivo passa em `python -m ifcopenshell.validate --rules` sem nenhuma violação. Isso garante que ele atende ao BEP? Justifique com a Tabela 12.1.
2. Quais são as seis facetas do IDS? Qual delas não admite cardinalidade quando usada como requisito, e qual admite apenas dois valores?
3. O que significam, na aplicabilidade, `minOccurs="0" maxOccurs="0"`? E `minOccurs="1" maxOccurs="unbounded"`?
4. Um modelo em milímetros tem `NominalDiameter = 100`, gravado como `IfcPositiveLengthMeasure`. Que valor deve constar do IDS para exigir DN100? O que acontece se o `dataType` do IDS for `IFCLENGTHMEASURE`?
5. Por que a comparação de revisões e o BCF dependem da estabilidade do GlobalId? O que a comparação mostra se o exportador gerar novos GlobalId a cada exportação?

Aplicação

6. Copie para a pasta do capítulo os arquivos `rede_proxies.ifc` e `rede_reclassificada.ifc` do Capítulo 10 e verifique ambos com `requisitos_bep.ids` (script `c12_03`). Explique os resultados, em especial por que o arquivo que só contém *proxies* atende a seis das oito especificações.
7. Escreva um IDS para a amostra `Infra-Bridge.ifc` (IFC 4.3) com três especificações: (a) pilares (`IfcColumn`) e sapatas (`IfcFooting`) devem ter material, e o modelo deve ter ao menos um desses elementos; (b) todo pilar deve estar contido numa parte de ponte (`IfcBridgePart`) do tipo `PIER`; (c) toda viga (`IfcBeam`) deve ter a quantidade `NetVolume` no conjunto `Qto_BeamBaseQuantities`. Verifique a amostra e interprete o resultado.
8. Compare `rede_proxies.ifc` com `rede_reclassificada.ifc` usando o script `c12_04`. O que mudou? O GlobalId foi preservado?

Desafio

9. Um fornecedor gravou `PressaoProjeto = 600` (em kPa), sem converter para pascal. A especificação R2 detecta o erro? Proponha uma alteração do IDS que detecte esse tipo de erro, sabendo que as pressões de projeto do empreendimento ficam entre 100 kPa e 2 MPa.
10. Adapte o script `c12_03` para rotinas automáticas: ele deve terminar com código de saída 0 quando todas as especificações forem atendidas e 1 caso contrário. Por que isso é útil?

12.10 Respostas dos exercícios

1. Não. A validação cobre os níveis 1 e 2 (sintaxe, esquema e regras EXPRESS); os requisitos do BEP são o nível 4. O arquivo `rede_rev1.ifc` não apresentou nenhuma violação do esquema, mas não atende à especificação R2, porque o tubo TB-002 está sem pressão de projeto.
2. `entity`, `partOf`, `classification`, `attribute`, `property` e `material`. Como requisito, `entity` não tem cardinalidade (é sempre obrigatória), e `partOf` admite apenas `required` e `prohibited`.
3. `0` e `0`: especificação proibida, isto é, não pode existir nenhum elemento aplicável. `1` e `unbounded`: especificação obrigatória, isto é, deve existir ao menos um elemento aplicável, e todos devem atender aos requisitos.
4. `0.1`, porque os valores do IDS estão em metros (SI), e o IfcTester converte o valor do arquivo antes de comparar. Com `dataType="IFCLENGTHMEASURE"`, a verificação falha pelo tipo, mesmo com o valor certo: `The property's data type "IfcPositiveLengthMeasure" does not match the required data type of "IFCLENGTHMEASURE"` (resultado obtido no teste).
5. Porque o GlobalId é a chave que liga o mesmo objeto nas duas revisões e liga o apontamento BCF ao elemento. Com identificadores novos a cada exportação, a comparação mostra todos os objetos como removidos e adicionados, sem nenhuma alteração detalhada, e os tópicos BCF deixam de localizar os elementos.
6. Comandos: `copy ..\cap10\rede_proxies.ifc .`, `copy ..\cap10\rede_reclassificada.ifc .`, `python c12_03_validar_ids.py requisitos_bep.ids rede_proxies.ifc --base ex6_proxies` e o equivalente para `rede_reclassificada.ifc`. Resultados obtidos:

```
Especificações atendidas: 6 de 8 | não conformidades: 9
Relatórios: ex6_proxies.html, ex6_proxies.bcf, ex6_proxies.xlsx
  Componentes: TAG no formato do BEP
```

```
    (nenhum elemento) | especificação obrigatória: nenhum elemento aplicável encontrado
  Elementos genéricos proibidos
    TQ-001 | elemento proibido: o elemento não deveria existir
    TB-001 | elemento proibido: o elemento não deveria existir
    VG-001 | elemento proibido: o elemento não deveria existir
    FT-001 | elemento proibido: o elemento não deveria existir
    TB-002 | elemento proibido: o elemento não deveria existir
    VR-001 | elemento proibido: o elemento não deveria existir
    B-001 | elemento proibido: o elemento não deveria existir
    SUPORTE-01 | elemento proibido: o elemento não deveria existir
```

No arquivo de *proxies*, não existe nenhum tubo, válvula, bomba ou tanque. A especificação R1 é obrigatória e falha por falta de elementos aplicáveis; R8 falha pelos oito *proxies*. As outras seis são opcionais e, sem elementos aplicáveis, contam como atendidas sem verificar nada: é a armadilha 4 da seção 12.4.4. No arquivo reclassificado, o resumo foi `Especificações atendidas: 1 de 8 | não conformidades: 52`: só a TAG (R1) é atendida. Faltam os conjuntos de processo e de controle, o material, a classificação, o diâmetro nominal e o fabricante; os elementos estão contidos no terreno (`IfcSite`), e não num pavimento; e o suporte `SUPORTE-01` continua como *proxy*. As 52 falhas somam 15 em R2 (3 requisitos x 5 elementos), 12 em R3, 12 em R4, 2 em R5, 6 em R6, 4 em R7 e 1 em R8. O IDS mede exatamente o que falta fazer, e é esse o papel do preenchimento em lote do Capítulo 10.

7. Uma solução, gravada como `ex12_ponte.ids`:

```
<?xml version="1.0" encoding="UTF-8"?>
<ids:ids xmlns:ids="http://standards.buildingsmart.org/IDS"
         xmlns:xs="http://www.w3.org/2001/XMLSchema"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://standards.buildingsmart.org/IDS http://standards.buildingsmart.org/IDS/1.0/
ids.xsd">
  <ids:info>
    <ids:title>Requisitos de informação: pontes (Capítulo 12, exercício 7)</ids:title>
  </ids:info>
  <ids:specifications>
    <ids:specification name="Pilares e fundações: material" ifcVersion="IFC4X3_ADD2" identifier="P1">
      <ids:applicability minOccurs="1" maxOccurs="unbounded">
        <ids:entity>
          <ids:name>
            <xs:restriction base="xs:string">
              <xs:enumeration value="IFCCOLUMN"/>
              <xs:enumeration value="IFCFOOTING"/>
            </xs:restriction>
          </ids:name>
        </ids:entity>
      </ids:applicability>
      <ids:requirements>
        <ids:material/>
      </ids:requirements>
    </ids:specification>
    <ids:specification name="Pilares: contidos em parte de ponte do tipo PIER" ifcVersion="IFC4X3_ADD2"
identifier="P2">
      <ids:applicability minOccurs="1" maxOccurs="unbounded">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCCOLUMN</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:applicability>
      <ids:requirements>
        <ids:partOf relation="IFCRELCONTAINEDINSPATIALSTRUCTURE">
          <ids:entity>
            <ids:name><ids:simpleValue>IFCBRIDGEPART</ids:simpleValue></ids:name>
            <ids:predefinedType><ids:simpleValue>PIER</ids:simpleValue></ids:predefinedType>
          </ids:entity>
        </ids:partOf>
      </ids:requirements>
    </ids:specification>
    <ids:specification name="Vigas: volume líquido informado" ifcVersion="IFC4X3_ADD2" identifier="P3">
```

```
      <ids:applicability minOccurs="0" maxOccurs="unbounded">
        <ids:entity>
          <ids:name><ids:simpleValue>IFCBEAM</ids:simpleValue></ids:name>
        </ids:entity>
      </ids:applicability>
      <ids:requirements>
        <ids:property dataType="IFCVOLUMEMEASURE">
          <ids:propertySet><ids:simpleValue>Qto_BeamBaseQuantities</ids:simpleValue></ids:propertySet>
          <ids:baseName><ids:simpleValue>NetVolume</ids:simpleValue></ids:baseName>
        </ids:property>
      </ids:requirements>
    </ids:specification>
  </ids:specifications>
</ids:ids>
```

Resultado de `python c12_03_validar_ids.py ex12_ponte.ids "<caminho da amostra>\Infra-Bridge.ifc" --base ex7_ponte`:

```
Especificações atendidas: 2 de 3 | não conformidades: 3
Relatórios: ex7_ponte.html, ex7_ponte.bcf, ex7_ponte.xlsx
  Vigas: volume líquido informado
    road river bridge - main girder | propriedade Qto_BeamBaseQuantities.NetVolume: The required property set
does not exist
    road river bridge - main girder | propriedade Qto_BeamBaseQuantities.NetVolume: The required property set
does not exist
    road river bridge - main girder | propriedade Qto_BeamBaseQuantities.NetVolume: The required property set
does not exist
```

Os 14 pilares e sapatas têm material, e os 7 pilares estão contidos em partes de ponte do tipo `PIER`. Das 8 vigas, as 3 longarinas principais da ponte rodoviária (`road river bridge - main girder`) não têm o conjunto de quantidades: o exportador gravou quantidades só para parte das vigas. Observe o uso de `predefinedType` dentro de `partOf` para exigir o tipo da parte de ponte; a classe `IfcBridgePart` e o tipo `PIER` só existem no IFC 4.3.

8. Comando: `python c12_04_comparar_revisoes.py rede_proxies.ifc rede_reclassificada.ifc --saida ex8_comparacao.xlsx`. Saída obtida:

```
 - 9 item(s) are in the old model
 - 9 item(s) are in the new model
 - 0 item(s) were added
 - 0 item(s) were deleted
 - 9 item(s) are common to both models
 - 7 item(s) had simple changes
 - 7 item(s) were changed

Elementos alterados segundo o ifcdiff: 7
Detalhamento: 14 diferença(s) -> ex8_comparacao.xlsx
  B-001 | atributo | PredefinedType: (vazio) -> ENDSUCTION
  B-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcPump
  FT-001 | atributo | PredefinedType: (vazio) -> STRAINER
  FT-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcFilter
  TB-001 | atributo | PredefinedType: (vazio) -> RIGIDSEGMENT
  TB-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcPipeSegment
  TB-002 | atributo | PredefinedType: (vazio) -> RIGIDSEGMENT
  TB-002 | classe | classe IFC: IfcBuildingElementProxy -> IfcPipeSegment
  TQ-001 | atributo | PredefinedType: (vazio) -> STORAGE
  TQ-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcTank
  VG-001 | atributo | PredefinedType: (vazio) -> ISOLATING
  VG-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcValve
  VR-001 | atributo | PredefinedType: (vazio) -> CHECK
  VR-001 | classe | classe IFC: IfcBuildingElementProxy -> IfcValve
```

Sete elementos mudaram de classe (de `IfcBuildingElementProxy` para `IfcPump`, `IfcFilter`, `IfcPipeSegment`, `IfcTank` e `IfcValve`) e receberam o `PredefinedType`. O GlobalId foi preservado: o ifcdiff encontrou os mesmos 9 itens (os 8 elementos e o terreno) nas duas revisões, sem nenhum adicionado ou removido, o que confirma o comportamento de `reassign_class` visto no Capítulo 10. O suporte não aparece porque não mudou.

9. Não detecta: R2 exige pressão maior que 0 e até 2 MPa, e 600 Pa está nessa faixa. Basta trocar o limite inferior por 100 kPa, em pascal: `value=ids.Restriction(options={"minInclusive": 1.0e5, "maxInclusive": 2.0e6}, base="double")`. No teste, com a pressão do TB-001 alterada para 600, a especificação corrigida apontou `The property value "600.0" does not match the requirements`, enquanto a original não apontou nada. Limites de plausibilidade (a faixa de valores fisicamente esperados no empreendimento) são uma forma simples de detectar erros de unidade.
10. Acrescente ao fim da função `main` a linha `raise SystemExit(0 if aprovadas == total else 1)`. No teste, o script terminou com código 1 para `rede_rev1.ifc` (sete de oito especificações atendidas) e com código 0 para um IDS contendo apenas a especificação R1. O código de saída permite que um arquivo de lote (`.bat`) ou uma rotina de integração interrompa a entrega quando a verificação falha, do mesmo modo que o validador do IfcOpenShell (seção 12.2.1).

12.11 Referências

- buildingSMART International. Information Delivery Specification (IDS): https://www.buildingsmart.org/standards/bsi-standards/information-delivery-specification-ids/
- buildingSMART International. Repositório oficial do IDS, versão 1.0.0, com documentação e exemplos: https://github.com/buildingSMART/IDS
- buildingSMART International. Esquema XSD do IDS 1.0: https://standards.buildingsmart.org/IDS/1.0/ids.xsd
- buildingSMART International. Implementações de IDS em software: https://technical.buildingsmart.org/ids-software-implementations/
- buildingSMART International. IDS Audit Tool: https://github.com/buildingSMART/IDS-Audit-tool
- buildingSMART International. Serviço de validação: https://validate.buildingsmart.org; documentação: https://buildingsmart.github.io/validate/
- buildingSMART International. BIM Collaboration Format (BCF): https://technical.buildingsmart.org/standards/bcf/
- IfcOpenShell. Validação de arquivos IFC: https://docs.ifcopenshell.org/ifcopenshell-python/validation.html
- IfcOpenShell. IfcTester: https://docs.ifcopenshell.org/ifctester.html
- IfcOpenShell. IfcDiff: https://docs.ifcopenshell.org/ifcdiff.html
- IfcOpenShell. Documentação geral: https://docs.ifcopenshell.org/

13 Interoperabilidade com softwares Autodesk

A qualidade de um arquivo IFC depende menos do formato e mais de quem o escreveu: o exportador do software de autoria e a configuração usada. Este capítulo percorre os caminhos IFC dos programas Autodesk mais usados em projetos de edificações, infraestrutura e plantas industriais (Revit, Civil 3D, Plant 3D, Navisworks, Forma e Tandem), mostra o que configurar em cada um e ensina um método para descobrir, no próprio arquivo, o que se perdeu na tradução.

Versões consideradas. Revit 2025, Civil 3D 2026 e Navisworks 2026; para o Plant 3D, a documentação de suporte e as notas de novidades das versões 2026 e 2027; e os serviços em nuvem Autodesk Forma e Autodesk Tandem na situação de setembro de 2026. Nomes de comandos, opções e recursos mudam entre versões e atualizações. As informações deste capítulo vêm da ajuda oficial e de artigos de suporte da Autodesk, indicados nas referências; confira sempre a ajuda da versão instalada. Os nomes de menus e opções estão em inglês, como na documentação da Autodesk.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- explicar por que a configuração do exportador determina o conteúdo do IFC;
- diagnosticar um arquivo exportado: versão, unidades, georreferenciamento, classes, propriedades, classificação, geometria, camadas e GlobalId;
- configurar, no Revit, a classe IFC, o tipo predefinido, os conjuntos de propriedades e as coordenadas da exportação;
- escolher, no Civil 3D, entre a exportação herdada do AutoCAD e a extensão IFC 4.3, e conhecer seus arquivos de configuração;
- contornar a falta de exportação IFC no Plant 3D e aproveitar as camadas como dado;
- conhecer o papel do Navisworks, do Forma e do Tandem na leitura de modelos IFC;
- relacionar sintomas frequentes a suas causas e correções.

13.1 Introdução: exportar é traduzir

13.1.1 O exportador como tradutor

O software de autoria tem objetos próprios: famílias e categorias no Revit; alinhamentos, superfícies, corredores e redes no Civil 3D; componentes de catálogo e especificações no Plant 3D. Ao exportar, o **exportador** traduz cada objeto para uma classe IFC, com seus atributos, conjuntos de propriedades, material, classificação, geometria e posicionamento. Como toda tradução, ela pode perder ou distorcer sentido em seis pontos:

1. **Classe**: objeto sem mapeamento vira `IfcBuildingElementProxy` (Capítulo 3), e o tipo predefinido fica vazio;
2. **Propriedades**: não exportadas, exportadas com outro nome ou com outro tipo de valor (número gravado como texto), ou em outra unidade;
3. **Classificação e material**: não associados ou associados só a parte dos elementos;

4. **Georreferenciamento**: ausente, com sistema de coordenadas errado ou com escala incoerente (Capítulo 7);
5. **Geometria**: simplificada, convertida em malha ou ausente;
6. **Identidade**: GlobalId que muda a cada exportação, o que invalida comparações e apontamentos BCF (Capítulo 12).

SOFTWARE	CAMINHO IFC	ESQUEMAS	LEITURA DE IFC
Revit 2025	Exportador integrado, desenvolvido pela Autodesk em projeto de código aberto (revit-ifc)	IFC2x3 e IFC4 (lista da ajuda); IFC 4.3 nas versões recentes do exportador	Abrir e vincular IFC
Civil 3D 2026	`IFCEXPORT` (herdado do AutoCAD) e a extensão IFC 4.3 (`IFCInfraExport`)	IFC2x3 e IFC4 (`IFCEXPORT`); IFC4 e IFC4X3_ADD2 (extensão)	`IFCIMPORT` e `IFCInfraImport`
Plant 3D	Sem exportação IFC nativa	Via exportação para AutoCAD e outro programa	Não se aplica
Navisworks 2026	Usado como leitor de IFC	Leitor v4 com suporte a IFC 4.3	Revisão, federação e interferências
Autodesk Forma	Forma Site Design importa IFC e OBJ	Exportação IFC 4.3 em versão beta	Visualização e gestão de documentos
Autodesk Tandem	Recebe modelos Revit e IFC	Não documentado nas fontes consultadas	Gêmeo digital

Tabela 13.1: Caminhos IFC nos programas Autodesk considerados.

13.1.2 Três regras de trabalho

1. **Teste no início.** Exporte um modelo pequeno e representativo, diagnostique o resultado (seção 13.2), ajuste a configuração e registre-a no BEP. Os arquivos de configuração (conjuntos de propriedades, mapeamentos) passam a ser documentos controlados do projeto.
2. **Diagnostique cada entrega.** Antes de distribuir um IFC, rode o diagnóstico, a validação do esquema e a verificação IDS (Capítulo 12).
3. **Corrija na origem sempre que possível.** A correção por script (Capítulo 10) é legítima quando a origem não permite o resultado exigido, mas precisa ser refeita a cada exportação e deve ficar registrada.

13.2 Método de diagnóstico: o raio X do arquivo exportado

Visualizadores mostram o que conseguem ler, e cada um lê de um jeito. A referência é o **arquivo**. O script abaixo resume, num só relatório, os pontos da Tabela 13.2. Grave-o como `C:\IFC_Curso\exercicios\cap13\c13_01_diagnostico_exportacao.py`.

ASPECTO	O QUE O DIAGNÓSTICO MOSTRA	SINTOMA DE PERDA
Cabeçalho	Esquema, MVD declarada, programa e versão do exportador	Versão ou MVD diferentes das contratadas

ASPECTO	O QUE O DIAGNÓSTICO MOSTRA	SINTOMA DE PERDA
Unidades	Unidades de comprimento, área, volume e ângulo	Unidade inesperada; valores mil vezes maiores ou menores
Georreferenciamento	`IfcMapConversion`, `IfcRigidOperation`, `IfcProjectedCRS` (EPSG), escala; no IFC2x3, conjuntos `ePSet_`	Ausente; código EPSG errado; escala incoerente com a unidade
Estrutura espacial	Classes espaciais e elementos sem contêiner	Tudo no terreno; falta pavimento, parte de rodovia ou de ponte
Classes	Classes mais frequentes e proporção de *proxies*	Muitos *proxies*
Tipos	Tipo predefinido vazio; elementos sem `IfcTypeObject`	Classe certa, mas sem o tipo predefinido
Propriedades	Conjuntos padrão e customizados; prefixo `Pset_` fora do padrão; tipos de valor	Conjuntos ausentes; números gravados como texto
Classificação e material	Sistemas e cobertura	Sem classificação ou material
Geometria	Tipos de representação `Body`; elementos sem forma	Tudo em malha quando se esperava extrusão; elementos sem geometria
Camadas	Camadas CAD (`IfcPresentationLayerAssignment`)	Organização do desenho perdida ou, ao contrário, única fonte de identificação
GlobalId	Repetidos e com formato inválido	Identificadores duplicados

Tabela 13.2: O que o diagnóstico verifica.

```
# -*- coding: utf-8 -*-
"""
c13_01_diagnostico_exportacao.py | Capítulo 13: raio-X de um arquivo IFC exportado.

Resume, num relatório de texto, os pontos em que as exportações costumam perder informação:
cabeçalho (software e MVD), unidades, georreferenciamento, estrutura espacial, classes e
proxies, tipos predefinidos, tipos (IfcTypeObject), conjuntos de propriedades (padrão e
customizados), tipos de valor, classificação, material, representações geométricas, camadas
e GlobalId.
O arquivo IFC é aberto somente para leitura.

Uso: python c13_01_diagnostico_exportacao.py <arquivo.ifc> [--relatorio diagnostico.txt]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import argparse
from collections import Counter
from pathlib import Path
import ifcopenshell
import ifcopenshell.util.classification as util_class
import ifcopenshell.util.element as util_elem
import ifcopenshell.util.pset as util_pset
import ifcopenshell.util.unit as util_unid
import ifcopenshell.validate

TEMPLATES = {"IFC2X3": "IFC2X3", "IFC4": "IFC4", "IFC4X3_ADD2": "IFC4X3"}   # modelos de Pset por esquema


class Relatorio:
```

```
    """Acumula as linhas do diagnóstico para imprimir e gravar."""

    def __init__(self):
        self.linhas = []

    def secao(self, titulo):
        self.linhas.append(f"\n[{titulo}]")

    def item(self, texto):
        self.linhas.append(f"  {texto}")


def pct(parte, total):
    return f"{100 * parte / total:.0f}%" if total else "-"


def diagnosticar(modelo, rel):
    cab = modelo.header
    rel.secao("Cabeçalho")
    rel.item(f"Esquema: {modelo.schema_identifier}")
    rel.item(f"Descrição (MVD): {', '.join(cab.file_description.description)}")
    rel.item(f"Sistema de origem: {cab.file_name.originating_system}")
    rel.item(f"Pré-processador: {cab.file_name.preprocessor_version}")

    rel.secao("Unidades")
    for tipo in ("LENGTHUNIT", "AREAUNIT", "VOLUMEUNIT", "PLANEANGLEUNIT"):
        unidade = util_unid.get_project_unit(modelo, tipo)
        nome = util_unid.get_full_unit_name(unidade) if unidade else "não declarada"
        rel.item(f"{tipo}: {nome}")

    rel.secao("Georreferenciamento")
    achou = False
    for classe in ("IfcMapConversion", "IfcMapConversionScaled", "IfcRigidOperation", "IfcProjectedCRS"):
        try:
            instancias = modelo.by_type(classe, include_subtypes=False)
        except RuntimeError:                            # classe inexistente nesta versão do esquema
            continue
        for inst in instancias:
            achou = True
            if classe == "IfcProjectedCRS":
                rel.item(f"{classe}: Name={inst.Name}")
            elif classe in ("IfcMapConversion", "IfcMapConversionScaled"):
                escala = 1.0 if inst.Scale is None else inst.Scale        # Scale omitida vale 1,0
                rel.item(f"{classe}: E={inst.Eastings} N={inst.Northings} H={inst.OrthogonalHeight} "
                         f"Escala={escala}")
                if classe == "IfcMapConversionScaled":                    # fatores por eixo (IFC 4.3)
                    rel.item(f"Fatores: X={inst.FactorX} Y={inst.FactorY} Z={inst.FactorZ} "
                             f"(confira a escala resultante manualmente)")
                else:
                    esperada = util_unid.calculate_unit_scale(modelo)      # unidade do arquivo -> metro
                    if abs(escala - esperada) > 1e-9:                      # supõe mapa em metros
                        rel.item(f"ATENÇÃO: com a unidade do arquivo, a escala esperada seria {esperada}")
            else:
                rel.item(f"{classe}: presente")
    for obj in modelo.by_type("IfcProject") + modelo.by_type("IfcSite"):   # IFC2x3: conjuntos ePSet_
        for nome in util_elem.get_psets(obj):
            if nome.startswith("ePSet_"):
                achou = True
                rel.item(f"{nome} em {obj.is_a()} '{obj.Name}'")
    if not achou:
        rel.item("nenhuma informação de georreferenciamento encontrada")

    rel.secao("Estrutura espacial")
    espaciais = Counter(e.is_a() for e in modelo.by_type("IfcSpatialStructureElement"))
    rel.item(", ".join(f"{c}: {n}" for c, n in sorted(espaciais.items())) or "nenhum elemento espacial")
    elementos = [e for e in modelo.by_type("IfcElement") if not e.is_a("IfcFeatureElement")]
    soltos = [e for e in elementos if not util_elem.get_container(e) and not util_elem.get_aggregate(e)]
    rel.item(f"Elementos: {len(elementos)} | sem contêiner e sem conjunto: {len(soltos)}")

    rel.secao("Classes")
    classes = Counter(e.is_a() for e in elementos)
    for classe, n in classes.most_common(8):
        rel.item(f"{classe}: {n}")
```

```
    if len(classes) > 8:
        rel.item(f"... e mais {len(classes) - 8} classe(s)")
    proxies = classes.get("IfcBuildingElementProxy", 0)
    rel.item(f"Proxies: {proxies} ({pct(proxies, len(elementos))} dos elementos)")

    rel.secao("Tipos predefinidos e tipos")
    com_atributo = [e for e in elementos if hasattr(e, "PredefinedType")]   # no IFC2x3, poucas classes o têm
    sem_tipo_pred = [e for e in com_atributo if util_elem.get_predefined_type(e) in (None, "NOTDEFINED")]
    com_tipo = [e for e in elementos if util_elem.get_type(e)]
    rel.item(f"Sem tipo predefinido, na ocorrência e no tipo (vazio ou NOTDEFINED): {len(sem_tipo_pred)} "
             f"de {len(com_atributo)} elementos com o atributo PredefinedType")
    rel.item(f"Com tipo associado (IfcTypeObject): {len(com_tipo)} de {len(elementos)}")

    rel.secao("Conjuntos de propriedades e quantidades")
    modelos_pset = util_pset.get_template(TEMPLATES.get(modelo.schema_identifier, "IFC4"))
    nomes = Counter(p.Name for p in modelo.by_type("IfcPropertySet"))
    qtos = Counter(q.Name for q in modelo.by_type("IfcElementQuantity"))
    padrao = sorted(n for n in nomes if n and n.startswith("Pset_"))
    desconhecidos = [n for n in padrao if not modelos_pset.get_by_name(n)]
    rel.item(f"Conjuntos distintos: {len(nomes)} (padrão Pset_: {len(padrao)}; "
             f"outros: {len(nomes) - len(padrao)})")
    rel.item(f"Quantidades distintas: {len(qtos)} ({', '.join(sorted(q for q in qtos if q)) or 'nenhuma'})")
    if desconhecidos:
        rel.item(f"Com prefixo Pset_ mas fora do padrão desta versão: {', '.join(desconhecidos)}")
    tipos_valor = Counter(p.NominalValue.is_a() for p in modelo.by_type("IfcPropertySingleValue")
                          if p.NominalValue)
    total_valores = sum(tipos_valor.values())
    texto = tipos_valor.get("IfcLabel", 0) + tipos_valor.get("IfcText", 0)
    rel.item(f"Valores simples: {total_valores}; como texto (IfcLabel/IfcText): {texto} "
             f"({pct(texto, total_valores)})")
    rel.item("Tipos de valor: " + (", ".join(f"{t} {n}" for t, n in tipos_valor.most_common(6)) or "-"))

    rel.secao("Classificação e material")
    sistemas = [c.Name for c in modelo.by_type("IfcClassification")]
    classificados = [e for e in elementos if util_class.get_references(e)]   # inclui a herdada do tipo
    com_material = [e for e in elementos if util_elem.get_material(e)]
    rel.item(f"Sistemas de classificação: {', '.join(sistemas) or 'nenhum'}")
    rel.item(f"Elementos classificados (na ocorrência ou no tipo): {len(classificados)} de {len(elementos)}")
    rel.item(f"Elementos com material: {len(com_material)} de {len(elementos)}")

    rel.secao("Geometria")
    representacoes = Counter(r.RepresentationType for r in modelo.by_type("IfcShapeRepresentation")
                             if r.RepresentationIdentifier == "Body")
    resumo = ", ".join(f"{t} {n}" for t, n in representacoes.most_common())
    rel.item("Representações Body: " + (resumo or "nenhuma"))
    sem_forma = [e for e in elementos if not e.Representation]
    rel.item(f"Elementos sem representação: {len(sem_forma)}")

    rel.secao("Camadas (layers)")
    camadas = sorted(c.Name for c in modelo.by_type("IfcPresentationLayerAssignment") if c.Name)
    lista = ", ".join(camadas[:8]) + (", ..." if len(camadas) > 8 else "")
    rel.item(f"Camadas: {len(camadas)}" + (f" ({lista})" if camadas else ""))

    rel.secao("GlobalId")
    gids = Counter(r.GlobalId for r in modelo.by_type("IfcRoot"))
    repetidos = [g for g, n in gids.items() if n > 1]
    invalidos = [g for g in gids if ifcopenshell.validate.validate_guid(g)]
    rel.item(f"Objetos: {sum(gids.values())} | repetidos: {len(repetidos)} | "
             f"com formato inválido: {len(invalidos)}")


def main():
    leitor = argparse.ArgumentParser(description="Raio-X de um arquivo IFC exportado.")
    leitor.add_argument("arquivo")
    leitor.add_argument("--relatorio", help="grava o diagnóstico também em arquivo de texto")
    args = leitor.parse_args()

    modelo = ifcopenshell.open(args.arquivo)
    rel = Relatorio()
    rel.linhas.append(f"Diagnóstico de {Path(args.arquivo).name}")
    diagnosticar(modelo, rel)
    texto = "\n".join(rel.linhas)
    print(texto)
```

```
    if args.relatorio:
        Path(args.relatorio).write_text(texto + "\n", encoding="utf-8")


if __name__ == "__main__":
    main()
```

Execução com a amostra rodoviária em IFC 4.3:

```
cd C:\IFC_Curso\exercicios\cap13
python c13_01_diagnostico_exportacao.py "C:\IFC_Curso\amostras\bSI\Certification-datasets-main\IFC 4.3.2.0 (IFC
4.3 ADD2)\Simple-Scene\Infra-Road.ifc"
```

```
Diagnóstico de Infra-Road.ifc

[Cabeçalho]
  Esquema: IFC4X3_ADD2
  Descrição (MVD): ViewDefinition [ReferenceView]
  Sistema de origem: BIM_Tools - Sketchup_IFC_manager - 5.6.0
  Pré-processador: Sketchup-IFC-manager 5.6.0 / SketchUp 2026 (26.2.242)

[Unidades]
  LENGTHUNIT: MILLIMETRE
  AREAUNIT: SQUARE_METRE
  VOLUMEUNIT: CUBIC_METRE
  PLANEANGLEUNIT: não declarada

[Georreferenciamento]
  IfcMapConversion: E=729011.2258823584 N=9063960.607644705 H=0.0 Escala=0.001
  IfcProjectedCRS: Name=EPSG:32760

[Estrutura espacial]
  IfcRoad: 5, IfcRoadPart: 26, IfcSite: 6
  Elementos: 35 | sem contêiner e sem conjunto: 0

[Classes]
  IfcCourse: 16
  IfcEarthworksFill: 16
  IfcElementAssembly: 2
  IfcBuildingElementProxy: 1
  Proxies: 1 (3% dos elementos)

[Tipos predefinidos e tipos]
  Sem tipo predefinido, na ocorrência e no tipo (vazio ou NOTDEFINED): 0 de 35 elementos com o atributo
PredefinedType
  Com tipo associado (IfcTypeObject): 19 de 35

[Conjuntos de propriedades e quantidades]
  Conjuntos distintos: 0 (padrão Pset_: 0; outros: 0)
  Quantidades distintas: 0 (nenhuma)
  Valores simples: 0; como texto (IfcLabel/IfcText): 0 (-)
  Tipos de valor: -

[Classificação e material]
  Sistemas de classificação: CCI Construction
  Elementos classificados (na ocorrência ou no tipo): 0 de 35
  Elementos com material: 33 de 35

[Geometria]
  Representações Body: Tessellation 45, SurfaceModel 20
  Elementos sem representação: 2

[Camadas (layers)]
  Camadas: 0

[GlobalId]
  Objetos: 147 | repetidos: 0 | com formato inválido: 0
```

Leitura do diagnóstico:

- o cabeçalho mostra que a amostra foi exportada pelo SketchUp, com o complemento IFC Manager, declarando a MVD Reference View;
- o georreferenciamento está completo e coerente: `IfcMapConversion` com escala 0,001, compatível com o arquivo em milímetros e o mapa em metros, e `IfcProjectedCRS` com código EPSG;
- a estrutura espacial usa as classes de rodovia do IFC 4.3 (`IfcRoad`, `IfcRoadPart`), e as camadas do pavimento rodoviário são `IfcCourse`, classe que só existe no IFC 4.3;
- não há nenhum conjunto de propriedades: um BEP que exija propriedades não seria atendido;
- a classificação existe (sistema CCI Construction), mas está associada aos elementos espaciais, não aos elementos construtivos;
- a geometria está toda em malhas (`Tessellation` e `SurfaceModel`).

A amostra de arquitetura exportada em IFC2x3 (`python c13_01_diagnostico_exportacao.py "<pasta IFC 2.3.0.1>\Simple-Scene\Building-Architecture.ifc"`) mostra outro efeito da tradução:

```
[Classes]
  IfcBuildingElementProxy: 5
  IfcWall: 4
  IfcSlab: 3
  IfcRoof: 1
  IfcFurnishingElement: 1
  Proxies: 5 (36% dos elementos)

[Tipos predefinidos e tipos]
  Sem tipo predefinido, na ocorrência e no tipo (vazio ou NOTDEFINED): 0 de 3 elementos com o atributo
PredefinedType
  Com tipo associado (IfcTypeObject): 13 de 14

[Conjuntos de propriedades e quantidades]
  Conjuntos distintos: 1 (padrão Pset_: 1; outros: 0)
  Quantidades distintas: 2 (Qto_SlabBaseQuantities, Qto_WallBaseQuantities)
  Valores simples: 6; como texto (IfcLabel/IfcText): 4 (67%)
  Tipos de valor: IfcLabel 4, IfcBoolean 2
```

A versão IFC 4.3 dessa amostra tem 2 *proxies* (os marcos `origin` e `geo-reference`); a versão IFC2x3 tem 5. Os outros três são a chaminé, o lastro de areia e o volume bruto da casa (`house - gross volume`), exportados em IFC 4.3 como `IfcChimney`, `IfcEarthworksFill` e `IfcSpatialZone`: essas classes não existem no IFC2x3 (`IfcChimney` e `IfcSpatialZone` surgiram no IFC4, e `IfcEarthworksFill`, no IFC 4.3). Na tubulação, o efeito é outro: `IfcPipeSegment` e `IfcValve` também não existem no IFC2x3, e tubos e válvulas são gravados com as classes genéricas `IfcFlowSegment` e `IfcFlowController`, com o tipo específico (`IfcPipeSegmentType`, `IfcValveType`) na definição de tipo (Capítulo 6). **A escolha da versão limita as classes disponíveis**: para infraestrutura e plantas industriais, use IFC4 ou IFC 4.3, conforme o que o contrato exigir e o exportador suportar.

13.3 Revit 2025

13.3.1 Onde fica e o que exporta

A exportação fica em **File > Export > IFC**. O diálogo oferece configurações salvas (*setups*) e abas com as opções. O exportador do Revit é desenvolvido pela Autodesk num projeto de código aberto (repositório revit-ifc, no GitHub), e suas atualizações trazem correções e recursos novos entre as versões do Revit.

A ajuda do Revit 2025 lista as versões IFC 2x2 Coordination View, IFC 2x3 Coordination View, IFC 2x3 Coordination View 2.0 (padrão), IFC 2x3 GSA Concept Design BIM 2010, IFC 2x2 Singapore BCA e-Plan Check, IFC2x3 Basic FM Handover View, IFC2x3 COBie 2.4 Design Deliverable View, IFC4 Reference View e IFC4 Design Transfer View, e os formatos IFC, IFC XML, IFC compactado e IFC XML compactado. Essa lista não inclui o IFC 4.3, mas o manual IFC da Autodesk registra, entre as novidades do Revit 2025, o suporte à versão oficial IFC 4.3.2.0 e a escolha do tipo de instalação (*facility type*) além de edificação, e as notas das versões do exportador no GitHub citam requisitos de troca da Reference View do IFC 4.3. Confira, no diálogo da sua instalação, o nome exato da opção e a versão do exportador.

13.3.2 Classe IFC e tipo predefinido

Desde o Revit 2023, a classe e o tipo predefinido são definidos por parâmetros próprios, que substituíram os antigos parâmetros compartilhados `IfcExportAs` e `IfcExportType`:

PARÂMETRO	NÍVEL	USO
`Export to IFC As`	Ocorrência	Classe IFC da ocorrência, com o tipo predefinido opcional: `IfcValve.CHECK`
`IFC Predefined Type`	Ocorrência	Tipo predefinido, quando a classe vem do mapeamento da categoria
`Export Type to IFC As`	Tipo	Classe IFC do tipo
`Type IFC Predefined Type`	Tipo	Tipo predefinido do tipo

Tabela 13.3: Parâmetros de classe e tipo predefinido no Revit (nomes conforme os artigos de suporte da Autodesk; o manual IFC da Autodesk grafa o último como `IFC Type Predefined Type`; confira na sua interface).

Exemplos de valores para uma planta industrial, com tipos predefinidos conferidos no esquema IFC4 e IFC 4.3:

FAMÍLIA DO REVIT	VALOR DE `EXPORT TO IFC AS`
Válvula de retenção	`IfcValve.CHECK`
Bomba centrífuga de sucção frontal	`IfcPump.ENDSUCTION`
Tanque de armazenamento	`IfcTank.STORAGE`
Filtro tipo Y	`IfcFilter.STRAINER`
Curva de tubulação	`IfcPipeFitting.BEND`

Tabela 13.4: Exemplos de mapeamento por parâmetro.

Para o padrão de cada categoria, sem preencher parâmetro por parâmetro, use o mapeamento de categorias do exportador (tabela de mapeamento na configuração de exportação). Um artigo de suporte da Autodesk registra que, a partir do Revit 2025.4.1, o botão de edição (`...`) desses parâmetros não abre; nesse caso, digite o valor diretamente.

13.3.3 Conjuntos de propriedades

A aba **Property Sets** do diálogo de exportação reúne as opções: exportar os conjuntos do Revit (*Export Revit property sets*), os conjuntos comuns do IFC (*Export IFC common property sets*), as quantidades básicas (*Export base quantities*), tabelas como conjuntos (*Export schedules as property sets*), conjuntos definidos pelo usuário (*Export user defined property sets*), a tabela de mapeamento de parâmetros (*Export parameter mapping table*) e as configurações de classificação (*Classification Settings*).

Para os metadados do BEP, o recurso central são os **conjuntos definidos pelo usuário**: um arquivo de texto que diz quais conjuntos criar, em quais classes e com quais parâmetros. O formato, segundo o arquivo de exemplo distribuído pela Autodesk (`DefaultUserDefinedParameterSets.txt`) e a ajuda do Revit, é:

```
PropertySet:<TAB><nome do conjunto><TAB>I<TAB><classes IFC separadas por vírgula>
<TAB><nome da propriedade><TAB><tipo de dado><TAB><parâmetro do Revit, se o nome for diferente>
```

Regras: colunas separadas por tabulação; arquivo em UTF-8; linhas iniciadas por `#` são comentários; o nome do conjunto não deve começar por `Pset_`, prefixo reservado aos conjuntos padrão. Segundo a ajuda do Revit, a partir da versão 2024.1 a indicação I (ocorrência) ou T (tipo) deixou de ser considerada, e a lista de classes deve incluir também as classes de tipo, como em `IfcColumn, IfcColumnType`. Pelos nomes da lista do arquivo de exemplo, o tipo de dado corresponde, na maioria dos casos, ao nome do tipo IFC sem o prefixo `Ifc` e sem o sufixo `Measure`: `Label`, `Text`, `Length`, `Pressure`, `ThermodynamicTemperature`, `Mass`, `Boolean`, entre outros. O script abaixo aplica essa regra e confere o resultado com a lista.

Escrever esse arquivo à mão é fonte de erro. O script abaixo o gera a partir de uma tabela de requisitos em CSV, conferindo no esquema IFC as classes e os tipos. Grave a tabela como `requisitos_psets.csv` (separador ponto e vírgula, UTF-8):

```
Conjunto;Propriedade;TipoIFC;Classes;ParametroRevit
ARV_Identificacao;TAG;IfcLabel;IfcPipeSegment,IfcValve,IfcPump,IfcTank;ARV_TAG
ARV_Identificacao;Revisao;IfcLabel;IfcPipeSegment,IfcValve,IfcPump,IfcTank;ARV_REVISAO
ARV_Processo;Fluido;IfcLabel;IfcPipeSegment,IfcValve,IfcPump;ARV_FLUIDO
ARV_Processo;PressaoProjeto;IfcPressureMeasure;IfcPipeSegment,IfcValve,IfcPump;ARV_PRESSAO_PROJETO
ARV_Processo;TemperaturaProjeto;IfcThermodynamicTemperatureMeasure;IfcPipeSegment,IfcValve,IfcPump;ARV_TEMPERATU
RA_PROJETO
ARV_Mecanica;Peso;IfcMassMeasure;IfcValve,IfcPump,IfcTank;ARV_PESO
```

E o script como `c13_02_gerar_psets_revit.py`:

```
# -*- coding: utf-8 -*-
"""
c13_02_gerar_psets_revit.py | Capítulo 13: gera o arquivo de conjuntos de propriedades definidos
pelo usuário do exportador IFC do Revit a partir de uma tabela de requisitos do BEP (CSV).

Entrada (CSV, separador ';', UTF-8): Conjunto;Propriedade;TipoIFC;Classes;ParametroRevit
Saída: arquivo de texto no formato do exemplo distribuído pela Autodesk
(DefaultUserDefinedParameterSets.txt), com colunas separadas por TAB:
```

```
    PropertySet:<TAB><nome do conjunto><TAB>I<TAB><classes IFC separadas por vírgula>
    <TAB><nome da propriedade><TAB><tipo de dado><TAB><parâmetro do Revit>

O script confere, antes de gravar, se as classes e os tipos IFC existem no esquema informado
e se o tipo de dado resultante está na lista aceita pelo exportador.
Atenção: o arquivo gerado segue o formato documentado; a importação no Revit não foi testada
neste material. Compare-o com o arquivo de exemplo da sua instalação antes de usar.

Uso: python c13_02_gerar_psets_revit.py <requisitos.csv> [--saida psets_revit.txt] [--esquema IFC4]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import argparse
import csv
from pathlib import Path
import ifcopenshell.ifcopenshell_wrapper as wrapper

# Tipos de dado aceitos pelo exportador (lista do arquivo de exemplo da Autodesk, repositório revit-ifc)
TIPOS_REVIT = {
    "Acceleration", "AngularVelocity", "Area", "AreaDensity", "Boolean", "ClassificationReference",
    "ColorTemperature", "Count", "Currency", "DynamicViscosity", "ElectricCurrent", "ElectricVoltage",
    "Energy", "ElectricalEfficacy", "Force", "Frequency", "HeatFluxDensity", "HeatingValue", "Identifier",
    "Illuminance", "Integer", "IonConcentration", "IsothermalMoistureCapacity", "Label", "Length",
    "LinearForce", "LinearMoment", "LinearStiffness", "LinearVelocity", "Logical", "LuminousFlux",
    "LuminousIntensity", "Mass", "MassDensity", "MassFlowRate", "MassPerLength", "ModulusOfElasticity",
    "MoistureDiffusivity", "MomentOfInertia", "NormalisedRatio", "Numeric", "PlanarForce", "PlaneAngle",
    "PositiveLength", "PositivePlaneAngle", "PositiveRatio", "Power", "Pressure", "Ratio", "Real",
    "RotationalFrequency", "SoundPower", "SoundPressure", "SpecificHeatCapacity", "Text",
    "ThermalConductivity", "ThermalExpansionCoefficient", "ThermalResistance", "ThermalTransmittance",
    "ThermodynamicTemperature", "Time", "Torque", "VaporPermeability", "Volume", "VolumetricFlowRate",
    "WarpingConstant",
}


def tipo_revit(tipo_ifc):
    """IfcPressureMeasure -> Pressure; IfcLabel -> Label (regra de nomes do exportador)."""
    nome = tipo_ifc[3:] if tipo_ifc.startswith("Ifc") else tipo_ifc
    return nome[:-len("Measure")] if nome.endswith("Measure") else nome


def existe(esquema, nome):
    try:
        esquema.declaration_by_name(nome)
        return True
    except Exception:
        return False


def main():
    leitor = argparse.ArgumentParser(description="Gera os conjuntos definidos pelo usuário do Revit.")
    leitor.add_argument("requisitos")
    leitor.add_argument("--saida", default="psets_revit.txt")
    leitor.add_argument("--esquema", default="IFC4",
                        help="versão usada na exportação: IFC4 ou IFC4X3_ADD2")
    args = leitor.parse_args()
    esquema = wrapper.schema_by_name(args.esquema)

    conjuntos, erros = {}, []                              # conjunto -> {"classes": [...], "props": [...]}
    with open(args.requisitos, encoding="utf-8-sig", newline="") as arquivo:
        for n, linha in enumerate(csv.DictReader(arquivo, delimiter=";"), start=2):
            tipo = tipo_revit(linha["TipoIFC"].strip())
            if not existe(esquema, linha["TipoIFC"].strip()):
                erros.append(f"linha {n}: o tipo {linha['TipoIFC']} não existe no {args.esquema}")
            elif tipo not in TIPOS_REVIT:
                erros.append(f"linha {n}: o tipo de dado {tipo} não é aceito pelo exportador")
            conjunto = conjuntos.setdefault(linha["Conjunto"].strip(), {"classes": [], "props": []})
            for classe in (c.strip() for c in linha["Classes"].split(",")):
                for nome in (classe, classe + "Type"):      # ocorrência e tipo (ex.: IfcPump, IfcPumpType)
                    if not existe(esquema, nome):
                        erros.append(f"linha {n}: a classe {nome} não existe no {args.esquema}")
                    elif nome not in conjunto["classes"]:
                        conjunto["classes"].append(nome)
            prop = (linha["Propriedade"].strip(), tipo, linha["ParametroRevit"].strip())
            iguais = [p for p in conjunto["props"] if p[0] == prop[0]]
```

```
            if not iguais:
                conjunto["props"].append(prop)
            elif iguais[0] != prop:                 # mesma propriedade com tipo ou parâmetro diferente
                erros.append(f"linha {n}: {prop[0]} já definida em {linha['Conjunto']} "
                             f"com outro tipo ou parâmetro")

    for nome in conjuntos:
        if nome.startswith("Pset_"):
            erros.append(f"conjunto {nome}: o prefixo Pset_ é reservado aos conjuntos padrão")
    if erros:
        raise SystemExit("Requisitos com problemas:\n  " + "\n  ".join(erros))

    linhas = ["# Conjuntos de propriedades definidos pelo usuário (exportador IFC do Revit)",
              f"# Gerado por c13_02_gerar_psets_revit.py a partir de {Path(args.requisitos).name}"]
    for nome, dados in conjuntos.items():
        linhas.append("\t".join(["PropertySet:", nome, "I", ", ".join(dados["classes"])]))
        for propriedade, tipo, parametro in dados["props"]:
            linhas.append("\t".join(["", propriedade, tipo, parametro]))
    Path(args.saida).write_text("\n".join(linhas) + "\n", encoding="utf-8")
    total = sum(len(d["props"]) for d in conjuntos.values())
    print(f"Arquivo gravado: {args.saida} | conjuntos: {len(conjuntos)} | propriedades: {total}")
    print(Path(args.saida).read_text(encoding="utf-8").replace("\t", " -> "))


if __name__ == "__main__":
    main()
```

Execução e saída obtida (na impressão de conferência, cada tabulação aparece como `->`):

```
python c13_02_gerar_psets_revit.py requisitos_psets.csv
```

```
Arquivo gravado: psets_revit.txt | conjuntos: 3 | propriedades: 6
# Conjuntos de propriedades definidos pelo usuário (exportador IFC do Revit)
# Gerado por c13_02_gerar_psets_revit.py a partir de requisitos_psets.csv
PropertySet: -> ARV_Identificacao -> I -> IfcPipeSegment, IfcPipeSegmentType, IfcValve, IfcValveType, IfcPump,
IfcPumpType, IfcTank, IfcTankType
 -> TAG -> Label -> ARV_TAG
 -> Revisao -> Label -> ARV_REVISAO
PropertySet: -> ARV_Processo -> I -> IfcPipeSegment, IfcPipeSegmentType, IfcValve, IfcValveType, IfcPump,
IfcPumpType
 -> Fluido -> Label -> ARV_FLUIDO
 -> PressaoProjeto -> Pressure -> ARV_PRESSAO_PROJETO
 -> TemperaturaProjeto -> ThermodynamicTemperature -> ARV_TEMPERATURA_PROJETO
PropertySet: -> ARV_Mecanica -> I -> IfcValve, IfcValveType, IfcPump, IfcPumpType, IfcTank, IfcTankType
 -> Peso -> Mass -> ARV_PESO
```

> **ATENÇÃO**
>
> O arquivo gerado segue o formato documentado pela Autodesk, mas a importação no Revit não foi testada neste material. Antes de usá-lo num projeto, compare-o com o arquivo de exemplo da sua instalação, exporte um modelo de teste e confira o resultado com o diagnóstico da seção 13.2 e com o script de exportação para planilha do Capítulo 9. Confira também as unidades: o valor exportado de um parâmetro de pressão depende das unidades do projeto e do exportador.

A **tabela de mapeamento de parâmetros** (*parameter mapping table*) resolve outro caso: preencher uma propriedade de um conjunto padrão com um parâmetro de nome diferente, desde que os tipos de dado sejam compatíveis. A sintaxe, segundo o manual IFC da Autodesk, é `<conjunto padrão><TAB><propriedade IFC><TAB><parâmetro do Revit>`.

13.3.4 Classificação

As **Classification Settings** guardam a origem, a edição e os demais dados do sistema de classificação principal. Segundo o manual IFC da Autodesk, os códigos são lidos dos parâmetros `ClassificationCode`, `ClassificationCode(2)` até `ClassificationCode(10)`, com o valor no formato `[Sistema]Código:Título`, e o código de montagem (*Assembly Code*, Uniformat) é exportado automaticamente. Para a NBR 15965 (Capítulo 5), um valor como `[ABNT NBR 15965]3E.EX.02:Tubo` (com o código fictício do exemplo do Capítulo 10) seguiria esse formato.

13.3.5 Coordenadas e GlobalId

A opção **Coordinate Base** define a origem da exportação: *Shared Coordinates*, *Survey Point*, *Project Base Point*, *Internal Origin*, *Project Base Point oriented in True North* ou *Internal Origin oriented in True North*. Para georreferenciar, a documentação do exportador recomenda o ponto de levantamento ou o ponto base do projeto, e o sistema de coordenadas projetado é informado por código EPSG, recurso que, segundo a ajuda, exige a exportação em IFC4. Para obras no Brasil, o código é o do SIRGAS 2000 na zona UTM do projeto (Capítulo 7).

A opção *Store the IFC GUID in an element parameter after export* grava, em parâmetros dos elementos, dos tipos e das informações do projeto, o GlobalId usado na exportação. Isso facilita localizar no Revit o elemento apontado num BCF ou numa planilha de não conformidades. Confirme a estabilidade dos identificadores entre exportações com o script do exercício 9.

13.3.6 Abrir e vincular IFC

O Revit abre (**Open IFC**) ou vincula (**Link IFC**) arquivos IFC. O mapeamento das classes IFC para categorias do Revit fica no arquivo `importIFCClassMapping.txt`, em que o valor `DontImport` descarta uma classe. Segundo o manual IFC da Autodesk, o vínculo cria três arquivos ao lado do IFC (`.ifc.RVT`, `.log.html` e `.sharedparameters.txt`); até o Revit 2025, o vínculo é posicionado origem a origem; e arquivos IFC 4.3 podem ser vinculados, mas não abertos.

13.4 Civil 3D 2026

13.4.1 Duas vias de exportação

ASPECTO	`IFCEXPORT` (HERDADO DO AUTOCAD)	EXTENSÃO IFC 4.3
Acesso	Menu do aplicativo > Export > IFC; `-IFCEXPORT` em scripts	Aba Add-ins > painel IFC Infrastructure > Export to IFC
Instalação	Nativa	Complemento gratuito, baixado da Autodesk Account
Esquemas	IFC2x3 e IFC4	IFC4 e IFC4X3_ADD2

ASPECTO	`IFCEXPORT` (HERDADO DO AUTOCAD)	EXTENSÃO IFC 4.3
Objetos do Civil 3D	Não exporta objetos de infraestrutura: superfícies, corredores e redes precisam antes ser convertidos em sólidos 3D	Exporta sistemas de coordenadas, pontos COGO e grupos, alinhamentos, superfícies TIN, *feature lines*, pontes e partes, *sites* do Civil 3D (que não devem ser confundidos com o `IfcSite`, o terreno do IFC) e lotes, corredores, redes de pressão e de gravidade, sólidos e blocos que contenham esses objetos
Georreferenciamento	Com IFC4, o sistema de coordenadas do desenho é incluído automaticamente	Recomenda sistema de coordenadas com código EPSG

Tabela 13.5: As duas vias de exportação do Civil 3D (segundo a ajuda do Civil 3D 2026 e artigos de suporte da Autodesk).

Para rodovias em IFC 4.3, com alinhamentos e corredores, a via é a extensão. Uma nota de suporte da Autodesk chegou a informar que a extensão para o Civil 3D 2026 ainda não estava disponível; revendedores registraram depois a publicação de versões para 2026. Verifique na Autodesk Account a versão disponível e, no Civil 3D, a instalada.

Comandos da extensão, conforme a ajuda:

COMANDO	FUNÇÃO
`IFCInfraExport`	Exporta o desenho
`IFCInfraExportSelected`	Exporta os objetos selecionados
`IFCInfraImport`	Importa um IFC
`IFCInfraSaveConfig`	Grava a configuração do projeto
`IFCInfraSaveMappingConfig`	Grava a configuração de mapeamento
`IFCInfraSavePropertyTemplates`	Grava os modelos de conjuntos de propriedades
`IFCInfraSetProjectBasePoint`	Define o ponto base do projeto
`IFCInfraAssignLinearPositioning` / `IFCInfraRemoveLinearPositioning`	Atribuem ou removem posicionamento linear
`IFCInfraHelp`	Abre a ajuda da extensão

Tabela 13.6: Comandos da extensão IFC 4.3 do Civil 3D.

13.4.2 Configuração e mapeamento

A extensão lê dois arquivos JSON: `IfcInfraConfiguration.json` (configurações do projeto) e `IfcInfraExportMapping.json` (regras de mapeamento de objetos para classes IFC). Eles são procurados, nesta ordem, em `C:\ProgramData\Autodesk\Civil3D\`, na pasta do desenho, na pasta do IFC e em arquivos com o prefixo do nome do desenho ou do IFC. Isso permite uma configuração padrão da empresa e ajustes por projeto.

O mapeamento é aplicado em quatro etapas: regras internas; nome da camada; códigos de objeto do corredor (dos pontos, das ligações, os *links*, e das formas, os *shapes*, das seções-tipo); e mapeamento de conjuntos de propriedades. Uma regra de exemplo da ajuda é `"IfcExportAs": "IfcAnnotation.StringLine"`; a ajuda não detalha onde o texto após o ponto é gravado (`StringLine` não é um tipo predefinido de `IfcAnnotation`), o que pode ser conferido no arquivo exportado. Objeto sem regra vira `IfcBuildingElementProxy`.

Os **conjuntos de propriedades** do Civil 3D são exportados. O comando `IFCInfraSavePropertyTemplates` grava um arquivo de definição de propriedades IFC e um mapa de parâmetros do Civil 3D. Segundo a ajuda, dados de propriedades não são acrescentados a objetos em camadas congeladas ou bloqueadas, nem dentro de definições de bloco: se uma propriedade "sumiu" na exportação, confira a camada do objeto.

A ajuda da extensão não traz uma tabela de qual classe IFC cada objeto recebe. Use o diagnóstico da seção 13.2 no seu arquivo exportado para conhecer o resultado real, e registre-o no BEP.

13.4.3 Georreferenciamento

A extensão recomenda um sistema de coordenadas baseado em EPSG no desenho e emite aviso para outros sistemas; o IFC usa as unidades do desenho. A ajuda não informa quais entidades de georreferenciamento são gravadas. Há relatos de usuários, no fórum da Autodesk, de exportações IFC 4.3 com `IfcRigidOperation` em vez de `IfcMapConversion`, e a Autodesk publicou um artigo sobre localização incorreta após a exportação, corrigida por ajuste no `IfcInfraConfiguration.json`. O diagnóstico da seção 13.2 identifica qual entidade foi gravada e se a escala é coerente com a unidade.

13.4.4 Limitações documentadas

- só o desenho ativo é exportado;
- as subpartes de corredores recebem **novos GlobalId a cada exportação**; os demais objetos guardam o GlobalId num valor de conjunto de propriedades;
- um alinhamento com descontinuidades (*gaps*) é omitido na exportação IFC 4.3;
- o pé topográfico americano (*US survey foot*) é suportado, mas outras ferramentas podem não lê-lo.

A segunda limitação tem consequência direta: para as subpartes de corredores, a comparação de revisões e os apontamentos BCF (Capítulo 12) não funcionam por GlobalId. Nesses casos, use chaves estáveis gravadas como propriedade (TAG, estaca, código do elemento) para vincular dados e apontamentos.

13.4.5 Onde entram os metadados do BEP

Há dois caminhos para os metadados do BEP num IFC exportado do Civil 3D:

1. **Na origem**: os dados ficam nos conjuntos de propriedades do Civil 3D (preenchidos manualmente, por Dynamo ou por outra rotina) e são exportados pela extensão;
2. **Depois da exportação**: o IFC é preenchido a partir da planilha, por TAG, com o script do Capítulo 10.

O primeiro mantém o dado no modelo de autoria; o segundo independe do exportador e dá o registro linha a linha. Em ambos, a verificação final é a mesma: diagnóstico e IDS.

13.5 Plant 3D

Segundo o suporte da Autodesk, o AutoCAD Plant 3D **não tem exportação IFC nativa**. A solução oficial é exportar o modelo para AutoCAD comum (comando `EXPORTTOAUTOCAD`) e gerar o IFC em outro programa (AutoCAD Architecture, AutoCAD MEP ou Civil 3D), com perda dos dados de planta: especificações, catálogos e propriedades dos componentes. As novidades das versões 2026 e 2027 não mencionam IFC. Para estruturas metálicas modeladas com o Advance Steel, o comando de exportação IFC do Advance Steel exporta apenas os elementos do Advance Steel. Há complementos de terceiros para exportação IFC a partir do Plant 3D; eles não foram avaliados neste material.

Pelo caminho oficial, o que se pode esperar no IFC é a geometria dos sólidos, sem os dados de planta. O que mais chega ao arquivo depende do programa que faz a exportação final: a classe dos elementos e a preservação das **camadas** (`IfcPresentationLayerAssignment`) devem ser conferidas com o diagnóstico da seção 13.2 no arquivo real. Se as camadas forem preservadas e o modelo de tubulação estiver organizado com uma camada por linha, a camada pode ser o único vínculo entre cada sólido e a lista de linhas do projeto. O script a seguir cria um arquivo de estudo que simula esse resultado. Grave-o como `c13_03a_exportacao_simulada.py`.

```
# -*- coding: utf-8 -*-
"""
c13_03a_exportacao_simulada.py | Capítulo 13: cria um arquivo de estudo que simula a exportação
de sólidos de um desenho CAD (por exemplo, um modelo de tubulação levado a um programa com
exportação IFC genérica): todos os elementos como IfcBuildingElementProxy, com o mesmo nome
genérico ("Sólido"), e a única informação de origem é a camada (layer) de cada sólido. O nome é
obrigatório em proxies no IFC4 (regra HasObjectName). É uma simulação didática, não a
reprodução do resultado de um exportador específico.

Uso: python c13_03a_exportacao_simulada.py [saida.ifc]    (padrão: exportacao_cad.ifc)
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import numpy as np
import ifcopenshell
import ifcopenshell.api.aggregate
import ifcopenshell.api.context
import ifcopenshell.api.geometry
import ifcopenshell.api.layer
import ifcopenshell.api.project
import ifcopenshell.api.root
import ifcopenshell.api.spatial
import ifcopenshell.api.unit
from ifcopenshell.util.shape_builder import ShapeBuilder

# (camada, forma, x, y, z em metros): trechos de tubo, equipamentos, suportes e um sólido na camada 0
PECAS = [
    ("TUB-L001", "tubo", 0.0, 0.0, 1.0), ("TUB-L001", "tubo", 2.0, 0.0, 1.0),
    ("TUB-L001", "tubo", 4.0, 0.0, 1.0),
    ("TUB-L002", "tubo", 0.0, 3.0, 1.0), ("TUB-L002", "tubo", 2.0, 3.0, 1.0),
    ("MEC-EQUIPAMENTOS", "caixa", 7.0, 0.0, 0.0), ("MEC-EQUIPAMENTOS", "caixa", 7.0, 3.0, 0.0),
    ("EST-SUPORTES", "caixa", 1.0, 0.0, 0.0), ("EST-SUPORTES", "caixa", 1.0, 3.0, 0.0),
    ("0", "caixa", 10.0, 0.0, 0.0),
]

saida = sys.argv[1] if len(sys.argv) > 1 else "exportacao_cad.ifc"
modelo = ifcopenshell.api.project.create_file(version="IFC4")
criar = ifcopenshell.api.root.create_entity
projeto = criar(modelo, ifc_class="IfcProject", name="Exportação CAD simulada")
ifcopenshell.api.unit.assign_unit(modelo)                          # milímetros (padrão)
modelo3d = ifcopenshell.api.context.add_context(modelo, context_type="Model")
corpo = ifcopenshell.api.context.add_context(modelo, context_type="Model", context_identifier="Body",
                                             target_view="MODEL_VIEW", parent=modelo3d)
terreno = criar(modelo, ifc_class="IfcSite", name="Terreno")
edificio = criar(modelo, ifc_class="IfcBuilding", name="Unidade")
```

```
pavimento = criar(modelo, ifc_class="IfcBuildingStorey", name="Nível 0")
ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)
ifcopenshell.api.aggregate.assign_object(modelo, products=[edificio], relating_object=terreno)
ifcopenshell.api.aggregate.assign_object(modelo, products=[pavimento], relating_object=edificio)

construtor = ShapeBuilder(modelo)                                  # dimensões na unidade do arquivo (mm)
camadas, elementos = {}, []
for camada, forma, x, y, z in PECAS:
    elem = criar(modelo, ifc_class="IfcBuildingElementProxy", name="Sólido")   # nome genérico
    matriz = np.eye(4)
    matriz[:3, 3] = (x, y, z)
    ifcopenshell.api.geometry.edit_object_placement(modelo, product=elem, matrix=matriz)   # metros
    if forma == "tubo":                                             # cilindro DN100 de 2 m ao longo de X
        solido = construtor.extrude(construtor.circle(radius=57.15), magnitude=2000.0,
                                    position_z_axis=(1.0, 0.0, 0.0), position_x_axis=(0.0, 1.0, 0.0))
    else:                                                           # caixa de 1,0 x 1,0 x 1,5 m
        solido = construtor.extrude(construtor.rectangle(size=(1000.0, 1000.0)), magnitude=1500.0)
    representacao = construtor.get_representation(corpo, [solido])
    ifcopenshell.api.geometry.assign_representation(modelo, product=elem, representation=representacao)
    if camada not in camadas:
        camadas[camada] = ifcopenshell.api.layer.add_layer(modelo, name=camada)
    ifcopenshell.api.layer.assign_layer(modelo, items=[representacao], layer=camadas[camada])
    elementos.append(elem)

ifcopenshell.api.spatial.assign_container(modelo, products=elementos, relating_structure=pavimento)
modelo.write(saida)
print(f"Arquivo gravado: {saida} | elementos: {len(elementos)} | camadas: {', '.join(camadas)}")
```

```
python c13_03a_exportacao_simulada.py
```

```
Arquivo gravado: exportacao_cad.ifc | elementos: 10 | camadas: TUB-L001, TUB-L002, MEC-EQUIPAMENTOS, EST-
SUPORTES, 0
```

O próximo script transforma a camada em dado: grava o nome da camada numa propriedade de cada elemento e, quando a camada segue o padrão de TAG informado, grava-a também no atributo `Tag`, que passa a servir de chave. Grave-o como `c13_03_camadas.py`.

```
# -*- coding: utf-8 -*-
"""
c13_03_camadas.py | Capítulo 13: aproveita as camadas (layers) de uma exportação CAD como dado.

Para cada elemento, lê a camada da sua geometria (IfcPresentationLayerAssignment) e:
  - grava o nome da camada numa propriedade (padrão: ARV_Origem.Camada, IfcLabel);
  - se a camada seguir o padrão de TAG informado (--padrao-tag), grava a camada também no
    atributo Tag do elemento, que passa a servir de chave para o preenchimento em lote.
Lista, ao final, as camadas encontradas e o número de elementos de cada uma.
O arquivo de entrada não é alterado; o resultado é gravado num arquivo novo.

Uso: python c13_03_camadas.py <entrada.ifc> <saida.ifc> [--padrao-tag "TUB-.*"] [--conjunto ARV_Origem]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import argparse
import re
from collections import Counter
from pathlib import Path
import ifcopenshell
import ifcopenshell.api.attribute
import ifcopenshell.api.pset
import ifcopenshell.util.element as util_elem


def main():
    leitor = argparse.ArgumentParser(description="Transforma as camadas CAD em dados do IFC.")
    leitor.add_argument("entrada")
    leitor.add_argument("saida")
    leitor.add_argument("--padrao-tag", help="expressão regular das camadas que são TAG (ex.: 'TUB-.*')")
    leitor.add_argument("--conjunto", default="ARV_Origem")
```

```
    args = leitor.parse_args()
    if Path(args.saida).resolve() == Path(args.entrada).resolve():
        raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")

    modelo = ifcopenshell.open(args.entrada)
    padrao = re.compile(args.padrao_tag) if args.padrao_tag else None
    contagem, com_tag, sem_camada, varias = Counter(), 0, 0, 0

    for elem in modelo.by_type("IfcElement"):
        nomes = sorted({camada.Name for camada in util_elem.get_layers(modelo, elem)})
        if not nomes:
            sem_camada += 1
            continue
        if len(nomes) > 1:                                  # geometria dividida entre camadas: só registra
            varias += 1
        camada = nomes[0] if len(nomes) == 1 else " | ".join(nomes)
        contagem[camada] += 1
        pset = ifcopenshell.api.pset.add_pset(modelo, product=elem, name=args.conjunto)
        ifcopenshell.api.pset.edit_pset(modelo, pset=pset,
                                        properties={"Camada": modelo.create_entity("IfcLabel", camada)})
        if padrao and len(nomes) == 1 and padrao.fullmatch(camada):
            ifcopenshell.api.attribute.edit_attributes(modelo, product=elem, attributes={"Tag": camada})
            com_tag += 1

    modelo.write(args.saida)
    print(f"Arquivo gravado: {args.saida}")
    print(f"Elementos com camada: {sum(contagem.values())} | sem camada: {sem_camada} | "
          f"em mais de uma camada: {varias} | Tag preenchido: {com_tag}")
    for camada, n in sorted(contagem.items()):
        marca = " (TAG)" if padrao and padrao.fullmatch(camada) else ""
        print(f"  {camada}: {n}{marca}")


if __name__ == "__main__":
    main()
```

```
python c13_03_camadas.py exportacao_cad.ifc exportacao_cad_camadas.ifc --padrao-tag "TUB-.*"
python -m ifcopenshell.validate --rules exportacao_cad_camadas.ifc
```

```
Arquivo gravado: exportacao_cad_camadas.ifc
Elementos com camada: 10 | sem camada: 0 | em mais de uma camada: 0 | Tag preenchido: 5
  0: 1
  EST-SUPORTES: 2
  MEC-EQUIPAMENTOS: 2
  TUB-L001: 3 (TAG)
  TUB-L002: 2 (TAG)
```

```
Validating exportacao_cad_camadas.ifc
No validation issues found.
```

Os cinco trechos de tubo receberam a TAG da linha no atributo `Tag`; os equipamentos, os suportes e o sólido da camada `0` receberam apenas a propriedade `ARV_Origem.Camada`, porque suas camadas não identificam um objeto. A partir daqui:

- os dados de linha (fluido, pressão, temperatura, especificação) podem ser gravados a partir da planilha, usando `Tag` como chave (Capítulo 10). Como várias peças compartilham a mesma TAG, o script do Capítulo 10 as marcaria como `DUPLICADO`; o Capítulo 14 trata esse caso, aplicando os dados de uma linha da planilha a todos os elementos da mesma chave;
- a camada não diz se a peça é tubo, curva ou válvula. Sem essa informação, reclassificar todos os sólidos de uma linha como `IfcPipeSegment` seria errado; mantenha-os como *proxies* com a TAG até que a classe possa ser determinada (por nome, por outra camada ou na origem).

> **OBSERVAÇÃO**
>
> O validador acusa *proxies* sem nome: o IFC4 exige o atributo `Name` em `IfcBuildingElementProxy` (regra `HasObjectName`). Por isso a simulação atribui um nome genérico a todos os sólidos. Numa exportação real, confira a regra com o validador (Capítulo 12).

13.6 Navisworks 2026

No fluxo IFC, o Navisworks é usado para **ler** arquivos IFC, na revisão, na federação de disciplinas e na detecção de interferências. As opções do leitor ficam em **Options > File Readers > IFC**. A mais importante é o método de conversão (*Conversion Method*):

MÉTODO	DESCRIÇÃO, SEGUNDO A AJUDA DO NAVISWORKS 2026
Legacy	Conversor antigo do Navisworks
Modern	Conversor baseado no Revit
v3	Versão 3 do leitor
v4 (padrão)	Usa o *Autodesk Translation Framework*, com suporte ao IFC 4.3 e melhor tratamento de coordenadas grandes

Tabela 13.7: Métodos de conversão do leitor IFC do Navisworks.

Outras opções: nível de detalhe da representação (*Representation Detail*), hierarquia espacial (*Show Spatial Hierarchy*), cores por propriedade, conversão de espaços e eixos, criação de conjuntos de seleção, coordenadas compartilhadas e precisão da triangulação (*Max Facet Deviation*, *Faceting Factor*). O leitor lê conjuntos de propriedades, inclusive propriedades complexas; segundo a ajuda, não suporta curvas B-spline, superfícies varridas e B-rep com vazios.

Se uma propriedade não aparece no Navisworks, não conclua que ela não existe: confira o arquivo com o script de exportação para planilha do Capítulo 9 e teste outro método de conversão. Um artigo de suporte da Autodesk registra, por exemplo, propriedades IFC ausentes na ferramenta de busca (*Find Items*) com o leitor Legacy; e, no fórum da Autodesk, a equipe da empresa confirmou a correção de um problema do leitor v4 da versão 2025 que renomeava conjuntos de propriedades customizados.

13.7 Autodesk Forma e Autodesk Tandem

Em 25 de março de 2026, a Autodesk renomeou o Autodesk Construction Cloud para **Autodesk Forma**. O produto de estudo de implantação que antes se chamava Forma passou a se chamar **Forma Site Design**; o antigo Docs passou a Forma Data Management, e o Build, a Forma Build. Na documentação e nos fóruns anteriores a essa data, "Forma" se refere ao produto de implantação.

- O **Forma Site Design** importa modelos IFC e OBJ e, segundo a página do produto, exporta propostas em IFC 4.3, em versão beta. As versões IFC aceitas na importação não constam das fontes consultadas.
- No **visualizador de documentos** do Forma (antigo ACC), para elementos ausentes ou exibidos incorretamente, a Autodesk recomenda IFC4 com hierarquia Projeto > Terreno > Edificação > Pavimento, ou o processamento do arquivo pelo leitor v4 do Navisworks 2025 ou posterior. Modelos

de infraestrutura em IFC 4.3, sem edificação e pavimento, devem ser testados no visualizador antes de serem adotados como entrega.

O **Autodesk Tandem**, plataforma de gêmeo digital, recebe modelos Revit ou IFC, por envio de arquivo ou a partir do serviço de documentos da Autodesk (Autodesk Docs, hoje Forma Data Management). Nele, os **modelos de instalação** (*Facility Templates*) combinam sistemas de classificação com parâmetros: os parâmetros são associados a códigos de classificação e herdados na árvore da classificação, e sistemas próprios podem ser carregados a partir de uma planilha modelo. Por isso, um IFC com classificação consistente (Capítulo 5) facilita a organização dos dados no Tandem. As versões IFC aceitas e a forma como propriedades e classificações do IFC são levadas aos parâmetros do Tandem não constam das fontes consultadas; teste com um modelo pequeno antes de definir o fluxo.

13.8 Problemas frequentes e correções

SINTOMA	CAUSA PROVÁVEL	DIAGNÓSTICO	CORREÇÃO NA ORIGEM	CORREÇÃO POR SCRIPT
Muitos *proxies*	Categoria ou objeto sem mapeamento; versão IFC sem a classe	Seção Classes do diagnóstico	`Export to IFC As` (Revit); `IfcInfraExportMapping.json` (Civil 3D); versão IFC4 ou 4.3	Reclassificação (Capítulo 10)
Propriedades do BEP ausentes	Conjuntos não configurados; objetos em camada congelada ou bloqueada (Civil 3D)	Diagnóstico; IDS	Conjuntos definidos pelo usuário; modelos de propriedades da extensão	Preenchimento em lote (Capítulo 10)
Números gravados como texto	Parâmetro de texto no software; tipo errado no arquivo de conjuntos	Tipos de valor no diagnóstico; `dataType` no IDS	Parâmetro numérico; tipo correto no arquivo de conjuntos	Regravação com o tipo IFC correto (Capít
Modelo fora de posição	Base de coordenadas errada; sistema sem EPSG	Georreferenciamento no diagnóstico; visualizador	*Coordinate Base* e EPSG (Revit); sistema EPSG no desenho (Civil 3D)	Edição de `IfcMapConversion` (Capítulo 7
Escala incoerente	Escala escrita na unidade errada	Aviso do diagnóstico	Configuração do exportador	Correção do atributo `Scale`

SINTOMA	CAUSA PROVÁVEL	DIAGNÓSTICO	CORREÇÃO NA ORIGEM	CORREÇÃO POR SCRIPT
GlobalId muda a cada exportação	Objetos derivados recriados na exportação (subpartes de corredor)	Script do exercício 9	Não há, na versão considerada	Vincular dados por chave estável (TAG)
Elementos fora da estrutura espacial	Objetos sem nível ou fora da hierarquia exportada	"sem contêiner" no diagnóstico; `partOf` no IDS	Associar nível ou estrutura na origem	`ifcopenshell.api.spatial.assign_co`
Propriedade some no visualizador	Limitação ou erro do leitor	Exportação do arquivo para planilha (Capítulo 9)	Outro método de conversão (Navisworks)	Não se aplica

Tabela 13.8: Sintomas, causas e correções.

13.9 Resumo do capítulo

- O conteúdo do IFC é definido pelo exportador e pela configuração: teste no início, registre a configuração no BEP e diagnostique cada entrega.
- O diagnóstico do arquivo (cabeçalho, unidades, georreferenciamento, estrutura, classes, tipos, propriedades, classificação, geometria, camadas e GlobalId) é a referência; visualizadores são apenas uma forma de leitura.
- A versão IFC limita as classes: o IFC2x3 não tem classes de ocorrência específicas de tubulação (usa classes genéricas de fluxo), nem classes de infraestrutura, nem várias classes de edificação.
- No Revit, a classe vem de `Export to IFC As`, e os metadados do BEP, dos conjuntos definidos pelo usuário, que podem ser gerados a partir da tabela de requisitos.
- No Civil 3D, objetos de infraestrutura em IFC 4.3 exigem a extensão, configurada por arquivos JSON; subpartes de corredores não mantêm GlobalId.
- O Plant 3D não exporta IFC; pelo caminho oficial, perdem-se os dados de planta, e, se as camadas forem preservadas, a camada pode ser o único vínculo com a lista de linhas, a ser transformado em dado.
- Navisworks, Forma e Tandem leem IFC com regras próprias: teste com modelos pequenos antes de definir o fluxo.

13.10 Vocabulário técnico

TERMO EM INGLÊS	TERMO EM PORTUGUÊS
exporter / setup	exportador / configuração salva
mapping	mapeamento
user defined property sets	conjuntos de propriedades definidos pelo usuário

TERMO EM INGLÊS	TERMO EM PORTUGUÊS
parameter mapping table	tabela de mapeamento de parâmetros
coordinate base / survey point / project base point	base de coordenadas / ponto de levantamento / ponto base do projeto
shared coordinates	coordenadas compartilhadas
add-in / extension	complemento / extensão
corridor / alignment / feature line	corredor / alinhamento / linha característica
layer	camada
file reader / conversion method	leitor de arquivo / método de conversão
facility template	modelo de instalação
digital twin	gêmeo digital

Tabela 13.9: Vocabulário do capítulo.

13.11 Exercícios

Fixação

1. Por que duas exportações IFC do mesmo modelo Revit podem ter conteúdos muito diferentes?
2. No Revit, que valor de `Export to IFC As` exporta uma família de acessório de tubulação como válvula de retenção? Para que serve o parâmetro `IFC Predefined Type`?
3. No Civil 3D 2026, qual via de exportação usar para alinhamentos e corredores em IFC 4.3, e por que não o `IFCEXPORT`?
4. Qual é o caminho oficial para obter IFC de um modelo do Plant 3D, e o que se perde nele?
5. Por que a versão IFC2x3 da amostra de arquitetura tem mais *proxies* que a versão IFC 4.3?

Aplicação

6. Execute o diagnóstico com a amostra `Infra-Bridge.ifc` (IFC 4.3) e escreva um parecer curto: o que está adequado e o que faltaria para atender a um BEP que exija conjuntos de propriedades e classificação dos elementos.
7. Execute `c13_02_gerar_psets_revit.py` com a opção `--esquema IFC2X3`. Explique as mensagens e o que fazer se o contrato exigir IFC2x3.
8. Execute `c13_03_camadas.py` sobre `exportacao_cad.ifc` sem a opção `--padrao-tag`. O que muda no resultado? Por que seria um erro usar `--padrao-tag ".*"`?

Desafio

9. Escreva um script que meça a estabilidade do GlobalId entre duas exportações do mesmo modelo, por classe IFC. Teste com duas execuções de `c13_03a_exportacao_simulada.py` e com os arquivos `rede_proxies.ifc` e `rede_reclassificada.ifc` do Capítulo 10.

10. Execute o diagnóstico com a amostra de georreferenciamento `geographic-referencing-utm.ifc` (Capítulo 7). Interprete o aviso sobre a escala.

13.12 Respostas dos exercícios

1. Porque o conteúdo depende da configuração do exportador: versão e MVD, mapeamento de categorias, conjuntos de propriedades exportados, base de coordenadas, nível de detalhe e opções de GlobalId. O formato é o mesmo; a tradução é que muda.
2. `IfcValve.CHECK`. O `IFC Predefined Type` informa só o tipo predefinido, quando a classe já vem do mapeamento da categoria ou de `Export to IFC As` sem o sufixo.
3. A extensão IFC 4.3 (`IFCInfraExport`, na aba Add-ins, painel IFC Infrastructure). O `IFCEXPORT` exporta apenas IFC2x3 e IFC4 e não reconhece objetos de infraestrutura do Civil 3D: superfícies, corredores e redes precisariam ser convertidos em sólidos, perdendo a semântica de alinhamento e corredor.
4. Exportar para AutoCAD comum (`EXPORTTOAUTOCAD`) e gerar o IFC no AutoCAD Architecture, no AutoCAD MEP ou no Civil 3D. Perdem-se os dados de planta (especificações, catálogos, propriedades dos componentes); resta a geometria, e a preservação das camadas e a classe dos elementos devem ser conferidas no arquivo, com o diagnóstico.
5. Porque a chaminé, o lastro de areia e o volume bruto da casa usam classes que não existem no IFC2x3 (`IfcChimney` e `IfcSpatialZone`, do IFC4, e `IfcEarthworksFill`, do IFC 4.3); na versão 2x3, o exportador recorreu a `IfcBuildingElementProxy`.
6. Resultado obtido:

```
Diagnóstico de Infra-Bridge.ifc

[Cabeçalho]
  Esquema: IFC4X3_ADD2
  Descrição (MVD): ViewDefinition [ReferenceView]
  Sistema de origem: BIM_Tools - Sketchup_IFC_manager - 5.6.0
  Pré-processador: Sketchup-IFC-manager 5.6.0 / SketchUp 2026 (26.2.242)

[Unidades]
  LENGTHUNIT: MILLIMETRE
  AREAUNIT: SQUARE_METRE
  VOLUMEUNIT: CUBIC_METRE
  PLANEANGLEUNIT: não declarada

[Georreferenciamento]
  IfcMapConversion: E=729011.2258823584 N=9063960.607644705 H=0.0 Escala=0.001
  IfcProjectedCRS: Name=EPSG:32760

[Estrutura espacial]
  IfcBridge: 3, IfcBridgePart: 18, IfcSite: 6
  Elementos: 50 | sem contêiner e sem conjunto: 0

[Classes]
  IfcBeam: 8
  IfcMember: 8
  IfcColumn: 7
  IfcFooting: 7
  IfcEarthworksFill: 4
  IfcWall: 4
  IfcSign: 4
  IfcSlab: 3
  ... e mais 3 classe(s)
  Proxies: 1 (2% dos elementos)

[Tipos predefinidos e tipos]
  Sem tipo predefinido, na ocorrência e no tipo (vazio ou NOTDEFINED): 0 de 50 elementos com o atributo
```

```
PredefinedType
  Com tipo associado (IfcTypeObject): 46 de 50

[Conjuntos de propriedades e quantidades]
  Conjuntos distintos: 0 (padrão Pset_: 0; outros: 0)
  Quantidades distintas: 3 (Qto_BeamBaseQuantities, Qto_ColumnBaseQuantities, Qto_SlabBaseQuantities)
  Valores simples: 0; como texto (IfcLabel/IfcText): 0 (-)
  Tipos de valor: -

[Classificação e material]
  Sistemas de classificação: nenhum
  Elementos classificados (na ocorrência ou no tipo): 0 de 50
  Elementos com material: 48 de 50

[Geometria]
  Representações Body: Tessellation 48
  Elementos sem representação: 2

[Camadas (layers)]
  Camadas: 0

[GlobalId]
  Objetos: 175 | repetidos: 0 | com formato inválido: 0
```

Parecer: a estrutura espacial usa as classes de ponte do IFC 4.3 (`IfcBridge`, `IfcBridgePart`), com todos os elementos contidos; há só um *proxy* (o marco de georreferenciamento); o georreferenciamento é coerente; quase todos os elementos têm tipo e material. Faltam conjuntos de propriedades (nenhum), classificação dos elementos (nenhum sistema) e quantidades na maior parte das classes: só lajes, pilares e 5 das 8 vigas têm conjuntos `Qto_`; não há quantidades nas sapatas, paredes, barras, guarda-corpos, aterros, placas, conjuntos, no *proxy* nem nas 3 longarinas vistas no Capítulo 12. Para o BEP, seria preciso configurar os conjuntos e a classificação na origem ou preenchê-los por script.

7. Resultado obtido (primeiras linhas; a lista segue para todas as linhas da tabela):

```
Requisitos com problemas:
  linha 2: a classe IfcPipeSegment não existe no IFC2X3
  linha 2: a classe IfcValve não existe no IFC2X3
  linha 2: a classe IfcPump não existe no IFC2X3
  linha 2: a classe IfcTank não existe no IFC2X3
  linha 3: a classe IfcPipeSegment não existe no IFC2X3
  linha 3: a classe IfcValve não existe no IFC2X3
  linha 3: a classe IfcPump não existe no IFC2X3
```

No IFC2x3 não existem `IfcPipeSegment`, `IfcValve`, `IfcPump` nem `IfcTank`; nessa versão, tubos e válvulas são `IfcFlowSegment` e `IfcFlowController`, com o tipo dado por `IfcPipeSegmentType` e `IfcValveType` (Capítulo 6). Se o contrato exigir IFC2x3, a tabela de requisitos precisa usar as classes dessa versão, e o BEP deve registrar que a classe da ocorrência será genérica. Antes disso, vale propor ao contratante o IFC4 ou o IFC 4.3.

8. Resultado obtido sem `--padrao-tag`:

```
Arquivo gravado: exportacao_cad_sem_tag.ifc
Elementos com camada: 10 | sem camada: 0 | em mais de uma camada: 0 | Tag preenchido: 0
  0: 1
  EST-SUPORTES: 2
  MEC-EQUIPAMENTOS: 2
  TUB-L001: 3
  TUB-L002: 2
```

Todas as camadas continuam gravadas em `ARV_Origem.Camada`, mas nenhum `Tag` é preenchido. Com `".*"`, todas as camadas seriam tratadas como TAG, e os equipamentos e suportes receberiam `MEC-EQUIPAMENTOS` e `EST-SUPORTES` como identificação, o que é falso: vários objetos diferentes com a mesma "TAG" e o sólido perdido na camada `0` com TAG `0`. O padrão deve descrever apenas as camadas que de fato são TAG.

9. Uma solução, gravada como `c13_04_estabilidade_guid.py`:

```
# -*- coding: utf-8 -*-
"""
c13_04_estabilidade_guid.py | Capítulo 13, exercício 9 (solução comentada).

Mede a estabilidade do GlobalId entre duas exportações do mesmo modelo: quantos elementos
da exportação antiga reaparecem, com o mesmo GlobalId, na nova, por classe IFC.
Os dois arquivos são abertos somente para leitura.

Uso: python c13_04_estabilidade_guid.py <exportacao_antiga.ifc> <exportacao_nova.ifc>
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
from collections import defaultdict
import ifcopenshell


def por_classe(modelo):
    """Classe IFC -> conjunto de GlobalId dos elementos dessa classe."""
    grupos = defaultdict(set)
    for elem in modelo.by_type("IfcElement"):
        grupos[elem.is_a()].add(elem.GlobalId)
    return grupos


antigo, novo = ifcopenshell.open(sys.argv[1]), ifcopenshell.open(sys.argv[2])
g_antigo, g_novo = por_classe(antigo), por_classe(novo)
todos_novos = set().union(*g_novo.values()) if g_novo else set()
total_antigo = total_mantido = 0
print(f"{'Classe':28s} {'antiga':>7s} {'nova':>6s} {'mesmo GlobalId':>15s}")
for classe in sorted(set(g_antigo) | set(g_novo)):
    antigos = g_antigo.get(classe, set())
    mantidos = len(antigos & todos_novos)                    # inclui elementos que mudaram de classe
    total_antigo += len(antigos)
    total_mantido += mantidos
    print(f"{classe:28s} {len(antigos):7d} {len(g_novo.get(classe, set())):6d} {mantidos:15d}")
taxa = 100 * total_mantido / total_antigo if total_antigo else 0
print(f"GlobalId preservados: {total_mantido} de {total_antigo} ({taxa:.0f}%)")
```

Resultados com `python c13_03a_exportacao_simulada.py exportacao_cad_2.ifc` seguido de `python c13_04_estabilidade_guid.py exportacao_cad.ifc exportacao_cad_2.ifc`, e com `python c13_04_estabilidade_guid.py ..\cap10\rede_proxies.ifc ..\cap10\rede_reclassificada.ifc`:

```
Classe                       antiga   nova  mesmo GlobalId
IfcBuildingElementProxy          10     10               0
GlobalId preservados: 0 de 10 (0%)
```

```
Classe                       antiga   nova  mesmo GlobalId
IfcBuildingElementProxy           8      1               8
IfcFilter                         0      1               0
IfcPipeSegment                    0      2               0
IfcPump                           0      1               0
IfcTank                           0      1               0
IfcValve                          0      2               0
GlobalId preservados: 8 de 8 (100%)
```

As duas "exportações" simuladas não têm nenhum GlobalId em comum (0%), porque o script cria identificadores novos a cada execução: é o comportamento documentado para as subpartes de corredores do Civil 3D. Entre o arquivo de *proxies* e o reclassificado, os 8 GlobalId foram preservados (100%), embora 7 elementos tenham mudado de classe; por isso o script procura o GlobalId entre todos os elementos novos, e não só na mesma classe. Com taxa baixa, comparação de revisões e BCF por GlobalId não funcionam, e os vínculos devem usar chaves estáveis.

10. Trecho do resultado:

```
[Georreferenciamento]
  IfcMapConversion: E=458657.3 N=5438232.25 H=113.7 Escala=1000.0
  ATENÇÃO: com a unidade do arquivo, a escala esperada seria 0.001
  IfcProjectedCRS: Name=EPSG:25832
```

O arquivo está em milímetros e o sistema de coordenadas (EPSG:25832, UTM) em metros. Pela fórmula do Capítulo 7, a escala que converte milímetros em metros é 0,001; o valor 1000 multiplicaria as distâncias por um milhão em relação ao correto. Um visualizador que aplique a escala gravada posicionaria o modelo de forma incoerente. Por isso o diagnóstico sinaliza a escala; a correção, se confirmada com o autor, é gravar `Scale = 0.001`.

13.13 Referências

- Autodesk. Ajuda do Revit 2025, exportação IFC (opções e versões): https://help.autodesk.com/cloudhelp/2025/ENU/Revit-DocumentPresent/files/GUID-E029E3AD-1639-4446-A935-C9796BC34C95.htm
- Autodesk. Ajuda do Revit 2025, conjuntos de propriedades definidos pelo usuário: https://help.autodesk.com/cloudhelp/2025/ENU/Revit-DocumentPresent/files/GUID-0968B3FF-832B-4155-8B36-E848ACC97B95.htm
- Autodesk. Exportador IFC do Revit (código aberto e versões): https://github.com/Autodesk/revit-ifc
- Autodesk. Manual IFC da Autodesk (Revit): https://autodesk.ifc-manual.com/revit/
- Autodesk. Artigo de suporte sobre os parâmetros `Export to IFC As` e `IFC Predefined Type`: https://www.autodesk.com/support/technical/article/caas/sfdcarticles/sfdcarticles/How-to-map-old-IfcExportAs-and-IfcExportType-parameter-values-to-new-Export-to-IFC-As-and-IFC-Predefined-Type-parameters.html
- Autodesk. Ajuda da extensão IFC do Civil 3D 2026 (comandos): https://help.autodesk.com/cloudhelp/2026/ENU/Civil3D-IFC/files/GUID-FBA4D6BE-3879-4D95-9DC3-B479F7FCFF41.htm
- Autodesk. Ajuda da extensão IFC do Civil 3D 2026 (exportação e limitações): https://help.autodesk.com/cloudhelp/2026/ENU/Civil3D-IFC/files/GUID-D2711E3E-F990-4D76-B31C-30E321697C04.htm
- Autodesk. Ajuda da extensão IFC do Civil 3D 2026 (arquivos de configuração): https://help.autodesk.com/cloudhelp/2026/ENU/Civil3D-IFC/files/GUID-E277BF8B-9A39-4A4B-AE15-19D568E02E82.htm
- Autodesk. Ajuda do Civil 3D 2026, `IFCEXPORT`: https://help.autodesk.com/cloudhelp/2026/ENU/Civil3D-UserGuide/files/GUID-C5C9DEEE-2C46-4094-B350-05829C1ED5DC.htm

- Autodesk. Artigo de suporte sobre IFC no Plant 3D: https://www.autodesk.com/support/technical/article/caas/sfdcarticles/sfdcarticles/How-to-generate-IFC-file-for-AutoCAD-Plant3D-model.html
- Autodesk. Ajuda do Navisworks 2026, leitor IFC: https://help.autodesk.com/cloudhelp/2026/ENU/Navisworks/files/GUID-E1CD4663-D460-4EA5-ABEC-FD24885BA3A6.htm
- Autodesk. Autodesk Construction Cloud passa a se chamar Autodesk Forma: https://www.autodesk.com/blogs/construction/autodesk-construction-cloud-is-now-autodesk-forma-heres-what-that-means-for-you/
- Autodesk. Forma, detalhes do produto: https://www.autodesk.com/products/forma/product-details
- Autodesk. Ajuda do Tandem, arquivos e modelos de instalação: https://help.autodesk.com/cloudhelp/ENU/Tandem-Facilities/files/tandem-files.html

14 Projeto integrador: do BEP ao relatório de não conformidades

Este capítulo reúne o conteúdo dos Capítulos 8 a 13 numa ferramenta única para o problema que motiva toda a apostila: inserir, num modelo IFC de tubulação e equipamentos, os metadados exigidos pelo BEP a partir de uma planilha, verificar o resultado com IDS e emitir um relatório de não conformidades. O projeto é apresentado com um modelo e uma planilha de exemplo, mas foi escrito para ser adaptado a modelos reais: toda a regra de negócio fica na planilha, e o código não precisa ser alterado de um projeto para outro.

Objetivos de aprendizagem. Ao concluir o capítulo, você será capaz de:

- organizar os requisitos de informação de um BEP numa planilha única, que serve tanto para preencher quanto para verificar;
- distinguir dados por elemento (chave única) de dados por grupo (chave de grupo, como a linha de tubulação);
- gerar automaticamente o IDS a partir dos requisitos, com as conversões de unidade corretas;
- executar o fluxo completo: conferência da planilha, preenchimento, validação do esquema, verificação IDS e relatórios;
- interpretar os resultados e conduzir o ciclo de correção até a conformidade;
- adaptar a ferramenta a um projeto real.

14.1 O problema e a solução

14.1.1 O problema

Num projeto industrial típico, o BEP exige que cada equipamento tenha TAG, descrição, fabricante, peso, material, classificação e revisão, e que cada peça de tubulação tenha fluido, pressão e temperatura de projeto, classe de tubulação, diâmetro nominal, material e revisão. Os dados estão em documentos do projeto (folhas de dados, lista de linhas) e são reunidos numa planilha. Duas características do problema orientam a solução:

- **equipamentos** são identificados um a um pela TAG;
- **tubulação** é organizada por **linha**: tubos, conexões e válvulas de uma mesma linha compartilham os dados de processo e de especificação. Na planilha, há uma linha de dados por linha de tubulação, e não por peça.

14.1.2 A solução em seis etapas



Figura 14.1: Fluxo do projeto integrador.

ETAPA	O QUE FAZ	CAPÍTULO DE ORIGEM
1. Conferência da planilha	Verifica abas, colunas, destinos, tipos e classes antes de alterar qualquer coisa	8 e 10
2. Geração do IDS	Converte cada linha de requisito numa especificação IDS	12
3. Preenchimento	Grava atributos, propriedades, material e classificação, por chave única ou de grupo	10
4. Validação do esquema	Executa `ifcopenshell.validate` com as regras EXPRESS no arquivo gravado	12
5. Verificação IDS	Verifica o modelo preenchido com o IfcTester	12
6. Relatórios	Planilha com resumo, preenchimento, não conformidades e esquema; relatórios HTML e BCF	9 e 12

Tabela 14.1: Etapas do projeto integrador.

A ideia central é que **a mesma planilha governa o preenchimento e a verificação**. Se o BEP mudar, muda a planilha; o IDS acompanha automaticamente, e o que se grava é exatamente o que se exige.

14.2 A planilha do BEP

A planilha tem quatro abas. As duas primeiras descrevem as regras; as demais, os dados.

14.2.1 Aba Chaves

Diz como cada aba de dados encontra os elementos no modelo:

COLUNA	SIGNIFICADO	EXEMPLO
`Aba`	Nome da aba de dados	`Linhas`
`Coluna`	Coluna da aba de dados que contém a chave	`LINHA`
`Atributo`	Atributo IFC do elemento que contém a chave	`Tag`
`Modo`	`unico`: a chave identifica um elemento; `grupo`: a chave identifica vários elementos, que recebem os mesmos dados	`grupo`

Tabela 14.2: Colunas da aba Chaves.

No modo `unico`, uma chave repetida no modelo é um erro (situação `DUPLICADO`), como no Capítulo 10. No modo `grupo`, é o comportamento esperado: todas as peças da linha `P-101` recebem os dados da linha `P-101` da planilha.

14.2.2 Aba Requisitos

Cada linha descreve uma informação do BEP: de onde vem, onde é gravada e o que se exige dela.

COLUNA	SIGNIFICADO
`Aba`, `Coluna`	Origem do valor: aba e coluna de dados

COLUNA	SIGNIFICADO
`Destino`	`Atributo`, `Pset`, `Material` ou `Classificacao`
`Conjunto`	Nome do conjunto de propriedades (`Pset`) ou do sistema de classificação (`Classificacao`)
`Propriedade`	Nome da propriedade (`Pset`) ou do atributo (`Atributo`)
`TipoIFC`	Tipo do valor, para `Pset` (`IfcLabel`, `IfcPressureMeasure`, `IfcPositiveLengthMeasure`...)
`Fator`, `Soma`	Conversão da unidade da planilha para a do IFC: valor gravado = valor x `Fator` + `Soma`
`Classes`	Classes IFC às quais o requisito se aplica, separadas por vírgula
`Obrigatorio`	`sim` ou `não` (cardinalidade do requisito no IDS)
`Padrao`, `Valores`, `Minimo`, `Maximo`	Restrição do valor: expressão regular, lista separada por barras verticais ou limites (use um só tipo)

Tabela 14.3: Colunas da aba Requisitos.

Os limites `Minimo` e `Maximo` são escritos **na unidade da planilha**, como os dados. O integrador aplica a eles a mesma conversão dos dados e, depois, a escala da unidade do arquivo para o SI, exigida pelo IDS (Capítulo 12). Assim, quem preenche a planilha pensa em kPa, °C, kg e mm, e o IDS recebe pascal, kelvin, quilograma e metro.

14.2.3 Abas de dados

As abas `Equipamentos` e `Linhas` contêm os dados, com uma coluna de chave e uma coluna para cada linha de requisito. O script abaixo grava a planilha de exemplo. Os dados contêm, de propósito, seis problemas típicos, que o projeto deve encontrar. Grave-o como `C:\IFC_Curso\exercicios\cap14\c14_01_planilha_bep.py`.

```
# -*- coding: utf-8 -*-
"""
c14_01_planilha_bep.py | Capítulo 14: planilha de exemplo do projeto integrador.

Grava uma planilha com quatro abas:
  Chaves       -> como cada aba de dados encontra os elementos no IFC
                  (Aba | Coluna | Atributo | Modo), Modo = unico ou grupo
  Requisitos   -> o que gravar e o que exigir, uma linha por informação do BEP
                  (Aba | Coluna | Destino | Conjunto | Propriedade | TipoIFC | Fator | Soma |
                   Classes | Obrigatorio | Padrao | Valores | Minimo | Maximo)
  Equipamentos -> dados por equipamento (chave: TAG no atributo Name)
  Linhas       -> dados por linha de tubulação (chave: TAG da linha no atributo Tag)
Os dados contêm, de propósito, problemas típicos, para exercitar a verificação.

Uso: python c14_01_planilha_bep.py [saida.xlsx]     (padrão: bep_unidade.xlsx)
Testado com: Python 3.11 e 3.13; pandas 3.0; openpyxl 3.1.
"""
import sys
import pandas as pd

saida = sys.argv[1] if len(sys.argv) > 1 else "bep_unidade.xlsx"

chaves = pd.DataFrame([
    ["Equipamentos", "TAG", "Name", "unico"],
    ["Linhas", "LINHA", "Tag", "grupo"],
], columns=["Aba", "Coluna", "Atributo", "Modo"])
```

```
EQUIP = "IfcTank,IfcPump"
TUB = "IfcPipeSegment,IfcPipeFitting,IfcValve"
requisitos = pd.DataFrame([
    # Aba, Coluna, Destino, Conjunto, Propriedade, TipoIFC, Fator, Soma,
    # Classes, Obrigatorio, Padrao, Valores, Minimo, Maximo
    ["Equipamentos", "TAG", "Atributo", None, "Name", None, None, None,
     EQUIP, "sim", "[A-Z]{1,3}-[0-9]{3}[A-Z]?", None, None, None],
    ["Equipamentos", "Descricao", "Atributo", None, "Description", None, None, None,
     EQUIP, "sim", None, None, None, None],
    ["Equipamentos", "Fabricante", "Pset", "ARV_Equipamento", "Fabricante", "IfcLabel", None, None,
     EQUIP, "sim", None, None, None, None],
    ["Equipamentos", "PesoOperacao_kg", "Pset", "ARV_Equipamento", "PesoOperacao", "IfcMassMeasure",
     None, None, EQUIP, "sim", None, None, 1, 50000],
    ["Equipamentos", "Material", "Material", None, None, None, None, None,
     EQUIP, "sim", None, None, None, None],
    ["Equipamentos", "Classificacao", "Classificacao", "ABNT NBR 15965", None, None, None, None,
     EQUIP, "sim", r"3E\.[A-Z]{2}\.[0-9]{2}", None, None, None],
    ["Equipamentos", "Revisao", "Pset", "ARV_Controle", "Revisao", "IfcLabel", None, None,
     EQUIP, "sim", "[A-Z]", None, None, None],
    ["Linhas", "LINHA", "Atributo", None, "Tag", None, None, None,
     TUB, "sim", "P-[0-9]{3}", None, None, None],
    ["Linhas", "Fluido", "Pset", "ARV_Processo", "Fluido", "IfcLabel", None, None,
     TUB, "sim", None, "Água|Produto A|Produto B", None, None],
    ["Linhas", "PressaoProjeto_kPa", "Pset", "ARV_Processo", "PressaoProjeto", "IfcPressureMeasure",
     1000, None, TUB, "sim", None, None, 100, 2000],
    ["Linhas", "TemperaturaProjeto_C", "Pset", "ARV_Processo", "TemperaturaProjeto",
     "IfcThermodynamicTemperatureMeasure", None, 273.15,
     TUB, "sim", None, None, 0, 200],
    ["Linhas", "ClasseTubulacao", "Pset", "ARV_Tubulacao", "ClasseTubulacao", "IfcLabel", None, None,
     TUB, "sim", "[A-Z][0-9][A-Z]", None, None, None],
    ["Linhas", "DN_mm", "Pset", "ARV_Tubulacao", "DiametroNominal", "IfcPositiveLengthMeasure", None, None,
     TUB, "sim", None, None, 15, 600],
    ["Linhas", "Material", "Material", None, None, None, None, None,
     TUB, "sim", None, None, None, None],
    ["Linhas", "Revisao", "Pset", "ARV_Controle", "Revisao", "IfcLabel", None, None,
     TUB, "sim", "[A-Z]", None, None, None],
], columns=["Aba", "Coluna", "Destino", "Conjunto", "Propriedade", "TipoIFC", "Fator", "Soma", "Classes",
            "Obrigatorio", "Padrao", "Valores", "Minimo", "Maximo"])

equipamentos = pd.DataFrame([
    ["TQ-101", "Tanque de água bruta", "Fabricante X", 18500, "Aço-carbono ASTM A283 Gr. C",
     "3E.EX.01", "A"],
    ["TQ-102", "Tanque de água tratada", None, 17900, "Aço-carbono ASTM A283 Gr. C",
     "3E.EX.01", "A"],
    ["B-101A", "Bomba de transferência", "Fabricante Z", 640, "Ferro fundido", "3E.EX.04", "A"],
    ["B-101B", "Bomba de transferência (reserva)", "Fabricante Z", "n/d", "Ferro fundido", "3E.EX.04", "A"],
    ["B-102", "Bomba dosadora", "Fabricante W", 85, "Aço inoxidável AISI 316", "3E.EX.05", "A"],
], columns=["TAG", "Descricao", "Fabricante", "PesoOperacao_kg", "Material", "Classificacao", "Revisao"])

linhas = pd.DataFrame([
    ["P-101", "Água", 600, 40, "A1A", 100, "Aço-carbono ASTM A106 Gr. B", "B"],
    ["P-102", "Água", 2500, 40, "A1A", 80, "Aço-carbono ASTM A106 Gr. B", "B"],
    ["P-199", "Produto A", 400, 60, "B2A", 50, "Aço inoxidável AISI 316", "A"],
], columns=["LINHA", "Fluido", "PressaoProjeto_kPa", "TemperaturaProjeto_C", "ClasseTubulacao", "DN_mm",
            "Material", "Revisao"])

with pd.ExcelWriter(saida) as livro:
    chaves.to_excel(livro, sheet_name="Chaves", index=False)
    requisitos.to_excel(livro, sheet_name="Requisitos", index=False)
    equipamentos.to_excel(livro, sheet_name="Equipamentos", index=False)
    linhas.to_excel(livro, sheet_name="Linhas", index=False)
print(f"Planilha gravada: {saida} | requisitos: {len(requisitos)} | equipamentos: {len(equipamentos)} "
      f"| linhas: {len(linhas)}")
```

14.3 O modelo de exemplo

O modelo de exemplo é uma pequena unidade de bombeamento, sem geometria (o foco é a informação), com dois tanques, duas bombas e três linhas de tubulação. Grave o script como `c14_00_modelo_exemplo.py`.

```
# -*- coding: utf-8 -*-
"""
c14_00_modelo_exemplo.py | Capítulo 14: modelo de exemplo do projeto integrador.

Cria uma pequena unidade de bombeamento em IFC 4.3, sem geometria (o foco é a informação):
  - equipamentos identificados pela TAG no atributo Name (tanques e bombas);
  - tubulação organizada por linha: cada tubo, conexão e válvula traz a TAG da LINHA no
    atributo Tag; o Name identifica a peça (válvulas com TAG própria, tubos e curvas genéricos).
Os GlobalId são gerados de forma determinística, para que os resultados sejam reprodutíveis.

Uso: python c14_00_modelo_exemplo.py [saida.ifc]     (padrão: modelo_unidade.ifc)
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5.
"""
import sys
import uuid
import ifcopenshell
import ifcopenshell.api.aggregate
import ifcopenshell.api.attribute
import ifcopenshell.api.project
import ifcopenshell.api.root
import ifcopenshell.api.spatial
import ifcopenshell.api.unit
import ifcopenshell.guid

# (classe, tipo predefinido, Name, Tag)
ELEMENTOS = [
    ("IfcTank", "STORAGE", "TQ-101", None), ("IfcTank", "STORAGE", "TQ-102", None),
    ("IfcPump", "ENDSUCTION", "B-101A", None), ("IfcPump", "ENDSUCTION", "B-101B", None),
    # linha P-101: sucção das bombas
    ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-101"), ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-101"),
    ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-101"), ("IfcPipeFitting", "BEND", "Curva 90", "P-101"),
    ("IfcPipeFitting", "BEND", "Curva 90", "P-101"), ("IfcValve", "ISOLATING", "VG-101", "P-101"),
    # linha P-102: recalque das bombas
    ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-102"), ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-102"),
    ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-102"), ("IfcPipeFitting", "BEND", "Curva 90", "P-102"),
    ("IfcValve", "CHECK", "VR-102", "P-102"), ("IfcValve", "ISOLATING", "VG-102", "P-102"),
    # linha P-103: desvio (bypass)
    ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-103"), ("IfcPipeSegment", "RIGIDSEGMENT", "Tubo", "P-103"),
]

saida = sys.argv[1] if len(sys.argv) > 1 else "modelo_unidade.ifc"
modelo = ifcopenshell.api.project.create_file(version="IFC4X3")
contador = iter(range(1, 1000))


def criar(classe, nome, tipo=None):
    """Cria a entidade e troca o GlobalId aleatório por um determinístico (apenas para o exemplo)."""
    entidade = ifcopenshell.api.root.create_entity(modelo, ifc_class=classe, name=nome, predefined_type=tipo)
    semente = f"exemplo-capitulo-14-{next(contador)}"               # mesma semente, mesmo GlobalId
    entidade.GlobalId = ifcopenshell.guid.compress(uuid.uuid5(uuid.NAMESPACE_URL, semente).hex)
    return entidade


projeto = criar("IfcProject", "Unidade de bombeamento (exemplo)")
ifcopenshell.api.unit.assign_unit(modelo)
terreno = criar("IfcSite", "Terreno")
edificio = criar("IfcBuilding", "Casa de bombas")
pavimento = criar("IfcBuildingStorey", "Térreo")
ifcopenshell.api.aggregate.assign_object(modelo, products=[terreno], relating_object=projeto)
ifcopenshell.api.aggregate.assign_object(modelo, products=[edificio], relating_object=terreno)
ifcopenshell.api.aggregate.assign_object(modelo, products=[pavimento], relating_object=edificio)

elementos = []
for classe, tipo, nome, tag in ELEMENTOS:
```

```
    elem = criar(classe, nome, tipo)
    if tag:
        ifcopenshell.api.attribute.edit_attributes(modelo, product=elem, attributes={"Tag": tag})
    elementos.append(elem)
ifcopenshell.api.spatial.assign_container(modelo, products=elementos, relating_structure=pavimento)

modelo.write(saida)
print(f"Arquivo gravado: {saida} | elementos: {len(elementos)}")
```

Execução dos dois scripts:

```
cd C:\IFC_Curso\exercicios\cap14
python c14_00_modelo_exemplo.py
python c14_01_planilha_bep.py
```

```
Arquivo gravado: modelo_unidade.ifc | elementos: 18
Planilha gravada: bep_unidade.xlsx | requisitos: 15 | equipamentos: 5 | linhas: 3
```

LINHA	PEÇAS	SITUAÇÃO NA PLANILHA
P-101	3 tubos, 2 curvas, válvula VG-101	Dados completos
P-102	3 tubos, 1 curva, válvulas VR-102 e VG-102	Pressão de projeto de 2.500 kPa, acima do limite de 2.000 kPa
P-103	2 tubos	Linha ausente da planilha
P-199	Nenhuma	Linha da planilha sem elementos no modelo

Tabela 14.4: Linhas de tubulação do exemplo.

Nos equipamentos, o tanque TQ-102 está sem fabricante, a bomba B-101B tem peso "n/d" e a bomba B-102 consta da planilha, mas não do modelo.

> **OBSERVAÇÃO**
>
> O script substitui o GlobalId aleatório de cada objeto por um valor calculado a partir de um texto fixo (`uuid5`), apenas para que os resultados deste capítulo sejam reprodutíveis. Em modelos reais, o GlobalId vem do software de autoria e nunca deve ser alterado.

14.4 O módulo da planilha

A leitura e a conferência da planilha ficam num **módulo** separado, importado pelo integrador (Capítulo 8). Isso mantém cada arquivo com uma responsabilidade e permite reaproveitar as funções. Grave-o como `c14_bep.py`, na mesma pasta.

```
# -*- coding: utf-8 -*-
"""
c14_bep.py | Capítulo 14: leitura e conferência da planilha do BEP (módulo do projeto integrador).

Funções usadas por c14_integrador.py:
  ler_bep(caminho, esquema)  -> lê as abas Chaves, Requisitos e as abas de dados, e confere tudo
                                antes de qualquer alteração no modelo
  converter(valor, tipo_ifc, fator, soma) -> valor da planilha no tipo e na unidade do IFC
  restricao(requisito, escala) -> restrição IDS (ifctester) da linha de requisito, ou None
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; ifctester 0.8.5; pandas 3.0.
"""
import pandas as pd
import ifcopenshell.ifcopenshell_wrapper as wrapper
from ifctester import ids
```

```
DESTINOS = ("Atributo", "Pset", "Material", "Classificacao")
TIPOS_TEXTO = {"IfcLabel", "IfcText", "IfcIdentifier"}
TIPOS_INTEIROS = {"IfcInteger", "IfcCountMeasure"}


def vazio(valor):
    """Célula vazia da planilha (None, NaN ou texto em branco)."""
    return valor is None or (isinstance(valor, float) and pd.isna(valor)) or str(valor).strip() == ""


def numero(valor, padrao):
    """Número de uma célula opcional (Fator, Soma, Minimo, Maximo)."""
    return padrao if vazio(valor) else float(valor)


def converter(valor, tipo_ifc, fator=None, soma=None):
    """Converte o valor da planilha para o tipo Python adequado ao tipo IFC (valor * Fator + Soma)."""
    if vazio(tipo_ifc) or tipo_ifc in TIPOS_TEXTO:
        return str(valor).strip()
    if tipo_ifc == "IfcBoolean":
        texto = str(valor).strip().lower()
        if texto in ("sim", "s", "true", "verdadeiro", "1"):
            return True
        if texto in ("não", "nao", "n", "false", "falso", "0"):
            return False
        raise ValueError(f"valor lógico não reconhecido: {valor!r}")
    try:
        convertido = float(str(valor).replace(",", ".")) * numero(fator, 1.0) + numero(soma, 0.0)
    except ValueError:
        raise ValueError(f"valor numérico inválido: {valor!r}") from None
    return int(round(convertido)) if tipo_ifc in TIPOS_INTEIROS else convertido


def existe(esquema, nome):
    try:
        esquema.declaration_by_name(nome)
        return True
    except Exception:
        return False


def restricao(req, escala=1.0):
    """Restrição IDS da linha: Valores (enumeração), Padrao ou Minimo/Maximo.

    Os limites passam pela mesma conversão dos dados (Fator e Soma), que leva o valor à unidade do
    arquivo IFC, e depois pela escala dessa unidade para o SI (por exemplo, 0,001 de mm para m),
    porque o IDS compara valores em unidades SI.
    """
    if not vazio(req["Valores"]):
        return ids.Restriction(options={"enumeration": [v.strip() for v in str(req["Valores"]).split("|")]})
    if not vazio(req["Padrao"]):
        return ids.Restriction(options={"pattern": str(req["Padrao"])})
    limites = {}
    if not vazio(req["Minimo"]):
        limites["minInclusive"] = converter(req["Minimo"], req["TipoIFC"], req["Fator"], req["Soma"]) * escala
    if not vazio(req["Maximo"]):
        limites["maxInclusive"] = converter(req["Maximo"], req["TipoIFC"], req["Fator"], req["Soma"]) * escala
    return ids.Restriction(options=limites, base="double") if limites else None


def ler_bep(caminho, nome_esquema):
    """Lê a planilha e devolve (chaves, requisitos, dados). Encerra com a lista de problemas, se houver."""
    esquema = wrapper.schema_by_name(nome_esquema)
    abas = pd.read_excel(caminho, sheet_name=None, dtype=object)          # todas as abas
    problemas = [f"aba '{a}' ausente" for a in ("Chaves", "Requisitos") if a not in abas]
    if problemas:
        raise SystemExit("Planilha do BEP com problemas:\n  " + "\n  ".join(problemas))
    chaves = {c["Aba"]: c for c in abas["Chaves"].to_dict(orient="records")}
    requisitos = abas["Requisitos"].to_dict(orient="records")

    for aba, chave in chaves.items():
        if aba not in abas:
```

```
            problemas.append(f"Chaves: a aba de dados '{aba}' não existe")
        elif chave["Coluna"] not in abas[aba].columns:
            problemas.append(f"Chaves: a coluna '{chave['Coluna']}' não existe na aba '{aba}'")
        if chave["Modo"] not in ("unico", "grupo"):
            problemas.append(f"Chaves: modo '{chave['Modo']}' inválido na aba '{aba}' (use unico ou grupo)")
    for n, req in enumerate(requisitos, start=2):
        onde = f"Requisitos, linha {n}"
        if req["Aba"] not in chaves or req["Aba"] not in abas:
            problemas.append(f"{onde}: aba de dados '{req['Aba']}' sem chave ou inexistente")
        elif req["Coluna"] not in abas[req["Aba"]].columns:
            problemas.append(f"{onde}: coluna '{req['Coluna']}' inexistente na aba '{req['Aba']}'")
        if req["Destino"] not in DESTINOS:
            problemas.append(f"{onde}: destino '{req['Destino']}' inválido")
        if req["Destino"] == "Pset" and not existe(esquema, str(req["TipoIFC"])):
            problemas.append(f"{onde}: tipo '{req['TipoIFC']}' inexistente no {nome_esquema}")
        for classe in str(req["Classes"]).split(","):
            if not existe(esquema, classe.strip()):
                problemas.append(f"{onde}: classe '{classe.strip()}' inexistente no {nome_esquema}")
        tipos_restricao = sum(not vazio(req[c]) for c in ("Valores", "Padrao")) + \
            (not vazio(req["Minimo"]) or not vazio(req["Maximo"]))
        if tipos_restricao > 1:
            problemas.append(f"{onde}: use só um tipo de restrição (Valores, Padrao ou Minimo/Maximo)")
        if str(req["Obrigatorio"]).strip().lower() not in ("sim", "não", "nao"):
            problemas.append(f"{onde}: Obrigatorio deve ser sim ou não")
    if problemas:
        raise SystemExit("Planilha do BEP com problemas:\n  " + "\n  ".join(problemas))
    dados = {aba: abas[aba] for aba in chaves}
    return chaves, requisitos, dados
```

Pontos a observar:

- `ler_bep` confere **tudo** antes de devolver os dados e, se houver qualquer problema, encerra com a lista completa. Nenhuma alteração é feita no modelo com uma planilha inconsistente;
- `converter` é a mesma função do Capítulo 10, agora com `Fator` e `Soma` opcionais;
- `restricao` monta a restrição IDS de cada linha e converte os limites para o SI.

14.5 O integrador

O programa principal executa as seis etapas. Grave-o como `c14_integrador.py`.

```
# -*- coding: utf-8 -*-
"""
c14_integrador.py | Capítulo 14: projeto integrador.

A partir de um modelo IFC e da planilha do BEP (ver c14_01_planilha_bep.py), executa:
  1. conferência da planilha (módulo c14_bep.py);
  2. geração do IDS a partir da aba Requisitos;
  3. preenchimento dos metadados, por chave única (um elemento) ou de grupo (todos os
     elementos com a mesma chave, por exemplo, todas as peças de uma linha);
  4. validação do esquema do modelo preenchido (ifcopenshell.validate, com as regras);
  5. verificação do modelo preenchido contra o IDS (IfcTester);
  6. relatórios: planilha com resumo, preenchimento, não conformidades e esquema; HTML e BCF.
Os arquivos de entrada não são alterados; tudo é gravado em arquivos novos com o prefixo --base.

Uso: python c14_integrador.py <modelo.ifc> <bep.xlsx> [--base projeto]
Testado com: Python 3.11 e 3.13; ifcopenshell 0.8.5; ifctester 0.8.5; pandas 3.0.
"""
import argparse
from collections import defaultdict
from datetime import datetime
from pathlib import Path
import pandas as pd
import ifcopenshell
import ifcopenshell.api.attribute
import ifcopenshell.api.classification
import ifcopenshell.api.material
```

```
import ifcopenshell.api.pset
import ifcopenshell.util.unit as util_unid
import ifcopenshell.validate
from ifctester import ids, reporter
import c14_bep as bep


def gerar_ids(requisitos, modelo):
    """Uma especificação por linha de requisito, aplicável às classes indicadas."""
    versao = modelo.schema_identifier
    documento = ids.Ids(title="Requisitos de informação do BEP (projeto integrador)", version="00")
    for req in requisitos:
        classes = [c.strip().upper() for c in str(req["Classes"]).split(",")]
        entidade = classes[0] if len(classes) == 1 else ids.Restriction(options={"enumeration": classes})
        rotulo = req["Propriedade"] if req["Destino"] in ("Atributo", "Pset") else req["Destino"]
        nome = f"{req['Aba']}: {req['Conjunto'] + '.' if req['Destino'] == 'Pset' else ''}{rotulo}"
        espec = ids.Specification(name=nome, ifcVersion=[versao], minOccurs=1, maxOccurs="unbounded")
        espec.applicability.append(ids.Entity(name=entidade))
        cardinalidade = "required" if str(req["Obrigatorio"]).strip().lower() == "sim" else "optional"
        escala = 1.0
        if not bep.vazio(req["Minimo"]) or not bep.vazio(req["Maximo"]):  # limites: unidade do arquivo -> SI
            escala = util_unid.calculate_unit_scale(modelo, util_unid.get_measure_unit_type(req["TipoIFC"]))
        valor = bep.restricao(req, escala)
        if req["Destino"] == "Atributo":
            faceta = ids.Attribute(name=req["Propriedade"], value=valor, cardinality=cardinalidade)
        elif req["Destino"] == "Pset":
            faceta = ids.Property(propertySet=req["Conjunto"], baseName=req["Propriedade"], value=valor,
                                  dataType=str(req["TipoIFC"]).upper(), cardinality=cardinalidade)
        elif req["Destino"] == "Material":
            faceta = ids.Material(value=valor, cardinality=cardinalidade)
        else:
            faceta = ids.Classification(system=req["Conjunto"], value=valor, cardinality=cardinalidade)
        espec.requirements.append(faceta)
        documento.specifications.append(espec)
    return documento


class Preenchedor:
    """Aplica uma linha de requisito a uma lista de elementos, reaproveitando conjuntos e materiais."""

    def __init__(self, modelo):
        self.modelo = modelo
        self.psets = {}
        self.materiais = {m.Name: m for m in modelo.by_type("IfcMaterial")}
        self.sistemas = {c.Name: c for c in modelo.by_type("IfcClassification")}

    def pset(self, elem, nome):
        if (elem.id(), nome) not in self.psets:
            self.psets[(elem.id(), nome)] = ifcopenshell.api.pset.add_pset(self.modelo, product=elem,
                                                                           name=nome)
        return self.psets[(elem.id(), nome)]

    def aplicar(self, elementos, req, valor):
        destino, texto = req["Destino"], str(valor).strip()
        if destino == "Atributo":
            for elem in elementos:
                ifcopenshell.api.attribute.edit_attributes(self.modelo, product=elem,
                                                           attributes={req["Propriedade"]: texto})
        elif destino == "Pset":
            convertido = bep.converter(valor, req["TipoIFC"], req["Fator"], req["Soma"])
            for elem in elementos:
                ifcopenshell.api.pset.edit_pset(
                    self.modelo, pset=self.pset(elem, req["Conjunto"]),
                    properties={req["Propriedade"]: self.modelo.create_entity(req["TipoIFC"], convertido)})
        elif destino == "Material":
            if texto not in self.materiais:
                self.materiais[texto] = ifcopenshell.api.material.add_material(self.modelo, name=texto)
            ifcopenshell.api.material.assign_material(self.modelo, products=elementos, type="IfcMaterial",
                                                      material=self.materiais[texto])
        else:                                          # Classificacao: uma referência por linha
            sistema = req["Conjunto"]
            if sistema not in self.sistemas:
                self.sistemas[sistema] = ifcopenshell.api.classification.add_classification(
```

```
                    self.modelo, classification=sistema)
            ifcopenshell.api.classification.add_reference(self.modelo, products=elementos,
                                                          identification=texto,
                                                          classification=self.sistemas[sistema])


def preencher(modelo, chaves, requisitos, dados):
    """Etapa 3: grava os dados das abas no modelo e devolve o registro, uma linha por linha de dados."""
    preenchedor, registro = Preenchedor(modelo), []
    for aba, chave in chaves.items():
        indice = defaultdict(list)                          # valor da chave -> elementos
        for elem in modelo.by_type("IfcElement"):
            valor_chave = getattr(elem, chave["Atributo"], None)
            if valor_chave:
                indice[str(valor_chave).strip()].append(elem)
        regras = [r for r in requisitos if r["Aba"] == aba]
        for posicao, linha in enumerate(dados[aba].to_dict(orient="records")):
            valor_chave = linha.get(chave["Coluna"])
            base = {"Aba": aba, "LinhaPlanilha": posicao + 2, "Chave": valor_chave}
            if bep.vazio(valor_chave):
                registro.append({**base, "Situacao": "SEM_CHAVE", "Elementos": 0, "Detalhe": "chave vazia"})
                continue
            elementos = indice.get(str(valor_chave).strip(), [])
            if not elementos:
                registro.append({**base, "Situacao": "NAO_ENCONTRADO", "Elementos": 0,
                                 "Detalhe": f"nenhum elemento com {chave['Atributo']} = {valor_chave}"})
                continue
            if chave["Modo"] == "unico" and len(elementos) > 1:
                registro.append({**base, "Situacao": "DUPLICADO", "Elementos": len(elementos),
                                 "Detalhe": "chave única repetida no modelo"})
                continue
            gravados, erros = 0, []
            for req in regras:
                valor = linha.get(req["Coluna"])
                if bep.vazio(valor):
                    if str(req["Obrigatorio"]).strip().lower() == "sim":   # pendência só se obrigatório
                        erros.append(f"{req['Coluna']}: vazio")
                    continue
                try:
                    preenchedor.aplicar(elementos, req, valor)
                    gravados += 1
                except (ValueError, TypeError) as erro:
                    erros.append(f"{req['Coluna']}: {erro}")
            detalhe = f"{gravados} valor(es) gravado(s)" + (" | " + "; ".join(erros) if erros else "")
            registro.append({**base, "Situacao": "OK" if not erros else "OK_COM_PENDENCIAS",
                             "Elementos": len(elementos), "Detalhe": detalhe})
    return registro


def validar_esquema(caminho):
    """Etapa 4: validação do esquema e das regras EXPRESS do arquivo gravado."""
    registro = ifcopenshell.validate.json_logger()
    ifcopenshell.validate.validate(str(caminho), registro, express_rules=True)
    return [{"Tipo": s.get("type"), "Instancia": s.get("instance"), "Atributo": s.get("attribute"),
             "Mensagem": s.get("message")} for s in registro.statements]


def verificar(documento, modelo):
    """Etapa 5: verificação IDS; devolve uma linha por falha."""
    documento.validate(modelo)
    falhas = []
    for espec in documento.specifications:
        if espec.minOccurs != 0 and not espec.applicable_entities:
            falhas.append({"Especificacao": espec.name, "Classe": None, "GlobalId": None, "Nome": None,
                           "Tag": None, "Motivo": "nenhum elemento aplicável encontrado"})
        for req in espec.requirements:
            for falha in req.failures:
                elem = falha["element"]
                falhas.append({"Especificacao": espec.name, "Classe": elem.is_a(), "GlobalId": elem.GlobalId,
                               "Nome": elem.Name, "Tag": getattr(elem, "Tag", None),
                               "Motivo": falha["reason"]})
    return falhas
```

```
def main():
    leitor = argparse.ArgumentParser(description="Projeto integrador: preenche e verifica um modelo IFC.")
    leitor.add_argument("modelo")
    leitor.add_argument("bep")
    leitor.add_argument("--base", default="projeto", help="prefixo dos arquivos gravados")
    args = leitor.parse_args()
    saida_ifc, saida_ids = Path(f"{args.base}_modelo.ifc"), Path(f"{args.base}_requisitos.ids")
    if saida_ifc.resolve() == Path(args.modelo).resolve():
        raise SystemExit("A saída deve ser um arquivo novo, diferente da entrada.")

    modelo = ifcopenshell.open(args.modelo)
    chaves, requisitos, dados = bep.ler_bep(args.bep, modelo.schema_identifier)       # etapa 1
    print(f"1. Planilha conferida: {len(requisitos)} requisitos | abas de dados: {', '.join(dados)}")

    documento = gerar_ids(requisitos, modelo)                                         # etapa 2
    if not documento.to_xml(str(saida_ids)):
        raise SystemExit("O IDS gerado não é válido segundo o XSD.")
    print(f"2. IDS gerado: {saida_ids} ({len(documento.specifications)} especificações)")

    registro = preencher(modelo, chaves, requisitos, dados)                           # etapa 3
    cabecalho = modelo.header.file_name
    cabecalho.name = saida_ifc.name
    cabecalho.time_stamp = datetime.now().replace(microsecond=0).isoformat()
    cabecalho.preprocessor_version = f"IfcOpenShell {ifcopenshell.version} (c14_integrador.py)"
    modelo.write(saida_ifc)
    situacoes = pd.Series([r["Situacao"] for r in registro]).value_counts()
    print(f"3. Preenchimento: {len(registro)} linhas de dados | "
          + ", ".join(f"{s}: {n}" for s, n in situacoes.sort_index().items()) + f" -> {saida_ifc}")

    esquema = validar_esquema(saida_ifc)                                              # etapa 4
    print(f"4. Validação do esquema: {len(esquema)} problema(s)")

    preenchido = ifcopenshell.open(saida_ifc)                                         # etapa 5
    falhas = verificar(documento, preenchido)
    atendidas = sum(1 for e in documento.specifications if e.status)
    print(f"5. Verificação IDS: {atendidas} de {len(documento.specifications)} especificações "
          f"atendidas | {len(falhas)} não conformidade(s)")

    html = reporter.Html(documento)                                                   # etapa 6
    html.report()
    html.to_file(f"{args.base}_verificacao.html")
    bcf = reporter.Bcf(documento)
    bcf.report()
    bcf.to_file(f"{args.base}_verificacao.bcf")
    resumo = pd.DataFrame([
        ["Modelo de entrada", Path(args.modelo).name], ["Planilha do BEP", Path(args.bep).name],
        ["Modelo preenchido", saida_ifc.name], ["IDS gerado", saida_ids.name],
        ["Linhas de dados", len(registro)], ["Linhas OK", int(situacoes.get("OK", 0))],
        ["Problemas de esquema", len(esquema)],
        ["Especificações atendidas", f"{atendidas} de {len(documento.specifications)}"],
        ["Não conformidades", len(falhas)],
    ], columns=["Item", "Valor"])
    with pd.ExcelWriter(f"{args.base}_relatorio.xlsx") as livro:
        resumo.to_excel(livro, sheet_name="Resumo", index=False)
        pd.DataFrame(registro).to_excel(livro, sheet_name="Preenchimento", index=False)
        colunas = ["Especificacao", "Classe", "GlobalId", "Nome", "Tag", "Motivo"]
        pd.DataFrame(falhas, columns=colunas).to_excel(livro, sheet_name="NaoConformidades", index=False)
        pd.DataFrame(esquema, columns=["Tipo", "Instancia", "Atributo", "Mensagem"]).to_excel(
            livro, sheet_name="Esquema", index=False)
    print(f"6. Relatórios: {args.base}_relatorio.xlsx, {args.base}_verificacao.html, "
          f"{args.base}_verificacao.bcf")

    print("\nPreenchimento com pendências:")
    pendentes = [r for r in registro if r["Situacao"] != "OK"]
    for r in pendentes:
        print(f"  {r['Aba']} linha {r['LinhaPlanilha']} ({r['Chave']}): {r['Situacao']} | {r['Detalhe']}")
    if not pendentes:
        print("  nenhuma")
    print("\nNão conformidades por especificação:")
    if not falhas:
        print("  nenhuma")
    contagem = defaultdict(list)
```

```
    for f in falhas:
        contagem[f["Especificacao"]].append(f["Tag"] or f["Nome"] or "(nenhum elemento)")
    for espec, itens in contagem.items():
        repeticoes = pd.Series(itens).value_counts(sort=False)          # chave -> número de falhas
        resumo_itens = ", ".join(f"{k} x{n}" if n > 1 else k for k, n in repeticoes.items())
        print(f"  {espec}: {len(itens)} | {resumo_itens}")


if __name__ == "__main__":
    main()
```

Destaques por etapa:

- **Geração do IDS** (`gerar_ids`): uma especificação por linha de requisito, obrigatória (`minOccurs=1`), para que a ausência de elementos das classes indicadas também seja apontada (armadilha 4 do Capítulo 12). A escala da unidade é obtida com `ifcopenshell.util.unit.get_measure_unit_type` e `calculate_unit_scale`: para `IfcPositiveLengthMeasure` num arquivo em milímetros, a escala é 0,001; para pressão, sem unidade declarada no projeto, é 1 (o valor já está em pascal);
- **Preenchimento** (`Preenchedor` e `preencher`): a regra é aplicada à **lista** de elementos da chave. Material e classificação são associados de uma vez a todo o grupo; uma célula vazia só é pendência se o requisito for obrigatório;
- **Validação do esquema** (`validar_esquema`): usa o registro em JSON do validador (`json_logger`), que devolve cada problema como um dicionário, pronto para a planilha;
- **Verificação** (`verificar`): percorre os resultados do IfcTester, como no Capítulo 12, incluindo a `Tag` de cada elemento, que identifica a linha de tubulação;
- **Relatórios**: a planilha tem as abas `Resumo`, `Preenchimento`, `NaoConformidades` e `Esquema`; o HTML e o BCF vêm do IfcTester.

14.6 Execução e leitura dos resultados

```
python c14_integrador.py modelo_unidade.ifc bep_unidade.xlsx
```

```
1. Planilha conferida: 15 requisitos | abas de dados: Equipamentos, Linhas
2. IDS gerado: projeto_requisitos.ids (15 especificações)
3. Preenchimento: 8 linhas de dados | NAO_ENCONTRADO: 2, OK: 4, OK_COM_PENDENCIAS: 2 -> projeto_modelo.ifc
4. Validação do esquema: 0 problema(s)
5. Verificação IDS: 6 de 15 especificações atendidas | 22 não conformidade(s)
6. Relatórios: projeto_relatorio.xlsx, projeto_verificacao.html, projeto_verificacao.bcf

Preenchimento com pendências:
  Equipamentos linha 3 (TQ-102): OK_COM_PENDENCIAS | 6 valor(es) gravado(s) | Fabricante: vazio
  Equipamentos linha 5 (B-101B): OK_COM_PENDENCIAS | 6 valor(es) gravado(s) | PesoOperacao_kg: valor numérico
inválido: 'n/d'
  Equipamentos linha 6 (B-102): NAO_ENCONTRADO | nenhum elemento com Name = B-102
  Linhas linha 4 (P-199): NAO_ENCONTRADO | nenhum elemento com Tag = P-199

Não conformidades por especificação:
  Equipamentos: ARV_Equipamento.Fabricante: 1 | TQ-102
  Equipamentos: ARV_Equipamento.PesoOperacao: 1 | B-101B
  Linhas: ARV_Processo.Fluido: 2 | P-103 x2
  Linhas: ARV_Processo.PressaoProjeto: 8 | P-102 x6, P-103 x2
  Linhas: ARV_Processo.TemperaturaProjeto: 2 | P-103 x2
  Linhas: ARV_Tubulacao.ClasseTubulacao: 2 | P-103 x2
  Linhas: ARV_Tubulacao.DiametroNominal: 2 | P-103 x2
  Linhas: Material: 2 | P-103 x2
  Linhas: ARV_Controle.Revisao: 2 | P-103 x2
```

Cada um dos seis problemas planejados foi encontrado, por um dos dois mecanismos:

PROBLEMA	REGISTRO DO PREENCHIMENTO	VERIFICAÇÃO IDS
TQ-102 sem fabricante	`OK_COM_PENDENCIAS` : `Fabricante: vazio`	Falta `ARV_Equipamento.Fabricante`
B-101B com peso "n/d"	`OK_COM_PENDENCIAS` : valor numérico inválido	Falta `ARV_Equipamento.PesoOperacao`
B-102 fora do modelo	`NAO_ENCONTRADO`	Não se aplica (não há elemento)
P-102 com 2.500 kPa	`OK` (o valor é válido como número)	Pressão fora do limite nas 6 peças da linha
P-103 fora da planilha	Não aparece (não há linha de dados)	Todas as exigências falham nas 2 peças
P-199 fora do modelo	`NAO_ENCONTRADO`	Não se aplica

Tabela 14.5: Os problemas e como foram detectados.

A tabela mostra por que as duas verificações são necessárias. O registro do preenchimento responde "o que da planilha foi gravado?"; ele não vê elementos que a planilha esqueceu (P-103) nem valores que são números válidos, mas violam o requisito (P-102). A verificação IDS responde "o modelo atende ao BEP?"; ela não vê linhas da planilha sem elemento correspondente (B-102, P-199). Juntas, cobrem os dois lados.

O IDS gerado também pode ser conferido. Trecho da especificação do diâmetro nominal, com os limites de 15 mm e 600 mm convertidos para metros:

```
<specification name="Linhas: ARV_Tubulacao.DiametroNominal" ifcVersion="IFC4X3_ADD2">
    <applicability minOccurs="1" maxOccurs="unbounded">
        <entity>
            <name>
                <xs:restriction base="xs:string">
                    <xs:enumeration value="IFCPIPESEGMENT" />
                    <xs:enumeration value="IFCPIPEFITTING" />
                    <xs:enumeration value="IFCVALVE" />
                </xs:restriction>
            </name>
        </entity>
    </applicability>
    <requirements>
        <property dataType="IFCPOSITIVELENGTHMEASURE" cardinality="required">
            <propertySet>
                <simpleValue>ARV_Tubulacao</simpleValue>
            </propertySet>
            <baseName>
                <simpleValue>DiametroNominal</simpleValue>
            </baseName>
            <value>
                <xs:restriction base="xs:double">
                    <xs:minInclusive value="0.015" fixed="false" />
                    <xs:maxInclusive value="0.6" fixed="false" />
                </xs:restriction>
            </value>
        </property>
    </requirements>
</specification>
```

A validação do esquema não apontou problemas: neste caso, o arquivo preenchido pela API continuou válido. As pendências são todas de **conteúdo**.

14.7 O ciclo de correção

Cada não conformidade é resolvida na origem do dado, depois de confirmada com o responsável:

ITEM	CORREÇÃO NA PLANILHA	ORIGEM DA INFORMAÇÃO
TQ-102	Fabricante = `Fabricante X`	Folha de dados do tanque
B-101B	Peso de operação = 640	Folha de dados da bomba
B-102	Linha removida: a bomba dosadora não faz parte deste modelo	Coordenação do projeto
P-102	Pressão de projeto = 1.000 kPa	Lista de linhas (valor digitado errado)
P-199	Linha removida: não existe no modelo	Lista de linhas
P-103	Linha incluída: Água; 600 kPa; 40 °C; A1A; DN 100; Aço-carbono ASTM A106 Gr. B; revisão B	Lista de linhas

Tabela 14.6: Correções aplicadas à planilha.

Com a planilha corrigida gravada como `bep_unidade_rev1.xlsx`, o integrador é executado de novo **sobre o modelo original**:

```
python c14_integrador.py modelo_unidade.ifc bep_unidade_rev1.xlsx --base projeto_rev1
```

```
1. Planilha conferida: 15 requisitos | abas de dados: Equipamentos, Linhas
2. IDS gerado: projeto_rev1_requisitos.ids (15 especificações)
3. Preenchimento: 7 linhas de dados | OK: 7 -> projeto_rev1_modelo.ifc
4. Validação do esquema: 0 problema(s)
5. Verificação IDS: 15 de 15 especificações atendidas | 0 não conformidade(s)
6. Relatórios: projeto_rev1_relatorio.xlsx, projeto_rev1_verificacao.html, projeto_rev1_verificacao.bcf

Preenchimento com pendências:
  nenhuma

Não conformidades por especificação:
  nenhuma
```

As 15 especificações foram atendidas. A entrega passa a ser composta pelo modelo preenchido (`projeto_rev1_modelo.ifc`), pelo IDS (`projeto_rev1_requisitos.ids`), que permite ao destinatário repetir a verificação, e pelo relatório.

> **ATENÇÃO**
>
> Execute sempre o integrador sobre o modelo **exportado**, nunca sobre um modelo já preenchido. Reprocessar a saída acumularia associações (uma segunda referência de classificação, por exemplo) e misturaria dados de revisões diferentes da planilha. A cada nova exportação ou revisão da planilha, o processo recomeça do arquivo exportado.

14.8 Como adaptar ao seu projeto

1. **Diagnostique a exportação** com o script do Capítulo 13: classes, estrutura, conjuntos existentes, unidades e GlobalId.

2. **Defina as chaves.** Qual atributo identifica cada equipamento e cada linha? Se a identificação estiver só na camada (exportações CAD), transforme-a em `Tag` com o script de camadas do Capítulo 13. Se estiver numa propriedade, adapte o índice da função `preencher`, que hoje lê atributos.
3. **Preencha a aba Requisitos** a partir do BEP: uma linha por informação, com a classe IFC correta. Se os elementos forem *proxies*, reclassifique-os antes (Capítulo 10) ou use `IfcBuildingElementProxy` nas classes, conscientemente.
4. **Monte as abas de dados** a partir dos documentos do projeto, com a mesma grafia de chave do modelo (maiúsculas, hifens, espaços).
5. **Execute numa cópia**, leia o registro de preenchimento e as não conformidades, corrija na origem (planilha ou modelo) e execute de novo, sempre a partir do arquivo exportado.
6. **Entregue** o modelo preenchido, o IDS e o relatório; registre no BEP a versão da planilha usada.

Limitações conhecidas desta versão, que servem como pontos de evolução:

- os dados são gravados nas ocorrências, não nos tipos (o Capítulo 10 mostrou a gravação em tipos e o cuidado com valores compartilhados);
- cada elemento recebe um único material simples (`IfcMaterial`);
- as chaves são lidas de atributos;
- a conversão de unidades usa fator e soma; temperaturas em graus Celsius declaradas no próprio projeto IFC (e não em kelvin) exigiriam tratamento adicional nos limites do IDS.

14.9 Resumo do capítulo

- Uma planilha única, com as abas Chaves, Requisitos e dados, governa o preenchimento e a verificação.
- Chave única identifica um elemento; chave de grupo aplica os mesmos dados a todos os elementos de uma linha de tubulação.
- O IDS é gerado a partir dos requisitos, com limites convertidos da unidade da planilha para o SI.
- O registro do preenchimento e a verificação IDS são complementares: um vê o que foi gravado, o outro vê o que falta no modelo.
- O ciclo de correção atua na origem dos dados e reprocessa sempre o arquivo exportado.
- A entrega inclui o modelo preenchido, o IDS e o relatório, o que permite ao destinatário repetir a verificação.

14.10 Exercícios

Fixação

1. Por que a aba `Linhas` usa chave de grupo e a aba `Equipamentos`, chave única? O que aconteceria se a aba `Linhas` usasse o modo `unico`?
2. Por que a linha P-103 não aparece no registro de preenchimento, mas aparece nas não conformidades?
3. Por que a pressão de 2.500 kPa da linha P-102 foi gravada, se viola o requisito?

4. Por que o integrador deve ser executado sobre o modelo exportado, e não sobre o modelo já preenchido?
5. Como o limite mínimo de 15 mm do diâmetro nominal chega ao IDS? Mostre o cálculo.

Aplicação

6. Aplique à planilha as correções da Tabela 14.6 e execute o integrador de novo. Confira o resultado com a seção 14.7 e abra as abas do relatório.
7. Acrescente um requisito **opcional** de potência das bombas: coluna `Potencia_kW` na aba `Equipamentos`; destino `Pset`, conjunto `ARV_Equipamento`, propriedade `Potencia`, tipo `IfcPowerMeasure`, fator 1000, classes `IfcPump`, obrigatório `não`. Preencha 15 kW para B-101A e deixe B-101B em branco. Qual o resultado?
8. Na planilha corrigida, troque o tipo da pressão para `IfcPressure` e o nome da coluna do fluido, na aba `Requisitos`, para `Fluído`. Execute o integrador. O que acontece com o modelo?

Desafio

9. Compare o modelo original com o modelo preenchido na seção 14.7, usando o script de comparação do Capítulo 12. Confirme que nenhum GlobalId mudou e explique o número de diferenças encontradas.
10. Escreva um script que converta as linhas de destino `Pset` da aba `Requisitos` na tabela de entrada do gerador de conjuntos do Revit (Capítulo 13), de modo que o mesmo BEP configure também a exportação do software de autoria.

14.11 Respostas dos exercícios

1. Porque todas as peças de uma linha compartilham os dados de processo e especificação, e a planilha tem uma linha por linha de tubulação; já cada equipamento tem dados próprios. Com o modo `unico`, as linhas P-101 e P-102, com várias peças cada, seriam marcadas como `DUPLICADO` e nada seria gravado nelas.
2. Porque o registro percorre as linhas da planilha, e P-103 não está na planilha. A verificação IDS percorre os elementos do modelo, e as duas peças de P-103 não têm os dados exigidos.
3. Porque 2.500 é um número válido, e o preenchimento só recusa valores que não podem ser convertidos. A adequação do valor ao requisito é papel da verificação IDS, que apontou a pressão fora do limite nas seis peças da linha.
4. Porque reprocessar a saída acumula associações (por exemplo, uma nova referência de classificação ao lado da anterior) e mistura dados de revisões diferentes da planilha. O arquivo exportado é a base estável; o preenchimento é sempre refeito a partir dele.
5. O limite passa pela conversão dos dados (fator e soma vazios: 15 x 1 + 0 = 15, na unidade do arquivo, milímetro) e pela escala do milímetro para o metro (15 x 0,001 = 0,015). O IDS recebe `minInclusive` igual a 0.015, como mostra o trecho da seção 14.6.
6. Resultado obtido: o da seção 14.7, com as 15 especificações atendidas, nenhuma pendência de preenchimento e nenhuma não conformidade. No relatório, a aba `Preenchimento` mostra as sete linhas com situação `OK`, e as abas `NaoConformidades` e `Esquema` ficam vazias.

7. Resultado obtido:

```
1. Planilha conferida: 16 requisitos | abas de dados: Equipamentos, Linhas
2. IDS gerado: ex7_requisitos.ids (16 especificações)
3. Preenchimento: 7 linhas de dados | OK: 7 -> ex7_modelo.ifc
4. Validação do esquema: 0 problema(s)
5. Verificação IDS: 16 de 16 especificações atendidas | 0 não conformidade(s)
6. Relatórios: ex7_relatorio.xlsx, ex7_verificacao.html, ex7_verificacao.bcf

Preenchimento com pendências:
  nenhuma

Não conformidades por especificação:
  nenhuma
```

A potência da B-101A foi gravada como 15.000 W (15 kW x 1000). A célula vazia da B-101B não gerou pendência, porque o requisito é opcional, e a especificação foi atendida: pela cardinalidade `optional`, a propriedade pode faltar, mas, se existir, deve ter o tipo `IfcPowerMeasure`. No modelo preenchido, a B-101A tem `ARV_Equipamento.Potencia = 15000.0`, e a B-101B não tem a propriedade.

8. Resultado obtido:

```
Planilha do BEP com problemas:
  Requisitos, linha 10: coluna 'Fluído' inexistente na aba 'Linhas'
  Requisitos, linha 11: tipo 'IfcPressure' inexistente no IFC4X3_ADD2
```

A conferência da planilha encontrou os dois erros e encerrou o programa antes de qualquer outra etapa: nenhum arquivo foi gravado, e o modelo de entrada, como sempre, não foi alterado. É o comportamento desejado: uma planilha inconsistente nunca chega ao modelo.

9. Comando: `python ..\cap12\c12_04_comparar_revisoes.py modelo_unidade.ifc projeto_rev1_modelo.ifc --saida ex9_comparacao.xlsx`. Início da saída obtida:

```
 - 21 item(s) are in the old model
 - 21 item(s) are in the new model
 - 0 item(s) were added
 - 0 item(s) were deleted
 - 21 item(s) are common to both models
 - 18 item(s) had simple changes
 - 18 item(s) were changed

Elementos alterados segundo o ifcdiff: 18
Detalhamento: 122 diferença(s) -> ex9_comparacao.xlsx
  B-101A | atributo | Description: (vazio) -> Bomba de transferência
  B-101A | propriedade | ARV_Controle.Revisao: (vazio) -> A
  B-101A | propriedade | ARV_Equipamento.Fabricante: (vazio) -> Fabricante Z
  B-101A | propriedade | ARV_Equipamento.PesoOperacao: (vazio) -> 640.0
  B-101A | relação | classificação: (vazio) -> 3E.EX.04
  B-101A | relação | material: (vazio) -> Ferro fundido
```

O ifcdiff encontrou os mesmos 21 objetos nas duas versões (18 elementos e 3 elementos espaciais), sem nenhum adicionado ou removido: os GlobalId foram preservados. As 122 diferenças correspondem exatamente ao preenchimento: 4 equipamentos x 6 informações (descrição, fabricante, peso, revisão, material e classificação) = 24, mais 14 peças de tubulação x 7 informações (fluido, pressão, temperatura, classe de tubulação, diâmetro nominal, revisão e material) = 98. Nenhuma linha de mudança de classe aparece: o preenchimento não alterou a classe IFC dos elementos.

10. Uma solução, gravada como `c14_03_requisitos_para_revit.py`:

```
# -*- coding: utf-8 -*-
"""
c14_03_requisitos_para_revit.py | Capítulo 14, exercício 10 (solução comentada).

Converte as linhas de destino "Pset" da aba Requisitos da planilha do BEP na tabela de entrada
do script c13_02_gerar_psets_revit.py (Capítulo 13). O nome do parâmetro do Revit é formado
por conjunto e propriedade (por exemplo, ARV_Processo_PressaoProjeto).

Uso: python c14_03_requisitos_para_revit.py <bep.xlsx> [--saida requisitos_revit.csv]
Testado com: Python 3.11 e 3.13; pandas 3.0.
"""
import argparse
import pandas as pd


def main():
    leitor = argparse.ArgumentParser(description="Gera a tabela de conjuntos do Revit a partir do BEP.")
    leitor.add_argument("bep")
    leitor.add_argument("--saida", default="requisitos_revit.csv")
    args = leitor.parse_args()

    requisitos = pd.read_excel(args.bep, sheet_name="Requisitos", dtype=object)
    psets = requisitos[requisitos["Destino"] == "Pset"]
    tabela = pd.DataFrame({
        "Conjunto": psets["Conjunto"],
        "Propriedade": psets["Propriedade"],
        "TipoIFC": psets["TipoIFC"],
        "Classes": psets["Classes"],
        "ParametroRevit": psets["Conjunto"] + "_" + psets["Propriedade"],
    })
    tabela.to_csv(args.saida, sep=";", index=False, encoding="utf-8")
    print(f"Tabela gravada: {args.saida} | propriedades: {len(tabela)}")


if __name__ == "__main__":
    main()
```

Execução e conversão com o gerador do Capítulo 13:

```
python c14_03_requisitos_para_revit.py bep_unidade_rev1.xlsx
python ..\cap13\c13_02_gerar_psets_revit.py requisitos_revit.csv --esquema IFC4X3_ADD2 --saida
psets_revit_bep.txt
```

```
Tabela gravada: requisitos_revit.csv | propriedades: 9
Arquivo gravado: psets_revit_bep.txt | conjuntos: 4 | propriedades: 8
# Conjuntos de propriedades definidos pelo usuário (exportador IFC do Revit)
# Gerado por c13_02_gerar_psets_revit.py a partir de requisitos_revit.csv
PropertySet: -> ARV_Equipamento -> I -> IfcTank, IfcTankType, IfcPump, IfcPumpType
 -> Fabricante -> Label -> ARV_Equipamento_Fabricante
 -> PesoOperacao -> Mass -> ARV_Equipamento_PesoOperacao
PropertySet: -> ARV_Controle -> I -> IfcTank, IfcTankType, IfcPump, IfcPumpType, IfcPipeSegment,
IfcPipeSegmentType, IfcPipeFitting, IfcPipeFittingType, IfcValve, IfcValveType
 -> Revisao -> Label -> ARV_Controle_Revisao
PropertySet: -> ARV_Processo -> I -> IfcPipeSegment, IfcPipeSegmentType, IfcPipeFitting, IfcPipeFittingType,
IfcValve, IfcValveType
 -> Fluido -> Label -> ARV_Processo_Fluido
 -> PressaoProjeto -> Pressure -> ARV_Processo_PressaoProjeto
 -> TemperaturaProjeto -> ThermodynamicTemperature -> ARV_Processo_TemperaturaProjeto
PropertySet: -> ARV_Tubulacao -> I -> IfcPipeSegment, IfcPipeSegmentType, IfcPipeFitting, IfcPipeFittingType,
IfcValve, IfcValveType
 -> ClasseTubulacao -> Label -> ARV_Tubulacao_ClasseTubulacao
 -> DiametroNominal -> PositiveLength -> ARV_Tubulacao_DiametroNominal
```

A propriedade `ARV_Controle.Revisao` aparece em duas linhas de requisito (equipamentos e tubulação); o gerador a declara uma só vez e reúne as classes das duas linhas no mesmo conjunto. Com isso, a planilha do BEP passa a configurar, de uma só fonte, a exportação do Revit, o preenchimento do IFC e a verificação IDS. Lembre a ressalva do Capítulo 13: a importação do arquivo gerado no Revit precisa ser testada na sua instalação.

14.12 Referências

As referências técnicas deste capítulo são as dos capítulos de origem de cada etapa: Capítulo 10 (API de edição do IfcOpenShell), Capítulo 12 (IDS, IfcTester, validação e ifcdiff) e Capítulo 13 (exportação dos softwares Autodesk). Documentação consultada para as funções novas:

- IfcOpenShell. Utilitários de unidades (`ifcopenshell.util.unit`): https://docs.ifcopenshell.org/autoapi/ifcopenshell/util/unit/index.html
- IfcOpenShell. Validação (`ifcopenshell.validate`): https://docs.ifcopenshell.org/autoapi/ifcopenshell/validate/index.html
- IfcOpenShell. IfcTester: https://docs.ifcopenshell.org/ifctester.html
- buildingSMART International. Unidades de valores no IDS (documentação do repositório oficial): https://github.com/buildingSMART/IDS

Controle de revisões

REVISÃO	DATA	DESCRIÇÃO
00	19/09/2026	Emissão inicial: Apresentação e Capítulos 0 a 14

Tabela R.1: Histórico de revisões do documento.